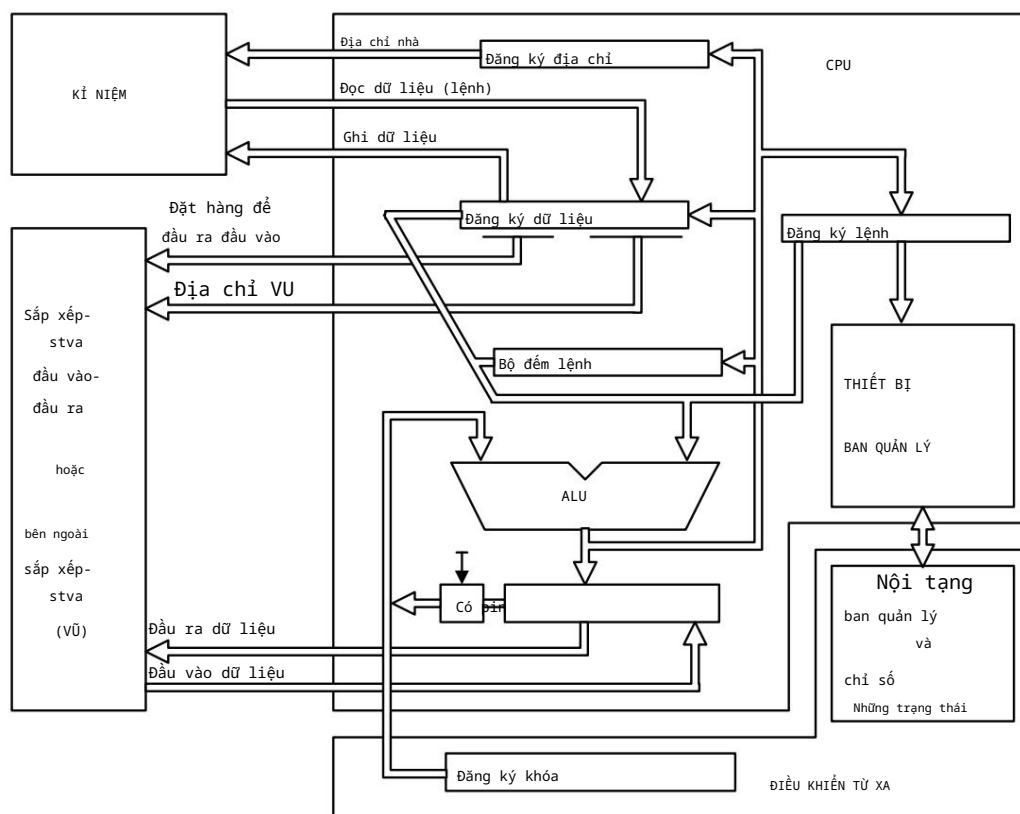


V.V. Kirillov

KIẾN TRÚC MÁY TÍNH CƠ BẢN

Hướng dẫn



St.Petersburg

2010

BỘ GIÁO DỤC VÀ KHOA HỌC LIÊN BANG NGÀ

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, CƠ KHÍ VÀ QUANG
HỌC SAINT PETERSBURG STATE

V.V. Kirillov

KIẾN TRÚC MÁY TÍNH CƠ BẢN

Hướng dẫn



St.Petersburg

2010

Kirillov V.V. Kiến trúc máy tính cơ bản

- St.Petersburg: SPbGU ITMO, 2010. - 144 tr.

Hướng dẫn thảo luận về các khái niệm cơ bản về hoạt động và cấu tạo của máy tính. Thông tin chung về biểu diễn và xử lý thông tin trong máy tính được đưa ra. Mô hình chức năng của một máy vi tính giả định (Máy tính cơ bản), có cấu trúc tương tự như bất kỳ máy tính đơn giản nào và hoạt động với mô hình này được mô tả. Trên đó, bạn có thể khám phá sự tương tác của các thiết bị máy tính trong quá trình lựa chọn và thực hiện các lệnh cũng như nghiên cứu hoạt động của nó ở cấp độ vi chương trình. Máy tính này lần đầu tiên được mô tả chi tiết trong [2,3] và kể từ đó đã được sử dụng để giảng dạy cho sinh viên tại Khoa Khoa học Máy tính, cũng như tại nhiều khoa của ITMO và một số trường đại học khác. Trong 25 năm, các mô tả trong [2,3] đã trở nên lỗi thời và cần phải sửa đổi chúng và xuất bản cuốn sách giáo khoa này, bao gồm các phần đã được chỉnh sửa trên Máy tính Cơ bản từ [2].

Sách hướng dẫn sẽ được sử dụng để chuẩn bị cho các cử nhân trong các lĩnh vực 230100 "Tin học và Kỹ thuật Máy tính", 231000 "Kỹ thuật Phần mềm" cũng như chuẩn bị cho các cử nhân trong các lĩnh vực khác, những người được giảng dạy các ngành liên quan đến công nghệ máy tính, cả hai giáo viên của Khoa Tin học và giáo viên các khoa khác.

Được đề nghị bởi Hội đồng Khoa KTiU vào ngày 13 tháng 10 năm 2009, Nghị định thư số 3



Năm 2009, Trường đã trở thành người chiến thắng trong một cuộc thi nhiều giai đoạn, kết quả là 12 trường đại học hàng đầu ở Nga đã được xác định, được trao giải "Đại học Nghiên cứu Quốc gia". Bộ Giáo dục và Khoa học Liên bang Nga đã phê duyệt Chương trình Phát triển Tổ chức Giáo dục Nhà nước về Giáo dục Chuyên nghiệp Đại học "Đại học Công nghệ Thông tin, Cơ học và Quang học St. Petersburg" cho năm 2009-2018.

GIỚI THIỆU

Vào năm 1980-1981 tại Khoa Khoa học Máy tính (CT) của Viện Cơ học và Quang học Leningrad (LITMO) đã có những cuộc thảo luận sôi nổi về cách cập nhật lớp học cho việc nghiên cứu Máy tính Điện tử (ECM). Các khán đài được tạo ra vào những năm 1960-70 đã lỗi thời về mặt đạo đức và vật lý, hơn nữa, với sự trợ giúp của chúng, không thể dạy sinh viên của nhiều khoa khác của trường đại học, những người mà Khoa Khoa học Máy tính đọc các bộ môn: "Khoa học Máy tính", "Máy tính Khoa học Kỹ thuật và Tính toán Kinh tế" và v.v. Hai cách đã được đề xuất: 1. tạo ra các tổ hợp phòng thí nghiệm giáo dục dựa trên bộ vi xử lý

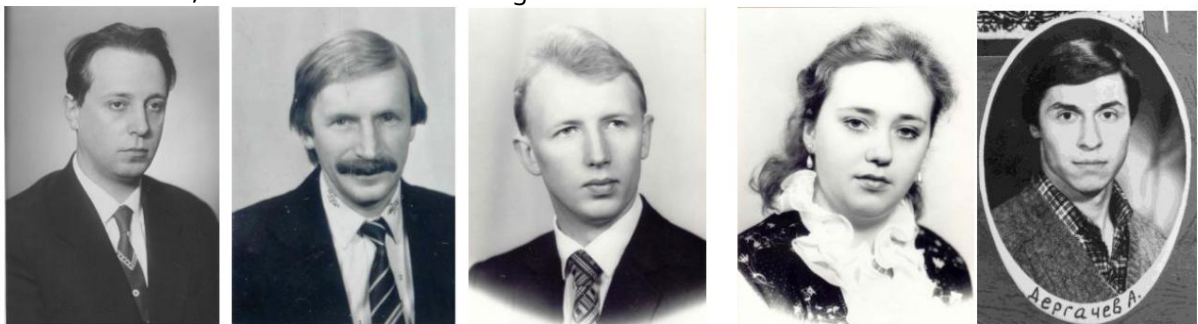
bộ dụng cụ

2. tạo ra các viết tắt cho mô phỏng máy tính và bất kỳ kỹ thuật số nào các thiết bị.

Cách đầu tiên có thể thực hiện một nghiên cứu vật lý về vi máy tính và thiết bị kỹ thuật số, nhưng yêu cầu thay thế định kỳ để phòng thí nghiệm. Vì vậy, kể từ năm 1983, khi đào tạo các tổ hợp phòng thí nghiệm dựa trên các bộ vi xử lý K584, K589, K580, một số tổ hợp phòng thí nghiệm mới đã được tạo ra và đã tiêu tốn khá nhiều tiền. Ngoài ra, các tổ hợp này rất tốt cho việc giảng dạy các chuyên gia, cử nhân và thạc sĩ trong lĩnh vực nghiên cứu "Tin học và Kỹ thuật máy tính", nhưng lại phức tạp và dư thừa để giảng dạy cho sinh viên các chuyên ngành và lĩnh vực đào tạo khác.

Cách thứ hai liên quan đến việc tạo ra một thiết bị hiển thị thông tin, ví dụ, thiết bị được mô tả trong đoạn 5.2 và mô hình phần mềm của máy tính đang nghiên cứu hoặc một thiết bị kỹ thuật số được triển khai trên bất kỳ máy tính nào. Một mô hình như vậy có thể trở thành (và đã trở thành) một máy tính cá nhân. Có thể dễ dàng thay đổi cả cấu trúc và chương trình để nghiên cứu hoạt động của máy tính và (hoặc) bất kỳ thiết bị kỹ thuật số nào trên đó.

Năm 1982-83 phó giáo sư Kirillov V.V. đã phát triển một mô hình phần mềm của "Máy tính cơ bản", được thực hiện trên "Iskra-226" bởi các sinh viên Gromov G.Yu., Gromova I.V. và Dergachev A.M.

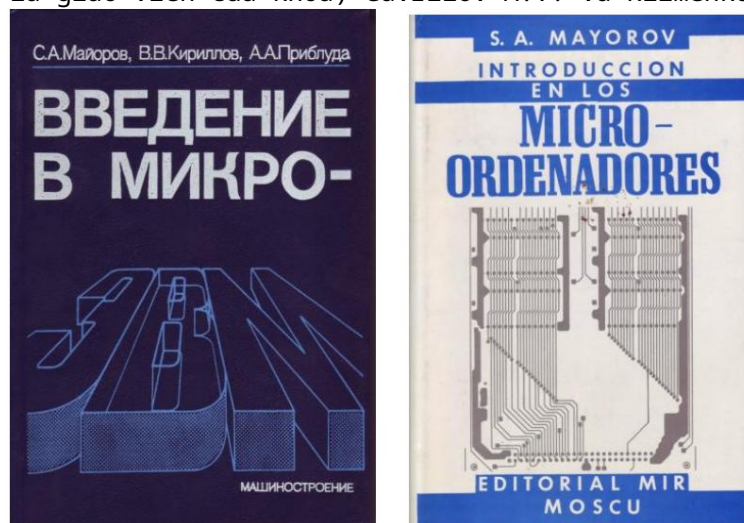


Các phó giáo sư Kirillov V.V., Pribluda A.A. và các sinh viên Gromov G.Yu., Gromova I.V., Dergachev A.M.
tại thời điểm tạo ra "Máy tính cơ bản"

Năm 1985, một lớp học được tổ chức tại khoa trên cơ sở tám máy tính Iskra-226, trong đó các lớp học về máy tính cơ bản được tổ chức bởi sinh viên của viện, giáo viên của khoa để đào tạo nâng cao cho giáo viên và gần một nghìn giáo viên. của các cơ sở giáo dục đại học của Liên Xô, như một phần của quá trình đang diễn ra trong giai đoạn 1985-86 gg. các chương trình đào tạo tin học phổ thông. Đây là động lực thúc đẩy sự ra đời của một máy tính cơ bản trong quá trình giáo dục của một số cơ sở giáo dục đại học của Liên Xô (KPI, LIIZhT, LIST, IMI, TPI, v.v.). Năm 1986, một lớp học mới được mở để nghiên cứu

cấu máy tính cơ bản trên các giá đỡ chuyên dụng, do Phó Giáo sư Pribluda A.A. phát triển, được điều khiển bởi máy vi tính Electronics 60.

Năm 1988, cuốn sách "Giới thiệu về máy vi tính" được xuất bản (do nhà xuất bản Mir dịch sang tiếng Tây Ban Nha) đã mô tả chi tiết mô hình này. Tại trường đại học của chúng tôi, sinh viên vẫn đang học những kiến thức cơ bản về cách máy tính hoạt động trên đó, bằng cách sử dụng phiên bản tiếp theo được tạo cho PC. Phiên bản này được phát triển bởi các sinh viên tốt nghiệp của khoa (bây giờ là giáo viên của khoa) Gavrilov A.V. và Klimenkov S.V.



Các phiên bản trước

Mặc dù đã được thư viện trường đại học mua vài trăm cuốn sách "Giới thiệu về máy tính vi mô" vào năm 1988, trong hơn hai mươi năm nó đã bị mòn và bị mất đi một phần bởi người đọc, do đó nó trở nên cần thiết để xuất bản những phần của cuốn sách có liên quan trực tiếp đến mô tả của máy tính cơ sở.

Sách giáo khoa được tạo ra sẽ được sử dụng cho cả sinh viên năm thứ nhất của hướng (chuyên ngành) "Tin học và Kỹ thuật máy tính", và sinh viên lớn hơn của các lĩnh vực và chuyên ngành khác, những người đang giảng dạy các ngành liên quan đến công nghệ máy tính, cả hai giáo viên của Khoa Máy tính. Khoa học và giáo viên của các bộ phận cung cấp đào tạo trong các lĩnh vực này.

1. THÔNG TIN CHUNG VỀ ĐẠI DIỆN VÀ QUÁ TRÌNH

THÔNG TIN TRONG MÁY TÍNH

1.1. Hai hình thức trình bày thông tin

Như bạn đã biết, máy tính hiện đại có thể giải quyết nhiều nhiệm vụ khác nhau. Để làm được điều này, chỉ cần sự trợ giúp của một chương trình "dạy" cho máy tính thuật toán giải một bài toán cụ thể và nhập dữ liệu ban đầu vào đó. Chương trình được viết bằng một ngôn ngữ thuật toán, đủ gần với ngôn ngữ tự nhiên (đặc biệt là tiếng Anh).

Tuy nhiên, máy tính không chỉ hiểu ngôn ngữ tự nhiên mà còn cả thuật toán. Để giải mã văn bản của chương trình được viết bằng ngôn ngữ thuật toán, trong máy phải có một chương trình dịch đặc biệt (từ tiếng Anh - dịch), chương trình này sẽ dịch văn bản của chương trình nguồn từ ngôn ngữ thuật toán sang ngôn ngữ máy tính. Nhưng ngôn ngữ máy tính trông như thế nào?

Máy tính là một thiết bị kỹ thuật trong đó thông tin về dữ liệu ban đầu của bài toán đang được giải, các quy tắc giải bài toán (thuật toán) và kết quả tính toán phải được đưa ra dưới dạng sự thay đổi của một số đại lượng vật lý:

góc quay hoặc chuyển động (chẳng hạn như đối với "truyền thông tin" đến TV để giảm âm lượng hoặc độ sáng);

từ hóa của tài liệu (ví dụ, để chơi một giai điệu bằng máy ghi âm hoặc lưu trữ dữ liệu về chứng nhận của sinh viên trong các ngành của chương trình giảng dạy trên đĩa từ);

sự chiếu sáng của màn hình (hiển thị) hoặc cảm biến quang (đầu vào thông tin từ băng đục lỗ hoặc thẻ đục lỗ).

Trong những thế kỷ trước, khi loài người chưa biết về các hiện tượng điện và từ hoặc chưa sử dụng được chúng, thì cách dễ tiếp cận nhất và do đó thuận tiện nhất là dạng biểu diễn thông tin cơ học trong các thiết bị máy tính. Trong máy đo tốc độ, các phép toán trên các số được thực hiện bằng cách sử dụng bánh xe, khi một bánh xe được thêm vào, sẽ quay 36° và với sự trợ giúp của một chốt, đặt bánh xe tiếp theo trong thâm niên bất cứ khi nào số 9 chuyển sang số 0 (mười tích lũy). Tuy nhiên, các thiết bị cơ khí cồng kềnh, đắt tiền và quá quán tính (chúng không thể được sử dụng để chế tạo máy tính phổ thông và tốc độ cao). Do đó, hiện nay trong tất cả các máy tính, tín hiệu điện (thông thường nhất là mức điện áp DC) đóng vai trò là hình thức biểu diễn thông tin chính. Để truyền tín hiệu điện, chỉ cần dây dẫn, những dây này với sự trợ giúp của nhiều loại

tín hiệu dễ dàng

biến đổi

mạch bán dẫn.

Khi sử dụng điện áp dòng điện một chiều làm vật mang thông tin, có thể có hai dạng biểu diễn giá trị số của một biến, ví dụ, X ,:

như một tín hiệu duy nhất - giá trị điện áp DC, có thể so sánh với giá trị của X (tương tự với nó).

Ví dụ, tại $X = 1845$ đơn vị, điện áp 1,845 V (thang biểu diễn 0,001 V / đơn vị) hoặc 9,225 V (thang biểu diễn 0,005 V / đơn vị) có thể được áp dụng cho đầu vào của thiết bị tính toán;

dưới dạng một số tín hiệu - một số giá trị điện áp Ví dụ, dòng điện một chiều có thể so sánh với số đơn vị trong X , số hàng chục trong X , số hàng trăm trong X , v.v.

Hình thức biểu diễn thông tin đầu tiên được gọi là tương tự hoặc liên tục (với sự trợ giúp của một giá trị tương tự - tương tự). Các giá trị được trình bày trong biểu mẫu này về cơ bản có thể nhận bất kỳ giá trị nào trong một phạm vi nhất định. Chúng có thể gần nhau một cách tùy ý, hầu như không thể phân biệt được, nhưng ít nhất về nguyên tắc vẫn có thể phân biệt được. Số lượng giá trị mà một giá trị đó có thể đảm nhận là vô hạn. Có vô số giá trị trong số chúng ngay cả trong trường hợp giá trị thay đổi trong một phạm vi giới hạn, ví dụ 0-2000 hoặc 0-0,0001. Do đó các tên - giá trị liên tục và thông tin liên tục. Từ liên tục làm nổi bật rõ ràng tính chất chính của các đại lượng như vậy - không có khoảng trống, khoảng cách giữa các giá trị mà một giá trị tương tự nhất định có thể lấy.

Hình thức biểu diễn thông tin thứ hai được gọi là kỹ thuật số hoặc rời rạc (sử dụng một tập hợp các điện áp, mỗi điện áp tương ứng với một trong các chữ số của giá trị được biểu diễn). Các đại lượng như vậy, không nhận tất cả các giá trị có thể có, mà chỉ có các giá trị khá xác định, được gọi là rời rạc (không liên tục). Không giống như một đại lượng liên tục, số lượng giá trị của một đại lượng rời rạc sẽ luôn là

cuối cùng.

So sánh các dạng biểu diễn thông tin liên tục và rời rạc, chúng ta dễ dàng nhận thấy rằng khi sử dụng dạng liên tục, người tạo ra máy tính sẽ cần một số lượng thiết bị ít hơn (mỗi giá trị được biểu diễn bằng một chữ không phải một số tín hiệu), nhưng các thiết bị này sẽ phức tạp hơn (chúng phải phân biệt một số lượng lớn hơn đáng kể các trạng thái tín hiệu). Ngoài ra, chúng tôi lưu ý rằng

các thiết bị xử lý tín hiệu liên tục có "trình độ" cao hơn (chúng có thể tích hợp tín hiệu, thực hiện bất kỳ chuyển đổi chức năng nào của nó, v.v.) và do đó, cũng như một số tính năng khác, có tốc độ cao. (Một số loại vấn đề được giải quyết nhanh hơn nhiều lần so với việc sử dụng các thiết bị có biểu diễn thông tin rời rạc.)

Tuy nhiên, do sự phức tạp của việc triển khai kỹ thuật của các thiết bị cho

các phép toán logic với các tín hiệu liên tục, lưu trữ lâu dài các tín hiệu đó, phép đo chính xác của chúng, dạng biểu diễn này chủ yếu được sử dụng trong máy tính tương tự (ACM). Những máy này được thiết kế để giải quyết các vấn đề:

- được mô tả bằng hệ thống phương trình vi phân: nghiên cứu
- hành vi của các đối tượng chuyển động (ô tô, rô bốt, tàu thủy, máy bay, v.v.);
- mô hình hóa các lò phản ứng hạt nhân, cấu trúc thủy lực, mạng khí, trường điện từ và hệ thống sinh học; giải quyết các vấn đề về tối ưu hóa tham số
- và điều khiển tối ưu; kiểm soát quá trình lọc dầu và nấu chảy

trở thành.

Nhưng các AVM không thể giải quyết các vấn đề liên quan đến lưu trữ và xử lý một lượng lớn thông tin, vốn có thể dễ dàng giải quyết bằng cách sử dụng một dạng biểu diễn thông tin kỹ thuật số (rời rạc) được thực hiện bởi máy tính điện tử kỹ thuật số (máy tính).

Bản tóm tắt:

1. Dữ liệu ban đầu, kết quả và thông tin khác do máy tính xử lý được trình bày dưới dạng bất kỳ đại lượng vật lý nào - thường là tín hiệu điện (điện áp DC).

2. Có hai hình thức biểu diễn thông tin (đại lượng vật lý) trong máy tính: tương tự (liên tục) và số (rời rạc). Trong tín hiệu đầu tiên, giá trị được biểu diễn dưới dạng một tín hiệu tỷ lệ với giá trị này, trong tín hiệu thứ hai - dưới dạng một số tín hiệu, mỗi tín hiệu tương ứng với một trong các chữ số của giá trị đã cho.

số lượng.

3. Dạng liên tục được sử dụng trong máy tính tương tự điện tử, và dạng rời rạc được sử dụng trong máy tính điện tử kỹ thuật số. Máy tính là cỗ máy vạn năng cho phép giải quyết mọi vấn đề của khoa học và công nghệ (phạm vi của những vấn đề đó ngày càng tăng). Tuy nhiên, những nhiệm vụ không yêu cầu phải lưu trữ và xử lý một lượng lớn thông tin được mô tả bằng hệ phương trình vi phân và phải được giải quyết trong thời gian tối thiểu, thì việc giải quyết với sự trợ giúp của AVM sẽ dễ dàng hơn. Họ cũng tạo ra các hệ thống điện toán lai (HCS), kết hợp các ưu điểm của cả máy tính tương tự và máy tính kỹ thuật số.

1.2. Các cách thể hiện thông tin rời rạc

Như đã trình bày ở trên, mỗi giá trị từ tập dữ liệu ban đầu của bài toán và (hoặc) kết quả của lời giải có thể được biểu diễn trong máy tính dưới dạng một số tín hiệu điện. Một trong những tín hiệu này

tương ứng với số đơn vị trong giá trị, khác - với số hàng chục, thứ ba - số lượng hàng trăm, v.v ... Tuy nhiên, cách biểu diễn tự nhiên như vậy đối với chúng tôi không phải là tốt nhất từ quan điểm kỹ thuật. Một thiết bị được thiết kế để xử lý các tín hiệu như vậy phải phân biệt được mười trạng thái trong mỗi trạng thái đó. Sẽ dễ dàng hơn nhiều khi xây dựng một thiết bị chỉ phân biệt hai trạng thái tín hiệu (có hoặc không có nó). Điều này càng hợp lý hơn, vì các thiết bị giá rẻ hiện có để nhập dữ liệu vào máy tính cũng mã hóa (đại diện) các thành phần riêng lẻ của thông tin đầu vào bằng hai trạng thái:

- lỗ có hoặc không có (băng đục lỗ và thẻ đục lỗ);
- vật liệu bị nhiễm từ hoặc khử từ (băng từ, đĩa);
- công tắc bật hoặc tắt (các khối riêng biệt của bảng điều khiển máy tính);
- nút có được nhấn hay không (bàn phím hiển thị).

Hoàn cảnh này đã thúc đẩy những người tạo ra máy tính đầu tiên sử dụng một hệ thống số khác để biểu diễn bên trong các số trong máy: thay vì hệ thống thập phân thông thường, được sử dụng trong các thiết bị máy tính cơ học, hệ thống số nhị phân đã được sử dụng. Giống như hệ thống số thập phân, hệ thống nhị phân (chỉ sử dụng các số 0 và 1) là một hệ thống số vị trí, nghĩa là, giá trị của mỗi chữ số của một số phụ thuộc vào vị trí (vị trí) của chữ số này trong ký hiệu của số. Mỗi vị trí được ấn định một trọng số nhất định. Vì vậy, số 371 có thể được viết

như:

$$3 \cdot 10^2 + 7 \cdot 10^1 + 1 \cdot 10^0 = (371), \\ 300 + 70 + 1$$

trong đó những con số có trọng lượng 101, hoặc trong ký hiệu nhị phân:

$$256 + 0 + 64 + 32 + 16 + 0 + 0 + 2 + 1 = \\ = 1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = \\ = (101110011)_2 = (371)_{10},$$

trong đó các chữ số có trọng lượng 2ⁱ.

Ở đây và bên dưới, cơ sở của hệ thống số được biểu thị dưới dạng chỉ số cho một số được đặt trong dấu ngoặc đơn,

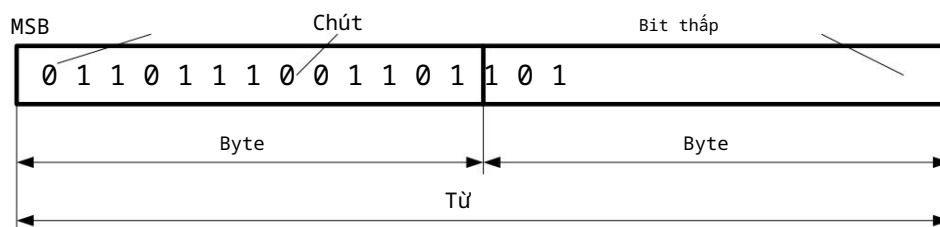
Có những thuật ngữ đặc biệt được sử dụng rộng rãi trong máy tính: bit, byte và word.

Một chữ số nhị phân thường được gọi là bit. Do đó, số 1001 là số nhị phân 4 bit và số tương đương nhị phân trước đây được coi là số 371, tức là 101110011, là số 9 bit. Bit ngoài cùng bên trái của số được gọi là bit có ý nghĩa nhất (nó có trọng số lớn nhất), bit ngoài cùng bên phải được gọi là bit ít quan trọng nhất (nó có trọng số nhỏ nhất). Một số nhị phân bao gồm 16 bit được biểu diễn trên

cơ. 1.1.

Sự phát triển của máy tính và công nghệ thông tin đã làm xuất hiện đơn vị 8-bit để trao đổi thông tin giữa các thiết bị. Đơn vị 8 bit này được gọi là byte. Nhiều loại máy tính mới và hệ thống điều khiển rời rạc xử lý thông tin ở dạng khối (từ) 8, 16, 32 hoặc 64 bit (1, 2, 4 và 8 byte). Một số nhị phân bao gồm 2 byte được hiển thị trong hình. 1.1.

Máy tính chứa một số lượng lớn các ô nhớ và thanh ghi (từ tiếng Latinh *registum* - được nhập, được ghi lại) để lưu trữ thông tin nhị phân. Hầu hết các ô này có cùng độ dài n , tức là chúng được sử dụng để lưu trữ n bit thông tin nhị phân. Thông tin được lưu trữ trong một ô như vậy được gọi là một từ. Từ của một máy tính 16-bit được hiển thị trong hình. 1.1.

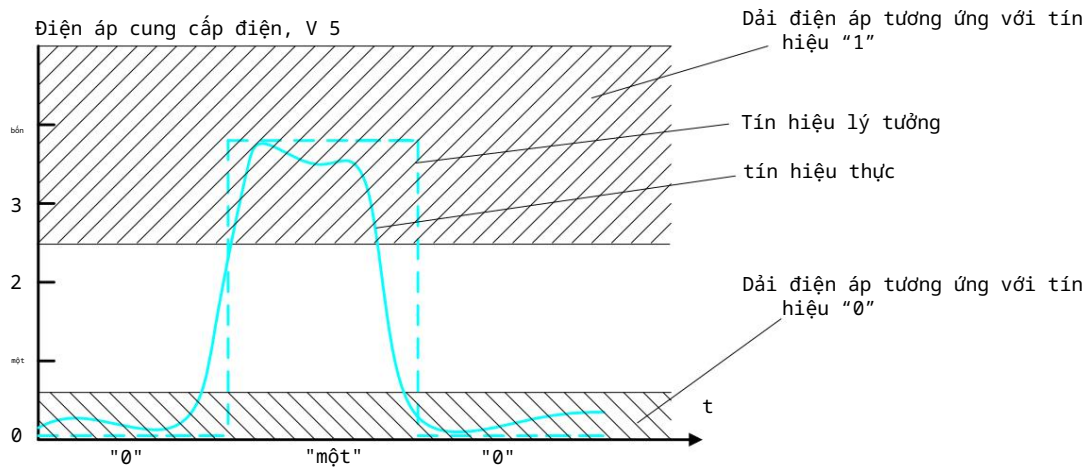


Cơ. 1.1. Bit, byte và từ

Các ô nhớ và các thanh ghi được tạo thành từ các phần tử bộ nhớ. Mỗi phần tử điện này có thể ở một trong hai trạng thái ổn định:

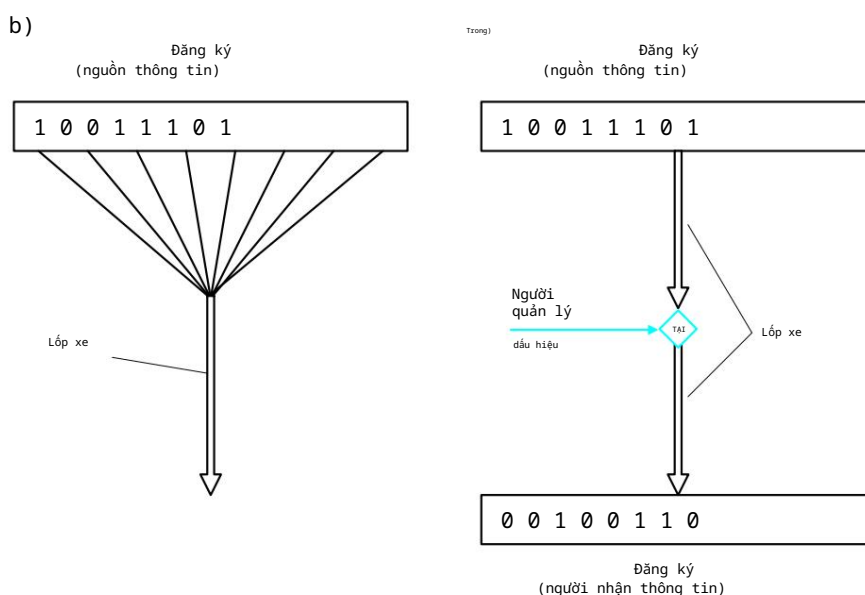
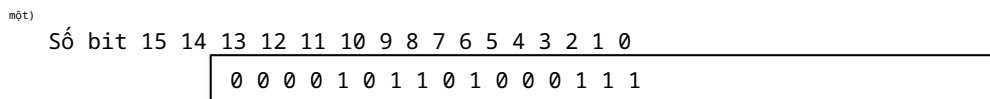
- tụ điện được sạc hoặc phóng điện,
- bóng bán dẫn ở trạng thái dẫn điện hoặc không dẫn điện,
- vật liệu bán dẫn đặc biệt có điện trở suất cao hoặc thấp, v.v ... Một trong những trạng thái vật lý này tạo ra mức điện áp đầu ra của phần tử bộ nhớ cao và mức còn lại - thấp. Trong các phần tử bộ nhớ của một số máy vi tính, đây là các điện áp có bậc tương ứng là 4 V và 0, với điện áp đầu tiên thường được coi là số nhị phân và điện áp thứ hai là số 0 nhị phân. (Mặc dù cũng có thể mã hóa ngược.)

Trên hình. 1.2 cho thấy tín hiệu đầu ra của phần tử bộ nhớ như vậy (ví dụ, một chữ số của thanh ghi) khi trạng thái của nó thay đổi (trong quá trình chuyển mạch) dưới ảnh hưởng của một số tín hiệu đầu vào. Mặc dù quá trình chuyển đổi từ 0 sang 1 và từ 1 sang 0 không xảy ra ngay lập tức, tuy nhiên, tại một số thời điểm nhất định, tín hiệu này đạt đến các giá trị được các phần tử máy tính cảm nhận là 0 hoặc 1.



CƠM. 1.2. Biểu diễn đồ thị của một tín hiệu nhị phân

Các thanh ghi thường được mô tả như trong Hình. 1.3a. Một thanh ghi được đặc trưng bởi một số duy nhất: số bit có thể được lưu trữ trong đó. Trên hình. 1.3, và các chữ số (bit) của thanh ghi 16 bit được đánh số theo lũy thừa của hai, tương ứng với vị trí của các bit của số nhị phân trong thanh ghi. Thông tin được đặt trong sổ đăng ký vẫn ở đó cho đến khi nó được thay thế bằng thông tin khác. Quá trình đọc thông tin từ sổ đăng ký không ảnh hưởng đến nội dung của sổ đăng ký sau này. Nói cách khác, thao tác đọc thông tin được lưu trữ trong thanh ghi được giảm xuống việc tạo ra một bản sao nội dung của nó, trong khi bản gốc được lưu trữ trong thanh ghi không thay đổi.



CƠM. 1.3. Thanh ghi, xe buýt và mạch cổng

Mạch điện kết nối thanh ghi với thanh ghi khác hoặc thiết bị máy tính khác được gọi là bus. Bus bao gồm các dây song song, mỗi dây được thiết kế để truyền bit tương ứng của thanh ghi (Hình 1.3, b). Hai thanh ghi 8 bit được kết nối với nhau bằng một bus tám dây. Một bus như vậy được cho là có chiều rộng là 8. Trong thực tế, bus thường chứa một số dây bổ sung được sử dụng để truyền tín hiệu đồng hồ và điều khiển, nhưng chúng tôi chưa quan tâm đến việc phân tích cấu trúc bus như vậy.

Mặc dù thực tế là dây dẫn điện cho phép bạn truyền tín hiệu theo bất kỳ hướng nào, bus chỉ được sử dụng để truyền thông tin theo hướng được chỉ ra bởi mũi tên trên bus (Hình 1.3, b, c). Việc truyền một chiều như vậy không phải do đặc tính của bus mà là do đặc tính của các mạch kết nối bus với thanh ghi hoặc các thiết bị máy tính khác. Các chương trình đặc biệt làm cho nó có thể, ví dụ, để chuyển thông tin trên xe buýt theo một hướng tại một số thời điểm và theo hướng ngược lại ở các điểm khác, tức là, để tổ chức một xe buýt hai chiều.

Mạch cổng là mạch khóa điện tử được thiết kế để điều khiển luồng thông tin từ thanh ghi đến bus và ngược lại. Trên hình. 1.3, trong cho thấy ứng dụng của mạch cổng B. Một mạch như vậy có hai đầu vào và một đầu ra. Một tín hiệu thông tin (dữ liệu từ thanh ghi) được áp dụng cho một đầu vào và một tín hiệu điều khiển cho đầu vào kia. Nếu tín hiệu điều khiển bằng một, thì dữ liệu đi qua mạch mà không có chướng ngại vật, như thể nó không tồn tại. Nếu tín hiệu điều khiển bằng 0, sẽ không có thông tin nào đi qua mạch. Để cung cấp tín hiệu thông tin cho đầu vào của mạch cổng, một bus nhiều dây thường được sử dụng. Đầu ra mạch cổng yêu cầu một bus có cùng số lượng dây. Nếu tín hiệu điều khiển bằng một, thì tín hiệu thông tin trên các bus đầu vào và đầu ra là như nhau. Do đó, khi một tín hiệu điều khiển duy nhất được áp dụng cho mạch cổng B, trạng thái của thanh ghi bộ thu thông tin sẽ thay đổi từ (00100110)₂ thành (10011101)₂, tức là nó sẽ nhận được một bản sao nội dung của thanh ghi nguồn thông tin. .

Xung đồng hồ (do bộ tạo xung nhịp máy tính tạo ra) được sử dụng để đồng bộ hóa các quá trình truyền thông tin giữa các thiết bị máy tính. Vì rất khó để minh họa nhu cầu phân phối thời gian của các hoạt động riêng lẻ để truyền thông tin trong các thiết bị kỹ thuật số bằng một ví dụ đơn giản, chúng tôi sẽ dành một ít thời gian để xem xét hoạt động của một mô hình đơn giản của máy tính vi mô trường học (Hình 1.4, a).

Máy tính bao gồm:

- kỹ thuật số (0, 1, ..., 9, ".") và chức năng (C, %, ×, ÷, -, √, +, =)

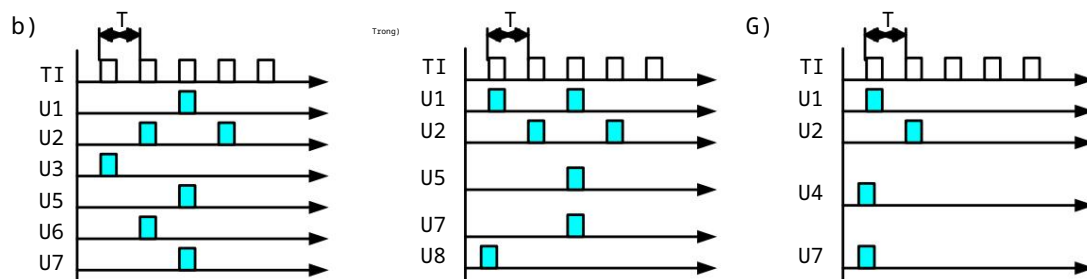
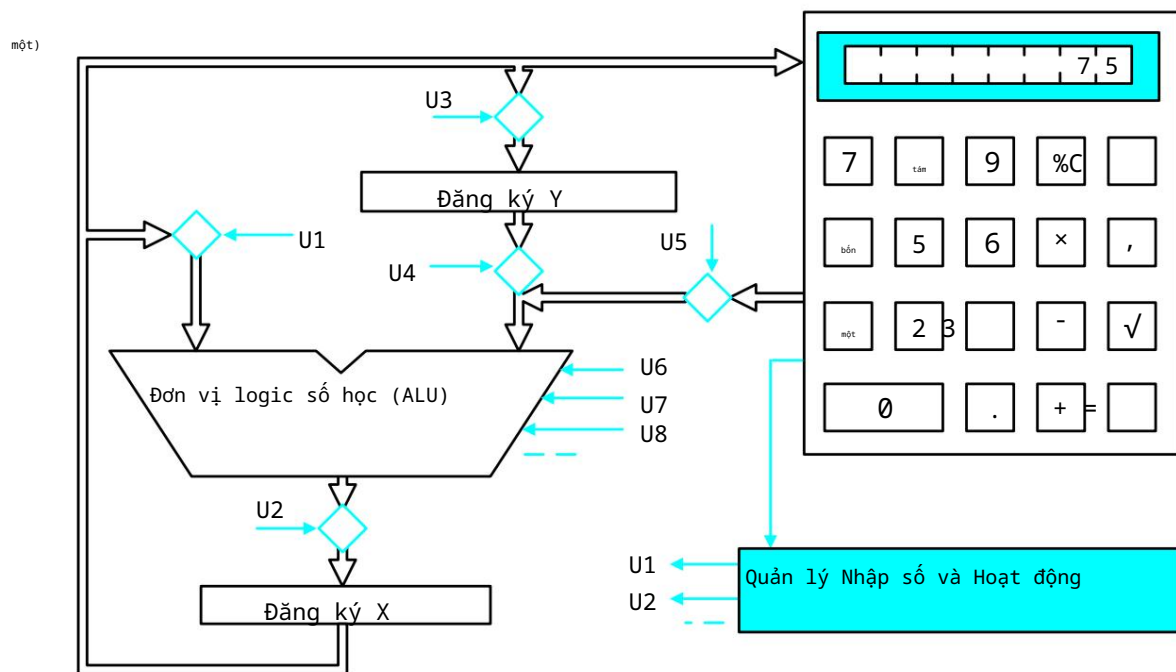
những bàn phím;

- thanh ghi chính X, nơi một toán hạng được nhập vào (một số được gõ trên bàn phím) và khi kết thúc bất kỳ thao tác nào, kết quả của nó (tổng, hiệu, v.v.) được gửi đi;
- bảng điểm để chỉ ra nội dung của thanh ghi X, tức là số đã nhập (tại thời điểm nhập nó trên bàn phím số) hoặc kết quả của các phép tính (sau khi nhấn phím chức năng);
- một thanh ghi phụ Y, trong đó nội dung của thanh ghi X được ghi đè khi một toán hạng mới được nhập vào thanh ghi cuối cùng, ví dụ, cho phép lưu trữ trong Y giá trị của số hạng trước đó hoặc số tiền tích lũy trước đó tại thời điểm một thuật ngữ mới được nhập vào X;
- đơn vị logic số học (ALU) được thiết kế để thực hiện các phép toán trên nội dung của thanh ghi X và Y (cộng, nhân, trừ và chia) hoặc chỉ thanh ghi X.

(khai thác căn bậc hai), cũng như một số hoạt động quản lý

(ví dụ, chuyển các chữ số đứng đầu của một số khi nhập chữ số mới của nó);

- một thiết bị điều khiển để nhập số và thực hiện các thao tác, sau khi nhấn phím số hoặc phím chức năng, tạo ra một chuỗi tín hiệu điều khiển (Ui) nhất định để thực hiện việc truyền thông tin cần thiết giữa các thiết bị của máy vi tính và (hoặc) cấu hình lại ALU cho một hoặc một hoạt động khác.



Cơ. 1.4. Về nhu cầu đồng bộ hóa các quá trình truyền thông tin giữa các thiết bị máy tính

Ngoài ra, lưu ý rằng mặc dù máy tính hoạt động với các tín hiệu được mã hóa bằng các chữ số nhị phân 0 và 1, ví dụ này sẽ sử dụng cách biểu diễn thập phân quen thuộc của các chữ số trong thanh ghi. Điều này cũng được chứng minh bởi thực tế là các con số được xử lý trong máy tính vì mô được mã hóa dưới dạng số thập phân được gọi là nhị phân

hệ thống số.

Điều gì sẽ xảy ra với các thanh ghi của máy tính vì mô trong quá trình nhập các số hạng và nhận tổng của các số $27 + 48 + 12976 + 38 + \dots$? Hãy bắt đầu từ thời điểm phím "+" được nhấn để hoàn thành việc nhập số hạng đầu tiên, tức là từ thời điểm khi các thanh ghi chứa các số sau:

Đăng ký Y	0
Đăng ký X	27

Bước đầu tiên. Nhập số 4 - chữ số cao nhất của số hạng thứ hai:

Thanh ghi	27
Y Thanh ghi	bốn

X Nội dung của thanh ghi X đã được viết lại để được lưu trữ trong thanh ghi Y,

thanh ghi X bị xóa và mã của phím được nhấn, tức là số 4, được viết thành chữ số có nghĩa nhỏ nhất của nó.

Bước thứ 2. Nhập số 8 - chữ số chính của số hạng thứ hai:

Thanh ghi	27
Y Thanh	48

ghi X Số có trong thanh ghi X đã được chuyển sang trái một chữ số và mã của phím được nhấn, tức là số 8, được viết thành chữ số bậc thấp còn trống.

Bước thứ 3. Nhấn phím "+":

Thanh ghi	27
Y Thanh	75

ghi X Đầu vào của chữ số cuối cùng của số hạng thứ hai (48) đã được cố định, thao tác thêm nội dung của thanh ghi X và Y ($48 + 27 = 75$) được thực hiện và thao tác chuyển kết quả sang thanh ghi X đã được thực hiện.

Bước thứ 4. Nhập số 1 - chữ số cao nhất của số hạng thứ ba:

Đăng ký Y	75
Đăng ký X	

Nội dung của thanh ghi X đã bị ghi đè ... (xem phần chú thích ở bước đầu tiên).

Các bước tiếp theo không được quan tâm, kể từ. chúng sẽ lặp lại các thao tác đã mô tả trước đó. Do đó, hãy chuyển sang việc xem xét các chuỗi tín hiệu ra Ui điều khiển được tạo kiểm soát thiết bị trong ba bước đầu tiên.

Nhập chữ số đầu tiên của bất kỳ toán hạng nào (lần nhấn phím số đầu tiên sau khi thực hiện bất kỳ thao tác nào) sẽ bắt đầu ghi lại nội dung của thanh ghi X vào thanh ghi Y, tức là tạo tín hiệu điều khiển Hoa Kỳ (Hình 1.4. 6). Đồng thời với tín hiệu này, tín hiệu U6 có thể được tạo ra, thiết lập 0 ở đầu ra của ALU. Bây giờ, thanh ghi X bị xóa, nội dung của nó được tính tổng bằng chữ số đầu vào và số tiền kết quả được ghi lại vào thanh ghi X. Để làm điều này, tín hiệu U2 được gửi đầu tiên (0 từ đầu ra ALU được ghi lại vào thanh ghi X), sau đó là đồng thời các tín hiệu U7, U1 và U5 (U7 để thiết lập ALU cho tính tổng) và cuối cùng là tín hiệu U2.

Các chữ số thứ hai (hoặc bất kỳ chữ số nào sau đây) được nhập bằng cách nhân với 10 (dịch sang trái một chữ số thập phân) nội dung của thanh ghi X và cộng giá trị đã dịch vào chữ số đầu vào. Để làm điều này, một chuỗi các tín hiệu điều khiển được tạo ra, được hiển thị trong Hình. 1,4, e (tín hiệu U8 làm nhiệm vụ tổ chức ca làm việc).

Tổng nội dung của thanh ghi X và Y (tín hiệu U7, U1 và U4) và sao chép tổng kết quả vào thanh ghi X (tín hiệu U2) được minh họa bằng biểu đồ thời gian trong hình. 1.4, thành phố

Cuối cùng chúng ta hãy thảo luận lý do tại sao một số tín hiệu điều khiển

sơ đồ thời gian trong hình. 1,4, b, c, d được tạo ra đồng thời, và những cái khác - trong những khoảng thời gian nhất định. Ví dụ, có thể tạo đồng thời các tín hiệu U1, U2, U4 U7 khi tính tổng nội dung của thanh ghi X và Y không? Phân tích đơn giản nhất cho phép chúng ta xác định rằng khi các tín hiệu U1 và U2 được tạo ra đồng thời, cũng như khi thiết lập ALU để thực hiện bất kỳ hoạt động nào với nội dung của thanh ghi X, sự không chắc chắn hoàn toàn phát sinh: nội dung của thanh ghi X nhập vào ALU, thông qua nó trở lại thanh ghi X, v.v. Mặt khác, nếu tổng của X

và Y sẽ không được cung cấp để tạo tín hiệu đồng thời U1, U4 và U7, khi đó tổng sẽ không hoạt động. Và điều gì xác định khoảng thời gian tối thiểu giữa

các tín hiệu điều khiển phải được dịch chuyển so với nhau (ví dụ, khoảng thời gian giữa các tín hiệu U1 và U2 trong Hình 1.4, d)? Khoảng thời gian T này được chọn dựa trên thời gian chuyển mạch tối đa của các phần tử máy tính (ví dụ, cho đến khi quá trình hình thành tổng $X + Y$ kết thúc, việc gửi tín hiệu điều khiển Y2 để viết lại tổng này trong thanh ghi X là vô nghĩa). Để duy trì các khoảng thời gian cần thiết, bộ tạo xung nhịp (GTI) được cài đặt trong các thiết bị điều khiển máy tính, đồng bộ hóa các tín hiệu điều khiển U_i . Có thể thấy từ các ví dụ trên, việc biểu diễn các chữ số nhị phân bằng các tín hiệu điện là khá dễ dàng. Các tín hiệu (số) này rất dễ gửi giữa các thiết bị máy tính. Chúng có thể thay đổi trạng thái của

thiết bị để lưu trữ các chữ số (phần tử bộ nhớ của thanh ghi và ô nhớ), v.v. Tuy nhiên, trong tất cả các ví dụ, chỉ các giá trị tương đương nhị phân của số nguyên thập phân không có dấu được xem xét và người ta có thể có ấn tượng rằng 0 và 1 không thể được sử dụng để mã hóa và xử lý thông tin khác. Điều này không phải như vậy, tập hợp các chữ số nhị phân (giống như tập hợp các dấu chấm và dấu gạch ngang trong mã Morse) cho phép bạn mã hóa bất kỳ thông tin nào (lên đến hình ảnh và âm thanh).

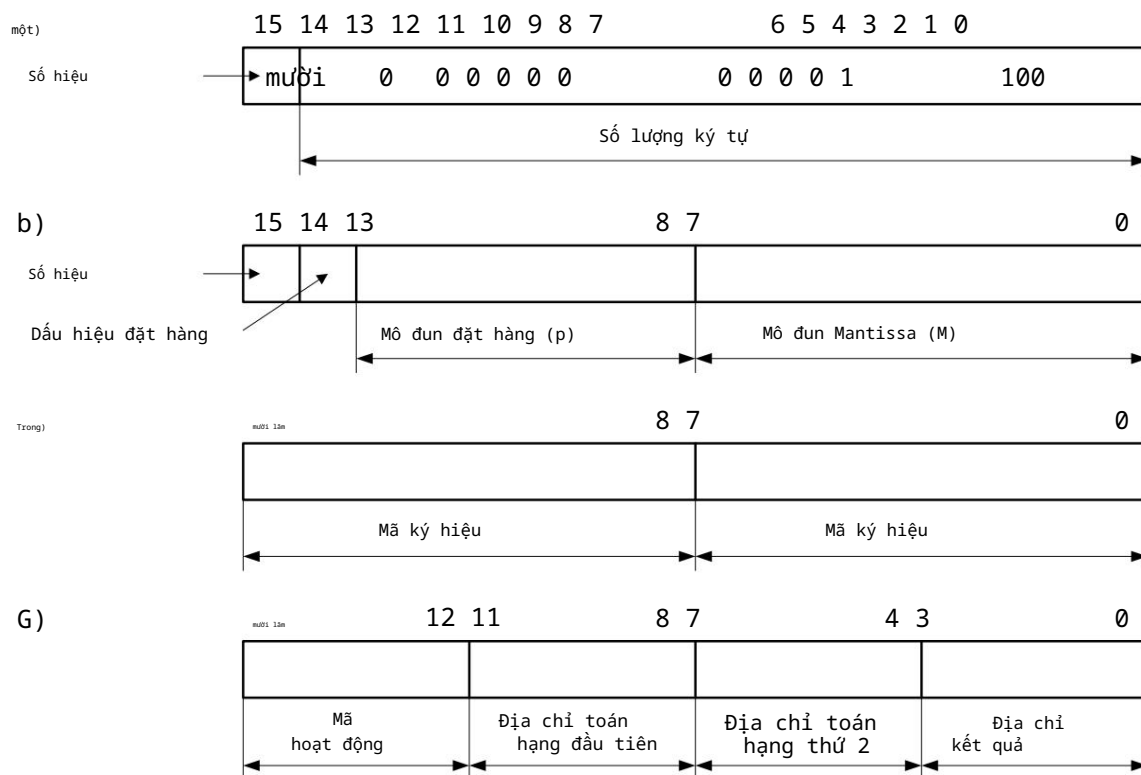
Nếu bạn cần mã hóa một số nguyên có dấu, thì bit quan trọng nhất của thanh ghi (ô nhớ) được sử dụng để lưu dấu (ví dụ: 0 cho dấu dương của số và 1 cho dấu âm). Trên hình. 1.5a cho thấy một biểu diễn như vậy của một số có dấu cho máy tính 16 bit. Các số có dạng $\pm M \cdot 10^{\pm p}$ (ví dụ: $384 \cdot 10^{-3}$ hoặc $-9 \cdot 10^{-5}$) có thể được mã hóa như trong Hình. 1,5, b. Tuy nhiên, trong 16 bit không thể lưu trữ ít nhất một số chữ số có thể chấp nhận được của phần

định trị, và thông thường các từ có độ dài từ 32 đến 64 bit được sử dụng để mã hóa các số như vậy. Các chữ cái trong bảng chữ cái tiếng Nga và tiếng Latinh, các dấu hiệu hoạt động và các ký hiệu khác được mã hóa bằng

sử dụng số 8 hoặc 16 bit (1 hoặc 2 byte) (Hình 1.5, c), cho phép bạn mã hóa $2^8 = 256$ ký tự khác nhau (xem Bảng B.1 và B.2

ứng dụng B).

Các lệnh được mã hóa theo cách tương tự (dưới dạng số nhị phân) - lệnh để thực hiện bất kỳ hoạt động nào với các từ thông tin. Vì vậy, trong bốn bit cao của lệnh trong Hình. 1,5, g, một phép toán (ví dụ, phép trừ) có thể được mã hóa, sau đó địa chỉ của các ô nhớ mà từ đó các toán hạng phải được lấy (giảm và trừ), và cuối cùng, địa chỉ của ô nhớ trong đó kết quả của hoạt động đã thực hiện được đặt (chênh lệch).



Cơ. 1.5. Các định dạng để trình bày thông tin trong máy tính 16 bit:

a - số nguyên; b - số dấu phẩy động; c - ký hiệu; d - đội

Khi chúng ta phải xử lý các thông tin như 50369, -15167, "Có" hoặc "C4C1; vào lệnh nằm trong ô nhớ có địa chỉ (4C1) 16 "", chúng ta dễ dàng phân biệt các đối tượng liệt kê bằng hình thức bên ngoài. Ở vị trí đầu tiên là số nguyên không dấu, ở vị trí thứ hai - số nguyên có dấu, ở vị trí thứ ba - từ tiếng Nga "Có" (vì ký hiệu của bảng chữ cái Nga - "D" được sử dụng) và ở thứ tư - lệnh của bước nhảy vô điều kiện của máy tính cơ sở với lời giải thích về cô ấy.

Tuy nhiên, nếu không có lời giải thích thì không thể hiểu được thông tin nào được vận chuyển bởi tập hợp các bit 1100 0100 1100 0001, với sự trợ giúp của tất cả các đối tượng được liệt kê ở trên được mã hóa trong máy tính. (Trong trường hợp này, số -15,167 được mã hóa trong cái gọi là mã bổ sung của hai, được mô tả trong Phần 2.4.)

Nhưng làm thế nào một máy tính phân biệt giữa chúng? Có hai cách để

thông báo cho máy tính về những gì được mã hóa trong chuỗi bit mà nó nhận được.

Cách đầu tiên được kết nối với việc bổ sung một chuỗi một số bit nhận dạng - metabit. Vì vậy, bằng cách đưa thêm một metabit vào chuỗi và viết vào đó, ví dụ: 0 cho các lệnh và 1 cho các loại thông tin khác, người ta có thể dễ dàng dạy máy tính phân biệt lệnh với dữ liệu. Một số siêu meta sẽ giúp xác định duy nhất các ký hiệu, số nguyên, v.v. Tuy nhiên, với phương pháp xác định chuỗi bit này, cần phải tăng dung lượng bộ nhớ máy tính để lưu trữ các siêu meta, làm phức tạp cấu trúc của nó bằng cách giới thiệu các sơ đồ phân tích metabit, tức là chi phí bổ sung được yêu cầu như nguồn nguyên liệu và thời gian (để phân tích metabits).

Cách thứ hai, được sử dụng khi làm việc với hầu hết các máy tính hiện đại, không liên quan đến chi phí tài nguyên vật liệu hoặc thời gian. Nó đòi hỏi người lập trình phải phân phối chi tiết các lệnh chương trình và dữ liệu trong bộ nhớ máy tính, chỉ báo chính xác địa chỉ của các toán hạng (kết quả hoặc chuyển tiếp) trong các lệnh và việc thực hiện một số biện pháp khác loại trừ khả năng sử dụng toán hạng như một lệnh, lệnh như một toán hạng và đưa ra các lỗi khác mà không thể phát hiện được trong quá trình thực hiện chương trình. Công việc của một lập trình viên có phần thuận lợi khi anh ta sử dụng một số ngôn ngữ thuật toán để viết chương trình, nhưng ngay cả trong trường hợp này, bạn phải mô tả chi tiết các loại biến, kích thước của mảng, v.v.

Bản tóm tắt

1. Thiết kế của máy tính cực kỳ đơn giản và máy tính hoạt động đáng tin cậy nhất (ổn định) nếu các tín hiệu lưu thông trong mạch điện tử của máy tính chỉ được sử dụng để biểu diễn hai giá trị: 0 và 1.

2. Bit (từ chữ số nhị phân tiếng Anh) là một chữ số nhị phân. Một bit chỉ có thể nhận một trong hai giá trị - 0 hoặc 1 (có hoặc không, bật hoặc tắt).

3. Một nhóm gồm tám chữ số (bit) liên kết, mà máy tính hoạt động như một thực thể duy nhất trong việc truyền, lưu trữ và xử lý thông tin, được gọi là byte. Một byte thường được sử dụng để đại diện cho các chữ cái và các ký tự đặc biệt, hoặc một cặp chữ số thập phân hoặc thập lục phân.

Các đơn vị thông tin lớn hơn là các từ. Chúng thường là bội số của một byte độ dài, và điều này giúp đơn giản hóa rất nhiều việc điều phối các quá trình xử lý thông tin trong máy tính.

4. Các chữ số nhị phân được biểu diễn trong máy tính bằng các tín hiệu (thường là điện áp), các phần tử có thể phân biệt rõ ràng của máy. Mặc dù những

các tín hiệu là liên tục, chúng có nghĩa là các ký hiệu nhị phân (0 hoặc 1) tại một số thời điểm rời rạc nhất định, thường được thiết lập bằng cách sử dụng xung đồng hồ.

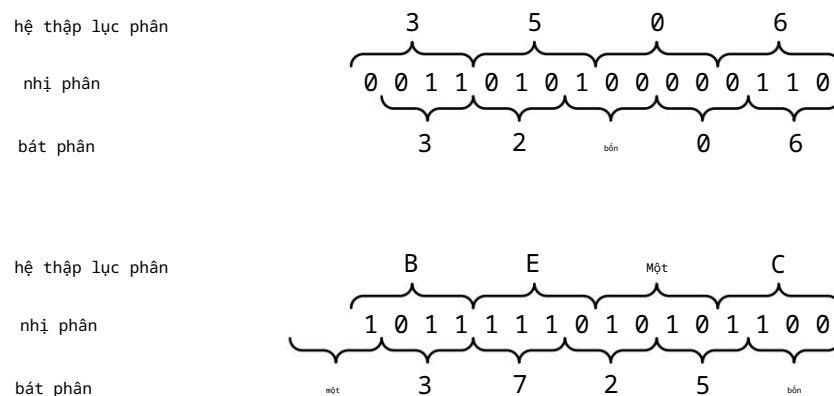
5. Với sự trợ giúp của một chuỗi các chữ số nhị phân, bất kỳ thông tin nào (số, văn bản, lệnh, v.v.) đều có thể được mã hóa, lưu trữ và xử lý trong máy tính. Các loại từ thông tin khác nhau thường khác nhau về cách chúng được sử dụng, nhưng không khác nhau về cách chúng được mã hóa.

1.3. Hệ thống số được sử dụng trong máy tính

Thuận tiện để sử dụng trong hệ thống nhị phân máy tính khá bất tiện cho việc viết và đọc số của một người. Thật vậy, thay vì, ví dụ, một số thập phân có bốn chữ số (8769) 10 bạn phải làm việc với số tương đương 14 bit của nó, số nhị phân (10001001000001) 2 . Đúng, điều này chỉ được sử dụng khi giao tiếp với máy tính ở cấp độ ngôn ngữ máy của nó. Tuy nhiên, hiện nay, một ngôn ngữ như vậy thường được sử dụng khi làm việc với bộ vi xử lý và máy vi tính.

Để giảm bớt sự phức tạp của việc xử lý thủ công các mã số, các lệnh, hệ thống số bát phân và hệ thập lục phân được sử dụng rộng rãi. Hệ thống số bát phân sử dụng 8 chữ số (0, 1, 2, 3, 4, 5, 6, 7), hệ thống số thập lục phân sử dụng 16 chữ số và 6 chữ cái Latinh viết hoa từ A đến F (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F). Vì cơ số của hệ bát phân là số $8 = 2^3$

, thì để chuyển số nhị phân sang bát phân, cần chia số nhị phân thành các nhóm 3 bit - bộ ba. Mỗi nhóm này có thể được biểu diễn bằng một chữ số bát phân, sử dụng bảng. Tương đương mã A.1 (xem Phụ lục A). Tương tự, các số nhị phân được chuyển đổi thành hệ thập lục phân (16 $\backslash$ u003d 2⁴). Chỉ trong trường hợp này, số nhị phân được chia thành các nhóm 4 bit (tứ phân), được biểu diễn bằng một chữ số thập lục phân. Chúng tôi đưa ra hai ví dụ về các phép biến đổi như vậy:



Việc chuyển đổi số nhị phân sang số thập phân có thể được thực hiện theo nhiều cách khác nhau. Đơn giản nhất của chúng theo sau trực tiếp từ định nghĩa số nhị phân. Việc chuyển đổi được thực hiện bằng cách cộng các giá trị của các lũy thừa của số 2, tương ứng với các chữ số đó của số nhị phân được dịch, chứa các đơn vị. Ví dụ: hãy chuyển số $(10111010)_2$ sang hệ thập phân:

$$\begin{array}{rcl}
 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 & 1 & 1 & 0 & 1 & & \\
 1 & 0 & 1 & & & & 0 & & & & & & & \\
 2^7 & 2^6 & 2^5 & 2^4 & 2^3 & 2^2 & 2^1 & 2^0 & & & & & & \\
 128 & + & 0 & + & 32 & + & 16 & + & 8 & + & 0 & + & 2 & + & 0 = 186 \text{ kết quả dịch}
 \end{array}$$

các chữ số của số đã dịch
số đã dịch
sức mạnh của 2

Do đó, $(10111010)_2 = (186)_{10}$.

Cụ thể, phép biến đổi ngược có thể được thực hiện như sau. Từ số thập phân cần dịch, giá trị lớn nhất của lũy thừa của số 2 bị trừ đi, không vượt quá số đã cho. Sau đó, hoạt động được lặp lại cho sự khác biệt kết quả (phần còn lại). Ngay sau khi số đã cho được phân rã hoàn toàn thành các giá trị của lũy thừa 2, biểu thức nhị phân mong muốn của nó có thể bao gồm các giá trị ở các vị trí bit tương ứng với các lũy thừa của 2 trong phép phân rã và các số không ở tất cả các vị trí khác. Ví dụ:

$$\begin{array}{rcl}
 53 & & 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0 \\
 \begin{array}{r} 53 \\ -32 \\ \hline 21 \end{array} & \rightarrow & 2^5 \leftarrow \begin{array}{|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 0 & 1 \\ \hline \end{array} \\
 \begin{array}{r} 21 \\ -16 \\ \hline 5 \end{array} & \rightarrow & 2^4 \leftarrow \begin{array}{|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 0 & 1 \\ \hline \end{array} \\
 \begin{array}{r} 5 \\ -4 \\ \hline 1 \end{array} & \rightarrow & 2^2 \leftarrow \begin{array}{|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 0 & 1 \\ \hline \end{array} \\
 \begin{array}{r} 1 \\ -1 \\ \hline 0 \end{array} & \rightarrow & 2^0 \leftarrow \begin{array}{|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 0 & 1 \\ \hline \end{array}
 \end{array}$$

thứ tư
một
một

Do đó, $(53)_{10} = (110101)_2$.

Các giá trị của lũy thừa của số 2, được sử dụng cho các phép dịch này, được đưa ra trong Bảng A.2 của Phụ lục A. Dòng đầu tiên của bảng này cho phép bạn lấy lũy thừa của số 16 ($16^0, 16^1, 16^2, 16^3, 16^4$).

Tương tự, bạn có thể chuyển đổi số bát phân và số thập lục phân sang số thập phân và ngược lại. Khi chuyển đổi số nhị phân và số bát phân sang hệ thập lục phân, chúng tôi khuyên bạn nên sử dụng các bảng trong Phụ lục A.

Các phép toán số học với các số nhị phân, bát phân và thập lục phân được thực hiện theo các quy tắc tương tự như

với các số thập phân, ngoại trừ các chữ số tiếp theo được thực hiện khi đạt đến 2, 8 và 16, chứ không phải 10 như ở số thập phân. Chúng tôi sẽ xem xét một số phép toán số học chi tiết trong đoạn 2.4 và ở đây chúng tôi đưa ra một ví dụ về việc cộng các số $(166)_{10}$ và $(47)_{10}$ trong hệ thống số nhị phân, bát phân và thập lục phân:

Chuyển	1 111		
kỳ 1 Kỳ 2 Kết	10100110	11	A6
quả	00101111		2F
	11010101	246 57 325	D5

Cuối cùng, chúng ta cũng nên đề cập đến hệ thống số thập phân-nhị phân, được sử dụng rộng rãi trong các thiết bị kỹ thuật số, trong đó phần chính của các hoạt động không liên quan đến việc xử lý và lưu trữ thông tin đầu vào, mà là đầu vào của chính nó và đầu ra cho bất kỳ các chỉ số với đại diện thập phân của kết quả thu được. (máy tính vi mô, máy tính tiền, v.v.).

Trong BCD, các chữ số thập phân từ 0 đến 9 được biểu diễn bằng các kết hợp nhị phân 4 bit từ 0000 đến 1001, tức là các số tương đương nhị phân của mười chữ số thập lục phân đầu tiên (xem Bảng A.1 của Phụ lục A). Chuyển đổi BCD sang thập phân (và chuyển đổi ngược lại) rất đơn giản và được thực hiện bằng cách thay thế trực tiếp bốn chữ số nhị phân bằng một chữ số thập phân (hoặc thay thế ngược). Hai chữ số BCD tạo nên 1 byte. Do đó, 1 byte có thể đại diện cho các giá trị từ 0 đến 99, thay vì 0 đến 255 hoặc 0 đến FF như với số thập lục phân 8 bit hoặc 2 byte. Bằng cách sử dụng 1 byte để đại diện cho mỗi hai chữ số thập phân, số BCD có thể được tạo với bất kỳ số vị trí thập phân mong muốn nào.

Vì vậy, nếu số

1001 0101 0011 1000

được coi là nhị phân, sau đó tương đương thập phân của nó

$$(1001\ 0101\ 0011\ 1000)_2 = (38200)_{10}$$

4 lần số thập phân tương đương của một số thập phân được mã hóa nhị phân

$$(1001\ 0101\ 0011\ 1000)_{2-10} = (9538)_{10}.$$

Thêm số thập phân được mã hóa nhị phân với một chữ số thập phân cũng giống như thêm số nhị phân 4 bit không dấu, ngoại trừ việc phải sửa lại nếu nhận được kết quả lớn hơn 1001. Kết quả được sửa bằng cách thêm mã nhị phân của số 6, tức là mã 0110.

Ví dụ:

$$\begin{array}{r}
 +^4 \\
 \hline
 5 \\
 \hline
 9
 \end{array}
 \quad
 \begin{array}{r}
 +^4 0100 \\
 \hline
 0101 \\
 \hline
 1001
 \end{array}
 \quad
 \begin{array}{r}
 +^5 \\
 \hline
 9 \ 14 \\
 \hline
 0110
 \end{array}
 \quad
 \begin{array}{r}
 +^5 0101 \\
 \hline
 1001 \\
 \hline
 +1110 \\
 \hline
 0110 - \text{sửa } 10 + 4 \ 1 \\
 0100
 \end{array}
 \quad
 \begin{array}{r}
 +^9 1001 \\
 \hline
 9 \ 1001 \\
 \hline
 18 \ 1 \ 0010 \\
 \hline
 + \\
 \hline
 0110 - \text{chỉnh sửa} \\
 10 + 8 \ 1 \ 1000
 \end{array}$$

Nếu phép cộng số nhị phân ban đầu hoặc phép cộng số hiệu chỉnh dẫn đến giá trị mang, thì khi thêm các số BCD có nhiều chữ số, giá trị sẽ được chuyển đến vị trí thập phân tiếp theo:

$$\begin{array}{r}
 +1889 \\
 \hline
 6376 \\
 \hline
 8265
 \end{array}
 \quad
 \begin{array}{r}
 +0001 \\
 \hline
 0110 \\
 \hline
 0111
 \end{array}
 \quad
 \begin{array}{r}
 +1000 \\
 \hline
 0011 \\
 \hline
 +1011 \\
 \hline
 0110 \\
 \hline
 + \\
 \hline
 10010
 \end{array}
 \quad
 \begin{array}{r}
 +1000 \\
 \hline
 0111 \\
 \hline
 +1111 \\
 \hline
 0110 \\
 \hline
 + \\
 \hline
 1 \ 0110
 \end{array}
 \quad
 \begin{array}{r}
 +1001 \\
 \hline
 0110 \\
 \hline
 +1111 \\
 \hline
 0110 \\
 \hline
 0101
 \end{array}$$

Bản tóm tắt

1. Để thuận tiện cho việc thực hiện các thao tác thủ công để chuẩn bị và gỡ lỗi các chương trình máy tính vi mô, mã nhị phân của lệnh và dữ liệu thường được thay thế bằng mã bát phân hoặc thập lục phân.

2. Việc chuyển đổi số nhị phân sang bát phân (thập lục phân) được thực hiện bằng cách tách số này thành các nhóm 3 bit (4 bit) và thay thế mỗi nhóm bằng một chữ số bát phân (thập lục phân).

3. Việc chuyển đổi các số bát phân (thập lục phân) sang nhị phân được thực hiện bằng cách thay thế mỗi chữ số bát phân (thập lục phân) bằng mã nhị phân 3-bit (4-bit) của nó.

4. Một số nhị phân có thể được chuyển đổi thành số thập phân bằng cách biểu diễn nó dưới dạng tổng các lũy thừa của 2.

5. Một số thập phân có thể được chuyển đổi thành nhị phân bằng cách trừ liên tiếp các lũy thừa của 2, bắt đầu với lũy thừa lớn nhất có trong số đó. Kết quả là các hệ số tìm được được sắp xếp theo một trình tự sao cho hệ số ở lũy thừa cao nhất của số 2 nằm trong bit cao nhất của số nhị phân.

6. Một số máy vi tính có thể xử lý thông tin được trình bày dưới dạng mã nhị phân của các chữ số thập phân, nghĩa là, như thể trong một hệ thống số thập phân. Hệ thống số này được gọi là hệ thập phân nhị phân. Chuyển đổi số từ hệ thập phân

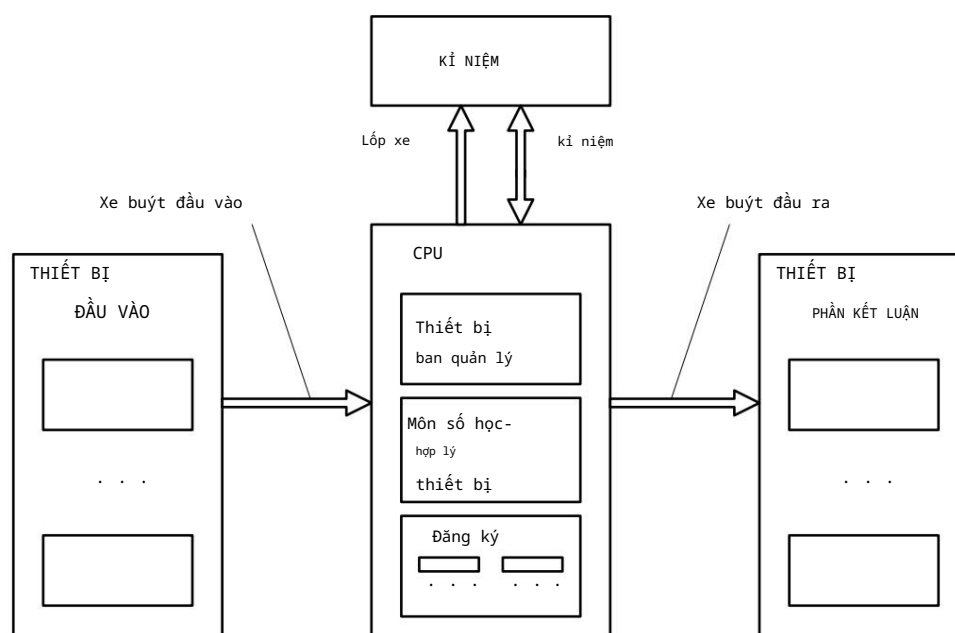
Chuyển đổi BCD (và chuyển đổi ngược) được thực hiện bằng cách thay thế trực tiếp từng chữ số thập phân của số bằng bốn chữ số nhị phân (hoặc thay thế ngược lại).

7. Cơ sở của hệ thống số thường được biểu thị dưới dạng chỉ số cho số được đặt trong dấu ngoặc đơn.

1.4. Cấu tạo và nguyên lý hoạt động của máy tính

Một máy tính điển hình bao gồm một bộ xử lý, bộ nhớ và các thiết bị đầu vào / đầu ra (Hình 1.6). Kể từ sự ra đời của những chiếc máy tính kỹ thuật số điện tử đầu tiên vào những năm 1940, công nghệ sản xuất của từng hệ thống trong số ba hệ thống con này đã được cải thiện đáng kể. Trong thập kỷ qua, nhờ sự phát triển của công nghệ tích hợp, hiệu suất của bộ vi xử lý và bộ nhớ đã được cải thiện đáng kể. Ngoài ra, chi phí của họ đã được giảm bớt. Ở thời điểm hiện tại, với giá của một chiếc tivi màu tốt, người ta có thể mua cho cá nhân một chiếc máy tính đủ mạnh, mà vào những năm 70, người ta sẽ phải trả nhiều hơn cho một số chiếc ô tô đắt tiền nhất.

Bất chấp những tiến bộ trong công nghệ, không có thay đổi đáng kể nào trong cấu trúc và hoạt động cơ bản của máy tính. Vì vậy, trở lại năm 1946, trong mô tả về chiếc máy tính được đề xuất đầu tiên có chương trình được lưu trữ trong bộ nhớ, người ta đã lưu ý rằng: "Chúng ta có ... hai loại bộ nhớ cá nhân: bộ nhớ các con số và bộ nhớ lệnh. Tuy nhiên, các hướng dẫn được hiển thị cho máy dưới dạng mã số, và nếu bằng cách nào đó máy có thể phân biệt các số với các lệnh, thì khối bộ nhớ có thể được sử dụng để lưu cả hai.



Cơ. 1.6. Sơ đồ khối tổng quát của máy tính

Bất chấp những tiến bộ trong công nghệ, không có thay đổi đáng kể nào trong cấu trúc và hoạt động cơ bản của máy tính. Vì vậy, trở lại năm 1946, trong mô tả về chiếc máy tính được đề xuất đầu tiên có chương trình được lưu trữ trong bộ nhớ, người ta đã lưu ý rằng: "Chúng ta có ... hai loại bộ nhớ cá nhân: bộ nhớ các con số và bộ nhớ lệnh. Tuy nhiên, các hướng dẫn được hiển thị cho máy dưới dạng mã số, và nếu bằng cách nào đó máy có thể phân biệt các số với các lệnh, thì khối bộ nhớ có thể được sử dụng để lưu cả hai.

Và hiện tại, trong hầu hết tất cả các máy tính, cùng một bộ nhớ được sử dụng để lưu trữ dữ liệu (số, văn bản) và lệnh. Điều này làm cho nó có thể tăng hiệu quả của việc sử dụng bộ nhớ máy tính khá đắt tiền, vì trong số các vấn đề mà nó giải quyết có các vấn đề về xử lý khá phức tạp (nhiều lệnh) một số lượng nhỏ dữ liệu ban đầu (ví dụ, một số tính toán khoa học) và các bài toán liên quan đến xử lý theo thuật toán đơn giản (ít lệnh) khối lượng dữ liệu rất lớn (xử lý tổng điều tra dân số, biên chế trong một doanh nghiệp lớn, v.v.).

"Trái tim" của máy tính là bộ xử lý, bao gồm:

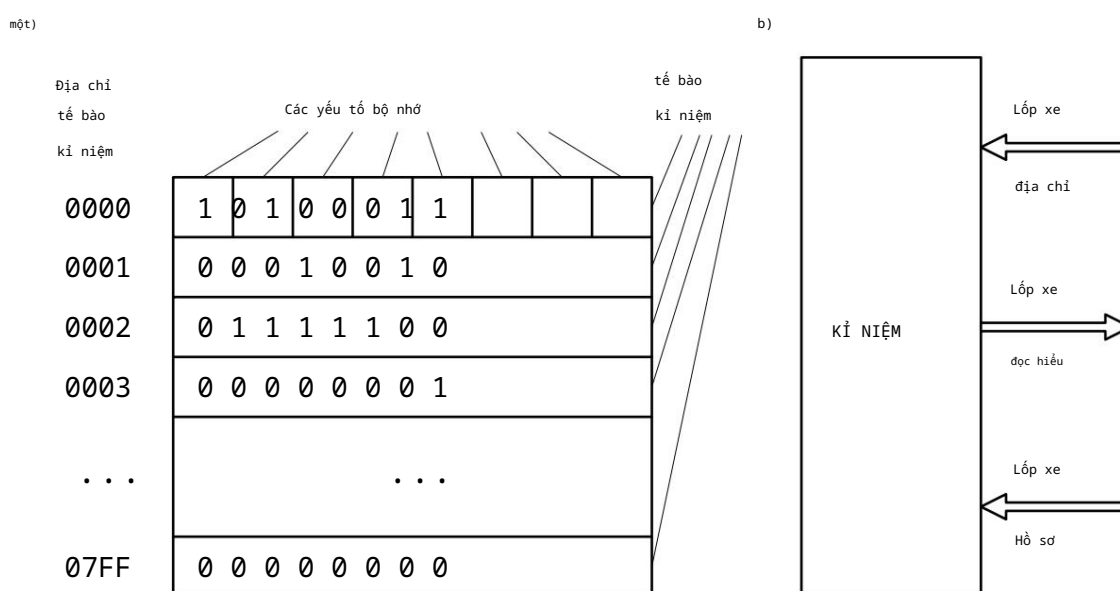
- thiết bị điều khiển để tìm nạp các lệnh từ bộ nhớ và việc thực thi chúng; đơn vị số học-lôgic thực hiện các phép toán trên dữ liệu;
-
- thanh ghi lưu trữ tạm thời dữ liệu và trạng thái bộ xử lý;
- mạch để điều khiển và giao tiếp với bộ nhớ và hệ thống con đầu vào đầu ra.

Chi tiết hơn về cấu trúc của bộ vi xử lý sẽ được thảo luận ở cuối chương này.

Thiết bị đầu vào cung cấp thông tin đọc (dữ liệu ban đầu và chương trình giải quyết vấn đề) từ một số vật mang thông tin nhất định (bàn phím, băng đục lỗ, băng từ hoặc đĩa, cảm biến trạng thái của các đối tượng được điều khiển, v.v.) và sự trình bày của nó dưới dạng tín hiệu điện nhận được bởi các thiết bị máy tính khác (bộ xử lý hoặc bộ nhớ).

Các thiết bị đầu ra trình bày kết quả xử lý thông tin dưới dạng thuận tiện cho việc cảm nhận trực quan (chỉ thị, máy in, máy vẽ, màn hình hiển thị, v.v.). Nếu cần, chúng cung cấp lưu trữ kết quả trên phương tiện mà từ đó có thể nhập lại các kết quả này vào máy tính để xử lý thêm (băng đục lỗ, băng từ, đĩa từ, v.v.) hoặc chuyển kết quả

trên các cơ quan điều hành của một đối tượng được điều khiển (ví dụ: rô bốt). Bộ nhớ máy tính bao gồm một thiết bị cung cấp chức năng lưu trữ các lệnh và dữ liệu. Thiết bị này bao gồm các khối có cùng kích thước - các ô nhớ được thiết kế để lưu trữ một từ thông tin (Hình. 1.7, a). Đổi lại, một ô nhớ bao gồm các phần tử bộ nhớ, trạng thái của mỗi phần tử trong số đó tương ứng với một chữ số nhị phân (được gọi là bit). Tập hợp các phần tử của một ô là nội dung của ô nhớ này.



Cơ. 1.7. Tổ chức bộ nhớ máy tính

Các ô được đánh số bằng các số 0, 1, 2, được gọi là địa chỉ tế bào. Nếu bạn cần ghi một từ vào bộ nhớ, bạn nên áp dụng các tín hiệu bus địa chỉ bộ nhớ (Hình 1.7, b) tương ứng với địa chỉ của ô mà bạn muốn đặt từ đang được viết và áp dụng chính từ đó vào xe buýt ghi. Bộ nhớ được sắp xếp theo cách mà từ đã cho sẽ được chuyển đến ô có địa chỉ xác định và có thể được lưu trữ ở đó trong một thời gian dài tùy ý. Bất cứ lúc nào, bằng cách truy cập vào bộ nhớ, bạn có thể lấy nội dung của từ được lưu trữ ở đó. Để thực hiện việc này, một địa chỉ được gửi đến bộ nhớ xác định vị trí của từ cần thiết và nó cấp một bản sao của từ trên bus đã đọc. Khi đọc, nội dung của ô không thay đổi, vì vậy khi bạn viết một từ, bạn có thể nhận được các bản sao của nó nhiều lần tùy thích. Điều này tương tự như ghi một bài hát trên băng. Bạn có thể nghe bài hát từ băng (đọc từ băng) bao nhiêu lần tùy thích, nhưng nếu bạn ghi lại một giai điệu khác vào vị trí của nó, giai điệu đầu tiên sẽ bị xóa. Tuy nhiên, thời gian truy cập thông tin trên băng từ phụ thuộc vào nơi ghi thông tin này (đôi khi bạn cần tua lại gần hết

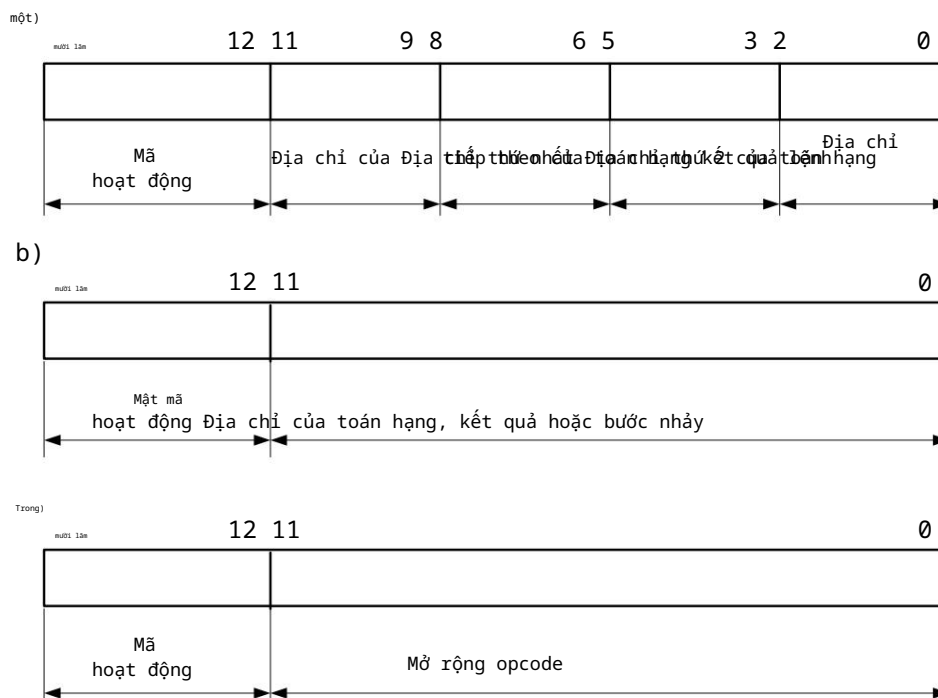
toàn bộ băng để nghe bất kỳ bài hát nào), trong khi thời gian truy cập vào bất kỳ ô nhớ nào luôn bằng nhau (không phụ thuộc vào số của nó).

Trên hình. 1.7a hiển thị một bộ nhớ có $4096 = 2^{12}$ từ 8 bit, tức là chứa 4096 byte thông tin. Với cùng số lượng phần tử lưu trữ, có thể tổ chức bộ nhớ gồm 32.768 từ 1 bit, 2048 từ 16 bit, 1024 từ 32 bit, v.v. Nếu bạn cần xử lý thông tin có thể được mã hóa chỉ bằng hai ký tự (ví dụ, một số dữ liệu điều tra dân số: giới tính, tình trạng hôn nhân, v.v.), sẽ có lợi khi sử dụng tổ chức bộ nhớ đầu tiên. Trong trường hợp tính toán chính xác, bộ nhớ gồm các từ 32 bit hoặc thậm chí 64 bit được chấp nhận nhiều hơn. Tuy nhiên, khi chọn dung lượng của ô nhớ, người ta phải tính đến việc chúng cũng phải lưu trữ các lệnh chương trình được bộ xử lý sử dụng để xử lý dữ liệu đó. Nhưng một đội máy tính được xây dựng như thế nào và năng lực của nó là gì?

Lệnh máy tính ban đầu chứa thông tin sau (Hình 1.8, a).

1. Mã hoạt động cho biết thao tác được thực hiện bởi máy tính (cộng, trừ, nhân, so sánh, đổi dấu, v.v.).

2. Địa chỉ của hai toán hạng - các đối số của phép toán, ví dụ, phụ, trừ và trừ, thừa số, v.v. Nếu bất kỳ toán hạng nào là hằng số, thì giá trị của chính toán hạng đó có thể được chỉ định trong lệnh thay vì địa chỉ của nó. Tuy nhiên, trường hợp này phải được phản ánh trong opcode để máy tính sử dụng phần tương ứng của lệnh làm toán hạng, chứ không phải địa chỉ của ô nhớ trong đó toán hạng này được lưu trữ.



Cơ. 1.8. Các định dạng
lệnh: a - bốn địa chỉ; b - unicast; c - không mặc quần áo

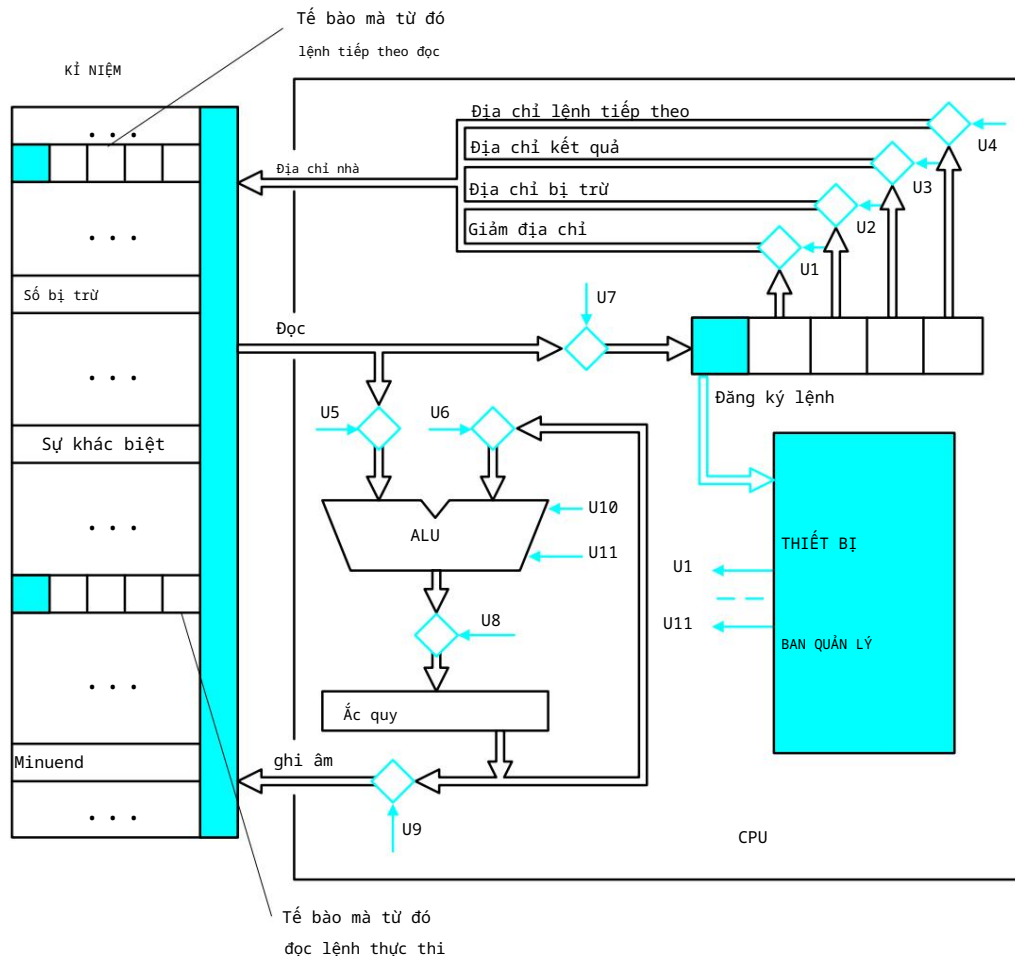
3. Địa chỉ của ô nhớ sẽ được đặt

kết quả hoạt động.

4. Địa chỉ của lệnh tiếp theo.

Một lệnh như vậy, bao gồm năm trường (mã hoạt động và bốn địa chỉ), có thể được thực thi bởi các bộ xử lý thuộc nhiều cấu trúc khác nhau.

Cấu trúc đơn giản nhất trong số các cấu trúc này (Hình 1.9) giống với cấu trúc bộ vi xử lý máy tính vi mô được thể hiện trong Hình 1.9. 1.4. CPU chứa một thiết bị điều khiển, một ALU, một thanh ghi để đặt một lệnh thực thi (thanh ghi lệnh) và một thanh ghi để đặt một trong các toán hạng hoặc kết quả của một thao tác trong quá trình thực hiện lệnh này - một bộ tích lũy (trong Hình 1.4, và vai trò của bộ tích lũy được thực hiện bởi thanh ghi X). Hãy xem xét điều gì xảy ra trong bộ xử lý sau khi một số lệnh được ghi lại từ bộ nhớ vào thanh ghi lệnh của nó, ví dụ, một lệnh trừ.



Cơ. 1.9. Một bộ xử lý đơn giản hoạt động trên lệnh 4 địa chỉ được tìm nạp hoàn toàn từ một vị trí bộ nhớ duy nhất

Vì trường của thanh ghi lệnh, nơi lưu trữ mã hoạt động, được kết nối bằng một bus với thiết bị điều khiển, nên sau này sẽ nhận được lệnh thực hiện thao tác "Trừ" và chuyển sang chế độ tạo tín hiệu điều khiển:

- việc tạo đồng thời các tín hiệu U1, U5, U10 và U8 cung cấp cho việc chuyển từ bộ nhớ giá trị số của giá trị giảm, tín hiệu của nó đi qua ALU (tín hiệu U10) và ghi vào pin;
- xung đồng hồ tiếp theo bắt đầu tạo ra các tín hiệu U2, U5, U6 và U11, dẫn đến việc chuyển giá trị số của giá trị bị trừ khỏi bộ nhớ, sang hoạt động trừ giá trị này (tín hiệu U11) khỏi nội dung của bộ tích lũy (từ một trong những giảm);
- trên tín hiệu U8, sự khác biệt nhận được được ghi vào bộ tích lũy thay vì giảm;

- khi đó tín hiệu US và U9 sẽ đảm bảo việc chuyển phần chênh lệch từ bộ tích lũy đến ô nhớ được chỉ đến bởi trường áp chót của lệnh;
- Cuối cùng, với sự trợ giúp của các tín hiệu U4 và U7, lệnh tiếp theo của chương trình sẽ được ghi lại vào thanh ghi lệnh.

Cấu trúc được xem xét của bộ xử lý và cấu trúc của lệnh bốn địa chỉ mà nó thực hiện có vẻ khá tự nhiên. Thật vậy, không làm phức tạp việc truy cập vào nội dung của ô nhớ, không thể nhận đồng thời cả hai toán hạng từ nó và do đó, toán hạng đầu tiên phải được lưu trữ trong bộ xử lý cho đến khi nhận được toán hạng tiếp theo. Do đó, bộ tích lũy được bao gồm trong bộ xử lý (nó được đặt tên như vậy vì nó có thể được sử dụng để tích lũy - tích lũy - các số hạng trong quá trình lấy tổng của nhiều số hạng hoặc tích lũy kết quả của các phép tính khác).

Một thanh ghi cũng cần thiết để lưu lệnh thực thi trong bộ xử lý (thanh ghi lệnh), vì trong quá trình thực hiện lệnh này, các thông tin khác nhau được sử dụng bởi thiết bị điều khiển và bộ nhớ phải được chọn từ nó. Tuy nhiên, có nên giữ tất cả các trường của lệnh trong thanh ghi lệnh cùng một lúc, nếu chúng vẫn được sử dụng tuần tự?

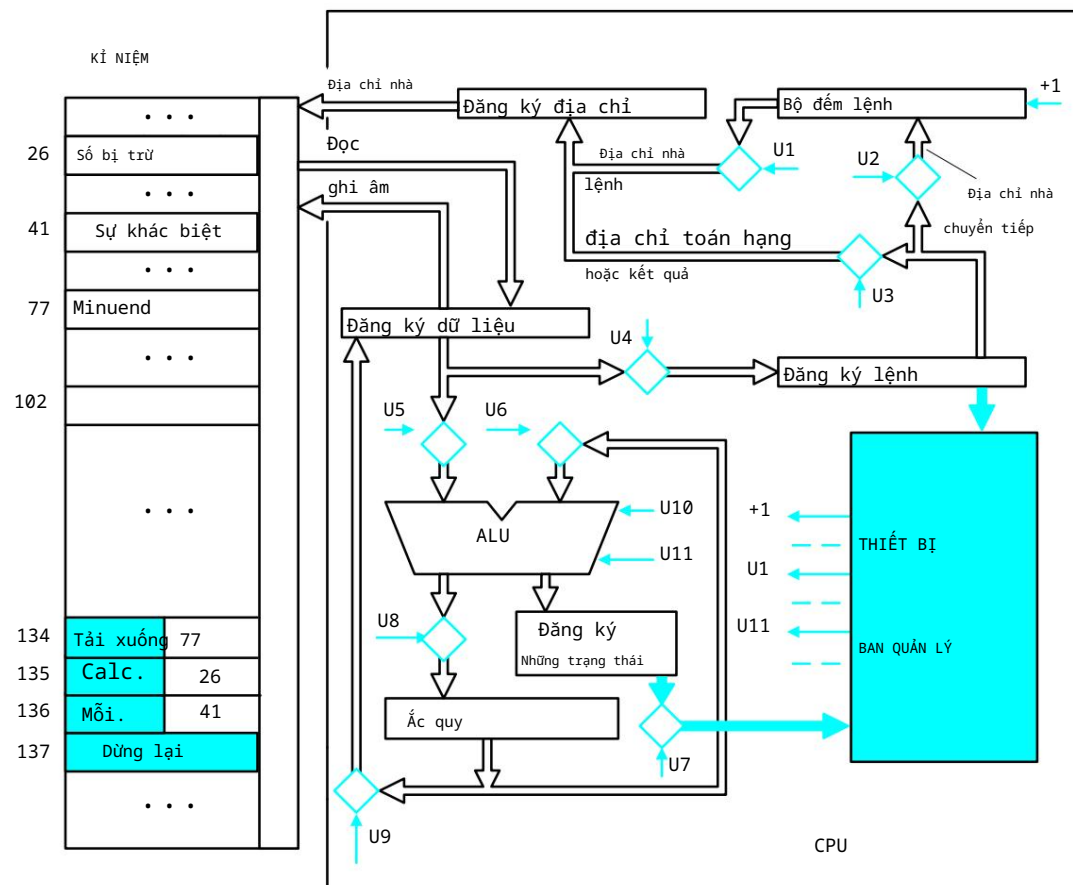
Khi một lệnh được lưu trữ trong một vị trí bộ nhớ, không có cách nào để đọc các trường riêng lẻ của lệnh đó từ bộ nhớ. Vì vậy, tốt hơn là nên đặt câu hỏi theo cách này: nên lưu một lệnh bốn địa chỉ trong một ô nhớ hay tốt hơn là lưu trữ các trường riêng lẻ của nó trong một số ô nhớ?

Một lệnh bốn địa chỉ ngắn (xem Hình 1.8, a) chắc chắn phải được lưu trữ hoàn toàn trong ô nhớ 16 bit. Nhưng một lệnh như vậy chỉ có thể giải quyết được $2^3 = 8$ ô nhớ. Máy vi tính hiện đại đánh địa chỉ bộ nhớ chứa ít nhất $2^{16} = 65.536$ ô. Nếu bạn tạo một lệnh bốn địa chỉ hoạt động với bộ nhớ có kích thước này, thì 16 bit sẽ phải được gán cho mỗi trường địa chỉ của lệnh và 80 bit cho toàn bộ lệnh (với mã hoạt động 16 bit). Tuy nhiên, một máy tính làm việc với các lệnh như vậy không còn được gọi là máy vi tính nữa, vì nó sẽ chứa rất nhiều thiết bị: 80-bit, ô nhớ, thanh ghi 80-bit, ALU 80-bit, mạch cổng 80-bit, 80- bus rộng, v.v ... e. Do đó, cần phải lưu trữ lệnh bốn địa chỉ trong các ô nhớ dưới dạng các trường riêng biệt của nó hoặc giảm số lượng trường địa chỉ của lệnh.

Trong số các lệnh của máy tính hiện đại, thực tế không có bốn địa chỉ. Các lệnh ít và ba địa chỉ (xem Hình 1.5, d), là kết quả của hoạt động này, bạn hầu như luôn có thể viết thay cho một trong các toán hạng đã được sử dụng. Ví dụ nhất

Vi thể

phần phối trong máy vi tính nhận được lệnh đơn địa chỉ và không địa chỉ (xem Hình 1.8, b, c), cho phép bạn xây dựng một bộ xử lý đơn giản hơn. Tuy nhiên, trước khi chuyển sang thảo luận về các lệnh ngắn như vậy, chúng ta hãy trả lời một số câu hỏi mà chúng tôi nghĩ rằng một người đọc tò mò có thể có.



Cơ. 1.10. Một bộ xử lý đơn giản hoạt động với hướng dẫn unicast và không địa chỉ

Có cần thiết phải lưu trữ các trường riêng biệt (hoặc các phần của trường) của các lệnh ba, hai hoặc một địa chỉ trong các ô nhớ khác nhau hay không và làm thế nào để làm việc với các lệnh đó?

Điều gì giải thích sự xuất hiện trong bộ xử lý được hiển thị trong Hình 1.10, các thanh ghi bổ sung (thanh ghi địa chỉ, bộ đếm chương trình và thanh ghi dữ liệu)? Tại sao các thanh ghi này được kết nối với nhau như trong Hình 1.10, nhưng không phải là khác?

Nếu các lệnh chương trình lần lượt được đặt trong bộ nhớ máy tính (và không theo một trình tự tùy ý), thì địa chỉ của lệnh tiếp theo thường sẽ khác với địa chỉ của lệnh thực thi (hoặc trường cuối cùng của nó) chỉ một, và thêm một địa chỉ vào địa chỉ hiện tại có thể được gán cho máy tính (bộ đếm lệnh trong Hình 1.10). Điều này làm cho nó có thể rút ngắn độ dài của lệnh (xóa địa chỉ của lệnh tiếp theo khỏi nội dung của nó), nhưng dẫn đến nhu cầu sử dụng

các lệnh chuyển tiếp đặc biệt được đặt ở những vị trí đó của chương trình, nơi có thể cần thiết để thay đổi trình tự thực hiện tự nhiên của các lệnh tùy thuộc vào kết quả của phép tính. Thông thường, các lệnh chuyển tiếp là các lệnh unicast (xem Hình 1.8, b), trong đó mã hoạt động chỉ định điều kiện được kiểm tra (dấu hiệu của kết quả của hoạt động trước đó, sự hiện diện của lệnh cao, v.v.) , và địa chỉ là địa chỉ của lệnh mà bạn muốn nhảy, nếu điều kiện được đáp ứng (nếu điều kiện không được đáp ứng, lệnh nằm sau lệnh nhảy được chọn).

Bây giờ chúng ta hãy tìm hiểu xem các trường địa chỉ còn lại luôn cần thiết các lệnh.

Phân tích các chương trình máy tính khác nhau cho thấy rằng trong nhiều trường hợp, kết quả của việc thực hiện lệnh trước được sử dụng như một toán hạng trong lệnh tiếp theo. Nếu kết quả của việc thực thi lệnh không được gửi đến bộ nhớ, nhưng được lưu trữ, chẳng hạn như trong bộ tích lũy, thì các lệnh unicast sau có thể được phối với:

- Các lệnh xử lý số học (logic) đưa ra lệnh thực hiện bất kỳ phép toán số học hoặc logic nào, sử dụng nội dung của bộ tích lũy và nội dung của ô nhớ được định địa chỉ làm toán hạng (chính xác hơn là nội dung của thanh ghi dữ liệu, trong đó nội dung của ô địa chỉ được viết lại);
- chuyển các lệnh đưa ra lệnh trao đổi thông tin giữa bộ tích lũy và bộ nhớ (thông qua thanh ghi dữ liệu), tức là tải bộ tích lũy với nội dung của ô nhớ được định địa chỉ hoặc ghi nội dung của bộ tích lũy (kết quả tính toán) vào ô này;
- các lệnh chuyển điều khiển cung cấp bước nhảy có điều kiện hoặc không điều kiện tới ô nhớ có địa chỉ (tức là tới một lệnh phải được đặt trong bộ xử lý khi một số điều kiện được đáp ứng hoặc bất kể kết quả của các phép tính trước đó), gọi một chương trình con và thoát khỏi nó;
- Các lệnh I / O.

Vậy, với sự trợ giúp của các lệnh như vậy, chẳng hạn, phép tính tổng theo công thức có dạng $A \leftarrow B + C + E + F + G + H$ sẽ được thực hiện như thế nào? Để làm điều này, trước tiên bạn phải tải giá trị của từ B vào bộ tích lũy, sau đó thực thi năm lệnh cộng giống hệt nhau và cuối cùng là lệnh chuyển nội dung của bộ tích lũy sang ô được phân bổ cho tổng A:

Lệnh chương trình

Tải địa chỉ của thuật ngữ B	
Thêm địa chỉ của addend C	
Thêm địa chỉ của addend E	
Thêm địa chỉ của thuật ngữ F	
Thêm địa chỉ của thuật ngữ G	
Thêm địa chỉ của addend H	
Chuyển tiếp địa chỉ kết quả A	
Dừng lại	

Nội dung của bộ tích lũy trong quá trình thực hiện chương trình

$$\begin{aligned}
 &B \\
 &B + C \\
 &B + C + E \\
 &B + C + E + F \\
 &B + C + E + F + G \\
 &B + C + E + F + G + H \\
 &B + C + E + F + G + H \\
 &B + C + E + F + G + H
 \end{aligned}$$

(Theo cách tương tự, bạn làm việc với máy tính vi mô khi nhập số hạng tiếp theo và nhấn phím "+", bạn thêm số hạng này vào số tiền đã tích lũy trước đó).

Lưu ý rằng lệnh cuối cùng của chương trình được xem xét hoàn toàn không có địa chỉ. Lệnh này được gửi tới thiết bị điều khiển chứ không phải bộ nhớ nhiều ô. Các lệnh như vậy được thiết kế để hoạt động với các thiết bị cụ thể (pin, thiết bị điều khiển, v.v.) được gọi là lệnh không giải quyết hoặc lệnh có địa chỉ ngầm. Chúng cho phép bạn tạo các lệnh để đặt lại bộ tích lũy, thêm một hằng số (+1 hoặc -1) vào nội dung của nó, chuyển máy tính sang trạng thái cho phép ngắt chương trình đang chạy và

vân vân.

Bộ xử lý thực hiện các chương trình từ lệnh đơn địa chỉ và không địa chỉ (xem Hình 1.8, b, c) là đơn giản nhất và cho phép bạn tạo ra các máy tính rẻ nhất. Trong một trong những cấu trúc có thể có của một bộ xử lý như vậy (xem Hình 1.10), các thiết bị đã quen thuộc với chúng ta được sử dụng:

- đơn vị logic số học thực hiện các phép toán trên hai giá trị 16 bit để thu được 16-bit kết quả bit và sự phát triển của một số dấu hiệu (kết quả nhỏ hơn 0, bằng 0 hoặc lớn hơn 0, có sự chuyển từ bậc cao nhất của kết quả);
- thanh ghi trạng thái - thanh ghi 4 bit lưu trữ các đặc điểm của kết quả của thao tác cuối cùng được sử dụng bởi các lệnh chuyển tiếp;
- bộ tích lũy - một thanh ghi 16 bit trong đó dữ liệu được xử lý hoặc kết quả của quá trình xử lý được đặt;
- thanh ghi lệnh - một thanh ghi 16 bit dùng để đặt một lệnh thực thi;

- thanh ghi địa chỉ - một thanh ghi 12 bit chứa địa chỉ của ô nhớ mà từ đó lệnh (toán hạng) sẽ được đọc hoặc kết quả xử lý sẽ được ghi;
- thanh ghi dữ liệu - thanh ghi 16 bit được sử dụng làm bộ đệm giữa bộ nhớ và các thanh ghi bộ xử lý khác; các lệnh (toán hạng) được gửi đến bộ xử lý thông qua nó và kết quả của quá trình xử lý được chuyển vào bộ nhớ;
- bộ đếm chương trình - một thanh ghi 12 bit, nội dung của nó được tăng lên bởi một thanh ghi tại thời điểm tìm nạp lệnh thực thi từ bộ nhớ và, nếu lệnh nhảy được chọn, có thể được thay thế bằng nội dung của phần địa chỉ của bước nhảy hướng dẫn; vào cuối chu trình thực thi lệnh, bộ đếm chương trình luôn lưu trữ địa chỉ của lệnh phải được thực hiện sau lệnh hiện tại và đây có thể là lệnh tiếp theo theo thứ tự hoặc lệnh mà bạn muốn chuyển đến khi các điều kiện được chỉ định bởi opcode hướng dẫn nhảy được đáp ứng.

Để giải quyết một vấn đề trên một máy tính như vậy, cần phải:

1) thông qua các thiết bị đầu vào thông tin, tải vào bộ nhớ máy tính chương trình giải quyết vấn đề (một thuật toán được viết bằng ngôn ngữ máy tính) và dữ liệu ban đầu (điều này được thực hiện theo nhiều cách khác nhau, sẽ được thảo luận trong các chương sau); cả chương trình và dữ liệu đều có thể được đặt trong bất kỳ vùng nào của bộ nhớ, bắt đầu từ vị trí 0 hoặc một vị trí khác tại bất kỳ địa chỉ nào (chương trình trước rồi đến dữ liệu, hoặc ngược lại).

2) "cho" bộ xử lý biết địa chỉ của ô nhớ nơi đặt lệnh đầu tiên của chương trình, cho mục đích nào nhập địa chỉ của ô này vào bộ đếm chương trình (một trong những cách để tải địa chỉ bắt đầu sẽ được thảo luận trong đoạn 3.1);

3) nhấn nút START, sẽ dẫn đến việc chuyển đến bộ nhớ địa chỉ của lệnh đầu tiên của chương trình và chuyển nội dung của nó từ bộ nhớ sang thanh ghi lệnh; Kể từ lúc này, bộ xử lý sẽ bắt đầu thực hiện một chuỗi các hoạt động khá đơn giản được thể hiện trong sơ đồ thuật toán thực thi lệnh (Hình 1.11).

Như có thể thấy từ sơ đồ thuật toán, hai hành động đầu tiên (khối 1 và 2) được thực hiện cho mỗi lệnh. Các hành động này (cũng như các hành động để xác định loại lệnh - khối 3, 6 và 9) thường được gọi là "Chọn lệnh". Các hành động tiếp theo của thuật toán ("Thực thi lệnh") hoàn toàn phụ thuộc vào loại

đội.

Xem xét việc thực hiện lệnh nằm trong ô 135 trong Hình. 1.10. Nó bắt đầu tại thời điểm khi pin

giá trị của lệnh giảm được ghi ở đó trong quá trình thực hiện lệnh trước đó được lưu trữ và địa chỉ 135 được lưu trong bộ đếm chương trình.

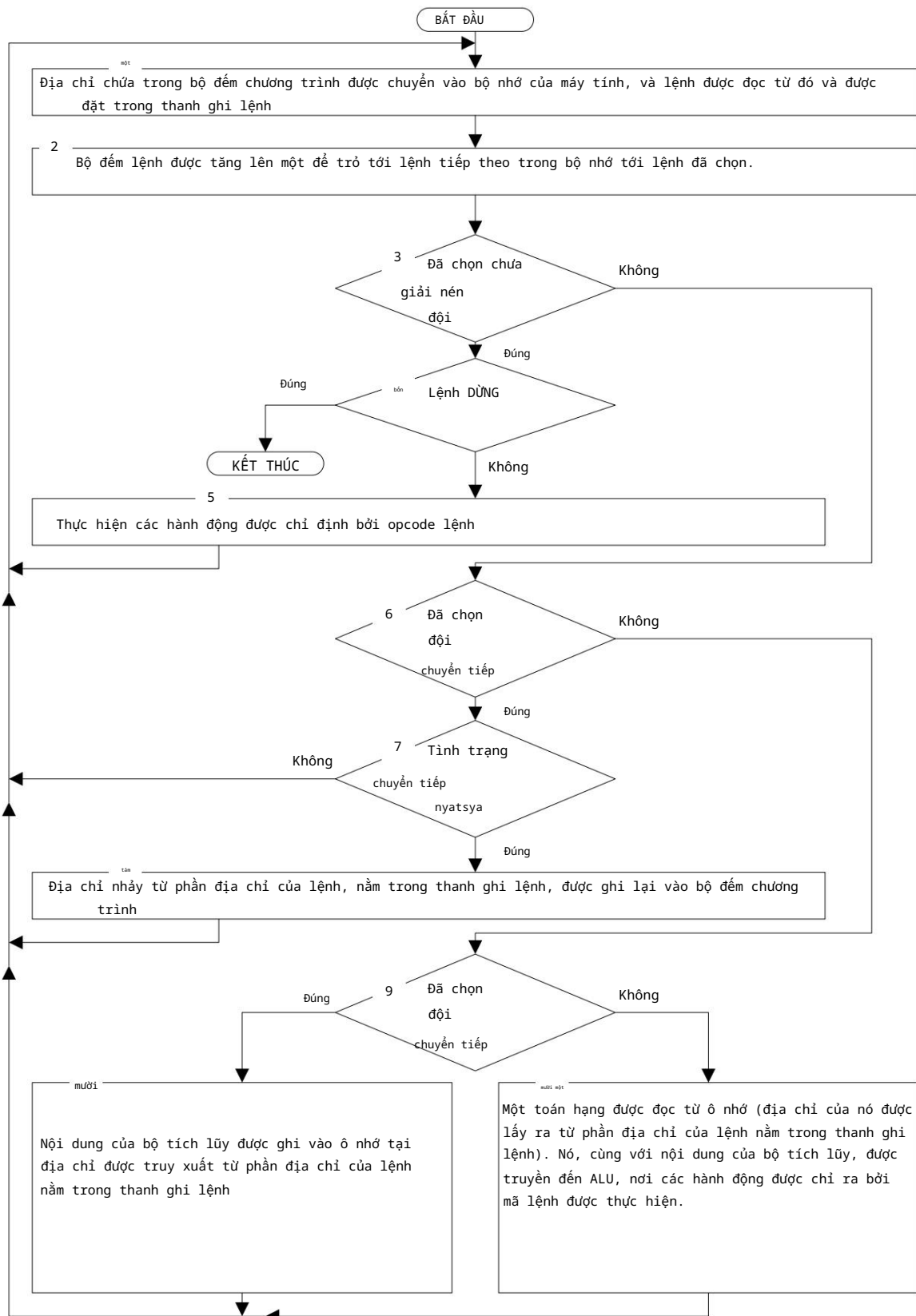
1. Địa chỉ chứa trong bộ đếm chương trình được ghi lại vào thanh ghi địa chỉ và sau đó vào bộ nhớ máy tính (tín hiệu U 1), và từ đó lệnh "Vych~~đ~~ được đặt trong thanh ghi lệnh (tín hiệu U4).

2. Bộ đếm chương trình được tăng lên một (tín hiệu + 1), để nó trở đến lệnh (địa chỉ 136) nằm trong bộ nhớ bên cạnh lệnh đã chọn.

3. Thiết bị điều khiển đọc mã lệnh đang được thực hiện từ thanh ghi lệnh, phát hiện đây là lệnh trừ và tiến hành thực hiện lệnh đó.

4. Địa chỉ của truyền con (26) được đọc từ thanh ghi lệnh (tín hiệu U2) và được chuyển đến bộ nhớ (thông qua thanh ghi địa chỉ), và giá trị số của truyền con được ghi lại từ bộ nhớ vào thanh ghi dữ liệu.

5. Hoạt động trừ nội dung của số đăng ký được thực hiện dữ liệu từ nội dung của bộ tích lũy (tín hiệu Y5, Y 6 và Y 1 1), sau đó sự khác biệt kết quả được ghi lại từ đầu ra ALU vào pin (tín hiệu Y 8), thay thế giá trị của giá trị đã giảm. một



Cơ. 1.11. Sơ đồ thuật toán để thực hiện các lệnh của một bộ xử lý đơn giản

Để dễ dàng nhận thấy rằng quá trình thực hiện một lệnh được rút gọn để xác định trình tự đóng mở.

các mạch van. Mô tả sơ đồ cổng nào sẽ mở và khi nào tạo thành một chương trình cho một máy có tập lệnh bao gồm lệnh "Sơ đồ cổng mở". Một chiếc máy như vậy là thiết bị điều khiển máy tính, nơi lưu trữ các chương trình để thực hiện tất cả các lệnh máy tính. Các chương trình này thường được gọi là chương trình cơ sở và hướng dẫn của chúng là

vi lệnh. Lệnh vi mô đơn giản nhất bao gồm một tập hợp các bit, mỗi bit điều khiển một trong các mạch cổng của bộ xử lý: một có nghĩa là mạch cổng mở, không -

đã đóng cửa. Mức phần sụn có thể được tìm thấy trong Chap. bốn.

Kết luận, chúng tôi lưu ý rằng bộ xử lý được hiển thị trong Hình. 1.10 không phải là bộ xử lý khả thi duy nhất được sử dụng để thực hiện các lệnh unicast. Vì vậy, tổng sản phẩm

$$AB + CE + FH + GK + \dots$$

tốt hơn là tính toán bằng cách sử dụng một bộ xử lý có hai bộ tích lũy (một bộ để tính sản phẩm và bộ tích lũy còn lại để cộng tổng). Điều này sẽ loại bỏ việc chuyển các kết quả trung gian vào bộ nhớ, tức là, tăng tốc độ tính toán và giảm số lượng ô nhớ được sử dụng. Có thể đưa ra các ví dụ khi nên sử dụng bộ xử lý có số lượng lớn các bộ tích lũy hoặc các thanh ghi khác (ví dụ, các thanh ghi chỉ mục). Bộ xử lý có nhiều hơn hai bộ tích lũy được gọi là bộ xử lý thanh ghi mục đích chung (RON).

Bản tóm tắt

1. Về mặt cấu trúc, bất kỳ máy tính nào cũng bao gồm một bộ xử lý, bộ nhớ và các thiết bị đầu vào - đầu ra.

2. Bộ nhớ máy tính bao gồm các ô có cùng kích thước, mỗi ô có số - địa chỉ duy nhất và có thể lưu trữ một lệnh hoặc số. Số lượng phần tử bộ nhớ (số bit) mà mỗi ô được cấu tạo sẽ xác định dung lượng của các từ máy - lệnh và dữ liệu hoặc các bộ phận của chúng.

3. Tất cả các lệnh như một (có thể là phần duy nhất) chứa một mã hoạt động chỉ định hành động sẽ được thực hiện trên lệnh này. Ngoài opcode, một lệnh có thể có một số phần địa chỉ chứa địa chỉ của các toán hạng, kết quả và lệnh tiếp theo.

4. Máy vi tính sử dụng các lệnh không địa chỉ, đơn địa chỉ và ít thường xuyên hơn là lệnh hai địa chỉ. Trong hướng dẫn unicast, một trong các toán hạng được chọn từ một thanh ghi đặc biệt - bộ tích lũy. Kết quả của hoạt động cũng được nhập vào nó. Các lệnh chưa được giải quyết hoặc đặt một số hành động với các thiết bị máy tính (ví dụ:

stop), hoặc được sử dụng để làm việc với các toán hạng có vị trí cố định (thường là với bộ tích lũy).

5. Bộ xử lý là sự kết hợp của thiết bị điều khiển, đơn vị logic số học và các thanh ghi liên quan. Nó tổ chức quá trình xử lý thông tin bằng cách chọn và thực hiện tuần tự các lệnh của chương trình nằm trong bộ nhớ của máy tính.

6. Khối logic số học thực hiện một số phép toán số học và logic trên một hoặc hai toán hạng được chuyển tới nó từ bộ nhớ và (hoặc) các thanh ghi của bộ xử lý. Kết quả của hoạt động được gửi đến một trong các thanh ghi của bộ xử lý.

7. Thiết bị điều khiển, sử dụng các thanh ghi của bộ xử lý, điều khiển việc trao đổi thông tin giữa bộ nhớ, các thiết bị đầu vào và đầu ra và ALU, cũng như việc thực hiện các hoạt động trong ALU.

8. Các thanh ghi của bộ xử lý được sử dụng để lưu trữ các kết quả trung gian của các phép tính và các thông tin điều khiển khác nhau. Quan trọng nhất trong số này là thanh ghi lệnh (chứa lệnh được thực hiện) và bộ đếm lệnh (chứa địa chỉ của lệnh tiếp theo).

Trong quá trình hoạt động, máy tính tuần tự thực hiện một tập hợp các thao tác khá đơn giản: tìm nạp một lệnh, xác định kiểu của nó, thực hiện lệnh và xác định địa chỉ của lệnh tiếp theo. Mức độ đa dạng của các tác vụ được giải quyết trên máy tính không phải do thiết bị của nó mà do chương trình và khả năng của các thiết bị đầu vào - đầu ra được kết nối với máy tính quyết định.

2. MÁY TÍNH CƠ BẢN

2.1. Mục đích và cấu trúc của máy tính cơ sở

Máy tính rất khác nhau về khả năng, kích thước và giá thành. Một số máy có thể thực hiện một số lượng rất hạn chế các thao tác đơn giản, trong khi những máy khác có thể thực hiện hàng trăm thao tác (lệnh) khác nhau. Về mặt lý thuyết, bất kỳ máy tính nào cũng có thể được sử dụng để giải quyết một vấn đề cụ thể. Lệnh nhỏ sẽ giải quyết vấn đề bằng cách thực hiện một số lượng lớn các lệnh rất đơn giản, và do đó có thể mất một khoảng thời gian dài để có được giải pháp. Một máy tính lớn có khả năng thực hiện nhiều thao tác sẽ giải quyết cùng một vấn đề nhanh hơn nhiều.

Chương này thảo luận về một máy giả định đơn giản có các tính năng điển hình của nhiều máy vi tính cụ thể. Kiến thức về nguyên lý cấu tạo và hoạt động của máy tính này là cơ sở tốt để làm chủ các hệ thống vi xử lý thuộc bất kỳ kiểu và mô hình nào, do đó nó được gọi là máy tính cơ sở. Đương nhiên, nên bắt đầu nghiên cứu máy tính với một máy tương tự cấp thấp, có thể được thực hiện một cách thực tế bằng cách sử dụng các mô hình của nó được xây dựng cho các loại máy tính cá nhân khác nhau. Khi chế tạo một máy tính cơ bản, các máy tính Elektronika 100 và Saratov, cũng như các máy tính PDP-8 tương tự như chúng, bộ vi xử lý chip đơn IM 6100 và máy tính cá nhân DECmate 11 đã được chọn làm nguyên mẫu [5, 6, 7] .

Trên hình. 2.1 cho thấy một cấu trúc đơn giản của máy tính cơ sở. Nó là một máy unicast hoạt động với các từ 16-bit. Nó thực hiện hai kiểu địa chỉ toán hạng: trực tiếp và gián tiếp. Chúng ta hãy xem xét các thành phần của máy tính cơ sở, không chạm vào các thiết bị đầu vào - đầu ra (I / O) và bảng điều khiển (PU).

Kí niệm. Bộ nhớ bao gồm 2048 ô, mỗi ô 16 bit có địa chỉ 0, 1, 7FE16, 7FF16. Một ô (hoặc từ) có thể được sử dụng để lưu trữ một số nhị phân hoặc một lệnh chương trình được mã hóa bằng mã nhị phân. Người dùng quyết định vùng nhớ nào sẽ được sử dụng cho dữ liệu và vùng nhớ nào cho chương trình. Trong trường hợp này, cần lưu ý rằng tám ô nhớ có địa chỉ 008, 00F

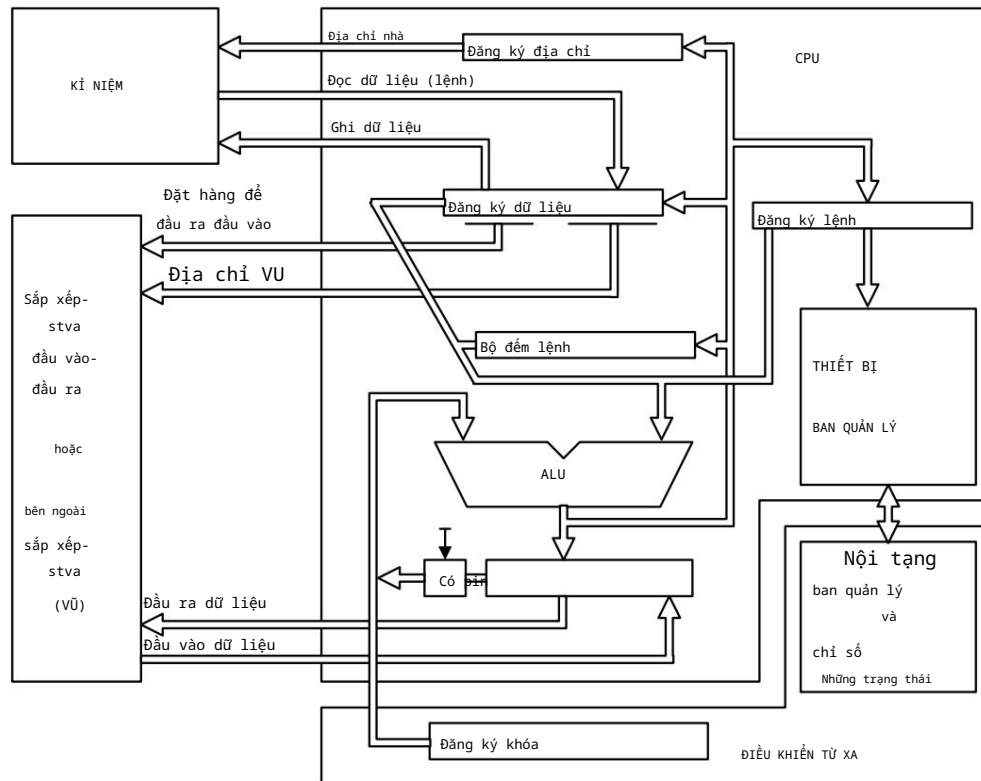
có phần khác biệt so với phần còn lại. Các ô này được gọi là ô chỉ mục và thường được sử dụng để tổ chức các vòng lặp trong chương trình.

Các lệnh của chương trình nguồn thường được người dùng đặt trong các ô nhớ liên tiếp. Máy đọc một lệnh từ bộ nhớ và thực hiện thao tác được chỉ định trong đó. Sau đó, nó đọc lệnh tiếp theo từ ô tiếp theo và thực hiện thao tác tiếp theo, v.v.

d.

CPU. Bộ xử lý bao gồm:

- sáu thanh ghi (thanh ghi lệnh 16 bit, bộ tích lũy và thanh ghi dữ liệu, bộ đếm lệnh 11 bit và thanh ghi địa chỉ, 1-thanh ghi chuyển bit);



Cơ. 2.1. Cấu trúc của máy tính cơ sở

- một đơn vị logic số học thực hiện các hoạt động đảo ngược bất kỳ toán hạng nào (hoặc hai toán hạng), thêm hai toán hạng 16 bit (hoặc các giá trị đảo ngược của chúng), nhân một cách hợp lý hai toán hạng 16 bit, thêm một vào kết quả và tham gia vào tổ chức của sự thay đổi theo chu kỳ của nội dung của bộ tích lũy và thanh ghi chuyển;

- một thiết bị điều khiển vi chương trình lưu trữ trong bộ nhớ của nó các vi chương trình để thực hiện 28 lệnh (xem Bảng 2.4), bốn hoạt động từ xa, phản hồi đối với tín hiệu ngắt của chương trình đang được thực thi từ các thiết bị đầu vào-đầu ra và một số vi chương trình khác.

Chúng ta hãy xem xét chi tiết hơn mục đích của tất cả các thanh ghi của bộ xử lý và ALU của máy tính cơ sở.

C o m a n d (S K) . Nó phục vụ để tổ chức truy cập vào các ô nhớ trong đó chương trình được lưu trữ. Vào cuối mỗi chu kỳ thực hiện lệnh, bộ đếm chỉ ra địa chỉ của vị trí bộ nhớ chứa lệnh chương trình tiếp theo. Vì các lệnh thường được nhập vào các ô liên tiếp và khi lệnh hiện tại được thực thi, nội dung của bộ đếm tăng lên một, khi đó

tự động chỉ định địa chỉ của lệnh tiếp theo. Trong một số trường hợp, nội dung của bộ đếm chương trình có thể được thay đổi bởi chính chương trình (ví dụ: khi thực hiện các lệnh rẽ nhánh). Bằng cách này, chuyển quyền kiểm soát sang một phần khác của chương trình. Máy tính cơ sở sử dụng bộ đếm chương trình 11 bit có thể xử lý bất kỳ bộ nhớ nào trong số 2048 từ của bộ nhớ.

Thanh ghi các lệnh (R C) . Thanh ghi 16-bit này được sử dụng để lưu trữ lệnh do máy trực tiếp thực hiện. Mã hoạt động lệnh (ví dụ, xem Hình 1.8, 1.10 hoặc 2.3) được gửi từ RC đến thiết bị điều khiển máy tính và được giải mã. Sau đó, các hành động được thực hiện để thực hiện lệnh: đọc toán hạng và (hoặc) thực hiện thao tác do lệnh quy định,

R e g i s t r a d r e s (RA). Nó chứa giá trị địa chỉ thực thi của ô nhớ và bao gồm 11 bit để định địa chỉ cho 2048 ô nhớ. Nếu máy tính thực hiện tìm nạp lệnh, thì nội dung của bộ đếm lệnh CK được gửi đến RA để chỉ ra địa chỉ của ô nơi lệnh được lưu trữ. Nếu máy tính tìm nạp dữ liệu, thì địa chỉ có thể đến từ thanh ghi lệnh.

Thanh ghi dữ liệu (RD). Nó được sử dụng để lưu trữ tạm thời 16-bit từ trong việc trao đổi thông tin giữa bộ nhớ và bộ xử lý. Khi đọc một lệnh, số hoặc ký tự từ bộ nhớ đến bộ xử lý, từ này đầu tiên đi vào thanh ghi dữ liệu, sau đó được chuyển đến thanh ghi lệnh (lệnh) hoặc đến các thanh ghi bộ xử lý khác. Khi chuyển dữ liệu từ bộ xử lý sang bộ nhớ, trước tiên chúng được đặt trong RD và chỉ sau đó mới được ghi vào ô nhớ mong muốn.

Khi một từ thông tin nằm trong một thanh ghi dữ liệu, nó có sẵn cho các phép toán số học hoặc logic. Ví dụ, nội dung của RD có thể được thêm vào nội dung của bộ tích lũy và kết quả thu được sẽ được nhập vào bộ tích lũy.

Pin (A). Thanh ghi A là một trong những phần tử chính của bộ xử lý. Một máy chỉ có thể thực hiện một phép toán số học hoặc logic trên hai toán hạng cùng một lúc. Thông thường, toán hạng đầu tiên được tìm nạp từ bộ nhớ vào thanh ghi dữ liệu trong khi toán hạng thứ hai nằm trong bộ tích lũy. Hoạt động được chỉ định bởi lệnh được thực hiện trên nội dung của RD và A, và kết quả của hoạt động được đặt trong bộ tích lũy.

Máy có thể kiểm tra kết quả trong pin. Tùy thuộc vào kết quả của việc kiểm tra, nó có thể đưa ra các quyết định khác nhau. Nó sử dụng bộ tích lũy 16 bit. Bất cứ khi nào xảy ra tràn bậc cao trong một thao tác cộng nhị phân, giá trị mang theo sẽ bị mất. Tuy nhiên, nó có thể được ghi vào thanh ghi mang một bit.

Thanh ghi mang (C). Đây là thanh ghi một bit hoạt động như một phần mở rộng của bộ tích lũy và được lấp đầy khi

tràn ra. Thanh ghi này cũng được sử dụng để tổ chức các ca làm việc theo chu kỳ. Trạng thái của thanh ghi mang có thể được kiểm tra để đưa ra quyết định.

Đơn vị logic số học (ALU). ALU của máy tính cơ sở cho phép bạn thực hiện các phép toán số học như cộng và trừ, có tính đến số mang (nội dung của thanh ghi C) có được do kết quả của hoạt động trước đó. Ngoài ra, nó có khả năng thực hiện các phép toán nhân logic (phép toán AND), đảo ngược, xoay và tăng 1.

Ví dụ, với sự trợ giúp của máy tính đang được xem xét, cần lấy tổng của các số 53 và 106, thì điều này có thể được thực hiện bằng cách sử dụng chương trình được hiển thị trong Bảng 2.1. Mặc dù cả hướng dẫn và dữ liệu phải được mã hóa ở dạng nhị phân (dạng duy nhất mà máy tính hiểu được), chương trình này được viết với các ký hiệu hướng dẫn để đơn giản hóa (xem Phần 2.2 để biết thêm về mã hóa ký hiệu).

Các lệnh sau được sử dụng:

CLA (Clear Accumulators - xóa pin) - lệnh này đặt pin về 0;

Bảng 2.1

Một chương trình để tính tổng của hai số		
Địa chỉ	Nội dung	Bình luận
20	0053	Điều khiển đầu tiên
21	0106	Kỳ hai
22	0000	Ô cho kết quả
23	CLA	Bộ tích lũy chứa 0000 (rõ ràng)
24	THÊM 20	Pin chứa 53
25	THÊM 21	Bộ tích lũy chứa 159 (53 + 106)
26	MOV 22	Trong ô có địa chỉ 22, 159 được viết
27	HLT	Tắt máy tính (chấm dứt lựa chọn lệnh)

ADD (ADD - add) - bằng lệnh này, nội dung của ô có số được viết sau ADD được thêm vào nội dung của bộ tích lũy và kết quả vẫn nằm trong bộ tích lũy;

MOV (MOVE - move) - với lệnh này, nội dung của bộ tích lũy được gửi đến ô có số được viết sau

MOV; nội dung của bộ tích lũy được bảo toàn;

HLT (HALT - dừng lại).

Chương trình được ghi vào bộ nhớ máy tính, bắt đầu từ ô 23 và dữ liệu số - từ ô 20. Ô số 22 được dành để ghi số tiền. Để thực hiện chương trình, cần thiết lập số 23 trong bộ đếm chương trình và khởi động máy tính. Sau đó, chương trình sẽ bắt đầu bằng cách đọc nội dung của ô 23. Lệnh CLA sẽ đặt bộ tích lũy bằng không. Khi lệnh này được thực thi, nội dung của bộ đếm chương trình sẽ tăng lên 1 và lệnh tiếp theo sẽ được đọc từ vị trí 24. Đây là lệnh ADD 20

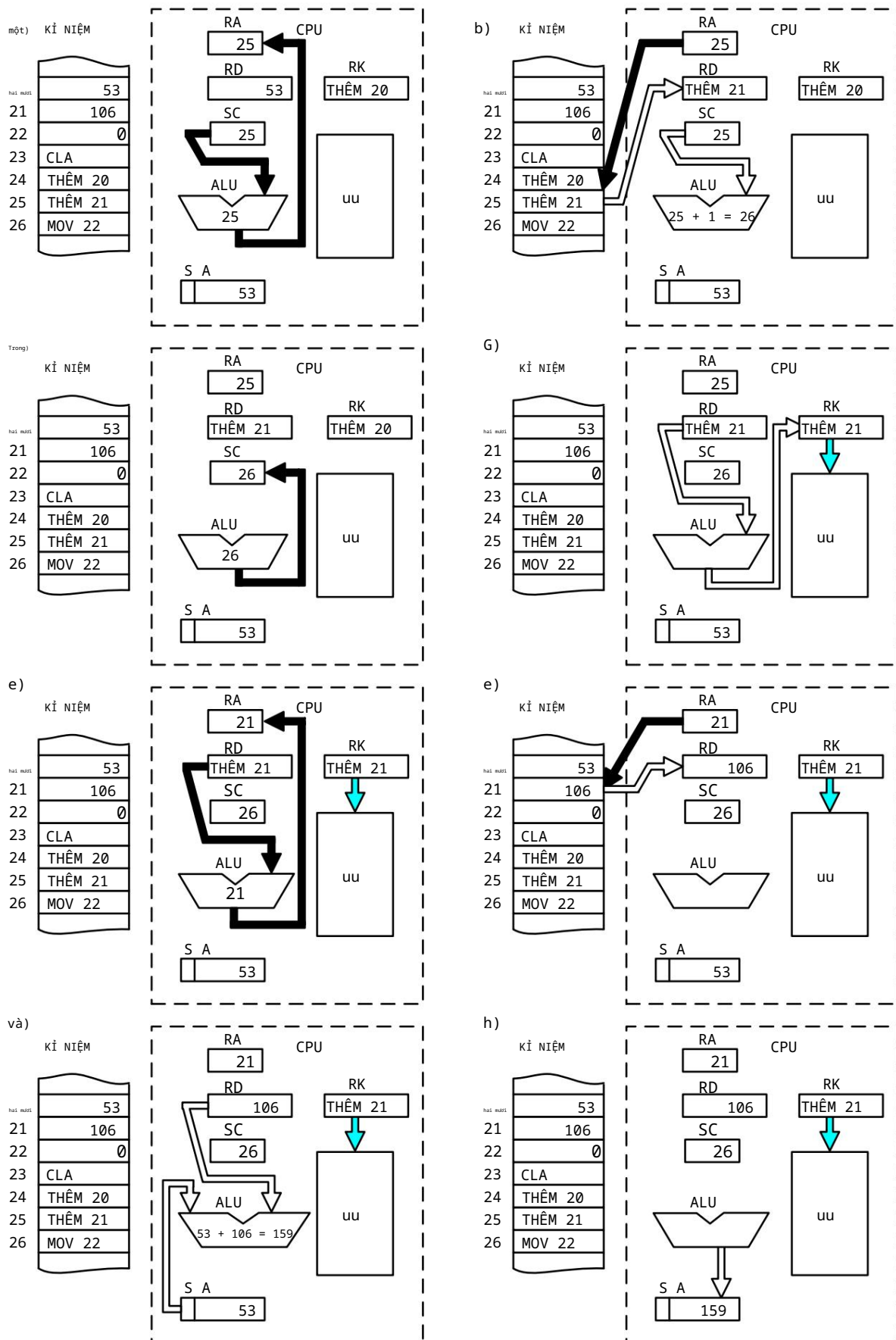
thêm nội dung của ô 20 vào nội dung của bộ tích lũy, tức là 53 với 0. Ở cuối lệnh, bộ tích lũy chứa số 53 và bộ đếm chương trình chứa 25 (nó lại tăng thêm 1).

Lệnh tiếp theo được đọc từ ô 25, nội dung của ô 21 và bộ tích lũy được tính tổng, tức là 106 và 53. Khi kết thúc lệnh, bộ tích lũy chứa tổng $106 + 53 = 159$ và bộ đếm chương trình chứa 26. Lệnh tiếp theo, MOV 22, chuyển bộ tích lũy nội dung đến địa chỉ ô 22. Bây giờ ô này và bộ tích lũy chứa số 159, và bộ đếm chương trình chứa 27. Từ ô 27, lệnh HLT được chọn, bộ đếm lệnh tăng 1 (nó chứa số 28), nhưng lệnh HLT dừng các lệnh lấy mẫu, tức là kết thúc thực thi chương trình được cố định (nội dung của ô 28 không được chọn vào thanh ghi lệnh và không được hiểu là lệnh).

Bây giờ chúng ta hãy xem xét chi tiết hơn việc thực hiện một trong các lệnh của chương trình này, ví dụ: lệnh ADD 21. Trước khi thực thi, lệnh trước đó (ADD 20) được lưu trữ trong thanh ghi lệnh, trong thanh ghi địa chỉ - địa chỉ của toán hạng của nó (20), trong thanh ghi dữ liệu và bộ tích lũy - giá trị của toán hạng này (53) và trong bộ đếm chương trình - địa chỉ của lệnh trong câu hỏi (25).

Đầu tiên, địa chỉ lệnh được chuyển từ bộ đếm lệnh sang thanh ghi địa chỉ (Hình 2.2, a). Ở đây cần lưu ý rằng trong máy tính cơ sở, tất cả các chuyển đổi giữa các thanh ghi được thực hiện thông qua ALU (nếu cần, giá trị được chuyển sẽ được đảo ngược, một giá trị được thêm vào nó, v.v.). Sau đó, nội dung của ô nhớ, địa chỉ nằm trong thanh ghi địa chỉ, được chọn từ bộ nhớ đến thanh ghi dữ liệu, và nội dung của bộ đếm chương trình được tăng lên một (Hình 2.2, b, c). Cuối cùng, nội dung của thanh ghi dữ liệu được chuyển đến thanh ghi lệnh, và thiết bị điều khiển bắt đầu phân tích nội dung này, tức là lệnh ADD 21 (Hình 2.2, d). Các hành động được liệt kê có liên quan đến "Lựa chọn đội".

Sau khi giải mã mã hoạt động của lệnh ADD 21, thiết bị điều khiển sẽ gửi phần địa chỉ của nó (địa chỉ 21) đến thanh ghi địa chỉ (Hình 2.2, e). Việc chuyển giao được thực hiện từ thanh ghi dữ liệu, nơi một bản sao của lệnh vẫn được lưu trữ. Sau đó, nội dung của ô nhớ, địa chỉ nằm trong thanh ghi địa chỉ, được đọc từ bộ nhớ vào thanh ghi dữ liệu (Hình 2.2, e). Hành động này tương tự như hành động trong Hình. 2.2, b, nhưng thiết bị điều khiển bây giờ sẽ coi nội dung của thanh ghi dữ liệu không phải là một lệnh, mà là một thuật ngữ, trong bước tiếp theo sẽ được thêm vào nội dung của bộ tích lũy (Hình 2.2, g). Cuối cùng, số tiền nhận được trong ALU được gửi đến bộ tích lũy (Hình 2.2, h) và do đó "Thực hiện lệnh" được hoàn thành. Tập hợp và thứ tự thực hiện các hoạt động trong chu trình này là duy nhất cho mỗi lệnh máy tính và sẽ được thảo luận dưới đây.



CƠM. 2.2. Thực thi lệnh ADD 21

2.2. Hệ thống lệnh và mã hóa chương trình

Tất cả các hoạt động trong máy tính được thực hiện trên các tín hiệu điện được mã hóa bằng chữ số nhị phân 0 hoặc 1. Do đó, trước khi đưa vào bộ nhớ, dữ liệu và lệnh phải được chuyển đổi thủ công hoặc phần cứng sang dạng nhị phân. Tuy nhiên, khi viết một chương trình, các lệnh có thể được viết dưới bất kỳ hình thức nào khác (ví dụ, ghi nhớ, như trong Bảng 2.1). Nói chung, một chương trình có thể được mã hóa theo bốn cách: nhị phân, bát phân, thập lục phân, và ký hiệu hoặc ghi nhớ.

Dạng nhị phân của lệnh. Biểu mẫu này là duy nhất mà máy hiểu được. Mỗi máy tính có một hệ thống lệnh bằng mã nhị phân, điều này có thể hiểu được. Thanh ghi lệnh, nơi lệnh được đặt từ bộ nhớ, được kết nối bằng các mạch điện với thiết bị điều khiển máy tính, nơi lệnh được giải mã và các tín hiệu được khởi tạo để thực hiện lệnh. Trong bảng 2.2 hiển thị nội dung của một phần bộ nhớ máy tính, nơi chương trình và dữ liệu được lưu trữ để tính tổng hai số từ ví dụ của đoạn 2.1. Cột bên trái là các địa chỉ ô ở dạng nhị phân. Cột bên phải thể hiện nội dung của các ô ở dạng nhị phân. Địa chỉ có 11 chữ số (bộ nhớ máy tính gồm $2048 = 2^{11}$ ô), và mỗi ô có 16 chữ số. Để thuận tiện cho việc đọc các số nhị phân và chuyển đổi chúng sang hệ thập lục phân, chúng được chia thành các tứ phân.

Mã hóa bát phân và thập lục phân. Từ ví dụ trước, rõ ràng là viết một chương trình bằng mã nhị phân là rất bất tiện và tẻ nhạt, cho rằng một chương trình trung bình bao gồm vài trăm hoặc hàng nghìn số không và số một. Để rút gọn bản ghi, nên sử dụng hệ thống số với cơ số $2^3 = 8$ hoặc $2^4 = 16$ (xem đoạn 1.3). Sự lựa chọn để mã hóa các lệnh của một hoặc một hệ thống số khác chủ yếu liên quan đến định dạng của lệnh máy tính. Trong trường hợp của chúng ta, hệ thập lục phân thuận tiện hơn, vì hầu hết các trường trong lệnh của máy tính cơ sở (xem Hình 2.2) bằng hoặc bội số của 4. Trong Bảng 2.3 hiển thị chương trình tương tự như trong Bảng 2.1 và 2.2, chỉ được mã hóa trong hệ thập lục phân. Đối với một lập trình viên, mã hóa như vậy thuận tiện hơn so với mã nhị phân.

hóa ghi nhớ. Mã hóa hệ thập lục phân (hoặc bát phân) có một lợi thế rõ ràng so với hệ nhị phân. Tuy nhiên, đối với các chương trình dài thì điều đó cũng bất tiện. Lập trình viên phải học thuộc lòng các mã thập lục phân của tất cả các lệnh được sử dụng trong máy (trong

một số máy tính có hơn một trăm). Để đơn giản hóa quá trình viết, gỡ lỗi và đọc một chương trình, một mã ghi nhớ hoặc mã ký hiệu được đề xuất: mỗi lệnh được biểu diễn bằng một ký hiệu ghi nhớ đơn giản gồm hai, ba hoặc bốn chữ cái. Các ký hiệu ghi nhớ dễ liên kết hơn với các hoạt động của máy vì chúng có thể được chọn giống với tên của một lệnh. Hầu hết các mã ghi nhớ là chữ viết tắt của tên lệnh tiếng Anh: SUB bằng cách SUBtract

(trừ), BR từ BRanch (nhảy), BPL từ Branch nếu PLus (nhảy theo số dương), v.v. Dễ dàng nhớ rằng bộ tích lũy đảo ngược (CoMplement Accumulator) được mã hóa bởi CMA hơn là nhớ mã nhị phân 1111010000000000 hoặc thậm chí hệ thập lục phân của nó tương đương với F400. Một ví dụ về mã hóa biểu tượng của một chương trình đã được đưa ra trong đoạn 2.1 (xem Bảng 2.1).

Bảng 2.2

Ví dụ về mã hóa nhị phân

nội dung bộ nhớ

Nội dung ô	000	0010	0000
	0000	0000	0101 0011
000 0010 0001	0000	0001	0000 0110
000 0010 0010	0000	0000	0000 0000
000 0010 0011	1111	0010	0000 0000
000 0010 0100	0100	0000	0010 0000
000 0010 0101	0100	0000	0010 0001
000 0010 0110	0011	0000	0010 0010
000 0010 0111	1111	0000	0000 0000

Bảng 2.3

ví dụ về hệ thập lục phân

mã hóa bộ nhớ

Nội dung ô	
020	0053
021	0106
022	0000
023	F200
024	4020
025	4021
026	3022
027	F000

Mặc dù mã hóa tượng trưng rất thuận tiện cho người lập trình, nhưng máy không thể hiểu được. Ngôn ngữ duy nhất được máy hiểu là ngôn ngữ của mã nhị phân. Do đó, cần phải dịch một chương trình tượng trưng thành tương đương nhị phân của nó. Điều này có thể được thực hiện thủ công bằng cách sử dụng các bảng tương ứng (như Bảng 2.4). Trong thực tế, việc dịch thuật được thực hiện bởi một chương trình máy đặc biệt.

Phân loại lệnh. Máy tính có khả năng hiểu và thực hiện một tập hợp các lệnh cụ thể. Khi biên dịch một chương trình, người lập trình bị giới hạn trong các hướng dẫn này. Số lượng và loại lệnh thay đổi tùy thuộc vào khả năng và mục đích của máy tính.

Tùy thuộc vào khối nào của máy mà một lệnh tham chiếu đến hoặc khối nào mà nó đề cập đến, các lệnh có thể được chia thành ba nhóm: truy cập bộ nhớ (lệnh địa chỉ); kiểu nài đăng ký (đăng ký hoặc không đăng ký); đầu vào

đầu ra.

Các lệnh truy cập bộ nhớ hướng dẫn máy thực hiện các hành động với nội dung của ô nhớ, địa chỉ của ô nhớ được chỉ định trong phần địa chỉ của lệnh. Ví dụ, lệnh ADD 20 từ Bảng. 2.1 là một lệnh truy cập bộ nhớ. Nó hướng dẫn máy đánh địa chỉ 20 và sử dụng nội dung của ô này làm toán hạng đầu tiên. Toán hạng thứ hai nằm trong bộ tích lũy. Hai toán hạng này là tổng.

Bảng 2.4

Hệ thống lệnh máy tính cơ sở Thuật			
Tên	ngữ Mã Mô tả	Các lệnh địa chỉ	
Phép nhân logic Phép chuyển Phép cộng Phép cộng với phép mang Phép trừ Nhảy nếu mang theo Nhảy nếu cộng Nhảy nếu trừ Nhảy	THÊM M 1XXX MOV M 3XXX THÊM M 4XXX ADC M 5XXX SUB M 6XXX		(M) & (A) A (A) M (M) + (A) A (M) + (A) + (C) A (A) - (M) A
nếu không Bước nhảy không điều kiện Tăng và bỏ qua	BCS M 8XXX BPL M 9XXX BMI M BEQ M BR M ISZ M 0XXX	Nếu (C) = 1 thì M SK Nếu (A) = 0 thì M SK Nếu (A) < 0, thì M SK Nếu (A) = 0, thì M SK M SK (M) + 1 M; Nếu (M) > = 0, thì (SK) + 1 SK	
Kêu gọi chương trình p /	JSR M 2XXX (SK)	M; M + 1 SC	
Các lệnh chưa được giải			
Vệ sinh pin Vệ sinh reg. chuyển khoản Đảo ngược pin Đảo ngược reg. chuyển khoản Xoay trái 1 bit Xoay phải 1 bit Bộ tích lũy tăng dần Bộ tích lũy giảm dần Không hoạt động Bật ngắt Tắt ngắt	CLA CLC CMA CMS ROL ROR INC THẮNG HUỖI HAI	F200 F300 0 C F400 F500 Nội dung F600 F700 Nội F800 F900	quyết 0 A — (A) A — (C) C A và C được chuyển sang trái, A (15) C và C A (0) dung của A và C được dịch sang phải, A (0) C và C A (15) (A) + 1 A (A) - 1 A
	HLT NOP EI DI	F000 F100 FA00 FB00	
I / O lệnh			
Cờ xóa Cờ thăm dò ý kiến Đầu vào	CLF B TSF B TRONG B	E0XX E1XX E2XX	0 cờ thiết bị B Nếu cờ thiết bị B là 1, thì (CK) + 1 CK (B) A

Sự kết luận	HẾT B	E3XX (A)	B
Truyền thuyết: (M), (A), (CK), (C) và (B) - nội dung của ô có địa chỉ M, bộ tích lũy, bộ đếm chương trình, thanh ghi truyền và thanh ghi dữ liệu của thiết bị I / O với địa chỉ B; A (X) - phóng điện của pin với số X; XXX - địa chỉ ô nhớ; XX là địa chỉ của thiết bị I / O.			

Các hướng dẫn chưa được giải quyết thực hiện các hành động khác nhau mà không cần tham chiếu đến vị trí bộ nhớ. Ví dụ, lệnh CLA từ Bảng. 2.1 Hướng dẫn vệ sinh pin cho máy. Hướng dẫn này đề cập đến một toán hạng nằm ở một nơi cụ thể - trong bộ tích lũy. Một ví dụ khác về lệnh không giải nén là lệnh HLT khởi bản. 2.1.

Các lệnh I / O thực hiện việc trao đổi dữ liệu giữa máy tính và các thiết bị bên ngoài. Chúng chỉ định địa chỉ (tên) của thiết bị đầu vào-đầu ra và mã của hoạt động mà thiết bị này phải thực hiện (lệnh cho đầu vào-đầu ra).

Trong bảng. 2.4 danh sách các lệnh của máy tính cơ sở được đưa ra. chi tiết các hành động được thực hiện bởi máy đối với các lệnh này được thảo luận trong các đoạn sau của chương này. Hãy giải thích một trong các mô tả: (CK)

+ 1 CK trong lệnh ISZ và TSF. Sau khi thực hiện thao tác này, bộ đếm chương trình sẽ không trở đến lệnh chương trình tiếp theo mà chỉ đến lệnh nằm sau nó. Điều này là do sau khi tìm nạp lệnh ISZ hoặc TSF, nội dung của bộ đếm lệnh đã được tự động tăng lên một.

Có một cách khác để chia các lệnh thành các nhóm. Nó dựa trên việc tính đến các chức năng được thực hiện bởi nhóm. Tám loại lệnh có thể được phân biệt: chuyển hoặc trao đổi, số học, logic, dịch chuyển, bước nhảy, lệnh gọi chương trình con, điều khiển và đầu vào-đầu ra

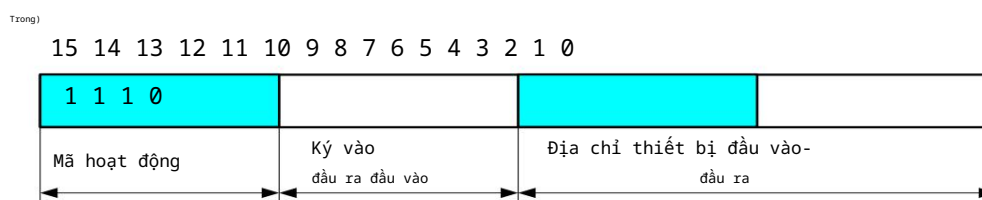
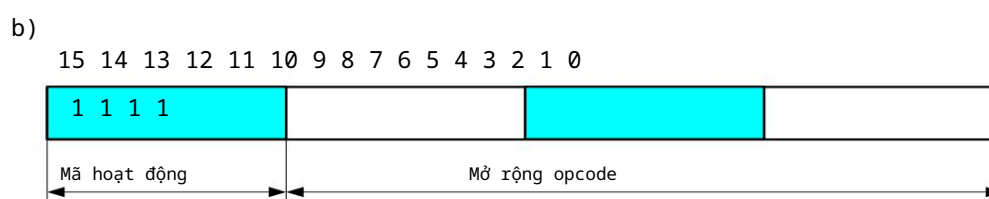
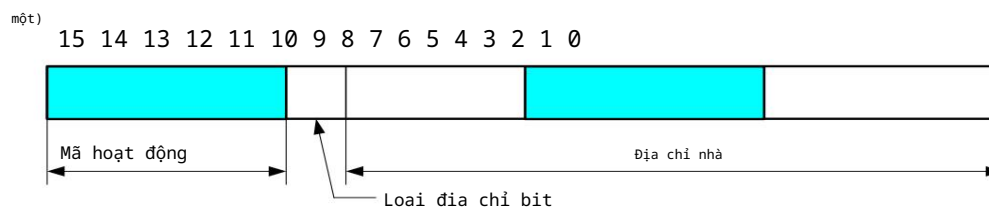
Các định dạng lệnh và phương pháp đánh địa chỉ. Phần 1.4 đã thảo luận về các định dạng (cấu trúc) lệnh khác nhau. Các nhà phát triển của máy tính cơ bản đã chọn ba định dạng của lệnh 16 bit (một từ) với opcode 4 bit (Hình 2.3).

Với số 4 bit, tối đa = 16 phép toán khác nhau có thể được mã hóa. Các nhà phát triển đã gán hai mã (1110 và 1111) cho các lệnh I / O và các lệnh chưa giải quyết. Dễ dàng nhận thấy rằng trong các lệnh này:

- phần địa chỉ ngắn hơn được sử dụng (địa chỉ 8 bit của thiết bị I / O trong Hình 2.3, c), phần này hoàn toàn không có (Hình 2.3, b) .
-

Do đó, có thể có tới $24 = 16$ lệnh I / O (thứ tự I / O 4 bit) và tối đa $212 = 4096$ lệnh không được giải nén (phần mở rộng opcode 12 bit).

Trong các lệnh truy cập bộ nhớ, 11 bit được gán cho địa chỉ, cho phép định địa chỉ trực tiếp cho tất cả 2048 (211) ô bộ nhớ máy tính cơ sở.



Cơ. 2.3. Các định dạng lệnh máy tính cơ bản

a) có thể định địa chỉ, b) không xác định được, c) đầu vào-đầu ra

Tuy nhiên, có những ứng dụng thích hợp hơn khi đặt vào một lệnh không phải địa chỉ của toán hạng (kết quả hoặc chuyển tiếp), mà là con trỏ của nó, tức là địa chỉ của ô nhớ trong đó địa chỉ của toán hạng (kết quả hoặc chuyển tiếp) được lưu trữ. Giải quyết gián tiếp như vậy

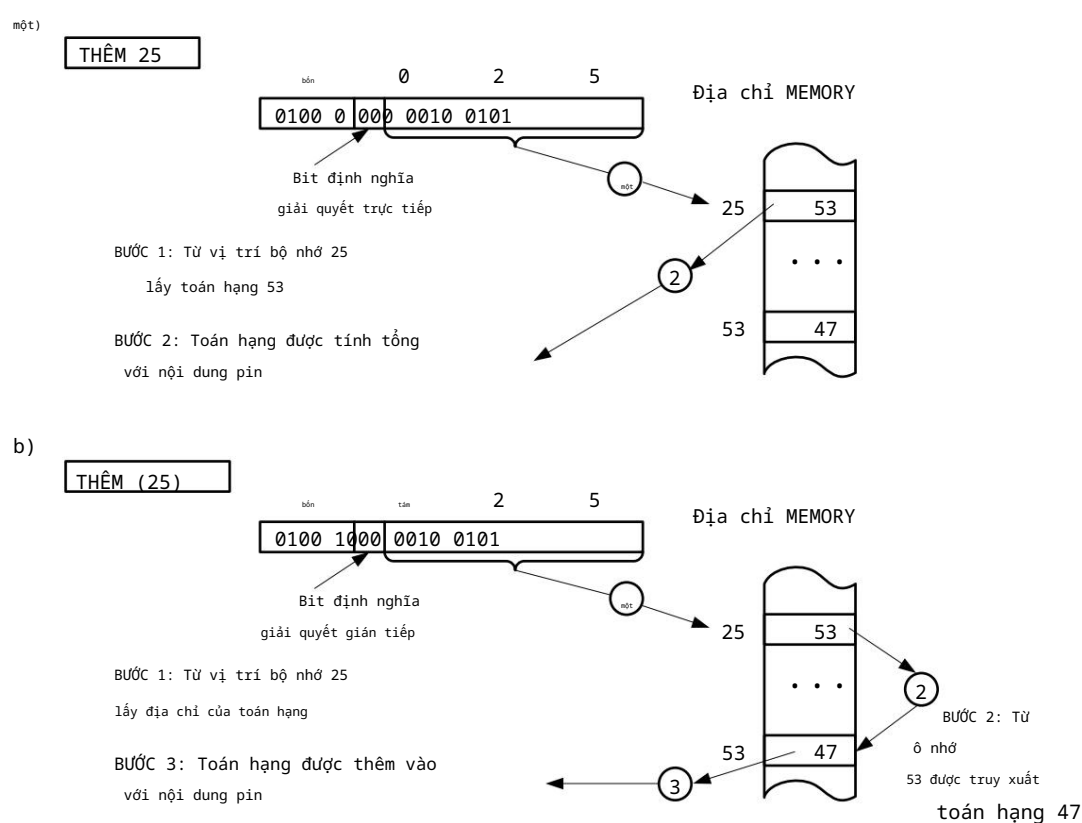
đơn giản hóa việc xây dựng các chương trình tuần hoàn, tổ chức công việc với các chương trình con, đồng thời tạo điều kiện để mở rộng không gian địa chỉ (định địa chỉ gián tiếp các ô 16 bit của máy tính cơ sở cho phép nó có bộ nhớ lên đến $216 = 65.536$ từ).

Để chỉ ra kiểu địa chỉ trong các lệnh, bit số 11 được sử dụng (Hình 2.3, a), trong đó 0 phải được viết cho địa chỉ trực tiếp và 1 cho địa chỉ gián tiếp. Trong thuật ngữ lệnh, để chỉ ra địa chỉ gián tiếp, toán hạng là được đặt trong ngoặc. Vì vậy, trong hình. 2.4, lệnh ADD 25 hoặc 4025 chỉ ra rằng số 53 nên được lấy từ ô 25, sẽ được thêm vào nội dung của bộ tích lũy. Lệnh ADD (25) hoặc 4825 chỉ ra rằng địa chỉ của ô 53 sẽ được lấy từ ô 25, trong

trong đó lưu trữ số 47, số này phải được thêm vào nội dung của bộ tích lũy. ("Và đây là một con chim tit vui vẻ, khéo léo ăn trộm lúa mì, được cất giữ trong một tủ tối trong ngôi nhà mà Jack đã xây dựng." Nghe như thế nào?)

2.3. Các phép tính toán học

Phần này thảo luận về các cách chính để viết số trong máy tính cơ bản, các phép toán số học được thực hiện với các số này và hướng dẫn thực hiện các phép toán đó. Chúng ta đã tự làm quen với một số lệnh này (CLA, ADD), lệnh di chuyển (MOV) và lệnh dừng (HLT).



Hình 2.4. Các phương pháp giải quyết: a - trực tiếp; b - gián tiếp

Số nguyên nhị phân không dấu. Những con số như vậy có thể được sử dụng để biểu diễn số nguyên dương và 0. Khi được đặt trong một từ 16 bit (xem Hình 1.1), chúng có thể thay đổi từ $(0000\ 0000\ 0000\ 0000)_2 = (0000)_{16} = 0$ đến $(1111\ 1111\ 1111\ 1111)_2 = (FFFF)_{16} = 2^{16} - 1 = \text{Năm } 65535$. Các số như vậy (cũng như các số nguyên nhị phân có dấu được thảo luận bên dưới) là các số có dấu chấm cố định, vì vị trí của dấu phẩy ngăn cách phần nguyên và phần thập phân của số được cố định nghiêm ngặt sau bit có ý nghĩa nhỏ nhất (xem Hình 1.1 và 1.5, a).

Tính tổng các số nguyên nhị phân không dấu tuân theo các quy tắc tương tự như phép cộng thập phân, ngoại trừ việc chuyển đổi xảy ra sau khi tổng đạt đến 2 (1 + 1) thay vì 10. Hãy xem một vài ví dụ

phép cộng.

Ví dụ 2.1. Tổng nhỏ hơn 65536 (216):

$ \begin{array}{r} 1\ 111\ 1\ 0000 \\ +\ 0110\ 0011\ 1010\ 0100 \\ \hline 0100\ 1001\ 1011\ 0100 \\ \hline 1010\ 1101\ 0101 \end{array} $	$ \begin{array}{r} +\ 11\ 1594 \\ \hline 17563 \\ \hline 19157 \end{array} $	Chuyển khoản
--	--	--------------

Ví dụ 2.2. Tổng là 65536:

$ \begin{array}{r} +\ 1000\ 0000\ 0000\ 0000 \\ \hline 1000\ 0000\ 0000\ 0000 \\ \hline 10000\ 0000\ 0000\ 0000 \end{array} $	$ \begin{array}{r} 111 \\ +\ 32768 \\ \hline 32768 \\ \hline 0? \end{array} $	Chuyển khoản
---	--	--------------

Ở đây, câu trả lời phải là số $32768 + 32768 = 65536$, nhưng vì độ dài từ chỉ là 16 bit, nên một phần của kết quả, tức là 0, được lưu trữ trong đó. Bên ngoài biểu diễn số (trong số 17 không tồn tại chút). Để lưu trữ đơn vị này trong máy tính, một thanh ghi mang 1 bit đặc biệt được sử dụng - C (xem đoạn 2.1). Người sử dụng máy tính phải soạn chương trình sao cho khi thực hiện hành động sau phần tổng kết, nội dung của số đăng ký chuyển giao được tính đến (sử dụng hoặc bỏ qua) khi cần thiết.

Ví dụ 2.3. Tổng lớn hơn 65536:

$ \begin{array}{r} 1\ 1111\ 1111\ 1111\ 111 \\ +\ 1111\ 1111\ 1111\ 1111 \\ \hline 0111\ 1111\ 1111\ 1111\ 1 \\ \hline 0111\ 1111\ 1111\ 1110 \end{array} $	$ \begin{array}{r} 111 \\ +\ 65535 \\ \hline 32767 \\ \hline 32766? \end{array} $	Chuyển khoản
--	--	--------------

Ở đây, do độ dài từ bị giới hạn, thay vì $65535 + 32767 = 98302$, một lần nữa (như trong Ví dụ 2.2), kết quả giảm đi 65536 (2^{16}), và một giá trị mang từ bit bậc cao xuất hiện.

Trong máy tính cơ sở, để thực hiện hoạt động được xem xét, lệnh ADD (ADD) được sử dụng. Một chương trình ví dụ để cộng hai số đã được thảo luận trong đoạn 2.1 và 2.2.

Số nguyên nhị phân số dương và số âm. Để phân biệt số âm với số dương, trong bất kỳ hệ thống số nào được xem xét ở trên, bạn có thể sử dụng một dấu hiệu (biểu diễn số như vậy được gọi là mã trực tiếp của chúng) hoặc

độ lớn của một số (một trong những biểu diễn này của các số được gọi là mã bổ sung của chúng).

Việc sử dụng các số có dấu (mã trực tiếp để biểu diễn các số) làm phức tạp cấu trúc của máy tính, vì phép toán cộng hai số có dấu hiệu khác nhau phải được thay thế bằng phép toán trừ một giá trị nhỏ hơn cho một số lớn hơn và gán một dấu hiệu lớn hơn. đến kết quả. Do đó, trong hầu hết các máy tính, số âm được biểu diễn dưới dạng gọi là phép cộng, khi tính tổng hai số có các dấu khác nhau, có thể thay phép trừ bằng phép cộng thông thường.

Phần bổ sung M của số nguyên n-bit K được gọi là
Sự khác biệt

$$M = b^N - \text{ĐẾN}, \quad (2.1)$$

trong đó b là cơ số của hệ thống số.

Vì số lượng các số nguyên n bit phân biệt là b^N (từ 0 đến $b^N - 1$), thì một nửa số này được coi là dương (từ 0 đến $b^N / 2 - 1$), và nửa còn lại - là âm (tức là, là phép cộng vào nửa đầu của các số). Tập hợp đầy đủ các số như vậy được gọi là các số được trình bày trong mã bổ sung (Bảng 2.5).

Có thể dễ dàng kiểm tra tính đúng đắn của việc lấy các phép cộng (mã số âm) trong các cột của Bảng. 2.5. Để thực hiện việc này, hãy thêm số từ nửa dưới của bảng với phần bù của nó nằm phía trên mã số 0. Kết quả của tổng như vậy sẽ là các số 105 = (1 00 000) 10, 164 = (1 0000) 16 hoặc 216 = (10 000 0000 0000 0000) 2. Do thực tế là số lượng bit trong Bảng. 2,5 được giới hạn ở 5, 4 và 16 chữ số tương ứng, 1 ở chữ số cao nhất của số tiền nhận được sẽ không được tính đến, vì nó "nằm ngoài" lưới bit. Do đó, tất cả các tổng đều bằng 0, tức là kết quả chính xác của việc cộng hai số có cùng độ lớn với các dấu khác nhau.

Bảng 2.5

Mã hàng trực tiếp 5-bit số thập phân	Mã số bổ sung		
	Mã số bổ sung		
	5-bit thập phân	4-bit hệ thập lục phân	16-bit nhị phân
-50 000	50.000	-	-
-49 999	50 001	-	-
-49 998	50 002	-	-
.	.	.	.
-32 769	67 231	-	-
-32 768	67 232	8000	1000000000000000
-32 677	67 233	8001	1000000000000001
.	.	.	.
-3	99 997	FFFD	1111 1111 1111 1101

-2	99 998	FFFE	
-1	99 999	FFFF	
0	00 000	0000	
+1	00 001	0001	
+2	00 002	0002	
+3	00 003	0003	111111111111111101111111111111
.	.	.	.
+32 766	32.766	7FFE	0 111 1111 1111 1110
+32 767	32.767	7FFF	0 111 1111 1111 1111
+32 768	32.768	-	-
.	.	.	.
+49 998	49 998	-	-
+49 999	49 999	-	-

Phần bù của một số có thể nhận được mà không cần phép trừ.
Chúng ta hãy viết lại công thức (2.1)

$$\text{dưới dạng } M = b^n - K = ((b^n - 1) - K) + 1. \quad (2.2)$$

Ở đây số $b^n - 1$ gồm n chữ số $(b - 1)$, tức là hệ số (đối với các số trong Bảng 2.5, đây là 99 999, FFFF hoặc 1111 1111 1111 1111). Do đó $(b$ bằng cách tạo thành các phép cộng với $b - 1$ cho từng chữ số của số K riêng biệt, và phép cộng tổng được - bằng cách cộng số này với 1. Vì trong hệ nhị phân, các phép cộng của các chữ số với 1 tương ứng với nghịch đảo của chúng giá trị (phép cộng 0 là 1 và phép cộng 1-0), thì một quy tắc đơn giản có thể được đưa ra để thu được phần bù của hệ nhị phân

những con số.

1. Lấy nghịch đảo của một số nhất định (thay tất cả các số 0 bằng 1 và tất cả các số 1 bằng số 0):

0 000 0010 1100 0101 số 1 111 1101

0011 1010 nghịch đảo của số 2. Tạo thành phần bù

hai của số đã cho bằng cách thêm 1 vào nghịch đảo của số này:

$$\begin{array}{r} + 1\ 111\ 1101\ 0011\ 1010\ 1\ 1 \\ \hline 111\ 1101\ 0011\ 1011 \end{array}$$

thuật ngữ đảo ngược
số 1 mã số bổ sung

Hãy kiểm tra tính đúng đắn của bản dịch:

$$\begin{array}{r} 1\ 1\ 111\ 1111\ 1111\ 111 \\ + 0\ 000\ 0010\ 1100\ 0101\ 1 \\ \hline 111\ 1101\ 0011\ 1011\ 1\ 0 \\ 000\ 0000\ 0000\ 0000 \end{array}$$

chuyển tiền
con số
mã bổ sung của số 0

Vì giá trị mang từ bit quan trọng nhất không được tính đến, kết quả của tổng là 0, điều này xác nhận tính đúng đắn của chuyển đổi.

Kết luận, chúng tôi lưu ý một đặc điểm nữa của mã bổ sung của các số nhị phân: bit quan trọng nhất của chúng thực hiện chức năng của dấu của số, nghĩa là, nó bằng 0 đối với các số dương và 1 đối với phần bổ sung của chúng (số âm). Do đó, bit quan trọng nhất của ô nhớ và thanh ghi có thể lưu trữ các số như vậy thường được gọi là bit dấu.

Phép cộng các số nguyên nhị phân có dấu. Nó được thực hiện theo các quy tắc tương tự và sử dụng lệnh ADD tương tự như đối với các số không có dấu. Ví dụ về việc cộng các số nguyên X và Y để thu được tổng Z được đưa ra trong Bảng 2.6.

Trong hai trường hợp (thứ hai và thứ tư), khả năng giới hạn của các con số dẫn đến sự sai lệch không chỉ về độ lớn, mà còn về dấu hiệu của kết quả. Bạn có thể sửa tình huống bằng cách thêm vào kết quả (hoặc trừ đi) số $216 = 65536$, nhưng đối với điều này, bạn cần phải đi đến đăng kết quả một cách tóm tắt.

Bảng 2.6

Các trường hợp khác nhau của việc thêm số nhị phân đã ký		
Trường hợp	Điều khoản và kết quả Nhận xét	
1	$ \begin{array}{r} X > 0, Y > 0, X + Y < 215 \\ + 0\ 000\ 0110\ 0011\ 1010\ X = + 1594 \\ 0\ 100\ 0100\ 1001\ 1011\ Y = + 17563 \\ \hline 0\ 100\ 1010\ 1101\ 0101\ Z = + 19157 \end{array} $	Kết quả là đúng
2	$ \begin{array}{r} X > 0, Y > 0, X + Y \geq 215 \\ + 0\ 100\ 0100\ 1001\ 1011\ X = + 17563 \\ 0\ 100\ 0010\ 1101\ 0101\ Y = + 19157 \\ \hline 1\ 000\ 1111\ 0111\ 0000\ Z = -28816 \end{array} $	Khi được thêm vào số dương nhận được một kết quả âm tính TRẦN RA!
3	$ \begin{array}{r} X < 0, Y < 0, X + Y < 215 \\ + 1\ 1111001\ 1100\ 0110\ X = - 1594 \\ 1\ 011\ 1011\ 0110\ 0101\ Y = -17563 \\ \hline 1\ 1\ 001\ 0101\ 0010\ 1011\ Z = -19157 \end{array} $	Kết quả là chính xác. Việc chuyển từ thứ tự cao nhất của chuỗi không được tính đến
bốn	$ \begin{array}{r} X < 0, Y < 0, X + Y > 215 \\ + 1\ 011\ 1011\ 0110\ 0101\ X = -17563 \\ 1\ 011\ 0101\ 0010\ 1011\ Y = -19157 \\ \hline 1\ 0\ 111\ 0000\ 1001\ 0000\ Z = + 28816 \end{array} $	Khi cộng các số âm, ta thu được kết quả dương - TRẦN RA!
5	$ \begin{array}{r} X > 0, Y < 0, X > Y \\ + 0\ 100\ 0100\ 1001\ 1011\ X = + 17563 \\ 1\ 1111001\ 1100\ 0110\ Y = - 1594 \\ \hline 0\ 011\ 1110\ 0110\ 0001\ Z = + 15969 \end{array} $	Kết quả là chính xác. Việc chuyển từ thứ tự cao nhất của chuỗi không được tính đến
6	$ \begin{array}{r} X > 0, Y < 0, X < Y \\ + 0\ 000\ 0110\ 0011\ 1010\ X = + 1594 \\ 1\ 011\ 1011\ 0110\ 0101\ Y = -17563 \\ \hline 1\ 100\ 0001\ 1001\ 1111\ Z = -15969 \end{array} $	Kết quả là đúng

Nếu các số hạng sao cho xác suất thu được các kết quả méo mó không bằng 0 thì trước khi tính tổng cần phân tích dấu hiệu của các số hạng và kết quả. Các dấu hiệu khác nhau của các số hạng hoặc sự trùng hợp của các dấu hiệu của các số hạng với dấu của tổng chỉ ra rằng kết quả sẽ đúng. Nếu không, quá trình tính toán phải được hủy bỏ và phát ra tín hiệu OVERFLOW.

Đặt một số nhị phân số nguyên trong bộ nhớ một vài từ. Nó được sử dụng để mở rộng phạm vi biểu diễn của những con số như vậy và do đó, để tăng độ chính xác của các phép tính. Vì vậy, bằng cách đặt một số trong hai từ 16 bit, chúng ta có thể làm việc với các số trong phạm vi

$$\begin{array}{l} \text{từ } (1 \ 000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000 \ 0000) \ 2 = 2147483648 \text{ đến } (0 \ 111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111 \ 1111) \ 2 = 2147483647 = 231 - 1 \\ \underbrace{\hspace{10em}}_{\text{từ cao cấp}} \quad \underbrace{\hspace{10em}}_{\text{từ thấp}} \end{array}$$

thay vì phạm vi từ -32768 đến +32767, trong đó các số nhị phân có dấu một từ thay đổi (xem Bảng 2.5).

Vì đơn vị số học-lôgic của máy tính cơ sở chỉ cho phép xử lý các toán hạng 16 bit, một chuỗi hành động nhất định phải được thực hiện để tổng hợp nhiều số từ:

1) để thêm các từ thấp hơn của các số, trong khi 1 hoặc 0 sẽ được tự động nhập vào số đăng ký chuyển giao, tùy thuộc vào sự có mặt hay vắng mặt của quá trình chuyển xảy ra khi thêm các chữ số cao hơn

lời tóm tắt;

2) viết tổng kết quả vào từ thấp của kết quả;

3) để thêm các từ tiếp theo theo thứ tự ưu tiên và nội dung của số đăng ký chuyển giao, khi kết thúc hoạt động, một chuyển giao được nhập vào số đăng ký chuyển giao từ chữ số cao nhất của các từ được tổng hợp (1 hoặc 0);

4) ghi số tiền nhận được vào từ tương ứng của kết quả;

5) lặp đoạn. 3 và 4 cho đến khi tất cả

các từ của các điều khoản.

Trong bảng. 2.7 là một ví dụ về chương trình thêm số X \ u003d 12345678 (BC614E) 16 và Y = 76543210 (48FF4EA) 16.

Bảng 2.7

Chương trình tính tổng các số có hai từ			
Địa chỉ nhà	Mã nội dung		Bình luận
	ghi nhớ		
0	F200	CLA	Xóa pin (pin chứa 0)
	400A	THÊM A	Bộ tích lũy chứa từ thấp của thuật ngữ X
2	400C	THÊM C	Bộ tích lũy chứa tổng của các từ thấp hơn X và Y, và thanh ghi ghi số là hành vi mang theo xảy ra

3	300E	DI CHUYỂN	khi chúng được thêm vào (trong trường hợp này, giá trị thực là 1)
bốn	F200	CLA	Tổng các từ thấp của X và Y được chuyển đến từ thấp
5	5009	ADC9	được phân bổ cho kết quả. Xóa (chuẩn bị bộ tích lũy bằng cách tổng hợp
6	400B	THÊM B	khoản)
7	300D	MOV D	Bộ tích lũy chứa tổng của từ cao X và nội dung của
8	F000	HLT	thanh ghi mang, lưu trữ số mang thu được bằng cách
9	00BC		thêm các từ thấp hơn.
Một	614E		Bộ tích lũy chứa tổng các từ cao X và
B	048F		Y, cũng như chuyển
C	F4EA		Tổng của các từ cao X và Y được chuyển đến từ cao được
D	0000		phân bổ cho kết quả.
E	0000		Tắt máy tính (chấm dứt lựa chọn lệnh)
			Từ cao của thuật ngữ X
			Từ thấp của thuật ngữ X
			Từ cao của thuật ngữ Y
			Từ thấp của thuật ngữ Y
			Ô dành cho từ cao của kết quả, số 1356 (054C) 16 sẽ
			được viết ở đây
			Ô dành riêng cho từ thấp của kết quả, số 22072 (5638)
			16 sẽ được viết ở đây

Chương trình này sử dụng lệnh ADC B để thêm nội dung của bộ tích lũy, vị trí B và thanh ghi mang, giữ giá trị mang (0 hoặc 1) từ thao tác trước đó. Kết quả của tổng được lưu trữ trong bộ tích lũy, và 1 hoặc 0 được nhập vào thanh ghi mang, tùy thuộc vào sự có mặt hay không có dấu từ bit quan trọng nhất của các từ được tổng hợp.

Tăng một (INCrement) và giảm một (DEC-rement). Lệnh INC thêm một vào bộ tích lũy và lệnh DEC trừ một. Nếu đồng thời có chuyển khoản từ lệnh cao thì 1 được nhập vào sổ đăng ký chuyển (C), ngược lại - 0.

Thay đổi dấu hiệu của một số. Ở trên đã nói rằng để thay đổi dấu hiệu của một số được viết trong một mã bổ sung, cần phải đảo ngược nó, và sau đó thêm một vào chữ số có nghĩa nhỏ nhất. Trên máy tính, thao tác như vậy có thể được thực hiện bằng cách sử dụng lệnh CMA (đảo ngược nội dung bộ tích lũy) và INC.

Trong bảng. 2.8 cho thấy một chương trình ví dụ để lấy $Y = -X$.

Bảng 2.8

Chương trình thay đổi dấu của một số			
Địa chỉ nhà	Mã nội dung		Bình luận
	ghi nhớ		
0	F200	CLA	Xóa pin (pin chứa 0)
	4006	6 sau công nguyên	Pin chứa X
2	F400	CMA	Bộ tích lũy có chứa đảo ngược X
3	F800	INC	1 được thêm vào nội dung của bộ tích lũy và nó tạo thành mã bổ sung của số X
bốn	3007	mov7	Số -X được đặt trong ô dành riêng cho Y
5	F000	HLT	Tắt máy tính (chấm dứt lựa chọn lệnh)
6	6844		Số X
7	0000		Ô dành riêng cho Y, sau khi thực hiện chương trình, sẽ được ghi ở đây -X \ u003d 97BC

Phép trừ ($X - Y$). Nó có thể được thực hiện bằng cách thêm minuend X và trừ mã phần bù của Y. Tuy nhiên, điều này yêu cầu viết và thực hiện một số lệnh (CLA, ADD Y, CMA, INC, ADD X). Để giảm chương trình và thời gian thực hiện phép trừ, máy tính cơ sở cung cấp lệnh trừ SUB Y

(CLA, ADD X, SUB Y), thực hiện các hành động tương tự trong thời gian ngắn hơn.

Khi thực hiện một phép trừ (giống như khi thực hiện một phép cộng), có thể xảy ra hiện tượng tràn. Dấu hiệu tràn là kết quả dương tính với $X < 0$ và $Y > 0$ hoặc kết quả âm tính đối với $X > 0$ và $Y < 0$ (xem Bảng 2.6).

Phép nhân và phép chia. Máy tính cơ sở không có hướng dẫn để thực hiện phép nhân và chia (ALU không thực hiện các phép toán này). Do đó, người dùng phải tổ chức việc nhận tác phẩm hoặc việc riêng tư theo lập trình.

Trong phép nhân nhị phân, tích từng phần được dịch sang trái một bit cho mỗi bit kế tiếp của hệ số nhân. Nếu cấp số nhân là 0, thì tích từng phần là 0; nếu cấp số nhân là 1, thì tích một phần bằng cấp số nhân.

Ví dụ, hãy xem xét thu được các tích của $5 \times 5 = 25$, $5 \times 10 = 50$ và $5 \times 3 = 15$:

$$\begin{array}{r} \times 101 \text{ (5)} \\ \underline{11 \text{ (3)}} \\ 101 \\ \underline{101} \\ 1111 \text{ (15)} \end{array}$$

$$\begin{array}{r} \times 101 \text{ (5)} \\ \underline{101 \text{ (5)}} \\ 101,000 \\ 101 \text{ 11001} \\ \underline{(25)} \end{array}$$

$$\begin{array}{r} \times 101 \text{ (5)} \\ \underline{1010 \text{ (10)}} \\ 000 \\ 101 \\ 000 \\ \underline{101} \\ 110010 \text{ (50)} \end{array}$$

2.4. Quản lý quy trình máy tính, sự thay đổi và hoạt động logic

Hầu hết tất cả các chương trình máy tính phải có khả năng kiểm tra dữ liệu ban đầu, kết quả trung gian và cuối cùng của phép tính, để thay đổi trình tự hoạt động theo đúng cách dựa trên kết quả kiểm tra. Ví dụ: bạn nên phản ứng đúng với tình huống khi cộng các số dương dẫn đến kết quả âm (xem đoạn

2.3).

Một vấn đề tương tự được giải quyết trong máy tính cơ sở với sự trợ giúp của các lệnh chuyển tiếp (BCS, BPL, BMI, BEQ, BR), lệnh INCREMENT và SKIP (ISZ), STOP (HLT). Tất cả các lệnh này (ngoại trừ HLT) là lệnh địa chỉ, tức là chúng chỉ ra địa chỉ của ô nhớ mà từ đó lệnh tiếp theo sẽ được chọn khi một điều kiện nhất định được đáp ứng. Nếu điều kiện không được đáp ứng, thì lệnh sau lệnh điều khiển đã cho phải được thực hiện. Như trong các lệnh địa chỉ khác, ở đây bạn có thể sử dụng địa chỉ gián tiếp, nghĩa là chỉ định địa chỉ của ô mà từ đó địa chỉ bước nhảy mong muốn sẽ được chọn.

Các nhánh trong chương trình được tổ chức bằng cách sử dụng các lệnh nhảy. Các hướng dẫn này không thay đổi trạng thái của bộ tích lũy và thanh ghi mang. Họ chỉ có thể thay đổi nội dung của bộ đếm chương trình bằng cách đặt vào đó địa chỉ được chỉ định bởi phần địa chỉ của lệnh.

BCS M (ĐI NẾU THẬT). Chuyển đến lệnh nằm trong ô có địa chỉ M, nếu nội dung của thanh ghi chuyển là 1.

BPL M (ĐI NẾU CỘNG HÒA). Chuyển đến lệnh nằm trong ô có địa chỉ M, nếu nội dung của bộ tích lũy lớn hơn hoặc bằng 0, tức là bit quan trọng nhất của nó chứa 0.

BMI M (ĐI NẾU TRỪ). Đi tới lệnh nằm trong ô có địa chỉ M, nếu nội dung của bộ tích lũy nhỏ hơn 0, tức là bit quan trọng nhất của nó chứa 1.

BEQ M (JUMP NẾU KHÔNG CÓ). Chuyển đến lệnh nằm trong ô có địa chỉ M, nếu nội dung của bộ tích lũy bằng 0, tức là tất cả các bit của nó đều chứa 0.

Nếu điều kiện được chỉ định trong các lệnh được liệt kê không được đáp ứng, thì không thể thực hiện rẽ nhánh và lệnh nằm sau lệnh rẽ nhánh có điều kiện được thực hiện.

BR M (THẮNG 6 KHÔNG ĐIỀU KIỆN). Việc chuyển đổi sang lệnh nằm trong ô có địa chỉ M, được thực hiện với bất kỳ giá trị nào của bộ tích lũy, thanh ghi chuyển và các thanh ghi khác của máy tính.

Ví dụ 2.4. Lấy mô-đun của số X chứa trong ô nhớ có địa chỉ 100. Đặt kết quả vào ô có địa chỉ 200. Chương trình lấy mô-đun được cho trong Bảng. 2.9.

Bảng 2.9

Chương trình truy xuất mô-đun			
Địa chỉ nhà	Nội dung ghi		Bình luận
	Mặt mã	nhớ	
0 ... 2	F200 4006 9005	CLA 100 AD BPL 5	Xóa pin (pin chứa 0) Bộ tích lũy chứa đối số X Nếu $X \geq 0$, thì chuyển đến lệnh nằm trong ô có địa chỉ 5
3 bỏ	F400 F800	CMA INC	Thay đổi dấu hiệu của số X, được biểu thị trong mã bổ sung của cả hai
5 6	3200 F000	MOV 200 HLT	Viết $ X $ vào ô có địa chỉ 200 Tắt máy tính (chấm dứt lựa chọn lệnh)

Các chương trình tuần hoàn được sử dụng trong những trường hợp đó khi yêu cầu thực hiện một tập hợp các hành động giống nhau nhiều lần trên một số thành phần thay đổi của dữ liệu.

Máy tính cơ sở có một số công cụ để đơn giản hóa việc tổ chức các chương trình tuần hoàn. Tính hiệu quả của việc giới thiệu các quỹ này được xem xét một cách thuận tiện nhất bằng các ví dụ.

Đầu tiên, xem xét thu được sản phẩm $Z = 50Y$. Vì không có lệnh nhân nào trong tập lệnh của máy tính cơ bản, chúng ta sẽ sử dụng phương pháp đơn giản nhất: chúng ta sẽ cộng giá trị Y . 50 lần. Đồng thời, một chương trình tuyến tính chứa 50 lệnh ADD Y khó có vị trí đầu tiên. trong cuộc thi chương trình nhân. Nó có thể sẽ bị vượt qua bởi chương trình được hiển thị trong Bảng. 2.10.

Có những cách nào để đơn giản hóa chương trình này? Đúng.

Ví dụ: thay vì đếm và kiểm tra số chu kỳ đã hoàn thành, bạn có thể đếm và kiểm tra số chu kỳ còn lại cho đến khi chương trình kết thúc, tức là giảm giá trị của M trong chu kỳ cho đến khi nó trở thành 0. Khi đó không cần sử dụng ô 8 và lệnh trừ (SUB 7) - các lệnh từ 15 đến 19 được thay thế bằng chuỗi lệnh: ADD 7, DEC, MOV 7, BEQ 1A, BR 10. Tuy nhiên, nếu giá trị âm của hệ số -50 được đặt trong ô 7 = (FFCE) 16, sau đó các lệnh từ 15 đến 19 có thể được thay thế bằng chuỗi lệnh: ADD 7, INC, MOV 7, BMI 10, HLT, tức là cũng miễn phí ô 1A.

Bảng 2.10

Phiên bản đầu tiên của chương trình có được Z = 50Y			
Địa chỉ nhà	Mã nội dung		Bình luận
	ghi nhớ		
5	0078	Y	Hệ số nhân (ở đây giá trị thập phân là 120) Ô được phân bổ để tích lũy kết quả. Nó sẽ luân phiên lưu trữ các giá trị Y, 2Y, 3Y, ... Sau 50 lần tổng kết, nó sẽ chứa kết quả mong muốn 50Y
6	0000	Z	
7	0032	M	
10	0000	C	Hệ số 50 = (32) 16 Ô được sử dụng để tích lũy số chu kỳ được thực hiện là bộ đếm chu kỳ _____
mười	F200	CLA	Đối với kết quả trung gian, nằm trong ô 6, một giá trị nữa của hệ số được thêm vào Y
mười một	4006	6 sau công nguyên	
12	4005	THÊM 5	
13	3006	MOV 6	
mười bốn	F200	CLA	Nội dung của bộ đếm chu kỳ được tăng lên 1 và bản sao của nó vẫn được lưu trữ trong bộ tích lũy
mười lăm	4008	THÊM 8	
16	F800	INC	
17	3008	MOV 8	
mười sáu	6007	SUB 7	Nếu nội dung của bộ đếm chu kỳ nhỏ hơn giá trị của hệ số nhân, thì một quá trình chuyển đổi được thực hiện cho các lệnh thực hiện tổng tiếp theo của Y với giá trị trung gian là Z. Ngược lại, lệnh tiếp theo (lệnh HLT) được thực hiện.
19	A010	BMI 10	
1A	F000	HLT	

Có thể đạt được sự đơn giản hóa chương trình nhiều hơn bằng cách tích lũy kết quả trực tiếp trong bộ tích lũy, chứ không phải trong ô 6. Sau đó, các lệnh từ 10 đến 13 có thể được thay thế chỉ bằng một lệnh ADD 5. Tuy nhiên, trong phiên bản đã thảo luận của chương trình (xem Bảng 2.10), bộ tích lũy phải được sử dụng nhiều hơn và thay đổi số chu kỳ, dẫn đến nhu cầu lưu trữ kết quả trung gian Z trong bộ nhớ máy tính. Vì vậy, để đơn giản hóa các chương trình chu kỳ, nên có một lệnh trong các lệnh máy tính cơ sở cho phép bạn tổ chức bộ đếm chu kỳ mà không ảnh hưởng đến pin. Một đội như vậy là ISZ.

Lệnh ISZ (INCREASE AND SKIP) được sử dụng để tăng 1 nội dung của vị trí bộ nhớ được định địa chỉ và chuyển đến một trong hai cách để tiếp tục chương trình, tùy thuộc vào dấu hiệu của nội dung này. Do đó, mỗi khi lệnh ISZ được thực thi

M, nằm ở địa chỉ A, 1 được thêm vào nội dung của ô có địa chỉ M, và nếu kết quả nhỏ hơn 0, thì lệnh sau ISZ M (lệnh nằm ở địa chỉ A + 1) được thực hiện; nếu không, lệnh này bị bỏ qua, tức là được thực thi

đội nằm ở $A + 2$. Một đặc tính rất có giá trị của lệnh ISZ M là nó không làm hỏng nội dung của bộ tích lũy và thanh ghi mang (việc tăng M được thực hiện mà không sử dụng các thanh ghi này).

Trong bảng. Hình 2.11 cho thấy một biến thể của chương trình để thu được $Z = 50Y$, trong đó tất cả các khả năng được thảo luận ở trên để giảm bộ nhớ và thời gian tích lũy kết quả được sử dụng (thời gian giảm do chỉ có ba lệnh được thực hiện trong mỗi lệnh trong số 50 chu kỳ thay vì mười).

Bảng 2.11

Phiên bản thứ hai của chương trình để có được $Z = 50Y$

Địa chỉ nhà	Mã nội dung		Bình luận
	ghi nhớ		
5	0078	Y	Multiplicand
6	0000	Z	Ô dành riêng cho kết quả
7	FFCE	M	Giá trị hệ số âm (-50)
mười	F200	CLA	Vệ sinh pin
mười một	4005	THÊM 5	Y được thêm vào nội dung của bộ tích lũy
12	0007	ISZ 7	Nội dung của M được tăng thêm 1 và nếu nó vẫn nhỏ
13	C011	BR11	hơn 0, thì lệnh BR 11 được thực hiện. Khi $M = 0$, lệnh BR 11 bị bỏ qua
mười bốn	3006	MOV 6	Kết quả của 50 phép cộng Y được ghi vào ô 6
mười lăm	F000	HLT	Tắt máy tính

Như ví dụ tiếp theo, hãy xem xét lấy trong ô 005 tổng 32 phần tử của một mảng nằm trong các ô nhớ từ 010 đến 02F.

Không giống như nhiệm vụ trước, trong đó nội dung của một ô (Y) được tổng hợp nhiều lần, ở đây nội dung của các ô khác nhau phải được tổng hợp. Đương nhiên, chúng ta không thể hài lòng với một chương trình chứa một chuỗi 32 lệnh cộng: 4010, 4011, 402E, 402F. Rõ ràng, yêu cầu tổ chức một quy trình tuần hoàn trong đó địa chỉ của phần tử mảng được tính tổng phải thay đổi với mỗi lần vượt qua vòng lặp (lần đầu tiên địa chỉ này phải bằng 10, ở lần truyền thứ hai của vòng lặp - 11, trên lần thứ ba - 12, v.v.). Làm thế nào để thay đổi địa chỉ của phần tử được tổng hợp?

Trong các máy tính đầu tiên, địa chỉ được thay đổi bằng cách sửa đổi phần địa chỉ của lệnh (chuyển hướng): sau khi nội dung của ô nhớ được máy tính sử dụng làm lệnh, các lệnh chương trình tiếp theo sử dụng nội dung này như một toán hạng (số) và đã chuyển đổi nó khi cần thiết (ví dụ: tăng lên 1). Hãy sử dụng kỹ thuật này để giải quyết vấn đề (Bảng 2.12).

Bảng 2.12

Phiên bản đầu tiên của chương trình để tính tổng các phần tử mảng Nội

Địa chỉ nhà	dung Mã Mnemonic		Bình luận
5	0000		Ô được phân bổ để tích lũy kết quả.
6	FFE0		Số phần tử âm của mảng (-32)
mười			
.	..		Giá trị số của các phần tử mảng
2F			
ba mươi	F200	CLA	Trung gian (ô 5) được thêm vào nội dung của phần tử mảng, địa chỉ của nó nằm trong phần địa chỉ của lệnh nằm trong ô 32 (đầu tiên địa chỉ này là 10, sau đó nó tăng lên 1 mỗi khi vòng lặp là được chuyển bằng cách sử dụng các lệnh 34-37)
31	4005	THÊM 5	
32	4010	THÊM 10 *	
33	3005	MOV 5	
34	F200	CLA	Chuyển bộ tích lũy của lệnh nằm ở địa chỉ 32, thêm 1 vào nội dung của nó và ghi lệnh đã sửa đổi vào vị trí cũ (trong ô 32)
35	4032	Cộng nguyên 32	
36	F800	INC	
37	3032	MOV 32	
38	0006	ISZ 6	
39	C030	BR 30	
3A	F000	HLT	Bộ đếm phần tử mảng tăng dần 1 và nhảy đến lệnh 30 miễn là nội dung của nó nhỏ hơn 0
*			Tắt máy tính

Hướng dẫn được sửa đổi trong quá trình thực hiện chương trình.

Chuyển hướng lệnh thực tế không được sử dụng trong các máy tính hiện đại. Để đưa ngôn ngữ lệnh gần hơn với các ngôn ngữ thuật toán, các công cụ định địa chỉ đặc biệt đã được phát triển (một số công cụ trong số chúng đã được thảo luận ngắn gọn trong Phần 2.2). Những công cụ tương tự này cho phép bạn tạo các chương trình hiệu quả được đặt trong các thiết bị bộ nhớ chỉ đọc, từ đó bạn có thể

chỉ đọc lệnh hoặc dữ liệu, nhưng không thể thay đổi chúng

(các chương trình máy tính vi mô điều khiển hoạt động của máy giặt, một số thiết bị trên xe hơi, rô bốt, v.v.). Một trong những phương tiện như vậy là địa chỉ gián tiếp (xem Hình 2.4).

Khi sử dụng cách đánh địa chỉ gián tiếp, bạn cần chọn một ô trong bộ nhớ máy tính (ví dụ, 007), ghi địa chỉ của phần tử đầu tiên của mảng được tổng hợp (địa chỉ 010) vào đó, thay thế Bảng. 2.12 lệnh 4010 đến lệnh 4807 (ô 32) và các lệnh chuyển hướng (CLA, ADD 32, INC, MOV 32) bằng các lệnh để tính toán địa chỉ hiện tại của phần tử mảng được tổng hợp (CLA, ADD

7, INC, MOV 7). Các dấu ngoặc trong lệnh ADD (7) và số 8 trong mã số 4807 của nó (một trong bit thứ 11 của lệnh) thông báo cho máy tính rằng nội dung của bộ tích lũy không được thêm vào nội dung

ô 7, nhưng với nội dung của ô có địa chỉ được lưu trữ trong ô 7. Do đó, địa chỉ của phần tử đầu tiên của mảng được tổng hợp trước tiên được đặt trong ô 7 và sau đó (với mỗi đoạn của vòng lặp) địa chỉ này là tăng 1.

Trong bảng. 2.13 cho thấy một phiên bản rút gọn của chương trình để tính tổng các phần tử mảng, trong đó, ngoài việc định địa chỉ gián tiếp, một công cụ mới được sử dụng để thay đổi địa chỉ hiện tại của phần tử mảng được tổng hợp. Công cụ này là lệnh ISZ 7, trong đó nội dung của ô 7 được tăng lên 1. Nhưng vì nội dung này (địa chỉ của phần tử mảng) là một giá trị dương, sau khi thực hiện ISZ 7, lệnh theo sau nó sẽ bị bỏ qua. Do đó, lệnh NOP (KHÔNG HOẠT ĐỘNG) được đặt trong ô 33, nhưng có thể đặt bất kỳ số nào.

Cuối cùng, hãy xem xét một công cụ khác cho phép chúng ta đơn giản hóa các chương trình tuần hoàn của máy tính cơ sở - các ô chỉ mục (ô có địa chỉ từ 008 đến 00F). Nếu bạn gián tiếp giải quyết bất kỳ ô nào trong số này, thì trước tiên nội dung của nó sẽ được sử dụng làm địa chỉ của toán hạng, và sau đó nó sẽ tự động tăng lên một. Với địa chỉ trực tiếp, các ô chỉ mục hoạt động giống như các ô thông thường (nội dung của chúng chỉ có thể thay đổi nếu thông tin được ghi vào ô).

Bảng 2.13

Phiên bản thứ hai của chương trình để tính tổng các phần tử mảng Nội			
Địa chỉ nhà	dung Mã Ghi nhớ		Bình luận
5	0000		Ô được phân bổ để tích lũy kết quả. Số phần tử âm của mảng (-32) Địa chỉ hiện tại của phần tử mảng (đầu tiên, địa chỉ của phần tử mảng đầu tiên là 10)
6	FFE0		
7	0010		
mười	..		Giá trị số của các phần tử mảng
2F			
ba mươi	F200	CLA	Vệ sinh pin
31	4807	THÊM (7)	Tính tổng của phần tử tiếp theo của mảng
32	0007	ISZ 7	Địa chỉ hiện tại của phần tử mảng được tăng thêm 1
33	F100	NOP	Lệnh KHÔNG HOẠT ĐỘNG
34	0006	ISZ 6	
35	C031	BR 31	Bộ đếm phần tử mảng tăng dần 1 và nhảy đến lệnh 31
36	3005	MOV 5	miễn là nội dung của nó nhỏ hơn 0
37	F000	HLT	Ghi kết quả vào ô 5 Tắt máy tính

Thuộc tính chỉ định của các ô chỉ mục cho phép bạn tạo một chương trình tối ưu để tính tổng các phần tử của mảng (Bảng 2.14).

~~Xử lý dữ liệu theo chiều bit được~~ cung cấp trong máy tính cơ sở bằng các lệnh nhân logic (AND), dịch chuyển vòng tròn sang trái (ROL) và sang phải (ROR), cũng như lệnh thanh ghi đảo ngược (CMC) và xóa (CLC). Các lệnh này thường được sử dụng để đóng gói thông tin trong bộ nhớ máy tính (ví dụ: đặt một số nhỏ hoặc mã trong một từ) và giải nén nó, xử lý thông tin văn bản, nhân (chia) với 2, 4, 8 và nội dung của ô nhớ. với địa chỉ M cho phép toán Boolean "AND". Kết quả thực hiện lệnh cho mỗi cặp bit của toán hạng ~~về~~ và bằng một khi cả hai bit đều bằng một; nếu không, bit kết quả bằng không. Vì kết quả tương tự thu được khi nhân bit theo chiều dọc (0x0 \ u003d 0, 0x1 \ u003d 0, 1x0 \ u003d 0 và 1x1 \ u003d 1), lệnh này được gọi là phép nhân logic. Các thuộc tính này cho phép bạn đánh dấu hoặc xóa các bit nhất định của một từ.

Bảng 2.14

Phiên bản thứ ba của chương trình để tính tổng các phần tử mảng Nội

Địa chỉ nhà	dung Mã Ghi nhớ		Bình luận
5 6 . F	0000 FFE0 . 0010	.	Ô được phân bổ để tích lũy kết quả. Số phần tử âm của mảng (-32) Địa chỉ hiện tại của phần tử mảng
mười . 2F	..		Giá trị số của các phần tử mảng
ba mươi 31	F200 480F	CLA THÊM (F)	Vệ sinh pin Tính tổng của phần tử tiếp theo của mảng. Vì đầu tiên địa chỉ của phần tử đầu tiên của mảng (10) được đặt trong ô chỉ mục F, sau đó sau lần thực hiện lệnh đầu tiên, nội dung của ô F sẽ tăng lên 1 và sẽ trở đến phần tử thứ hai của mảng, sau lần thực thi thứ hai - đến phần tử thứ ba, v.v.
32 33 34 35	0006 C031 3005 F000	ISZ 6 BR 31 MOV 5 HLT	Bộ đếm phần tử mảng tăng dần 1 và nhảy đến lệnh 31 miễn là nội dung của nó nhỏ hơn 0 Ghi kết quả vào ô 5 Tắt máy tính

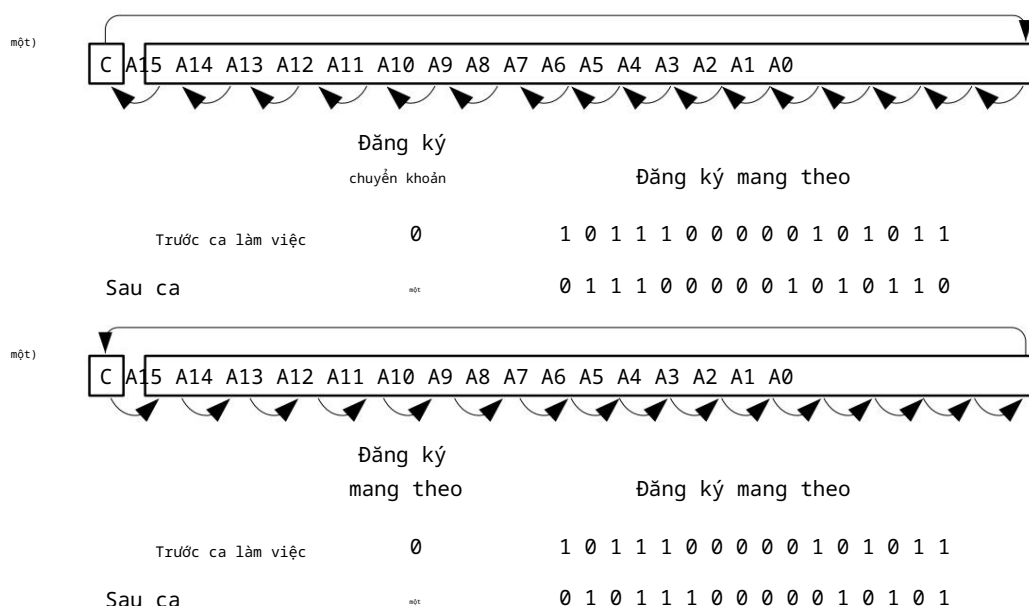
Ví dụ cuối cùng về việc đơn giản hóa các chương trình tuần hoàn, hãy xem xét việc lấy số phần tử lẻ của một mảng nằm trong các ô có địa chỉ từ 010 đến 02F. Chương trình giải quyết vấn đề này được đưa ra trong Bảng. 2,15.

Bảng 2.15

Chương trình xác định số phần tử lẻ trong một mảng Nội dung Mã Ghi nhớ

Địa chỉ nhà			Bình luận
5	0000		Ô được phân bổ để tích lũy kết quả.
6	0001		Mặt nạ để phát hiện các phần tử mảng lẻ.
7	FFE0		Khi nó được nhân một cách hợp lý với một số
tân	0010		lẻ, kết quả sẽ bằng 1 (nếu không thì - 0)
			Số phần tử âm của mảng (-32)
			Địa chỉ hiện tại của phần tử mảng
mười			
.	..		Giá trị số của các phần tử mảng
2F			
ba mươi	F200	CLA	Vệ sinh pin
31	4808	THÊM (8)	Pin chứa một phần tử thông thường
32	1006	VÀ 6	mảng
33	4005	THÊM 5	Bit có ý nghĩa nhỏ nhất của bộ tích lũy (của phần
34	3005	MOV 5	tử mảng) được chọn và thêm vào tổng số phần tử mảng
			lẻ. Nếu bit có ý nghĩa nhỏ nhất là 1 (phần tử lẻ),
35	0007	ISZ 7	thì tổng được tăng lên bởi bit đó
36	C030	BR 30	
37	F000	HLT	Sửa đổi nội dung của bộ đếm phần tử mảng và chuyển
			đến địa chỉ 30 trong khi nội dung này nhỏ hơn 0 Tắt
			máy tính

Hướng dẫn ROL (LEFT CYCLE BY 1 DITCH) và ROR (RIGHT CYCLE BY 1 DIT) đóng bộ tích lũy và thanh ghi mang thành một vòng và dịch chuyển tất cả các bit của vòng bằng một chữ số sang trái hoặc phải (Hình 2.5).



Cm. 2.5. Chuyển dịch theo chu kỳ: a - sang trái; sáng

Bằng cách chuyển các bit của một số sang trái hoặc phải, bạn có thể thực hiện các phép toán nhân hoặc chia cho 2 (một lần dịch chuyển), 4 (hai lần dịch chuyển), v.v. Thật vậy, sau khi chuyển số

$$(0000\ 0000\ 0011\ 1010)_2 = 58$$

một chữ số bên trái sẽ nhận được một số

$$(0000\ 0000\ 0111\ 0100)_2 = 116,$$

và sau một lần thay đổi nữa - số

$$(0000\ 0000\ 1110\ 1000)_2 = 232$$

(ở đây giả định rằng số đăng ký thực hiện đã được xóa trước khi thay đổi).

Hãy xem xét một ví dụ khác về việc đơn giản hóa chương trình thông qua việc sử dụng các lệnh shift: phân tích các phần tử của mảng X (các ô 010-02F) và, nếu X (1) là số âm, hãy gán giá trị 1 cho phần tử thứ H (1) của mảng H và nếu X (1) dương, giá trị 0 (Bảng 2.16).

Bảng 2.16

Dấu trích xuất Nội dung Mã Ghi nhớ			
Địa chỉ nhà			Bình luận
7 tán 9	FFE0 0010 0030		Số phần tử âm của mảng X và H Địa chỉ hiện tại của phần tử mảng X <u>Địa chỉ hiện tại của phần tử mảng H</u>
mười . 2F	..		Giá trị số của các phần tử mảng X
ba mươi . 4F	..		Giá trị số của các phần tử của mảng H
năm mươi 51 52 53 54 55 56 57 58	F200 4808 F600 F200 F600 3809 0007 C050 F000	CLA THÊM (8) ROL CLA ROL mov (9) ISZ 7 BR50 HLT	Vệ sinh pin Phần tử tiếp theo của mảng X được đưa vào bộ tích lũy Dịch chuyển bit dấu X (I) để thực hiện thanh ghi Vệ sinh pin Chuyển nội dung của thanh ghi mang sang chữ số thập của bộ tích lũy và ghi giá trị kết quả vào H (I) Sửa đổi nội dung của bộ đếm phần tử mảng và chuyển đến địa chỉ 50 trong khi nội dung này nhỏ hơn 0 Tắt máy tính

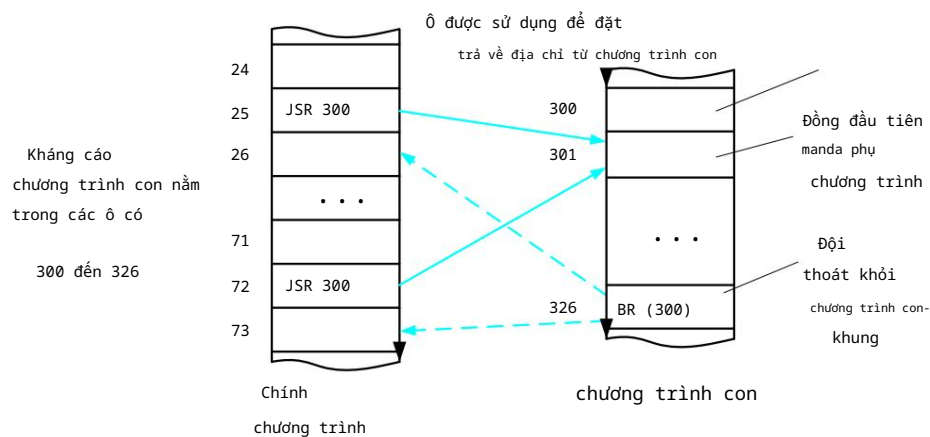
Trong một số ứng dụng, cần phải đảo ngược hoặc xóa một chút cụ thể của một từ. Để làm điều này, ví dụ, bạn có thể:

- đặt từ trong bộ tích lũy;
- chuyển từ để bit mong muốn nằm trong thanh ghi mang; thực hiện lệnh
- CMC ("Invert Carry Register") hoặc CLC ("Clear Carry Register"); chuyển các bit của từ về vị trí cũ.
-

2.5. Chương trình con và tham số

Khá thường xuyên có một tình huống khi các phần riêng biệt của chương trình phải thực hiện các hành động xử lý dữ liệu giống nhau (ví dụ, tính toán của một hàm lượng giác). Trong những trường hợp như vậy, các phần lặp lại của chương trình được tách ra thành một chương trình con và chỉ những hướng dẫn để gọi chương trình con này mới được nhập vào những vị trí thích hợp trong chương trình. Máy tính cơ sở sử dụng hướng dẫn JSR ("Giới thiệu chương trình con") cho mục đích này.

Trên hình. 2.6 cho thấy một phần của chương trình chính, chứa hai lệnh JSR 300, với sự trợ giúp của việc chuyển đổi sang thực thi các lệnh của chương trình con được thực hiện.



Cm. 2.6. Gọi một chương trình con từ chương trình chính

Bằng lệnh JSR 300, nằm trong ô 25, số $25 + 1 = 26$ được ghi vào ô có địa chỉ 300 và quá trình chuyển đổi sang lệnh thực thi đầu tiên của chương trình con, nằm trong ô 301, bắt đầu.) Nằm ở vị trí 326. Đây là một lệnh nhảy không điều kiện với địa chỉ gián tiếp. Nó hướng dẫn máy tính nhảy đến lệnh nằm ở địa chỉ được lưu trong ô 300. Vì số 26 đã được nhập trước đó trong ô này nên lệnh được viết trong ô 26 sẽ được thực hiện, tức là sau lệnh gọi chương trình con.

Bằng lệnh JSR 300, nằm trong ô 72, số $72 + 1 = 73$ được ghi vào ô có địa chỉ 300 và quá trình thực thi chương trình con bắt đầu (từ ô 301). Khi lệnh cuối cùng của chương trình con BR (300) được thực thi, một quá trình chuyển đổi sẽ được thực hiện tới lệnh nằm ở địa chỉ 73, tức là địa chỉ được lưu trong ô 300.

Do đó, khi thiết kế một chương trình con, trước lệnh đầu tiên của nó, bạn nên đặt một ô để gửi địa chỉ trả về từ chương trình con. Lệnh gọi chương trình con chỉ định địa chỉ của ô cụ thể này và lệnh JSR M thực hiện các hành động sau:

1) đặt vào ô M địa chỉ của lệnh theo sau JSR M, tức là lệnh phải được thực thi sau các lệnh của chương trình con;

2) chuyển quyền điều khiển đến lệnh nằm trong ô có địa chỉ M + 1.

Lệnh thực thi cuối cùng của chương trình con phải là lệnh thoát, tức là lệnh BR (M), lệnh này sẽ chuyển đến lệnh có địa chỉ được lưu trong ô M.

Ví dụ 2.5. Nhận mô đun kép của các số X, Y và Z được lưu trữ trong các vị trí bộ nhớ lần lượt là 58, 63 và 71. Kết quả (2 | X |, | Y |, 2 | Z |) được đặt trong các ô 74, 77 và 82.

Để giải quyết vấn đề này, bạn có thể lấy chương trình của ví dụ 2.4 làm cơ sở, sắp xếp đoạn chương trình dưới dạng một chương trình con (Bảng 2.17). Trong tab chương trình. 2.17, chỉ một giá trị được chuyển đến chương trình con: một số được đặt trong bộ tích lũy, mô đun của số đó phải được nhân đôi. Kết quả tính toán (lại một giá trị) được trả về chương trình chính thông qua cùng một bộ tích lũy. Nhưng làm thế nào để truyền và (hoặc) nhận hai hoặc nhiều tham số?

Bảng 2.17

Phiên bản đầu tiên của chương trình để lấy mô-đun kép của ba số Mã nội dung			
Địa chỉ nhà	Ghi nhớ		Bình luận
Chương trình chính			
mười	F200	CLA	Ghi giá trị của X vào bộ tích lũy, gọi chương trình con để lấy mô-đun nhân đôi (ghi vào ô 30 của địa chỉ 13 và chuyển đến lệnh ở địa chỉ 31), chuyển kết quả đến ô lưu trữ 2 X
mười một	4058	THÊM 58	
12	Năm 2030	JSR 30	
13	3074	tháng 74	
mười bốn	F200	CLA	Các hành động tương tự như trong lệnh 10-13, nhưng với đối số Y. Khi truy cập chương trình con, địa chỉ trả về 17 được gửi đến ô 30
mười lăm	4063	63 SCN	
16	Năm 2030	JSR 30	
17	3077	tháng 77	
mười sáu	F200	CLA	Các hành động tương tự như trong lệnh 10-13, nhưng với đối số Z. Khi truy cập chương trình con, địa chỉ trả về 1B được gửi đến ô 30
19	4071	THÊM 71	
1A	Năm 2030	JSR 30	
1B	3082	MOV 82	
1C	F000 HLT		Tắt máy tính
Chương trình con mô-đun kép			
ba mươi	0000	L34	Ô để đặt địa chỉ trả hàng Lệnh đầu tiên: kiểm tra dấu hiệu của số trong bộ tích lũy và nếu nó là số dương, sau đó chuyển quyền kiểm soát đến các hướng dẫn để nhân đôi nội dung của bộ tích lũy
31	9034 BPL		
32	F400	CMA	Ký thay đổi
33	F800	INC	
34	F600	ROL	Nhân đôi nội dung của bộ tích lũy Thoát khỏi chương trình con (chuyển sang thực hiện lệnh, địa chỉ của lệnh được lưu trong ô 30)
35	C830	BR (30)	

Trong máy tính vi mô có một số bộ tích lũy, hoặc tốt hơn, các thanh ghi đa năng, có thể chuyển một số lượng lớn hơn các tham số đến (hoặc nhận từ) một chương trình con. Nhưng số lượng thanh ghi mục đích chung cũng có giới hạn. Do đó, trong tất cả các máy vi tính (cũng như trong máy tính cơ sở) đều có khả năng trao đổi các tham số thông qua các ô ki niệm.

Trong bảng. 2.18 hiển thị phiên bản thứ hai của chương trình để giải ví dụ 2.7. Ở đây, một trong những cách khả thi để truyền tham số được sử dụng: đặt các tham số trực tiếp sau lệnh JSR. Địa chỉ trả về được viết bởi lệnh JSR thực sự là địa chỉ của tham số đầu tiên. Chương trình con phải tăng địa chỉ này để tìm nạp phần còn lại của các tham số và lấy địa chỉ trả về thực tế - địa chỉ của lệnh nằm sau tham số cuối cùng. Bạn có thể sử dụng hằng số hoặc biến (địa chỉ) làm tham số. Trong trường hợp thứ hai, việc xử lý các tham số này trở nên phức tạp hơn một chút (xem Bảng 2.18).

Bảng 2.18

Phiên bản thứ hai của chương trình để lấy mô-đun kép của ba số Nội dung Mã

		Ghi nhớ		Bình luận
Địa chỉ nhà				
Chương trình chính				
mười 12	Năm 2030 0058 0074	JSR 30	Gọi chương trình con, nơi chứa dữ liệu: địa chỉ X và các ô cho vị trí 2 $ X $. Địa chỉ được gửi đến ô 30	
13 mười bốn mười lăm	Năm 2030 0063 0077	JSR 30	Gọi đến chương trình con, nơi chứa dữ liệu: địa chỉ Y và các ô để đặt 2 $ Y $. Địa chỉ được gửi đến ô 30	
16 17 mười tám	Năm 2030 0058 0074	JSR 30	Gọi đến chương trình con, nơi chứa dữ liệu: địa chỉ Z và các ô để đặt 2 $ Z $. Địa chỉ được gửi đến ô 30 17	
19	F000 HLT		Tắt máy tính	
Chương trình con mô-đun kép				
bảy mươi 31 32 33 34 35	0000 F200 4830 3035 0030 0000	CLA THÊM (30) MOV 35 ISZ 30	Ô để đặt địa chỉ của đối số, kết quả và trả về từ chương trình con Giá trị của biến được trả về bởi nội dung của ô 30 được gửi đến ô 35 (khi một chương trình con được gọi, nội dung này là địa chỉ của đối số), và sau đó địa chỉ trong ô 30 được tăng lên 1 (bây giờ nó trở về địa chỉ của kết quả)	
36	F200 CLA		Địa chỉ của kết quả được gửi đến ô 3A, và	

37	4830	THÊM (30)	địa chỉ trong ô 30 lại được tăng lên 1 lần nữa (bây giờ nó trở đến địa chỉ của lệnh của chương trình chính, nằm sau lệnh gọi chương trình con này)
38	3035	MOV 3A	
39	0030	ISZ 30	
3A	0000		
3B	F200	CLA	Nhận mô-đun kép của số có địa chỉ được chỉ ra bởi nội dung của ô 35 và chuyển tiếp kết quả đến địa chỉ được chỉ ra bởi nội dung của ô 3A
3C	4835	THÊM (35)	
3D	9040	BPL 40	
3E	F400	CMA	
3F	F800	INC	
40	F600	ROL	
41	383A	MOV (3A)	
42	C830 BR	(30)	Thoát chương trình con

Cũng có thể đặt các tham số trong một vùng tham số đặc biệt với việc chuyển địa chỉ bắt đầu của vùng này vào chương trình con. Chương trình chính có thể sử dụng vùng này cho một số chương trình con, điều này sẽ làm giảm dung lượng bộ nhớ.

2.6. Thực hiện các hướng dẫn máy

Thời gian dành cho việc có được một chương trình máy tính khả thi sẽ giảm đáng kể nếu lập trình viên hiểu rõ về cách thực thi các lệnh của từng máy và sử dụng kiến thức này khi gỡ lỗi chương trình. Thông tin chi tiết về các quá trình xảy ra trong máy tính cơ sở trong quá trình thực hiện các lệnh riêng lẻ được đưa ra trong đoạn 4.2-4.4 và ở đây chúng tôi sẽ chỉ xem xét ngắn gọn các quá trình trong số đó, kiến thức về nó cho phép bạn nhanh chóng khoanh vùng các lỗi trong quá trình thực thi chương trình đang được gỡ lỗi.

Trong quá trình thực hiện các lệnh máy, thiết bị điều khiển máy tính phân tích và chuyển lệnh, các bộ phận riêng lẻ của nó (mã hoạt động, thuộc tính định địa chỉ và địa chỉ) hoặc toán hạng từ một thanh ghi của máy sang một thanh ghi khác, bộ nhớ hoặc thiết bị đầu vào-đầu ra.

Các hành động này (hoạt động vi mô) tiến hành theo một trình tự thời gian nhất định và được phối hợp với nhau. Để tạo ra một chuỗi thời gian (xem Hình 1.4), một bộ tạo xung đồng hồ (các xung có tần số vài trăm hoặc hàng nghìn MHz) được sử dụng.

Chu kỳ lệnh. Để thực hiện một lệnh, cần phải thực hiện một số lệnh vi mô nhất định, mỗi lệnh bắt đầu thực hiện đồng thời một hoặc nhiều lệnh vi mô trong một chu kỳ làm việc của máy tính. Tổng số xung đồng hồ (vi lệnh) cần thiết để thực hiện một lệnh xác định thời gian thực hiện của nó, được gọi là chu kỳ lệnh (Hình 2.7). Một chu kỳ lệnh thường bao gồm một hoặc nhiều chu kỳ máy.

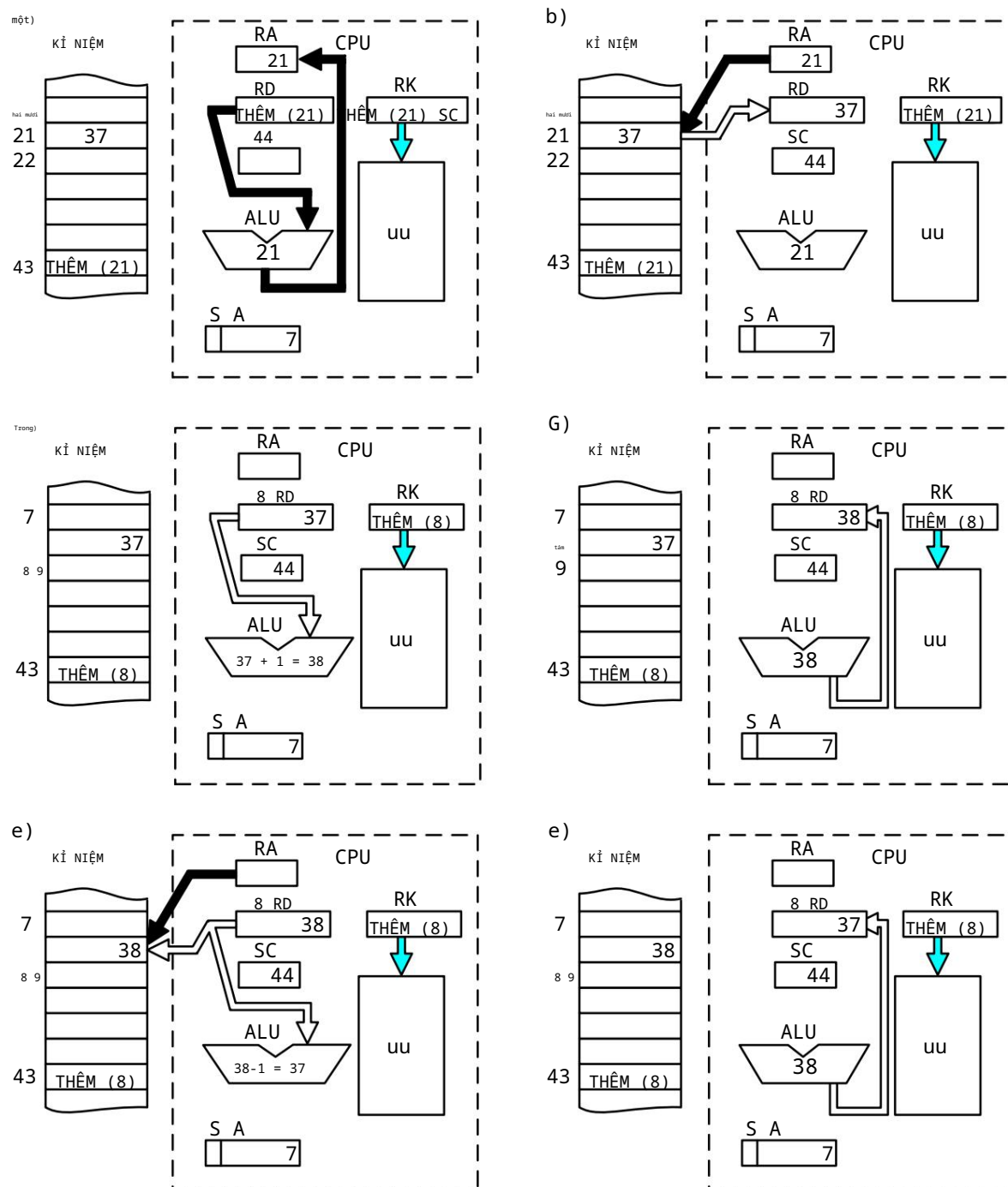
Lựa chọn địa chỉ. Trạng thái này (chu kỳ máy này) tuân theo một lệnh tìm nạp đối với các lệnh được định địa chỉ gián tiếp (bit kiểu địa chỉ được đặt thành một). Trạng thái được sử dụng để đọc địa chỉ toán hạng (kết quả hoặc địa chỉ bước nhảy) từ bộ nhớ và bao gồm các bước sau:

1) phần địa chỉ của lệnh được gửi từ thanh ghi dữ liệu (trong đó một bản sao của lệnh vẫn được lưu) vào thanh ghi địa chỉ (Hình 2.9, a);

2) nội dung của ô nhớ được chỉ ra bởi thanh ghi địa chỉ được đọc vào thanh ghi dữ liệu (Hình 2.9, b); bây giờ thanh ghi này chứa địa chỉ của toán hạng (địa chỉ của kết quả hoặc quá trình chuyển đổi) sẽ được sử dụng khi thực hiện lệnh (trong vòng lặp thực thi).

Để minh họa các hành động được thực hiện trong chu kỳ máy này, lệnh ADD (21) nằm trong ô 43 được chọn. Sau khi lệnh được tìm nạp, bản thân lệnh (4821) được lưu trữ trong thanh ghi dữ liệu và thanh ghi lệnh : thuộc tính địa chỉ (1) 2 và cuối cùng là địa chỉ (000 0010 0001) 2 = (21) 16. Bộ đếm chương trình chứa địa chỉ của lệnh tiếp theo (44), trong bộ tích lũy -

kết quả của các hoạt động trước đó (7) và trong thanh ghi địa chỉ 11 bit - địa chỉ của lệnh đang được thực thi (43). Vào đầu chu kỳ tìm nạp địa chỉ, nội dung của thanh ghi địa chỉ được thay thế bằng 11 chữ số có nghĩa nhỏ nhất của thanh ghi dữ liệu (Hình 2.9, a), tức là địa chỉ của địa chỉ toán hạng và sau đó là địa chỉ của chính toán hạng được đọc vào thanh ghi dữ liệu (Hình 2.9, b).



CƠM. 2.9. Chu kỳ tìm nạp địa chỉ

Nếu một trong các ô chỉ mục (ô có số từ 008 đến 00F) được định địa chỉ gián tiếp, thì chu trình tìm nạp địa chỉ của toán hạng (kết quả) sẽ tiếp tục:

- 3) nội dung của thanh ghi dữ liệu được tăng lên một (Hình 2.9, c, d);
- 4) nội dung đã thay đổi của thanh ghi dữ liệu được gửi đến ô bộ nhớ tại địa chỉ được chỉ ra bởi thanh ghi địa chỉ (Hình 2.9, e);
- 5) nội dung của thanh ghi dữ liệu giảm đi 1 (Hình 2.9, e, f).

Sau thao tác này, giá trị của địa chỉ có trong ô chỉ mục trước bước 3 được khôi phục trong thanh ghi dữ liệu. Nội dung của ô chỉ mục đã tăng lên một và trong lần truy cập tiếp theo, một địa chỉ mới của toán hạng (kết quả) sẽ được chọn.

Để minh họa các bước 3, 4 và 5, lệnh ADD (8), cũng nằm trong ô 43, được chọn. Do đó, hai bước đầu tiên để tìm nạp địa chỉ của toán hạng (kết quả) sẽ rất giống với khi thực hiện lệnh Lệnh ADD (21), được hiển thị trong Hình. 2,9, a, b. Sự khác biệt duy nhất là dữ liệu và thanh ghi lệnh ban đầu được lưu trữ lệnh ADD (8) thay vì ADD (21) và do đó thanh ghi địa chỉ trở đến vị trí 8 (thay vì vị trí 21).

Chấp hành. Chuỗi các hành động được thực hiện trong vòng lặp được xác định bởi loại lệnh địa chỉ đang được thực thi.

Đ. Đối với các lệnh yêu cầu tìm nạp toán hạng từ bộ nhớ máy tính (AND, ADD, ADC, SUB, ISZ), trạng thái thực thi được sử dụng để đọc toán hạng vào thanh ghi dữ liệu và thực hiện thao tác được chỉ ra bởi opcode lệnh.

Ví dụ về vòng lặp thực thi cho lệnh ADD 21 chi tiết được xem xét trong Hình. 2,2, e - 2,2, h. Và mặc dù trong Hình. 2.2 minh họa việc thực hiện một lệnh cộng với địa chỉ trực tiếp, cả hai chu trình (lựa chọn và thực thi) sẽ hoàn toàn giống nhau khi thêm với địa chỉ gián tiếp. Trong trường hợp thứ hai, giữa các chu kỳ này, một chu kỳ tìm nạp địa chỉ của toán hạng sẽ được thực hiện và địa chỉ này, giống như địa chỉ trực tiếp, sẽ được đặt trong thanh ghi dữ liệu.

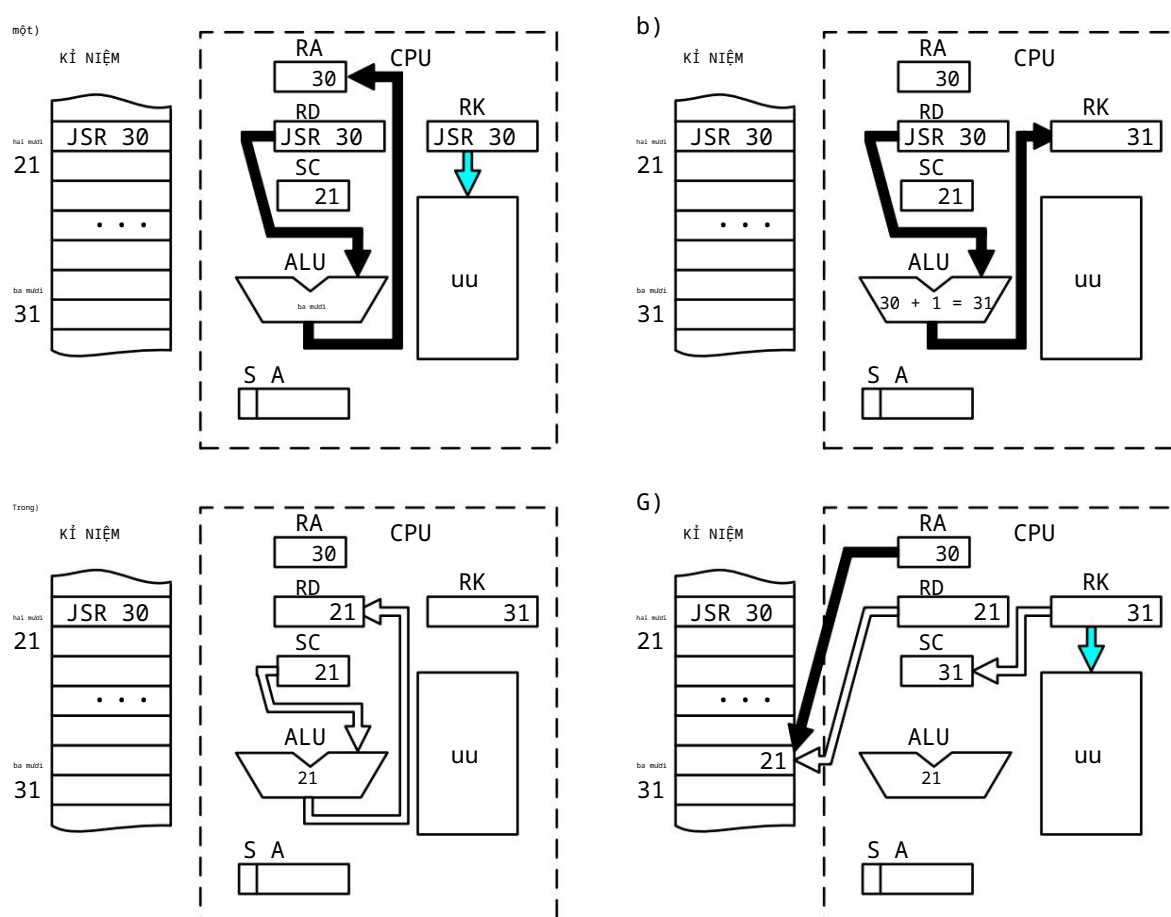
B. Tại lệnh truyền (MOV), trong chu kỳ máy này, nội dung của bộ tích lũy được ghi vào ô nhớ có địa chỉ nằm trong thanh ghi dữ liệu, nội dung của thanh ghi dữ liệu được chuyển đến thanh ghi địa chỉ, và nội dung của bộ tích lũy vào thanh ghi dữ liệu và sau đó đến ô nhớ, được chỉ ra bởi thanh ghi địa chỉ.

B. Khi thực hiện các lệnh nhảy (BCS, BPL, BMI, BEQ), điều kiện tương ứng được kiểm tra (1 trong thanh ghi mang, 0 trong bit dấu của bộ tích lũy, v.v.) và địa chỉ được chuyển từ thanh ghi dữ liệu sang bộ đếm chương trình khi điều kiện này được đáp ứng. Nếu một lệnh rẽ nhánh không điều kiện (BR) được thực hiện, địa chỉ nhánh được chuyển đến bộ đếm chương trình mà không cần kiểm tra.

Do đó, khi điều kiện được chỉ định bởi opcode lệnh được đáp ứng, hoặc khi lệnh BR được thực thi, lệnh tiếp theo sẽ được chọn từ vị trí bộ nhớ có địa chỉ nằm trong thanh ghi dữ liệu (địa chỉ nằm trong lệnh đang được thực thi hoặc được chọn từ vị trí được chỉ đến bởi địa chỉ này). Nếu không, lệnh bên cạnh lệnh đang được thực thi sẽ được chọn.

D. Đối với lệnh JSR, trong chu kỳ máy này, nội dung của bộ đếm chương trình được chuyển đến vị trí bộ nhớ có địa chỉ chứa trong thanh ghi dữ liệu và nội dung của thanh ghi dữ liệu tăng lên một sẽ được nhập vào bộ đếm chương trình.

Các hành động này được minh họa bằng ví dụ thực hiện lệnh JSR 30 (Hình 2.10). Ở đây, như một thanh ghi để lưu trữ tạm thời địa chỉ của lệnh đầu tiên của chương trình con, thanh ghi lệnh được sử dụng, nội dung của lệnh này, sau khi giải mã lệnh, không còn được sử dụng bởi thiết bị điều khiển.



Cơ. 2.10. Vòng lặp thực thi lệnh JSR 30

Đầu tiên, phần địa chỉ của lệnh được chuyển từ thanh ghi dữ liệu sang thanh ghi địa chỉ (Hình 2.10, a). Sau đó, địa chỉ tương tự được tăng thêm một (địa chỉ của lệnh đầu tiên của chương trình con được hình thành) và được gửi đến lưu trữ tạm thời trong thanh ghi lệnh (Hình 2.10, b). Sau đó, nội dung của bộ đếm chương trình (địa chỉ của lệnh được thực hiện sau khi trở về từ chương trình con) được chuyển đến thanh ghi dữ liệu (Hình 2.10, c) và sau đó đến ô nhớ được chỉ ra bởi thanh ghi địa chỉ (Hình 2.10, d). Cuối cùng, địa chỉ của lệnh đầu tiên của chương trình con được ghi lại từ thanh ghi lệnh vào bộ đếm chương trình (Hình 2.10, d).

Do đó, quá trình chuyển đổi sang thực hiện các lệnh của chương trình con được tổ chức (bắt đầu với lệnh trong ô 31) và ghi nhớ (trước lệnh đầu tiên của nó) địa chỉ trả về từ chương trình con. Khi lệnh BR (30) được thực hiện ở cuối chương trình con, địa chỉ 21 sẽ đi vào bộ đếm chương trình, được lưu ở vị trí 30 và máy sẽ tiếp tục thực hiện các lệnh sau lệnh JSR 30.

Như đã chỉ ra ở đầu đoạn này, thời gian gỡ lỗi của các chương trình máy có thể giảm xuống nếu bạn biết cách thực hiện các lệnh máy tính và nếu bạn có khả năng thực thi chương trình đang được gỡ lỗi bằng các lệnh riêng lẻ và đôi khi theo chu kỳ máy. . Cơ hội này được cung cấp cho người dùng máy tính cơ sở. Nó có thể thực thi chương trình không chỉ theo các lệnh và chu trình máy, mà còn theo các chu kỳ riêng lẻ, tức là theo dõi việc thực thi từng lệnh, tìm ra thanh ghi nào mà bộ xử lý sử dụng khi chúng được thực thi, thông tin nào được đặt trong các thanh ghi này và cách thiết bị phản ứng phản ứng điều khiển để thay đổi nội dung của các thanh ghi nhất định.

Quy trình gỡ lỗi có thể được tìm thấy trong đoạn 3.1, mô tả bảng điều khiển gỡ lỗi của máy tính cơ sở.

3. TỔ CHỨC THÔNG TIN ĐẦU VÀO.

TRONG MÁY TÍNH CƠ BẢN

3.1. Bảng điều khiển gỡ lỗi máy tính cơ sở

Một số lượng lớn các thiết bị đầu vào - đầu ra khác nhau hoặc như người ta thường nói, các thiết bị bên ngoài (ED) có thể được kết nối với máy tính. Trong ch. 8 và các đoạn tiếp theo của chương này xem xét việc tổ chức thông tin liên lạc giữa bộ xử lý và VU. Ở đây chúng ta sẽ nói về thiết bị đơn giản nhất trong số các thiết bị này - bảng điều khiển gỡ lỗi. Trên hình. Hình 3.1 và 4.7 cho thấy các bảng điều khiển gỡ lỗi trong cuộc sống thực mà cách đây một thời gian đã được đặt trong phòng thí nghiệm và được kết nối với các mô hình của máy tính cơ sở. Hiện tại, việc nghiên cứu hoạt động của máy tính cơ sở được thực hiện bằng trình giả lập. Các phiên bản của trình giả lập thay đổi định kỳ và mỗi phiên bản hiển thị bảng điều khiển gỡ lỗi trên màn hình theo cách riêng của nó. Do đó, mô tả của bảng điều khiển bên dưới khác với những gì được hiển thị trên màn hình khi trình giả lập đang chạy. Thay vì các chỉ báo với một dòng đèn sáng, các số và số không được hiển thị trên màn hình và bàn phím được sử dụng để nhập dữ liệu thay vì bảng điều khiển các nút và công tắc bật tắt. Tuy nhiên, mô tả của giao diện điều khiển đã được giữ nguyên để mỗi khi một phiên bản mới của trình giả lập xuất hiện, nó không phải viết lại. Và với sự trợ giúp của gợi ý trên màn hình giả lập, bạn có thể tìm hiểu cách thực hiện một thao tác điều khiển từ xa cụ thể trong một phiên bản cụ thể. Bảng điều khiển gỡ lỗi của máy tính cơ sở (Hình 3.1) cho phép bạn nhập chương trình và dữ liệu, làm quen chi tiết với quá trình thực thi chương trình này, xem xét sự tương tác của bộ xử lý với một số thiết bị bên ngoài, v.v.

Bàn điều khiển hiển thị các chỉ báo trạng thái của các thanh ghi chính của máy, 16 ô nhớ (trong số đó là ô được định địa chỉ hiện tại), hai thiết bị điều khiển (bộ điều khiển) của thiết bị đầu vào thông tin và một bộ điều khiển của thiết bị đầu ra thông tin, cũng như các chỉ báo trạng thái của thiết bị điều khiển máy tính cơ sở.

Chỉ báo của mỗi thanh ghi hoặc ô nhớ là một dãy bóng đèn nằm ngang, số của bóng đèn tương ứng với số chữ số trong thanh ghi (ô nhớ). Trạng thái đơn của phóng điện được biểu thị bằng đèn sáng: nếu nội dung của phóng điện bằng 0 thì đèn tương ứng không sáng. Để thuận tiện cho việc đọc nội dung của thanh ghi (ô nhớ), các chữ số của chúng được kết hợp thành các nhóm bằng các tứ phân (tương đương của các chữ số thập lục phân).

Dưới các chỉ báo là một bảng điều khiển với các công tắc bật tắt và các nút dùng để nhập lệnh và dữ liệu, cũng như điều khiển hoạt động của máy. Xem xét cách chúng có thể được sử dụng để làm đầu vào và

kiểm tra chương trình, đảm bảo chương trình thực hiện theo chu kỳ máy, lệnh hoặc ở chế độ tự động, v.v.

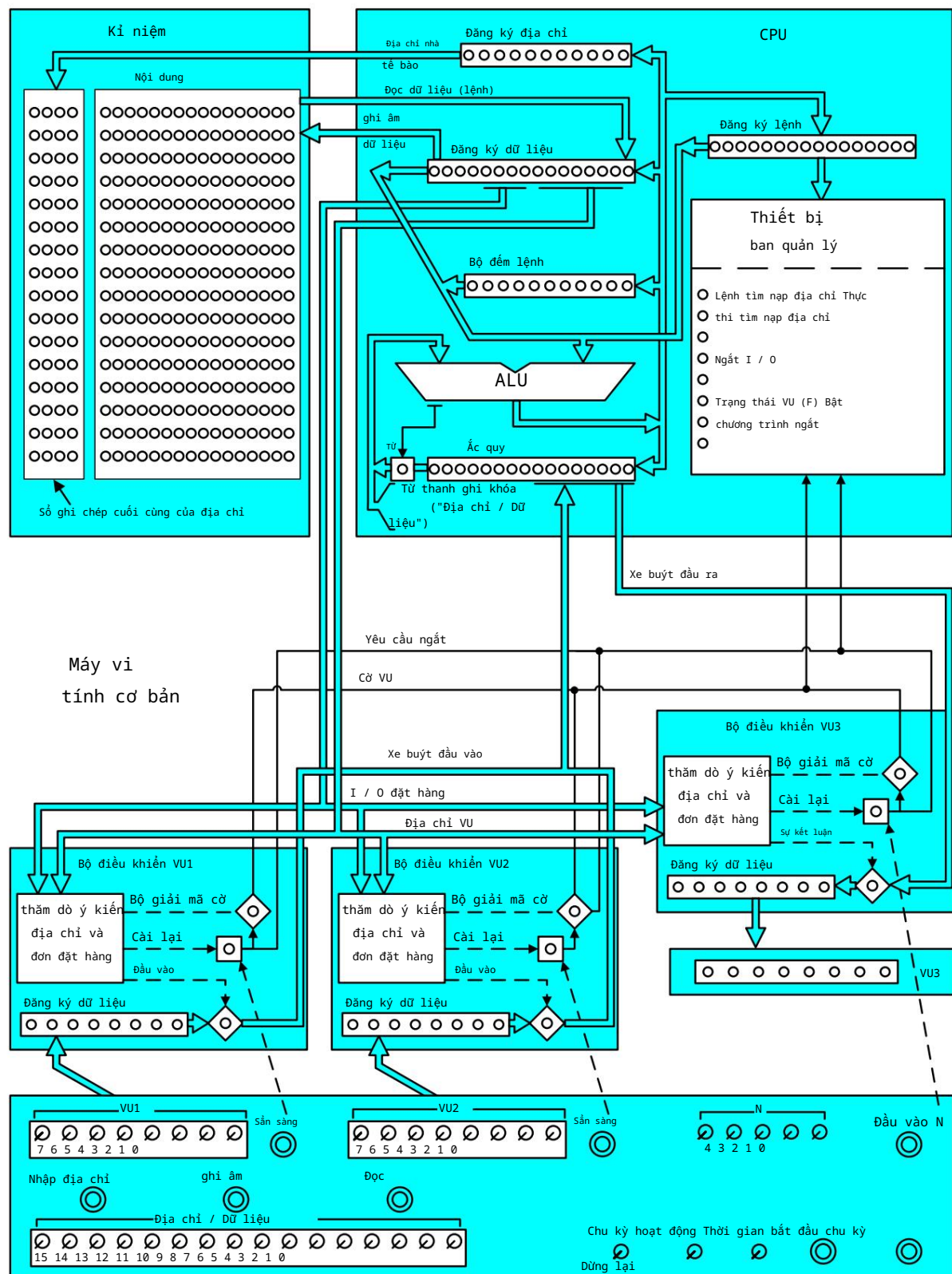
các hoạt động thủ công. Tất cả các thao tác thủ công chỉ được thực hiện khi công tắc chuyển đổi RUN / STOP ở vị trí STOP.

Chương trình và / hoặc đầu vào dữ liệu. Đồng thời, bước tiếp theo:

1) trên thanh ghi bàn phím (ĐỊA CHỈ / DỮ LIỆU) nhập địa chỉ của ô mà bạn muốn nhập lệnh hoặc số (để quay số một, công tắc bật tắt phải được đặt ở vị trí cách xa bạn và để quay số 0 - sang vị trí ngược lại);

2) nhấn nút NHẬP ĐỊA CHỈ (nút này sẽ ghi đè lên nội dung của thanh ghi khóa trong bộ đếm chương trình);

3) gõ mã của lệnh hoặc số trên thanh ghi bàn phím;



Cm. 3.1. Bảng điều khiển gỡ lỗi máy tính cơ sở

4) nhấn nút WRITE (thao tác này sẽ ghi đè nội dung của bộ đếm chương trình vào thanh ghi địa chỉ và nội dung của thanh ghi khóa vào

thanh ghi dữ liệu và sau đó đến vị trí bộ nhớ được chỉ đến bởi thanh ghi địa chỉ).

Vì nội dung của bộ đếm chương trình tự động tăng lên một tại thời điểm ghi, việc ghi lệnh hoặc dữ liệu vào các ô nhớ liên tiếp chỉ yêu cầu thực hiện các đoạn văn. 3 và 4. Để ghi vào các ô nhớ riêng biệt, mỗi lần bạn cần thực hiện cả 4 bước.

Đọc chương trình và / hoặc dữ liệu. Hai bước đầu tiên giống như khi nhập lệnh (dữ liệu). Sau đó, nút ĐỌC được nhấn, dẫn đến việc ghi lại nội dung của ô được chỉ ra bởi thanh ghi địa chỉ vào thanh ghi dữ liệu và để tăng nội dung của bộ đếm chương trình lên từng ô. Do đó, để xem nội dung của ô tiếp theo theo thứ tự, chỉ cần nhấn lại nút ĐỌC.

Chuẩn bị cho việc thực hiện chương trình. Trước khi gửi lệnh thực hiện một chương trình được nhập vào bộ nhớ máy tính, địa chỉ của lệnh thực thi đầu tiên của chương trình này (địa chỉ bắt đầu) phải được nhập vào bộ đếm chương trình. Để thực hiện việc này, công tắc chuyển đổi OPERATION / STOP được chuyển sang vị trí STOP, địa chỉ bắt đầu của chương trình được nhập trên thanh ghi phím và nút ENTER ADDRESS được nhấn.

Sau đó, một hoạt động được thực hiện để đặt chế độ thực thi chương trình được yêu cầu: tự động (lệnh theo lệnh có điểm dừng sau khi thực hiện lệnh HLT hoặc theo lệnh từ bảng điều khiển gỡ lỗi) hoặc từng bước (theo lệnh riêng lẻ, chu kỳ máy hoặc chu kỳ). Chế độ tự động được chuẩn bị bằng cách di chuyển công tắc chuyển đổi RUN / STOP sang vị trí RUN. Để chuẩn bị chế độ bước, bạn phải di chuyển công tắc chuyển đổi này sang vị trí DỪNG và đặt các công tắc chuyển đổi CYCLE và (hoặc) TACT đúng cách. Tắt công tắc bật tắt (cho chính mình) chuẩn bị chế độ hoạt động bằng lệnh, bật công tắc bật tắt CYCLE - theo chu kỳ máy, bật công tắc chuyển đổi TACT - theo chu kỳ, đồng thời bật công tắc chuyển đổi CYCLE và START - theo chu kỳ.

Thực hiện chương trình. Việc thực thi toàn bộ chương trình (chế độ tự động chuẩn bị) có thể được bắt đầu bằng hai cách: bằng cách nhấn nút BẮT ĐẦU hoặc nút TIẾP TỤC.

Khi nhấn nút START, bộ tích lũy, các thanh ghi địa chỉ, dữ liệu, lệnh, chuyển giao và cờ của VU lần đầu tiên được xóa (được đặt thành 0), trạng thái ngăn chặn ngắt, chế độ CHƯƠNG TRÌNH, đầu " Chu kỳ tìm nạp lệnh "và bắt đầu thực hiện tuần tự các lệnh chương trình.

Khi nhấn nút TIẾP TỤC, chế độ CHƯƠNG TRÌNH được đặt và việc thực thi chương trình bắt đầu với lệnh (chu kỳ máy đó và chu kỳ của nó) mà tại đó quá trình thực thi của nó bị gián đoạn. Nếu trước đó các thao tác được thực hiện để nhập địa chỉ bắt đầu

chương trình, sau đó việc thực thi sẽ bắt đầu bằng một lần tìm nạp lệnh nằm tại địa chỉ này, sử dụng nội dung "cũ" (không bằng 0) của các thanh ghi chính của máy tính.

Vì trong tất cả các chương trình được thảo luận trong Bảng. 2.1, 2.7, 2.8 và các ví dụ 2.4-2.6, lệnh bắt đầu (CLA) đặt lại pin, sau đó việc thực thi các chương trình này có thể được bắt đầu với thành công như nhau bằng cả nút BẮT ĐẦU và bằng nút TIẾP TỤC.

Trước khi thực hiện chương trình bằng các lệnh (chu kỳ máy và / hoặc chu kỳ), có thể nhấn nút START. Trong trường hợp này, bộ tích lũy, thanh ghi địa chỉ, dữ liệu, lệnh, chuyển và cờ của VU được đặt lại về 0, trạng thái cấm ngắt được thiết lập và chu kỳ "Tìm nạp lệnh" bắt đầu.

Thực hiện tuần tự các lệnh riêng lẻ hoặc chu kỳ máy (tùy thuộc vào chế độ đã chuẩn bị trước đó) được bắt đầu bằng cách nhấn nút TIẾP TỤC.

Nếu chế độ BẮT ĐẦU được đặt, thì việc nhấn nút TIẾP TỤC dẫn đến việc chỉ thực hiện chu kỳ đầu tiên của quá trình chuyển đổi sang lựa chọn lệnh, sau đó máy tính không phản hồi khi nhấn nút này, nhưng phản ứng với việc nhấn nút ENTER N. Sử dụng sau đó, bạn có thể thực hiện tất cả các lệnh vi mô của lệnh đang được triển khai và khi lệnh cuối cùng được thực thi, máy tính sẽ ngừng phản hồi khi nhấn nút ENTER N. Để thực hiện lệnh chương trình tiếp theo trong chu kỳ, bạn phải nhấn lại nút TIẾP TỤC một lần, rồi nhấn nút NHẬP VÀO N cho đến khi máy tính ngừng phản hồi các lệnh này.

Từ chế độ thực thi chương trình từng bước, bạn có thể chuyển sang chế độ thực thi tự động bất kỳ lúc nào. Để thực hiện việc này, công tắc chuyển đổi VẬN HÀNH / DỪNG được chuyển đến vị trí VẬN HÀNH và nhấn nút TIẾP TỤC.

Việc chuyển đổi từ chế độ tự động sang chế độ từng bước được thực hiện bằng cách chuyển công tắc chuyển đổi RUN / STOP sang vị trí STOP. Trong trường hợp này, máy tính hoàn thành việc thực hiện tất cả các chu kỳ của lệnh hiện tại và dừng lại. Bây giờ, sau khi xem trạng thái của các thiết bị máy tính chính trên các chỉ báo của bảng điều khiển từ xa, bạn có thể tiếp tục thực hiện chương trình ở chế độ tự động (đưa công tắc chuyển đổi WORK / STOP về vị trí WORK và nhấn nút TIẾP TỤC), hoặc bằng các lệnh, chu trình máy hoặc chu trình.

Chương trình gỡ lỗi. Ở trên (xem Phần 2.5), người ta đã nói rằng khả năng từng bước qua chương trình cho phép bạn tăng tốc quá trình gỡ lỗi nó. Để minh họa tuyên bố này, hãy xem xét gỡ lỗi chương trình trong Ví dụ 2.5. Chương trình này được thiết kế để nhân hai số (trong ví dụ, thừa số là 5 và $120 = (78) 16$). Trong bảng. 3.1 chương trình được trình bày dưới dạng biểu tượng (như trong ví dụ 2.5), dạng thập lục phân và dạng nhị phân, hai dạng cuối cùng là

đúng và sai (thay vì lệnh BR 1, lệnh BR 0 được viết và thay vì số -5 - số -3).

Bảng 3.1

Chương trình nhân hai số				
Địa chỉ nhà	Nội dung			
	Dạng ghi nhớ	Dạng thập lục phân	dạng nhị phân	
			Sửa lỗi	
0	CLA	F200	1111 0010 0000 0000	1111 0010 0000 0000
1	THÊM 10	4010	0100 0000 0001 0000	0100 0000 0001 0000
2	ISZ 11	0011	0000 0000 0001 0001	0000 0000 0001 0001
3	BR1	C001	1100 0000 0000 0001	1100 0000 0000 0000
4	HLT	F000	1111 0000 0000 0000	1111 0000 0000 0000
5	.	.	.	.
6	.	.	.	.
7	.	.	.	.
8	.	.	.	.
9	.	.	.	.
10	.	.	.	.
11	.	.	.	.
12	.	.	.	.
13	.	.	.	.
14	.	.	.	.
15	.	.	.	.
16	.	.	.	.
17	.	.	.	.
18	.	.	.	.
19	.	.	.	.
20	.	.	.	.
21	.	.	.	.
22	.	.	.	.
23	.	.	.	.
24	.	.	.	.
25	.	.	.	.
26	.	.	.	.
27	.	.	.	.
28	.	.	.	.
29	.	.	.	.
30	.	.	.	.
31	.	.	.	.
32	.	.	.	.
33	.	.	.	.
34	.	.	.	.
35	.	.	.	.
36	.	.	.	.
37	.	.	.	.
38	.	.	.	.
39	.	.	.	.
40	.	.	.	.
41	.	.	.	.
42	.	.	.	.
43	.	.	.	.
44	.	.	.	.
45	.	.	.	.
46	.	.	.	.
47	.	.	.	.
48	.	.	.	.
49	.	.	.	.
50	.	.	.	.
51	.	.	.	.
52	.	.	.	.
53	.	.	.	.
54	.	.	.	.
55	.	.	.	.
56	.	.	.	.
57	.	.	.	.
58	.	.	.	.
59	.	.	.	.
60	.	.	.	.
61	.	.	.	.
62	.	.	.	.
63	.	.	.	.
64	.	.	.	.
65	.	.	.	.
66	.	.	.	.
67	.	.	.	.
68	.	.	.	.
69	.	.	.	.
70	.	.	.	.
71	.	.	.	.
72	.	.	.	.
73	.	.	.	.
74	.	.	.	.
75	.	.	.	.
76	.	.	.	.
77	.	.	.	.
78	.	.	.	.
79	.	.	.	.
80	.	.	.	.
81	.	.	.	.
82	.	.	.	.
83	.	.	.	.
84	.	.	.	.
85	.	.	.	.
86	.	.	.	.
87	.	.	.	.
88	.	.	.	.
89	.	.	.	.
90	.	.	.	.
91	.	.	.	.
92	.	.	.	.
93	.	.	.	.
94	.	.	.	.
95	.	.	.	.
96	.	.	.	.
97	.	.	.	.
98	.	.	.	.
99	.	.	.	.
100	.	.	.	.
101	.	.	.	.
102	.	.	.	.
103	.	.	.	.
104	.	.	.	.
105	.	.	.	.
106	.	.	.	.
107	.	.	.	.
108	.	.	.	.
109	.	.	.	.
110	.	.	.	.
111	.	.	.	.
112	.	.	.	.
113	.	.	.	.
114	.	.	.	.
115	.	.	.	.
116	.	.	.	.
117	.	.	.	.
118	.	.	.	.
119	.	.	.	.
120	.	.	.	.
121	.	.	.	.
122	.	.	.	.
123	.	.	.	.
124	.	.	.	.
125	.	.	.	.
126	.	.	.	.
127	.	.	.	.
128	.	.	.	.
129	.	.	.	.
130	.	.	.	.
131	.	.	.	.
132	.	.	.	.
133	.	.	.	.
134	.	.	.	.
135	.	.	.	.
136	.	.	.	.
137	.	.	.	.
138	.	.	.	.
139	.	.	.	.
140	.	.	.	.
141	.	.	.	.
142	.	.	.	.
143	.	.	.	.
144	.	.	.	.
145	.	.	.	.
146	.	.	.	.
147	.	.	.	.
148	.	.	.	.
149	.	.	.	.
150	.	.	.	.
151	.	.	.	.
152	.	.	.	.
153	.	.	.	.
154	.	.	.	.
155	.	.	.	.
156	.	.	.	.
157	.	.	.	.
158	.	.	.	.
159	.	.	.	.
160	.	.	.	.
161	.	.	.	.
162	.	.	.	.
163	.	.	.	.
164	.	.	.	.
165	.	.	.	.
166	.	.	.	.
167	.	.	.	.
168	.	.	.	.
169	.	.	.	.
170	.	.	.	.
171	.	.	.	.
172	.	.	.	.
173	.	.	.	.
174	.	.	.	.
175	.	.	.	.
176	.	.	.	.
177	.	.	.	.
178	.	.	.	.
179	.	.	.	.
180	.	.	.	.
181	.	.	.	.
182	.	.	.	.
183	.	.	.	.
184	.	.	.	.
185	.	.	.	.
186	.	.	.	.
187	.	.	.	.
188	.	.	.	.
189	.	.	.	.
190	.	.	.	.
191	.	.	.	.
192	.	.	.	.
193	.	.	.	.
194	.	.	.	.
195	.	.	.	.
196	.	.	.	.
197	.	.	.	.
198	.	.	.	.
199	.	.	.	.
200	.	.	.	.
201	.	.	.	.
202	.	.	.	.
203	.	.	.	.
204	.	.	.	.
205	.	.	.	.
206	.	.	.	.
207	.	.	.	.
208	.	.	.	.
209	.	.	.	.
210	.	.	.	.
211	.	.	.	.
212	.	.	.	.
213	.	.	.	.
214	.	.	.	.
215	.	.	.	.
216	.	.	.	.
217	.	.	.	.
218	.	.	.	.
219	.	.	.	.
220	.	.	.	.
221	.	.	.	.
222	.	.	.	.
223	.	.	.	.
224	.	.	.	.
225	.	.	.	.
226	.	.	.	.
227	.	.	.	.
228	.	.	.	.
229	.	.	.	.
230	.	.	.	.
231	.	.	.	.
232	.	.	.	.
233	.	.	.	.
234	.	.	.	.
235	.	.	.	.
236	.	.	.	.
237	.	.	.	.
238	.	.	.	.
239	.	.	.	.
240	.	.	.	.
241	.	.	.	.
242	.	.	.	.
243	.	.	.	.
244	.	.	.	.
245	.	.	.	.
246	.	.	.	.
247	.	.	.	.
248	.	.	.	.
249	.	.	.	.
250	.	.	.	.
251	.	.	.	.
252	.	.	.	.
253	.	.	.	.
254	.	.	.	.
255	.	.	.	.

Sau khi chương trình được thực thi ở chế độ tự động, bộ tích lũy (nên chứa sản phẩm $120 \times 5 = 600 = (258)_{16}$) chứa số $(0000\ 0000\ 0111\ 1000)_2 = (78)_{16} = (120)_{10}$. Có vẻ như trong ô 11, thay vì số -5, số -1. Tuy nhiên, trong quá trình thực hiện chương trình, các đơn vị được thêm tuần tự vào nội dung của ô này và cuối cùng, nội dung của nó trở thành bằng số không. Do đó, không thể thiết lập nội dung ban đầu của ô 11 được nữa.

Thực hiện lại chương trình (sau khi khôi phục nội dung của ô 11) cho kết quả tương tự và người ta quyết định thực hiện chương trình bằng các lệnh, nhập kết quả thực hiện vào Bảng 3.2.

Trong quá trình thực hiện các lệnh, hóa ra số -3 đã được nhập vào ô 11 thay vì số -5. Tuy nhiên, điều này không thể cho kết quả bằng $(78)_{16}$, vì $(78)_{16} \times 3 \neq (78)_{16}$ tức là. Do đó, chúng tôi tiếp tục tìm kiếm lỗi.

Bảng 3.2

Thực hiện các lệnh của chương trình nhân hai số đã thực								
hiện nội dung của các thanh ghi sau					Bình luận			
đội		thực hiện lệnh						
Mã địa chỉ	SC	RA	RK	RD				
000	F200	0001	0000	F200	F200	0	0000	0 => A
001	4010	0002	0010	4010	0078	0	0078	(78) 16 => A
002	0011	0003	0011	0011	FFFE	0	0078	RD phải là -4, không phải -2, xuất hiện do -3 được nhập vào ô 11
003	C000	0000	0003	C000	C000	0	0078	
000	F200	0001	0000	F200	F200	0	0000	Chuyển đến địa chỉ 0000 Bộ tích lũy bị xóa do chuyển sai đến lệnh CLA

Sau khi kết thúc chu kỳ đầu tiên, hóa ra thay vì chuyển sang tính tổng nội dung của bộ tích lũy và ô 11, bộ tích lũy đã được đặt lại về không. Bây giờ nó trở nên rõ ràng tại sao kết quả bằng (78) 16. Bất kể chương trình chứa bao nhiêu chu kỳ, số tiền nhận được trong mỗi chu kỳ (trừ chu kỳ cuối cùng) sẽ bị ghi đè.

Các câu hỏi có thể nảy sinh: "Có cần thiết phải kiểm tra chương trình bằng cách thực hiện tuần tự các lệnh không? Sẽ tốt hơn nếu chỉ kiểm tra cẩn thận mã của các lệnh này?" Nếu người dùng có tùy chọn mã hóa chính xác, thì việc kiểm tra cẩn thận nội dung của bộ nhớ sẽ giúp tìm ra lỗi. Tuy nhiên, với khối lượng chương trình lớn, khi bạn phải kiểm tra hàng trăm, hàng nghìn chữ số 0 và 1, sự chú ý nhanh chóng trở nên buồn tẻ và các lỗi bị bỏ qua. Ngoài ra, vẫn có những lỗi phát sinh ở giai đoạn lập trình: các mã lệnh trong bộ nhớ máy tính và trên giấy trùng khớp với nhau, nhưng không tương ứng với thuật toán đã cho. Tình huống này chỉ xảy ra khi chương trình được thực thi trên máy tính.

3.2. Thiết bị I / O máy tính cơ sở

Khi phát triển các thiết bị bên ngoài, họ hiếm khi tập trung vào kết nối của chúng với một loại máy tính nhất định. Sự phát triển rất nhanh của công nghệ máy tính và do đó, sự đổi mới thường xuyên của nhóm máy tính trong trường hợp này sẽ dẫn đến nhu cầu thay đổi các mạch WU kết nối nó với một máy tính cụ thể, tức là thay đổi quy trình sản xuất của WU. Chi phí sản xuất VU phù hợp để sử dụng với các loại máy tính khác nhau (ngay cả với những máy tính chưa được phát triển) sẽ giảm đáng kể nếu khi phát triển VU, chúng tôi chỉ tập trung vào việc thực hiện tốt nhất các chức năng được chỉ định (in ấn, âm thanh tái sản xuất, v.v.). Đồng thời, có thể sử dụng các thiết bị kiểm soát đặc biệt - bộ điều khiển (tiếng Anh là controller - manager, auditor) để giao tiếp với máy tính. Sau đó, để kết nối VU với một loại máy tính mới, chỉ cần phát triển và sản xuất một bộ điều khiển mới.

Máy tính cơ bản, được tạo ra để dạy các nguyên tắc hoạt động của máy tính, sử dụng VU đơn giản nhất: hai thiết bị đầu vào (VU-1 và VU-2) và một thiết bị đầu ra (VU-3) được đặt trên bảng điều khiển gỡ lỗi (Hình 3.1). Thiết bị đầu vào là 8-một thanh ghi công tắc bật tắt bit và một nút (SN SÀNG) để phát tín hiệu rằng thông tin được nhập bằng công tắc bật tắt có thể được đọc vào máy tính. Thiết bị đầu ra được đại diện bởi một thanh ghi 8 bit, trạng thái của thanh ghi này được hiển thị trên bảng điều khiển gỡ lỗi và một nút (INPUT N) để phát tín hiệu rằng có thể nhận được thông tin trong thanh ghi này.

Các bộ điều khiển đơn giản nhất được cài đặt giữa VU và bộ xử lý, mỗi bộ điều khiển chứa:

- thanh ghi dữ liệu (để trao đổi dữ liệu giữa VU và bộ xử lý);
- bộ giải mã địa chỉ (cho phép bạn chọn một cuộc gọi tới VU này trong số tất cả các cuộc gọi tới các thiết bị I / O được kết nối với bộ xử lý);
- bộ giải mã lệnh (lệnh giải mã từ bộ xử lý để thực hiện các hoạt động nhất định);
- thanh ghi trạng thái (nơi lưu trữ thông tin về sự sẵn sàng của VU để trao đổi dữ liệu với bộ xử lý).

Trong các bộ điều khiển của các VU đơn giản nhất, các thanh ghi trạng thái một bit thường được sử dụng, chúng thường được gọi là cờ hoặc cờ. Tên này cũng áp dụng cho các bộ điều khiển của máy tính cơ sở.

Các bộ điều khiển được kết nối với bộ xử lý bằng các bus đầu vào và đầu ra thông tin, bus địa chỉ và bus điều khiển, qua đó các lệnh được truyền từ bộ xử lý đến các bộ điều khiển VU và thông tin về trạng thái của VU tới bộ xử lý.

3.3. Truyền thông tin được kiểm soát bằng phần mềm

Các thiết bị I / O truyền đến máy tính và nhận từ nó, như một quy luật, một lượng lớn thông tin không thể chỉ được đặt trong các thanh ghi của bộ xử lý. Do đó, thông tin được chuyển từ VU sang bộ nhớ máy tính và đi vào VU từ bộ nhớ của nó. Trong trường hợp này, việc trao đổi có thể chịu sự kiểm soát của một chương trình máy tính thông qua các thanh ghi của bộ xử lý (truyền dữ liệu được điều khiển bằng chương trình) hoặc dưới sự kiểm soát của một thiết bị bên ngoài đặc biệt (bộ điều khiển truy cập bộ nhớ trực tiếp), bỏ qua bộ xử lý (truyền dữ liệu với quyền truy cập bộ nhớ trực tiếp). Trao đổi được điều khiển theo chương trình được thực hiện theo từng phần nhỏ, với khả năng truy cập bộ nhớ trực tiếp, thông tin được truyền theo khối lớn.

Trong máy tính cơ sở, chỉ điều khiển bằng phần mềm truyền dữ liệu.

1. Hoạt động đầu ra cung cấp cho việc chuyển thông tin từ bộ tích lũy bộ xử lý đến thanh ghi dữ liệu của bộ điều khiển VU. Sau khi dữ liệu đã được nhận bởi thanh ghi này, các hoạt động được bắt đầu để chuyển đổi chúng trong VU được liên kết với bộ điều khiển (ví dụ: in ký tự tương ứng hoặc thay đổi vị trí của cơ quan điều hành). Trong máy tính cơ sở, trạng thái của chỉ báo của thanh ghi đầu ra VU-3 chỉ thay đổi.

2. Thao tác nhập bao gồm việc chuyển dữ liệu từ thanh ghi dữ liệu của bộ điều khiển VU sang pin máy tính. Trong máy tính cơ sở, thiết bị đầu vào được mô phỏng bằng cách sử dụng một bộ công tắc bật tắt, trạng thái của công tắc này được đọc vào thanh ghi dữ liệu khi một lệnh đầu vào được thực hiện.

Khi sử dụng bộ trao đổi được điều khiển bằng chương trình, một chương trình phải được biên dịch để chuyển dữ liệu từ bộ nhớ máy tính sang pin và sau đó đến thanh ghi dữ liệu của bộ điều khiển VU (hoặc từ thanh ghi dữ liệu của bộ điều khiển VU đến bộ tích lũy và sau đó đến bộ nhớ máy tính).

Trong chương trình này, một trong ba kiểu trao đổi như vậy có thể được thực hiện: đồng bộ, không đồng bộ và ngắt.

Trao đổi đồng bộ chỉ có thể được sử dụng để giao tiếp với các CL như vậy mà thời gian thực hiện của hoạt động được biết chính xác (ví dụ: thời gian tối đa có thể dành cho việc in một ký tự, v.v.). Trong trường hợp này, chương trình trao đổi phải được thiết kế sao cho các lệnh cho trao đổi đi trong khoảng thời gian không ít hơn thời gian thực hiện một thao tác trao đổi.

Đây là loại trao đổi đơn giản nhất, yêu cầu chi phí phần cứng và phần mềm tối thiểu. Tuy nhiên, khi nó được sử dụng để truyền dữ liệu giữa các máy tính và VU tương đối chậm, theo quy luật, không thể tải đầy đủ máy tính trong thời gian giữa hai lần truyền dữ liệu. Vì vậy, ví dụ, một máy đánh chữ điện cho phép bạn tổ chức đầu ra dữ liệu với tốc độ không quá 7 ký tự mỗi giây và trong 1/7 giây, một máy tính siêu nhỏ có thể thực hiện từ 10.000 đến 1.000.000 hoạt động.

Trao đổi không đồng bộ được sử dụng rộng rãi trong vi máy tính. Khi nó được thực hiện, khoảng thời gian giữa các lệnh truyền dữ liệu được thiết lập bởi chính thiết bị bên ngoài. Bộ điều khiển của các thiết bị này được cung cấp một thanh ghi trạng thái thông báo cho máy tính về sự sẵn sàng trao đổi thông tin của thiết bị (ví dụ: máy đánh chữ điện chỉ đặt tín hiệu sẵn sàng khi nó đã in xong ký tự trước đó, giấy và ruy băng mực. được nạp vào nó). Chương trình giao lưu được xây dựng như thế này.

1. Trạng thái của thiết bị I / O được kiểm tra.
2. Nếu thiết bị đã sẵn sàng để trao đổi, hãy chuyển sang bước 3. Nếu không, bước 1 được thực hiện lại.
3. Phần tiếp theo của dữ liệu được chuyển, các hành động để xử lý chúng và (hoặc) các hoạt động khác được thực hiện và nếu việc trao đổi dữ liệu vẫn chưa hoàn thành, thì quá trình chuyển đổi sang bước 1 sẽ được thực hiện.

Như vậy, với trao đổi không đồng bộ, người lập trình không cần biết về thời gian thực hiện của hoạt động trên VU. Bản thân nó thông báo cho máy tính về việc hoàn thành thao tác trước đó và sự sẵn sàng để thực hiện một thao tác mới. Tuy nhiên, máy tính phải dành thời gian chờ đợi thời điểm sẵn sàng, và vì độ sẵn sàng được kiểm tra bằng phần mềm (lệnh TSF B), máy tính không thể thực hiện bất kỳ công việc chuyển đổi dữ liệu nào khác tại thời điểm này.

Trao đổi ngắt chương trình khác với trao đổi không đồng bộ ở chỗ mức độ sẵn sàng của VU đối với trao đổi được kiểm tra bằng cách sử dụng

phần cứng máy tính, không phải phần mềm. Điều này cho phép bạn tổ chức công việc của máy tính như sau.

Máy thực thi một số chương trình (chúng ta hãy gọi nó là chương trình chính hoặc nền) không liên quan đến việc trao đổi. Khi VU đã sẵn sàng để nhận hoặc đưa ra thông tin, nó sẽ gửi một tín hiệu sẵn sàng đến máy tính, tín hiệu này được nhận biết bởi một bộ phận máy tính đặc biệt (ví dụ, một bộ điều khiển ngắt). Khối này đình chỉ (ngắt) việc thực hiện chương trình chính và chuyển quyền điều khiển cho chương trình con tổ chức kiểu trao đổi dữ liệu mong muốn. Khi thực hiện xong chương trình con, hoạt động của máy tính lại tiếp tục theo chương trình chính (tạm thời bị gián đoạn).

Sự khác biệt giữa các loại trao đổi được kiểm soát bởi chương trình có thể được minh họa bằng ví dụ sau.

Ví dụ 3.1. Bạn được hướng dẫn để đun sôi nước trong một ấm đun nước.

1. Hoạt động không điều kiện (trao đổi đồng bộ). Lúc 8 giờ bạn đặt một ấm nước lên bếp, biết rằng nó thường sôi trong 10 phút, đúng 8 giờ 10 phút bạn tắt bếp mà không cần quan tâm nước đã sôi hay chưa.

2. Hoạt động có điều kiện (trao đổi không đồng bộ). Bạn bắc một ấm nước lên bếp và nhìn khoảng một phút một lần. Khi ấm nước sôi, bạn lấy nó ra khỏi bếp.

3. Hoạt động gián đoạn. Bạn đặt một ấm nước lên bếp, đặt một cái còi trên vòi và bắt đầu đọc một cuốn sách (tác phẩm nền). Khi tiếng còi thổi, bạn sẽ lấy ấm ra khỏi bếp và tiếp tục đọc sách.

Trong đoạn 3.4 và 3.5, các loại trao đổi phổ biến nhất được coi là: không đồng bộ và gián đoạn chương trình.

Các lệnh I / O của máy tính cơ sở có cùng định dạng (xem Hình 2.2, c): trường 4 bit của mã hoạt động, trường 4 bit của mã lệnh và trường 8 bit của lựa chọn thiết bị I / O mã số.

Mã hoạt động (1110) 2 được xử lý trong bộ xử lý máy tính và phục vụ để phân biệt các lệnh I / O với các lệnh máy khác.

Mã đơn đặt hàng được sử dụng để gửi đơn đặt hàng đến bộ điều khiển WU để thực hiện một hành động. Vì bốn bit được dành riêng cho mã này, máy tính có thể gửi 16 lệnh khác nhau đến thiết bị bên ngoài. Việc giải mã lệnh được thực hiện bởi bộ điều khiển VU.

Ba thứ tự chính được thiết lập trong máy tính cơ sở: truyền dữ liệu (IN B - đầu vào và OUT B - đầu ra), kiểm tra mức độ sẵn sàng của máy phụ (TSF B) và đặt lại trạng thái sẵn sàng (CLF B), trong đó B là địa chỉ của nô lệ.

Mã lựa chọn thiết bị (địa chỉ VU) được sử dụng để tổ chức giao tiếp giữa máy tính và một trong các thiết bị bên ngoài được kết nối với nó. Mã này được truyền đồng thời tới tất cả VU (có thể có $2^8 = 256$ trong số chúng) và được phân tích trong bộ điều khiển của chúng. Khi mã sau nhận ra mã thuộc VU đã cho, nó sẽ kết nối VU với máy tính. Tất cả các thiết bị khác sẽ không được kết nối với máy tính và sẽ không đáp ứng lệnh của nó.

Các lệnh I / O (xem Bảng 2.4) sử dụng từ cờ. Đây thường được gọi là _____ thanh ghi trạng thái một bit của VU,

được đặt ở một trạng thái duy nhất khi thiết bị sẵn sàng trao đổi thông tin. Khi cờ RT bị xóa (được đặt thành 0), RT bận: thiết bị đầu ra vẫn đang xử lý lệnh trước đó và thiết bị đầu vào chưa hoàn tất việc chuẩn bị dữ liệu.

Lệnh CLF B (E0xx), trong đó xx là hai chữ số thập lục phân của mã mẫu VU) được sử dụng để đặt lại VU với mã mẫu B. Lệnh CLF 02 đặt lại (đặt thành 0) cờ trong bộ điều khiển VU với mã mẫu 02.

Bằng lệnh TSF B (E1xx), nội dung của cờ VU với mã mẫu B được đặt trong VU ĐĂNG KÝ NHÀ NƯỚC (Φ trong thiết bị điều khiển trong Hình 3.1). Sau đó, bộ xử lý thêm nội dung của thanh ghi Φ với nội dung của bộ đếm chương trình. Do đó:

khi $\Phi = 0$ (vùng nhớ B chưa sẵn sàng để trao đổi), lệnh chương trình theo sau TSF B được thực thi;

khi $F = 1$ (thiết bị B sẵn sàng trao đổi), lệnh theo sau TSF B bị bỏ qua và lệnh nằm sau TSF B được thực thi.

Lệnh IN B (E2xx) được sử dụng để gửi nội dung của thanh ghi dữ liệu bộ điều khiển VU với mã mẫu B đến tám bit quan trọng nhất của bộ tích lũy.

Lệnh OUT B (E3xx) được sử dụng để chuyển nội dung của tám bit quan trọng nhất của bộ tích lũy sang thanh ghi dữ liệu thiết bị với mã mẫu B.

Khi, trong quá trình vận hành máy tính, thiết bị điều khiển phát hiện thấy lệnh có mã hoạt động (1110) 2 nằm trong thanh ghi lệnh và đồng thời trong thanh ghi dữ liệu, quá trình chuyển đổi được thực hiện để Chế độ I / O và nội dung của 12 chữ số có nghĩa nhỏ nhất của thanh ghi dữ liệu được truyền tới tất cả các bộ điều khiển VU (xem Hình 3.1). Bộ giải mã địa chỉ của tất cả các bộ điều khiển VU giải mã nội dung này, nhưng một trong số chúng hoạt động, mã (địa chỉ) của nó khớp với địa chỉ được đặt trong lệnh.

Bộ giải mã này mở các cổng (xem đoạn 1.2) để chuyển thông tin đến bộ giải mã lệnh, bộ giải mã này sẽ giải mã nội dung của các bit thứ 8-11 của thanh ghi dữ liệu và ra lệnh thực hiện một trong các lệnh trên: mở cổng để truyền trạng thái của cờ (TSF) với máy tính, đặt lại cờ (CLF) hoặc mở các cổng kết nối bộ tích lũy với thanh ghi dữ liệu bộ điều khiển (IN hoặc OUT).

Để chuyển trạng thái cờ của tất cả VU sang BỘ ĐIỀU KHIỂN TRUYỀN CẢM HỨNG, dòng "Yêu cầu ngắt" được sử dụng và để chuyển trạng thái cờ của VU đã được thăm dò sang ĐĂNG KÝ TRẠNG THÁI (F) - dòng trạng thái cờ. Dữ liệu thường được truyền qua một bus dữ liệu duy nhất, trong máy tính cơ sở (để đơn giản hóa mô tả) được biểu diễn bằng hai bus: một bus đầu vào và một bus đầu ra.

Trao đổi do chương trình kiểm soát được thực hiện theo từng phần nhỏ - thường là byte. Một đơn vị thông tin 8 bit như vậy cho phép bạn mã hóa tối đa 256 (28) ký tự khác nhau được truyền giữa máy tính và VU: số, chữ cái trong bảng chữ cái Latinh và tiếng Nga, dấu hiệu của hành động toán học, v.v. (xem Phụ lục B). Do đó, các bus đầu vào và đầu ra của máy tính cơ sở liên kết các thanh ghi dữ liệu 8 bit của bộ điều khiển VU với 8 bit ít quan trọng nhất của bộ tích lũy.

Vì từ của máy tính cơ sở có 16 bit, nên trong một từ như vậy có thể đặt mã của hai ký tự để lưu trữ. Về vấn đề này, nên soạn các chương trình để tổ chức trao đổi giữa bộ nhớ máy tính và các VU khác nhau như sau.

1. Khi nhập, ký tự đầu tiên nằm trong các chữ số thấp hơn của bộ tích lũy được chuyển sang các chữ số cao hơn của nó. Sau đó, ký tự tiếp theo được nhận trong các bit ít quan trọng nhất của bộ tích lũy, và chỉ khi đó nội dung của bộ tích lũy (mã gồm hai ký tự) mới được chuyển vào bộ nhớ.

2. Khi xuất ra bộ tích lũy, một từ chứa hai ký tự đầu ra sẽ được chuyển từ bộ nhớ. Sau đó, biểu tượng được hiển thị trên VU, mã của nó nằm trong các bit bậc thấp của bộ tích lũy, và sau khi dịch chuyển - nhân vật thứ hai.

3.4. Trao đổi không đồng bộ

Với truyền dữ liệu không đồng bộ, trước tiên bạn phải đảm bảo rằng VU đã sẵn sàng cho việc trao đổi (được bật và đặt về trạng thái ban đầu hoặc đã hoàn thành thao tác trước đó) và chỉ sau đó đưa ra lệnh cho việc trao đổi (đầu vào hoặc đầu ra).

VU báo cáo sự sẵn sàng cho việc trao đổi bằng cách đặt cờ trong bộ điều khiển VU thành một trạng thái duy nhất. Để tìm hiểu trạng thái của cờ này, bạn nên thực hiện lệnh "Cờ thăm dò" (TSF), sau đó nội dung cờ của thiết bị có địa chỉ được chỉ định trong lệnh thăm dò sẽ được chuyển qua dòng "Trạng thái cờ" vào thanh ghi trạng thái của VU (xem Hình 3.1). Tùy thuộc vào nội dung này (0 hoặc 1), lệnh tiếp theo của chương trình hoặc lệnh nằm sau lệnh tiếp theo sẽ được thực hiện tương ứng (lệnh tiếp theo sẽ bị bỏ qua).

Ví dụ 3.2. Sử dụng VU-1, ghi lại các mã ký tự của từ "Có" trong ô 006.

Chương trình cho nhiệm vụ này được đưa ra trong Bảng 3.3. Khi làm việc trên một chương trình như vậy, máy tính sẽ đợi nút READY được nhấn (mô phỏng sự xuất hiện của sự sẵn sàng trao đổi thông tin của VU). Do đó, trước lần nhấn đầu tiên của nút này, cần phải quay mã ký hiệu D trên công tắc bật tắt VU-1, sau đó nhấn nút và tiến hành quay mã ký hiệu A. Trong quá trình thiết lập này, máy tính được bạn đặt lại cờ sẵn sàng của VU, gửi mã của ký hiệu D đến các chữ số thấp hơn của pin và chuyển mã này lên các bit cao để sẵn sàng nhận mã của ký tự tiếp theo.

Vì tốc độ của máy tính gấp nhiều lần tốc độ của con người nên vào thời điểm hoàn thành mã ký hiệu A, máy sẽ không chỉ hoàn thành quá trình xử lý ký hiệu D mà còn kiểm tra nội dung của Cờ VU-1 hàng nghìn lần. Về vấn đề này, sau khi quay mã, bạn có thể nhấn ngay nút READY, tức là đưa ra lệnh gửi mã này đến pin máy tính. Bây giờ bộ tích lũy chứa mã của từ "Có" và nội dung của nó có thể được gửi đến ô được chỉ định.

Ở đây cần nhắc lại rằng trạng thái cờ của bộ điều khiển kết nối với máy tính được gửi đến bộ xử lý qua hai dòng: "Yêu cầu ngắt" và "Trạng thái cờ" (xem Hình 3.1). Dòng "Yêu cầu ngắt" được kết nối trực tiếp với tất cả các cờ của bộ điều khiển VU và dòng "Trạng thái cờ" được kết nối thông qua các cổng chỉ mở trên các lệnh "Cờ thăm dò" (TSF). Trong trao đổi không đồng bộ, dòng trạng thái cờ được sử dụng. Do đó, việc nhấn bất kỳ nút SN SÀNG nào khác, ngoại trừ nút đề cập đến thiết bị có địa chỉ được chỉ định trong lệnh TSF, sẽ không dẫn đến việc thực hiện các hành động cho đầu vào (hoặc đầu ra) thông tin, mặc dù tín hiệu sẵn sàng sẽ xuất hiện trên dòng "Yêu cầu ngắt".

Bảng 3.3

Chương trình nhập mã gồm hai ký tự			
Địa chỉ nhà	Mã nội dung		Bình luận
	ghi nhớ		
5	FFF8		Hàng số -8, được sử dụng để chuyển ô để viết "CÓ"
6	0000		
	.		
hai mươi	E101	TSF 1	Thăm dò cờ bộ điều khiển VU-1 và lặp lại thao tác này nếu VU chưa sẵn sàng để trao đổi (cờ = 0)
21	C020	BR20	
22	E201	TRONG 1	Hành động này chỉ được thực hiện sau khi nhấn nút SN SÀNG cho VU-1, tức là khi thực thi TSF 1 tìm thấy cờ = 1 và bỏ qua BR
			20. Bằng lệnh IN 1, nội dung của thanh ghi dữ liệu của bộ điều khiển VU-1 được chuyển đến tám bit quan trọng nhất của bộ tích lũy
23	E001	CLF 1	Đặt lại mức độ sẵn sàng của VU-1 để ngăn việc đọc mã từ VU-1 cho đến khi mã ký tự A được đặt trên đó và điều này không được báo cáo bằng cách nhấn nút SN SÀNG
24	F600	ROL	Mã của ký tự đầu tiên được nhập (D) được chuyển sang trái tám chữ số, nhường chỗ cho ký tự tiếp theo được nhập.
25	0005		Bỏ phiếu cho cờ của bộ điều khiển VU-1 ... (xem chú thích cho các lệnh 20 và 21)
26	C024		Nhập mã ký tự được đặt trên công tắc bật tắt VU-1 (nếu tín hiệu sẵn sàng VU-1 được đưa ra)
27	E101		Đặt lại mức độ sẵn sàng cho VU-1
28	C027		Chuyển tiếp "CÓ" đến ô 006
29	E201		Tắt máy tính
2A	E001		
2B	3006		
2C	F000	HLT	

3.5. Trao đổi ngắt chương trình

Truyền dữ liệu có ngắt chương trình đặc biệt thuận tiện khi làm việc với các VU tốc độ thấp (máy đánh chữ, máy đục lỗ, máy viễn thông, v.v.), cũng như trong các tình huống mà thời điểm truyền dữ liệu sang máy tính là không thể đoán trước, chẳng hạn như khi làm việc với giao tiếp kênh truyền hình. Tính năng đặc trưng chính của quá trình truyền đang được xem xét là việc trao đổi dữ liệu giữa máy tính và WU được bắt đầu bởi một tín hiệu từ WU. Để thực hiện loại trao đổi này, cần phải thay thế chu kỳ chờ sẵn sàng phần mềm của VU trong quá trình truyền không đồng bộ (TSF, BR) bằng kiểm tra phần cứng để biết sự hiện diện của ngắt bên ngoài, tức là tín hiệu sẵn sàng dọc theo "Yêu cầu ngắt" (xem Hình 3.1).

Trong trường hợp này, máy tính có thể thực thi bất kỳ chương trình nào (chúng ta sẽ gọi nó là chương trình chính), và khi nhận được yêu cầu ngắt từ VU (yêu cầu truyền hoặc nhận dữ liệu), nó có thể tạm dừng (ngắt) việc thực hiện điều này. chương trình trong suốt thời gian trao đổi dữ liệu. Tất cả các hành động

về việc tạm ngừng chương trình và quá trình chuyển đổi sang chương trình con trao đổi được thực hiện trong máy tính cơ sở sử dụng bộ điều khiển ngắt, là một phần của thiết bị điều khiển (xem Hình 3.1).

Bằng các lệnh "Cho phép ngắt" (E1) hoặc "Tắt ngắt" (D1), bộ điều khiển ngắt đi vào một trong hai trạng thái trong đó nó tương ứng đáp ứng hoặc không đáp ứng các tín hiệu sẵn sàng của VU được truyền qua "Yêu cầu ngắt" hàng. Nếu bộ điều khiển được đặt ở trạng thái cho phép ngắt, thì trình tự các bước để thực hiện truyền dữ liệu được đề cập thường trông như thế này.

Bước 1. Sau khi hoàn thành việc thực hiện lệnh hiện tại của chương trình chính, điều khiển được chuyển đến bộ điều khiển ngắt. Nếu tại thời điểm này không có tín hiệu trên trạng thái đơn của cờ của bất kỳ VU trên dòng "Yêu cầu ngắt", thì quá trình tìm nạp và thực hiện lệnh tiếp theo của chương trình chính sẽ bắt đầu và bước này được lặp lại. Nếu có yêu cầu ngắt từ VU, bước thứ hai được thực hiện.

Bước 2. Bộ điều khiển ngắt đi vào trạng thái vô hiệu hóa ngắt, nội dung của bộ đếm chương trình (địa chỉ của lệnh tiếp theo của chương trình chính sẽ được thực thi trong trường hợp không có ngắt) được nhập vào ô nhớ có địa chỉ 0, và điều khiển được chuyển đến lệnh nằm trong ô 1.

Hai hành động cuối cùng bắt buộc việc thực hiện lệnh "Tham chiếu đến chương trình con" (JSR 0), như nó đã được chèn vào giữa lệnh vừa thực hiện và các lệnh tiếp theo của chương trình chính (xem Phần 2.4). Một chương trình con như vậy, văn bản phải được viết bởi chính người dùng, thường được gọi là chương trình con để xử lý hoặc sửa chữa các ngắt. Nó luôn bắt đầu từ ô có địa chỉ 1 (ô 0 lưu địa chỉ trả về từ chương trình con) và có thể có ký tự rất khác tùy thuộc vào tập VU được kết nối với máy tính và nguyên tắc xây dựng chương trình chính. Tuy nhiên, thông thường nhất, chương trình con này yêu cầu sử dụng bộ tích lũy, và vì vậy một trong những điều đầu tiên cần làm là ghi nhớ nội dung của bộ tích lũy và thanh ghi mang. Sau đó, nguồn của ngắt được tìm ra (bằng cách thăm dò các cờ, VU được xác định, cờ của nó được đặt thành một trạng thái duy nhất) và dữ liệu được chuyển. Chương trình con kết thúc bằng việc khôi phục nội dung của bộ tích lũy và thanh ghi chuyển, cho phép ngắt và thoát chương trình con tới lệnh có địa chỉ được lưu trong ô 0. Các hàm này xác định nội dung của các bước sau.

Bước 3. Nội dung của bộ tích lũy và thanh ghi chuyển được lưu trữ trong bộ nhớ. Điều này yêu cầu ít nhất ba hướng dẫn: chuyển nội dung của bộ tích lũy sang một ô đệm chuyên dụng (MOV B1), dịch chuyển theo chu kỳ (thường là sang trái) để

để nội dung của thanh ghi chuyển giao rơi vào bộ tích lũy (ROL), và ghi nội dung này vào một ô đệm khác (MOV B2). Do đó, thông tin tối thiểu cần thiết về tác vụ bị gián đoạn được cung cấp - ô 0 lưu trữ địa chỉ của nhiệm vụ tiếp tục và các ô đệm lưu trữ trạng thái của hai thanh ghi chính khác của máy tại thời điểm bị gián đoạn.

Bước 4. Nguồn của ngắt được tìm kiếm. Để làm điều này, trong bất kỳ trình tự nào (thích hợp nhất), các cờ của tất cả các VU được thăm dò (lệnh TSF). Khi phát hiện một VU có cờ đặt (cờ = 1), một quá trình chuyển đổi được thực hiện cho phần đó của chương trình con, phần này mô tả các hành động trao đổi thông tin giữa bộ xử lý và thiết bị bên ngoài này.

Bước 5: Dữ liệu được chuyển và cấu hình trước xử lý (nếu cần).

Bước 6. Nội dung của sổ đăng ký mang và bộ tích lũy được khôi phục. Điều này yêu cầu tối thiểu năm hướng dẫn: xóa bộ tích lũy (CLA), gọi nội dung của vị trí bộ đệm B2 đến bộ tích lũy đã xóa (ADD B2), xoay sang phải để khôi phục nội dung của thanh ghi mang (ROR), xóa bộ tích lũy (CLA), và gọi nội dung của vị trí đệm B1 thành pin đã được làm sạch (ADD B1).

Bước 7. Bộ điều khiển ngắt một lần nữa được chuyển sang trạng thái cho phép, và chương trình bị ngắt (chính) được quay trở lại để thực thi. Để thực hiện điều này, các lệnh EI ("Cho phép ngắt") và BR (0) được thực hiện tuần tự - quá trình chuyển đổi sang lệnh có địa chỉ được lưu trong ô 0 (xem đoạn 2.4). Ở đây cần lưu ý rằng lệnh EI nhất thiết phải được đặt ngay trước lệnh thoát khỏi chương trình con - BR (0). Điều này được thực hiện trong trường hợp có một yêu cầu ngắt khác trong quá trình xử lý yêu cầu đang được xem xét. Nếu ít nhất một lệnh được đặt giữa các lệnh EI và BR (0), thì sau khi thực thi, quá trình chuyển đổi sang xử lý ngắt có thể xảy ra, nội dung của ô 0 sẽ được thay thế bằng một địa chỉ trả về mới và do đó, đường dẫn trở lại chương trình bị gián đoạn (chính) sẽ bị hủy.

Để kết luận, chúng tôi nhớ lại những điều sau đây.

1. Sử dụng lệnh D1, bộ điều khiển ngắt được đặt ở trạng thái vô hiệu hóa ngắt và máy tính không phản hồi yêu cầu trao đổi từ bất kỳ thiết bị bên ngoài nào. Bộ điều khiển này cũng vào trạng thái tương tự sau khi nhấn nút START, do đó, để đảm bảo trao đổi về việc ngắt chương trình, bộ điều khiển sau nhất thiết phải chứa lệnh EI ("Bật ngắt").

2. Chương trình ngắt xảy ra khi có các điều kiện sau: bộ điều khiển ngắt ở trạng thái cho phép ngắt, yêu cầu ngắt đã xuất hiện (cờ của một số VU

được đặt thành một trạng thái duy nhất) và việc thực thi bất kỳ lệnh hiện tại nào đã kết thúc, ngoại trừ lệnh D1 ("Đã tắt ngắt") và lệnh E1. Nếu một lệnh E1 đang được thực hiện tại thời điểm yêu cầu ngắt, thì trạng thái ngắt sẽ xảy ra sau khi thực hiện lệnh E1 sau.

lệnh (xem bước 7).

3. Tùy thuộc vào các điều kiện cụ thể, quy trình phục vụ ngắt có thể ở bất kỳ dạng nào, nhưng nó phải luôn bắt đầu bằng lệnh nằm trong ô có địa chỉ 1 và kết thúc bằng lệnh E1 và BR (0) theo sau nhau. Lệnh E1 được sử dụng để khôi phục trạng thái cho phép ngắt, được tự động thay thế bằng trạng thái ngắt ngắt ở lối vào chương trình con, để trong quá trình xử lý ngắt hiện tại, máy tính không phản hồi các ngắt tiếp theo.

Ví dụ 3.3. Soạn một chương trình định kỳ (với khoảng thời gian ba chu kỳ lệnh) làm tăng nội dung của bộ tích lũy lên một. Trên các công tắc bật tắt của anh ta, nên được đặt trong ô có địa chỉ 25.

Chương trình chính để giải quyết một vấn đề như vậy là khá đơn giản. Nó có thể bắt đầu bằng cách thiết lập kích hoạt ngắt (E1) và xóa bộ tích lũy (CLA). Sau đó, một chu kỳ phải được tổ chức để tăng trên mỗi đơn vị nội dung của bộ tích lũy. Vòng lặp này chỉ có thể chứa hai lệnh: "Tăng dần bộ tích lũy" (INC) và một lệnh rẽ nhánh vô điều kiện (BR) để quay lại INC. Tuy nhiên, theo sự phân công, một chu trình như vậy phải bao gồm ba lệnh, và do đó giữa INC và BR bạn có thể đặt lệnh NOP ("Không hoạt động"). Một chương trình tương tự được hiển thị trong Bảng 3.4.

Nếu các lệnh của chương trình này được nhập vào bộ nhớ của máy tính, địa chỉ bắt đầu 0020 được đặt trong bộ đếm lệnh và nhấn nút BẮT ĐẦU, thì một chu kỳ tăng vô hạn của pin sẽ bắt đầu được thực hiện. Trong chu kỳ này, các lệnh sẽ được lặp lại mọi lúc: INC, NOP, BR 22, INC, NOP, BR 22, INC, v.v. Khi bất kỳ nút nào trong ba nút (READY, READY hoặc ENTER N), thì điều khiển sẽ được chuyển đến lệnh nằm trong ô 1 và ô 0 sẽ được ghi vào địa chỉ của một trong các lệnh trên sẽ được thực hiện trong trường hợp không có yêu cầu ngắt. Thao tác này sẽ chuyển đến quy trình ngắt, quy trình này sẽ được đặt bắt đầu từ vị trí 1.

Bảng 3.4

Chương trình chính để giải quyết vấn đề của ví dụ 3.3			
Địa chỉ nhà	Mã nội dung		Bình luận
	ghi nhớ		
hai mươi	FA00	E1	Đặt trạng thái cho phép ngắt Về sinh pin
21	F200	CLA	
22	F200	INC	Chu kỳ để tăng nội dung của bộ tích lũy
23	F100	NOP	
24	C022	BR22.	
25	0000		Ô để lưu trữ mã đến từ VII-1

Trong bảng. 3.5 cho thấy một trong những biến thể có thể có của một quy trình như vậy, được xây dựng theo sơ đồ tiêu chuẩn: lưu trữ nội dung của các thanh ghi chính (bước 3), tìm nguồn của ngắt (bước 4), truyền dữ liệu (bước 5), khôi phục nội dung của các thanh ghi chính (bước 6) và cuối cùng là tổ chức việc thoát khỏi chương trình con (bước 7). Chương trình con cung cấp phản hồi không chỉ cho các yêu cầu từ VU-1 và VU-3, mà còn cho một yêu cầu từ VU-2, có thể phát sinh từ việc vô tình nhấn nút SN SÀNG gần "thiết bị" này. Trong trường hợp sau, cờ CL-2 bị xóa, tương đương với việc hủy yêu cầu không lường trước.

Bảng 3.5

Biến thể đầu tiên của quy trình ngắt, ví dụ 3.3

Mã nội dung			Bình luận
Địa chỉ nhà	ghi nhớ		
0	0000		Một ô để lưu trữ địa chỉ trả hàng (địa chỉ này sẽ được nhập ở đây trong bước thứ 2)
01	C030	BR 30	Lệnh đầu tiên của chương trình con là sự chuyển đổi sang văn bản chính của nó, nằm trong các ô 30-4D
02	0000		
03	0000		
04	0000		
05	0000		
06	0000		
07	0000		
08	0000		
09	0000		
10	0000		
11	0000		
12	0000		
13	0000		
14	0000		
15	0000		
16	0000		
17	0000		
18	0000		
19	0000		
20	0000		
21	0000		
22	0000		
23	0000		
24	0000		
25	0000		
26	0000		
27	0000		
28	0000		
29	0000		
30	0000		
31	F600	ROL	Lưu trong các ô đệm 4C và 4D nội dung của bộ tích lũy và thanh ghi mang
32	304D	MOV 4D	
33	E101	TSF 1	Cờ thăm dò VU-1. Nếu nó được đặt lại, thì hãy chuyển đến bỏ phiếu cho cờ VU-2. Nếu không, hãy chuyển sang mục nhập dữ liệu từ Bước VU-1
34	C036	BR 36	
35	C039	BR 39	
36	E103	TSF 3	Cờ thăm dò VU-3. Nếu nó được đặt lại, thì chuyển tiếp 4 để bỏ phiếu cho cờ VU-2. Nếu không, hãy chuyển sang đầu vào dữ liệu từ VU-3 Đầu vào dữ liệu từ VU-1,
37	C043	DR43	
38	C03E	BR3E	
39	F200	CLA	chuyển chúng đến ô 25, đặt lại cờ VU-1, chuyển sang khôi phục nội dung của các thanh ghi chính và thoát chương trình con
3A	E201	TRONG 1	
3B	E001	CLF 1	
3C	3025	MOV 25	
3D	C044	BR44	
3E	F200	CLA	Chuyển nội dung của ô đệm 4C sang bộ tích lũy, xuất ra tám bit bậc thấp của bộ tích lũy thành VU-3, đặt lại cờ VU-3, chuyển sang khôi phục thanh ghi và thoát
3F	404C	ADD4C	
40	E303	HẾT 3	
41	E003	CLF 3	
42	C044	BR44	
43	E002	CLF2	Xóa cờ VU-2
44	F200	CLA	Khôi phục nội dung của thanh ghi mang và bộ tích lũy
45	404D	THÊM 4D	
46	E700	ROR	
47	F200	CLA	
48	F400	CMA	
49	104C	VÀ 4C	
4A	FA00	E1	Tiếp tục lại bước trạng thái đã bật ngắt và thoát chương trình con ô để lưu bộ tích lũy và thực hiện nội dung
4B	C800	BR (0)	
4C	0000		đăng ký
4D	0000		

Vì các vị trí chỉ mục 008-00F thường được sử dụng trong các chương trình tuần hoàn, nên không thích hợp để sử dụng chúng với các lệnh từ một quy trình ngắt. Do đó, trong ví dụ 3.3, văn bản chính của chương trình con như vậy được đặt bắt đầu từ ô 30 và quá trình chuyển đổi sang nó được thực hiện bằng lệnh BR 30 nằm trong ô 001.

Số lượng các lệnh thường xuyên ngắt có thể được giảm đáng kể, dựa trên các chi tiết cụ thể của vấn đề đang được giải quyết.

1. Nội dung của bộ tích lũy được tích lũy trong chương trình chính chỉ có thể bị hỏng (thay thế bằng giá trị khác) trong quá trình thực hiện thao tác nhập dữ liệu từ VU-1.

2. Đối với bất kỳ tương tác nào (cần thiết để giải quyết vấn đề) với VU-1, VU-2 hoặc VU-3, nội dung của sổ đăng ký chuyển nhượng không bị ảnh hưởng.

3. Nội dung tích lũy trong chương trình chính sẽ được hiển thị trên VU-3 pin và nó không thể bị hỏng trong quá trình thu hồi này.

Một biến thể của quy trình xử lý ngắt, được biên dịch có tính đến các tính năng được liệt kê, được đưa ra trong Bảng. 3.6. Nó ngắn hơn lệnh đầu tiên 10 lệnh (tiết kiệm hơn 30%).

Bảng 3.6

Quy trình ngắt biến thể thứ hai cho ví dụ 3.3 Nội dung Mã dễ nhớ

Quy trình ngắt điện theo thứ tự cho ví dụ 3.3 Nội dung Mã đề nhỏ			
Địa chỉ nhà			Bình luận
0	0000		Địa chỉ trả về từ chương trình con
	0030	BR 30	Chuyển đến thực thi chương trình con
	E101	TSF 1	
31	C03C	BR3C	Bỏ phiếu bằng cờ VU-1 và chuyển sang bỏ phiếu bằng cờ VU-3 nếu cờ VU-1 bị xóa
32	303B	MOV 3B	Tiết kiệm nội dung của pin trong một ô 3V
33	F200	CLA	
34	E201	TRONG 1	
35	E001	CLF 1	Nhập dữ liệu từ VU-1 và gửi chúng đến một địa chỉ nhất định (ô 25)
36	3025	MOV 25	
37	F200	CLA	
38	403B	THÊM 3 tỷ	Khôi phục nội dung của pin
39	FA00	E1	Sự hồi phục
3A	C800	BR (0)	Những trạng thái ngắt kích hoạt và thoát chương trình con
3B	0000		Bộ đệm để lưu trữ nội dung của bộ tích lũy
3C	E103	TSF 3	
3D	C041	BR41	Thăm dò cờ VU-3 và tiến hành đặt lại cờ VU-2 nếu cờ VU-3 được đặt lại
3E	E303	HẾT 3	
3F	E003	CLF 3	
40	C039	BR 39	Xuất ra VU-3 trong số tám chữ số có nghĩa nhỏ nhất của bộ tích lũy và chuyển sang các lệnh cuối cùng của chương trình con
41	E002	CLF2	
42	C039	BR 39	Đặt lại cờ VU-2 và chuyển sang các lệnh cuối cùng của chương trình con

4. MÁY TÍNH CƠ SỞ THIẾT BỊ ĐIỀU KHIỂN PHẦN MỀM

4.1. Máy tính đa cấp và mức vi chương trình của chúng

Hiện tại, hầu hết người dùng máy tính bắt đầu giao tiếp với máy này bằng một trong các ngôn ngữ thuật toán (SI, PASCAL, BASIC, v.v.), tin rằng ngôn ngữ đó là ngôn ngữ của máy. Rất có thể những cỗ máy như vậy sẽ sớm được thiết kế, nhưng hiện nay việc triển khai phần cứng của các máy tính với ngôn ngữ máy thuật toán là rất phức tạp và tốn kém. Do đó, các ngôn ngữ máy của hầu hết các máy tính hiện đại đều khá thô sơ (tương tự như ngôn ngữ của máy tính cơ sở), điều này làm cho việc sử dụng trực tiếp các ngôn ngữ đó trở nên bất tiện và khó khăn. Khả năng thực thi một chương trình được viết bằng ngôn ngữ thuật toán trên máy tính được cung cấp với sự trợ giúp của các chương trình hệ thống đặc biệt (trình biên dịch hoặc trình thông dịch) dịch các chương trình của người dùng sang ngôn ngữ máy của máy tính.

Có hai cách để tổ chức quá trình dịch chương trình nguồn từ thuật toán sang ngôn ngữ máy và quá trình

chấp hành.

Phương pháp đầu tiên, được gọi là biên dịch, bao gồm thực tế là quá trình thực thi thuật toán chỉ được thực hiện sau khi hoàn thành quá trình dịch của chương trình nguồn. Đồng thời, trong quá trình thực hiện chương trình máy kết quả, không chỉ chương trình gốc không còn cần thiết nữa mà còn có chương trình biên dịch đã dịch nó thành biểu diễn máy.

Phương pháp thứ hai, được gọi là thông dịch, là mỗi câu lệnh của chương trình nguồn được thay thế bằng chương trình thông dịch bằng một chuỗi lệnh máy tương đương. Trình tự này được thực hiện ngay lập tức, sau đó câu lệnh tiếp theo của chương trình nguồn được dịch, các lệnh nhận được trong quá trình dịch của nó được thực hiện, ... Do đó, trong khi giải quyết vấn đề, máy cần cả chương trình nguồn và chương trình thông dịch.

Chi phí tạo một trình biên dịch (thông dịch viên) và thời gian dành cho quá trình dịch một chương trình phần lớn được xác định bởi sự giống nhau của ngôn ngữ biên dịch và ngôn ngữ nhận. Do đó, các ngôn ngữ thuật toán phổ biến và các ngôn ngữ phức tạp hơn của hệ thống phần mềm không được dịch ngay lập tức sang ngôn ngữ của lệnh máy. Có một hệ thống phân cấp nhất định của các ngôn ngữ lập trình, trong đó mỗi ngôn ngữ phức tạp hơn dựa trên ngôn ngữ trước đó.

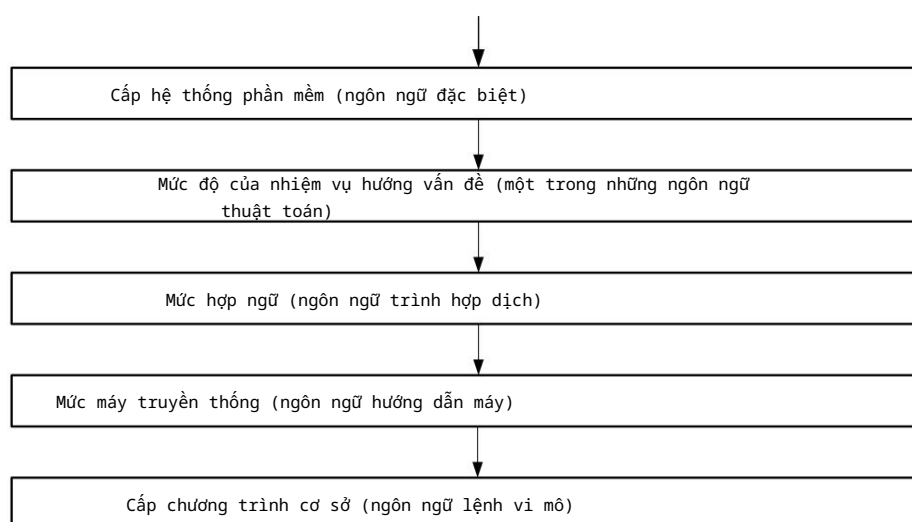
Chúng ta đã gặp một trong những ngôn ngữ đơn giản nhất, rất gần với các lệnh máy tính và thường được dùng như một ngôn ngữ trung gian.

giữa chúng và các ngôn ngữ thuật toán. Nó là một ngôn ngữ mã hóa hướng dẫn tượng trưng, thường được gọi là hợp ngữ. Hợp ngữ (được phát triển cho từng loại máy tính) là phương tiện đầu tiên để tự động hóa lập trình trong công nghệ máy tính. Chúng cho phép sử dụng tên và nhãn (địa chỉ) tượng trưng. Trình biên dịch từ các ngôn ngữ như vậy được gọi là trình biên dịch. Chúng cấp phát các vị trí bộ nhớ nhất định cho các biến tượng trưng, tổ chức liên kết giữa các phần khác nhau của chương trình, giúp đơn giản hóa việc lập trình so với lập trình ở cấp độ lệnh.

Khả năng thực hiện trên bất kỳ chương trình máy tính nào được viết bằng ngôn ngữ khác với ngôn ngữ máy của nó một lần nữa nhấn mạnh tính phổ quát của máy tính. Giờ đây, bất kỳ người dùng máy tính đủ điều kiện nào cũng có thể tạo ngôn ngữ của riêng mình để mô tả bất kỳ tác vụ cụ thể nào (mô phỏng máy tính, điều khiển một công cụ hoặc quá trình thử nghiệm, học chơi cờ vua, v.v.) và viết cho ngôn ngữ này, chẳng hạn như thông dịch viên trong ngôn ngữ SI hoặc PASCAL

Một người làm việc với máy tính bằng ngôn ngữ này hay ngôn ngữ khác (hoặc học chơi cờ vua) có thể không biết liệu chương trình của mình có được thực thi từng bước bởi một trình thông dịch hay không, chương trình này được thực thi bởi một thông dịch viên khác hay nó được thực hiện trực tiếp bằng phần cứng (của ông Thường chỉ quan tâm đến đầu ra của chương trình. Thông thường, đối với anh ta dường như ngôn ngữ mà anh ta giao tiếp với máy tính là ngôn ngữ máy của nó. Do đó, những người dùng khác nhau của cùng một máy tính có thể đang làm việc trên các máy tính khác nhau. Do đó xuất hiện các khái niệm: máy tính ảo (biểu kiến) và máy tính đa cấp.

Máy tính đa cấp là một máy tính có khả năng làm việc với n cấp độ ngôn ngữ lập trình khác nhau (Hình 4.1).

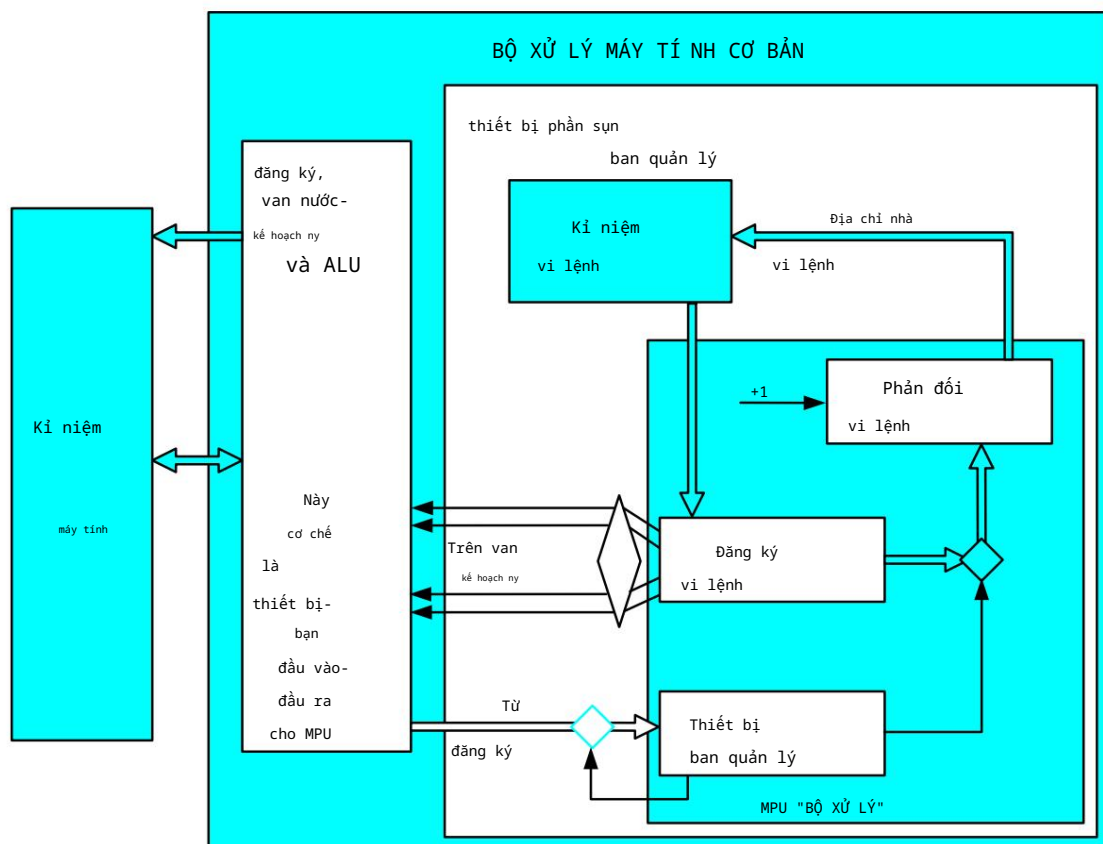


Hình 4.1. Máy tính đa cấp

Ngôn ngữ thấp hơn, hoặc cấp độ, là ngôn ngữ đơn giản nhất, ngôn ngữ trên khó khăn nhất. Một máy như vậy có thể được coi là n máy ảo khác nhau, mỗi máy có ngôn ngữ máy riêng. Độ phức tạp của việc triển khai phần cứng của các máy ảo này tăng lên khi độ phức tạp của ngôn ngữ (tăng số cấp).

Nếu chúng ta xem xét máy tính cơ sở từ các vị trí này, nó chỉ ra rằng ngôn ngữ của các lệnh được mô tả trong Ch. 2 không phải là ngôn ngữ cấp thấp nhất. Ở cấp độ thấp hơn của nó, các hành động cơ bản (hoạt động vi mô) được thực hiện trên các từ thông tin. Một số hành động này đã được xem xét khi mô tả việc thực hiện các lệnh trong đoạn 2.1 và 2.5 (chuyển nội dung của một thanh ghi này sang thanh ghi khác, kiểm tra một bit trong thanh ghi, v.v.).

Thứ tự của các hoạt động vi mô được điều khiển bởi bộ phận điều khiển của máy tính cơ sở, đến lượt nó, là một máy tính rất đơn giản. Đối với máy tính này, các thanh ghi và mạch cổng của máy tính cơ sở đóng vai trò là thiết bị đầu vào và đầu ra (Hình 4.2). Chương trình hoạt động của một máy tính như vậy - thiết bị điều khiển vi chương trình (MCD) - được gọi là vi chương trình và các lệnh của nó chứa thông tin về các hành động cơ bản sẽ được thực hiện trong một chu kỳ làm việc của máy tính được gọi là vi lệnh.



Hình 4.2. Cấu trúc của thiết bị điều khiển phần sụn và kết nối của nó với các thiết bị khác của máy tính cơ sở

Vi chương trình được lưu trữ trong một thiết bị lưu trữ vĩnh viễn - bộ nhớ vi lệnh .. (MP). Trong mỗi chu kỳ của máy tính từ MP, vi lệnh tiếp theo được gửi đến thanh ghi vi lệnh, tức là vi lệnh được bộ đếm vi lệnh trở tới (nó cũng là thanh ghi địa chỉ vi lệnh). Sau đó, bộ đếm vi lệnh được tăng lên một.

Nếu một lệnh gọi là vi hoạt động được chọn từ bộ nhớ, các bit trong đó xác định tập hợp các vi hoạt động mong muốn, thì trạng thái của các bit này được truyền tới các mạch cổng của bộ xử lý. ALU được cấu hình phù hợp (cho phép tính tổng, phép nhân logic, phép nghịch đảo, v.v.) và nội dung của một số thanh ghi được chuyển qua nó đến các thanh ghi khác (xem Hình 2.2, 2.8, 2.9 hoặc 2.10).

Khi một lệnh vi điều khiển được chọn, nội dung của thanh ghi máy tính được chỉ ra trong lệnh vi điều khiển được gửi đến thiết bị điều khiển MPU, bit được chỉ ra trong lệnh vi điều khiển được trích xuất từ nó và so sánh với một số bit nhất định của lệnh vi mô (với 0 hoặc 1). Nếu kết quả của phép so sánh là dương, thì bộ đếm vi lệnh sẽ được gửi từ lệnh vi mô đến địa chỉ mà lệnh vi mô tiếp theo của chương trình vi mô sẽ được chọn. Nếu không, không có quá trình chuyển giao nào được thực hiện và trong chu kỳ tiếp theo, vi lệnh nằm sau lệnh được thực hiện sẽ được thực hiện.

Cần rõ ràng máy tính có thể lập trình vi mô được mô tả ở đây có hai bộ nhớ, hai bộ lệnh và hai chương trình: cấp máy truyền thống (các lệnh trong Bảng 2.4) và cấp vi chương trình. Tuy nhiên, bộ xử lý thực thi phần cứng chỉ là một và kiến trúc của máy tạo thành lớp phần sụn.

Cấp độ máy truyền thống là một trong những cấp độ máy đó, bạn có thể tìm thấy mô tả về cấp độ này trong sách hướng dẫn sử dụng máy tính. Trong trường hợp này, máy tính cơ sở sẽ được quảng cáo là có dung lượng bộ nhớ 2048 từ 16 bit và tập lệnh gồm 28 lệnh. Trong số các lệnh này: phép nhân logic, cộng, trừ, nhảy có điều kiện và không điều kiện, lệnh gọi chương trình con, v.v. Nó có hai kiểu định địa chỉ (trực tiếp và gián tiếp), có thể kết nối tối đa 256 thiết bị bên ngoài với nó, v.v. Trong hướng dẫn sử dụng, bạn có thể tìm thấy đề cập rằng máy có thể lập trình vi mô, nhưng các mô tả về mức phần sụn, bộ nhớ của các vi lệnh và bản thân các vi lệnh thường không được cung cấp ở đó.

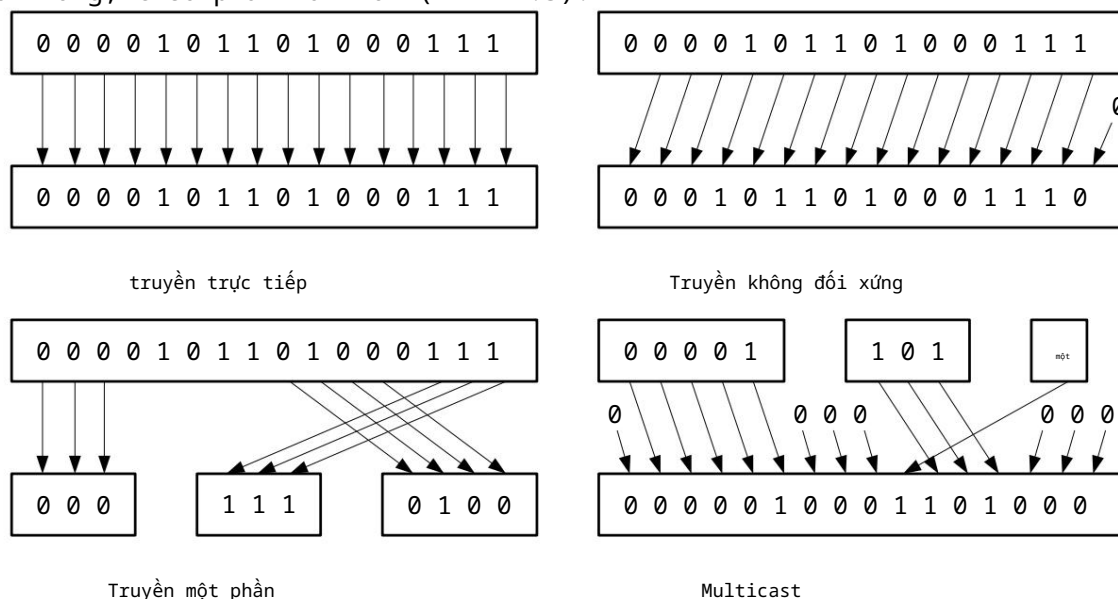
Người dùng muốn lập trình ở cấp độ máy truyền thống (và thậm chí hơn thế bằng các ngôn ngữ cấp cao) cuối cùng không quan tâm đến cấp độ truyền thống có được thực hiện bằng cách tạo ra các mạch điện đặc biệt hay được sử dụng cho mục đích này hay không.

chương trình cơ sở chạy ở cấp độ máy thấp hơn. Tuy nhiên, liên quan đến sự xuất hiện của một số bộ vi xử lý có một ngôn ngữ duy nhất - ngôn ngữ của các lệnh vi mô, việc người dùng làm quen với lập trình ở cấp độ lệnh vi xử lý trở nên cần thiết.

4.2. Các thành phần bộ xử lý và các hoạt động cơ bản

Phần 1.2 và 2.1 đã bao gồm hầu hết các thành phần của bộ xử lý: thanh ghi, bus, mạch cổng và đồng hồ. Ở đây chúng ta sẽ xem xét kỹ hơn các phương pháp truyền dữ liệu giữa các thanh ghi, hoạt động của ALU và một số mạch phụ trợ.

Truyền dữ liệu giữa các thanh ghi. Đây là hoạt động cấp phần sụn cơ bản. Bạn có thể chỉ định bốn kiểu truyền dữ liệu như vậy: trực tiếp, không đối xứng, theo phần và nhóm (Hình 4.3).



Cơ. 4.3. Truyền dữ liệu giữa các thanh ghi

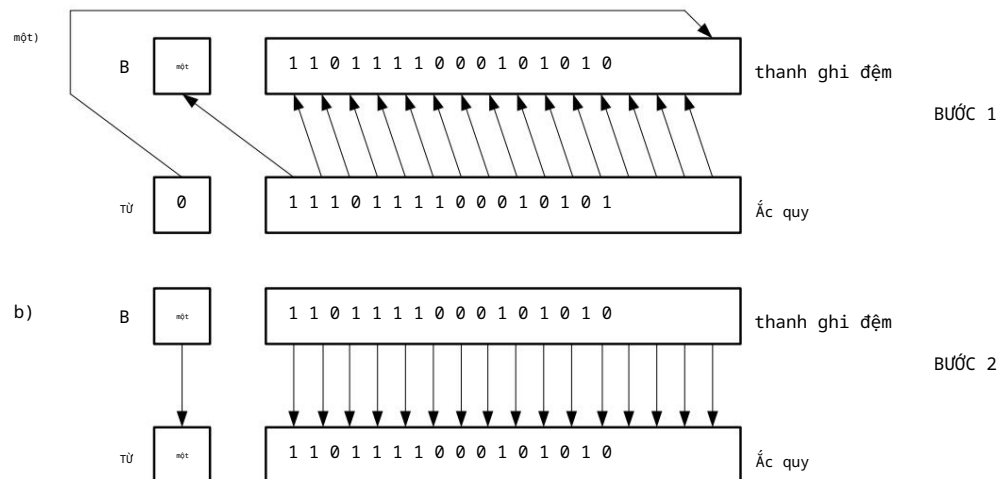
Trong trường hợp truyền trực tiếp, nội dung của thanh ghi N-bit được sao chép vào thanh ghi N-bit sao cho bit thứ i của bản sao tương ứng với thứ i bit của bản gốc. Trong truyền không đối xứng, bit thứ i của bản sao tương ứng với bit thứ $(i + K)$ của bản gốc, trong đó K là số nguyên dương hoặc âm. Trong trường hợp này, một hoặc nhiều bit của thanh ghi đích có thể không có các bit tương ứng từ thanh ghi nguồn. Các bit như vậy được lấp đầy bởi các số không.

Khi được truyền theo từng phần, nội dung của một thanh ghi được sao chép (hóa ra là "phân tán") trong một số thanh ghi và thứ tự của các bit có thể được thay đổi. Trong trường hợp phát đa hướng, các bản sao nội dung của một số thanh ghi được ghi vào các phần khác nhau của thanh ghi nhận, thứ tự của các bit cũng có thể

đã thay đổi. Các bit không có nguồn thông tin tương ứng được điền bằng các số không.

Truyền dữ liệu giữa các thanh ghi thay đổi nội dung của thanh ghi đích, nhưng giữ nguyên nội dung của thanh ghi nguồn không thay đổi. Để truyền dẫn, các bus có chiều rộng khác nhau được sử dụng với các mạch cổng được tích hợp sẵn bên trong chúng (Hình 1.3).

Trên hình. 4.4 cho thấy hoạt động của bộ dịch chuyển.



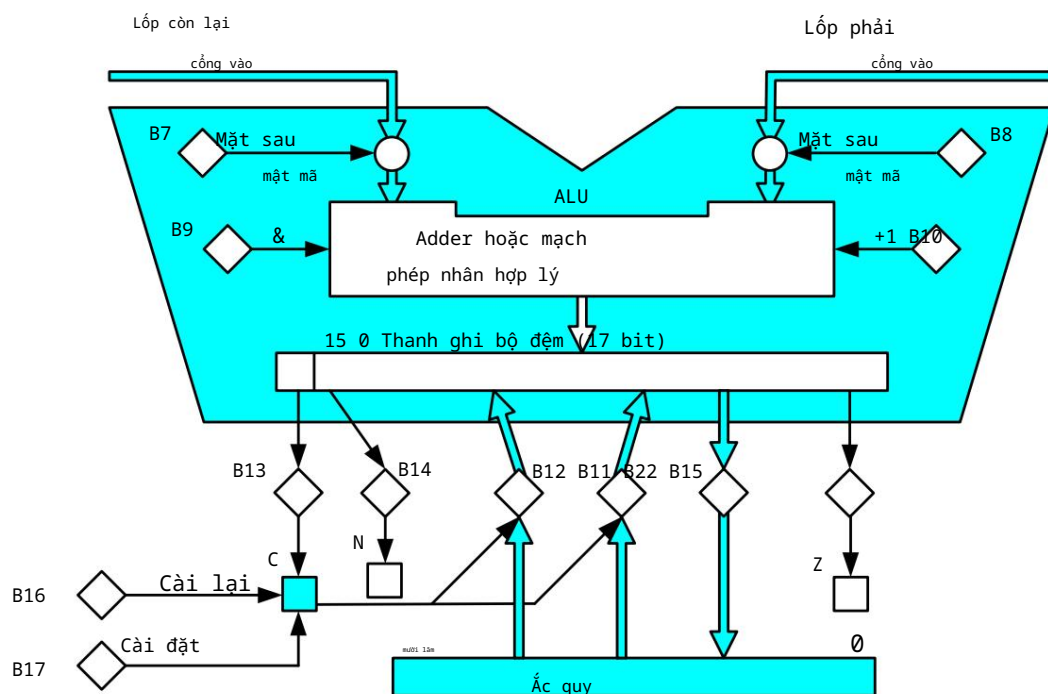
Cơ. 4.4. Thao tác thiết bị dịch chuyển sang trái 1-bit

Số được xoay 1 bit sang trái được đặt trong bộ tích lũy. Sau đó, một quá trình truyền không đối xứng được thực hiện tới các thanh ghi bên trong của ALU: thanh ghi đếm và thanh ghi một bit B. Trong trường hợp này, bit quan trọng nhất của thanh ghi đếm được điền không phải bằng 0 mà bằng nội dung của phần mang. thanh ghi C. Sau đó, một chuyển trực tiếp được thực hiện từ B và thanh ghi đếm đến C và bộ tích lũy.

Cần lưu ý rằng cả hai lần chuyển xe này đều được thực hiện bằng các loại lớp khác nhau. Một trong số chúng (với kết nối không đối xứng giữa các bit của thanh ghi nguồn với thanh ghi máy thu) dùng để chuyển thông tin từ bộ tích lũy sang thanh ghi bộ đếm, và phần còn lại từ thanh ghi đếm (BR) đến bộ tích lũy. Để cặp thanh ghi này có thể dịch chuyển sang phải, cần sử dụng thêm một kênh thông tin (một bus có mạch cổng) không đối xứng phải.

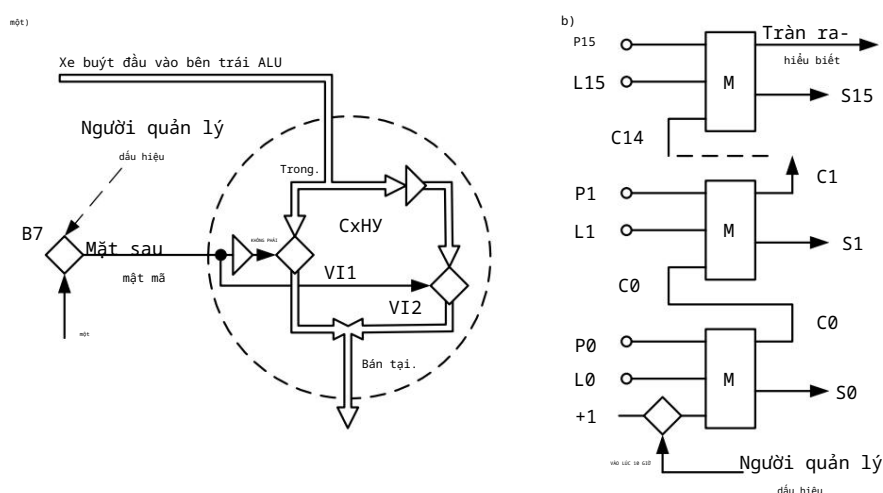
Công việc của đơn vị logic số học. ALU sử dụng nội dung của bộ tích lũy và thanh ghi dữ liệu làm toán hạng để tạo ra một kết quả được đặt trong bộ tích lũy. Tất cả các lệnh số học và logic của máy tính cơ sở (xem Bảng 2.4) và các phép toán số học hỗ trợ (ví dụ, tăng nội dung của bộ đếm chương trình lên một) có thể được thực hiện bằng ALU, sơ đồ khối của nó được thể hiện trong hình. 4.5. Chúng ta hãy xem xét ngắn gọn việc thực hiện mạch của các vi hoạt động riêng lẻ được thực hiện bởi ALU, bộ tích lũy và thanh ghi chuyển giao (C) theo các tín hiệu điều khiển U7-U17 và U22 đến cổng

sơ đồ V7-V17 và V22 (trong Hình 4.5 tín hiệu điều khiển U7-U17 và U22 không được hiển thị).



Cơ. 4.5. ALU, Accumulator và Carry Register

Mã trả về của các tín hiệu đầu vào ALU có thể được lấy theo sơ đồ của Hình. 4.6, a), trong đó việc thay đổi tất cả 0 thành 1 và tất cả 1 thành 0 được thực hiện bằng cách sử dụng biến tần CxNE. Trong trường hợp không có tín hiệu điều khiển (xem Hình 1.3), thiết bị không truyền đến đầu ra B7, van VI2 đóng, đầu ra của biến tần KHÔNG phải là một tín hiệu duy nhất và van VI1 mở. Trong trường hợp này, nút đầu ra chứa mã trực tiếp của toán hạng, tức là mã khớp với mã trong nút đầu vào. Khi một tín hiệu điều khiển đơn được áp dụng cho B7, 1 đi qua nó, được tạo ra bởi điện áp không đổi tương ứng (xem Hình 1.2), VI1 đóng, VI2 mở và đi vào nút Đầu ra. mã được cung cấp nghịch đảo với mã tại nút Trong.



Cơ. 4.6. Các kế hoạch để lấy mã trả lại (a) và số tiền (b)

Việc bổ sung được thực hiện bằng cách sử dụng một mạch cộng điển hình (Hình 4.6, b). Hàm được thực hiện bởi một trong các chữ số của bộ cộng này (chữ số 1) được trình bày dưới dạng Bảng. 4.1. Các biến đầu vào của bảng là C0, L1 và P1, các biến đầu ra là S1 và C1. Khi van B10 đóng, bộ cộng thực hiện sự phụ thuộc $S = L + P$.

Bảng 4.1

Chuyển từ Điều khoản trước bên phải C0			Tổng	Thực hiện tiếp theo loại C1
	Lỗi vào bên trái L1	Chữ số đầu vào P1		
0	0	0	0	0
0	0	001	001	0
0	001	0	001	0
0	001	001	0	001
001	0	0	001	0
001	0	001	0	001
001	001	0	0	001
001	001	001	001	001

Với sự trợ giúp của tín hiệu điều khiển được áp dụng cho van B10, có thể đảm bảo phép cộng vào tổng của một: $S \setminus L + P + 1$.

Một giá trị mang từ chữ số có nghĩa nhất (chữ số 15) cho biết bộ cộng bị tràn. Mang này được ghi vào chữ số quan trọng nhất (số bit 16) của thanh ghi đệm (xem Hình 4.5) và có thể được ghi lại vào thanh ghi mang C bằng cách áp dụng tín hiệu điều khiển đến cổng B13.

Cần lưu ý rằng bộ cộng, mạch cổng và bộ nghịch lưu được gọi là mạch tổ hợp, tức là mạch trong đó các giá trị các biến đầu ra hoàn toàn được xác định bởi các giá trị của các biến đầu vào tại một thời điểm nhất định. Do đó, một sự thay đổi trong bất kỳ đầu vào nào của ALU sẽ ngay lập tức thay đổi tổng. Điều này cho phép bạn thực hiện một số hoạt động vi mô khác nhau với sự trợ giúp của bộ cộng và các mạch để lấy mã trả về với các tổ hợp tín hiệu điều khiển khác nhau được áp dụng cho các mạch cổng B1-B8 và B10 (Hình 4.5 và 4.7):

- 1) tổng hợp nội dung của hai thanh ghi, một trong số đó được kết nối với đầu vào bên trái của ALU và thanh ghi kia ở bên phải.

Để thực hiện việc này, bạn cần áp dụng một tín hiệu điều khiển duy nhất cho B / hoặc B2, hoặc B3 và cùng một tín hiệu cho B4 hoặc B5 hoặc B6 (khi lệnh ADD được thực hiện, các tín hiệu điều khiển được áp dụng cho B1 và B4);

- 2) tính tổng nội dung của hai thanh ghi và 1 (ba số hạng).

Hoạt động vi mô này khác với hoạt động trước đó chỉ ở chỗ đồng thời với việc cung cấp tín hiệu điều khiển cho các van thiết lập kết nối giữa bộ cộng và thanh ghi (nơi lưu trữ mã của các điều khoản), tín hiệu tương tự phải được áp dụng cho van B10;

3) phép trừ.

Ví dụ, khi trừ nội dung của thanh ghi dữ liệu khỏi nội dung của bộ tích lũy (lệnh SUB trong Bảng 2.4), bạn phải đồng thời áp dụng các tín hiệu điều khiển đơn cho B1, B4, B8 và B10. Sau đó, mã nghịch đảo của chuỗi con sẽ được thêm vào giá trị nhỏ nhất, và một mã sẽ được thêm vào tổng này, tương đương với việc tính tổng giá trị nhỏ nhất với mã bổ sung của chuỗi con (xem đoạn 2.3);

4) thêm một vào nội dung của bất kỳ số đăng ký nào.

Hoạt động vi mô này là cần thiết để tăng nội dung của bộ đếm chương trình, thực hiện lệnh INC (xem Bảng 2.4) và trong một số trường hợp khác. Để thực hiện nó, cần áp dụng các tín hiệu điều khiển cho B10 và một mạch cổng thiết lập kết nối giữa bộ cộng và thanh ghi tương ứng. Trong trường hợp này, đầu vào khác của bộ cộng sẽ được kết nối với bus, tất cả các mạch cổng được đóng lại, tương đương với việc gửi mã số 0 qua bus;

5) trừ một trong các nội dung của bất kỳ số đăng ký nào.

Ví dụ, hoạt động vi mô này là cần thiết để thực hiện lệnh DEC (xem Bảng 2.4). Các tín hiệu điều khiển được áp dụng cho các mạch cổng B4 và B5, tức là nội dung của bộ tích lũy được thêm vào mã đảo ngược của số 0 hoặc (giống nhau) với mã bổ sung của số - 1 (xem đoạn 2.3);

6) đảo ngược nội dung của bất kỳ thanh ghi nào.

Sự biến đổi như vậy đã được sử dụng trong các hoạt động vi mô của Sec. 3 và 5. Để thực hiện, ví dụ, lệnh SMA (xem Bảng 2.4), các tín hiệu điều khiển nên được áp dụng cho B4 và B7;

7) xóa bất kỳ thanh ghi nào (đặt mã số 0 trong đó).

Để làm điều này, bạn cần thực hiện hai thao tác vi mô: gửi số 0 đến BR (trong chu trình này, tất cả các cổng từ B1 đến B10 đều bị đóng) và chuyển nội dung của BR đến thanh ghi mong muốn (cổng tương ứng được mở. trên xe buýt đầu ra BR, ví dụ, B22 khi lệnh CLA được thực thi).

Danh sách các hoạt động vi mô như vậy có thể được tiếp tục (ví dụ: nhập mã số - 1 vào thanh ghi, nhận tín hiệu chuyển từ chữ số có nghĩa nhất của bộ cộng khi cộng hai số bằng -1, v.v.), tuy nhiên, danh sách trên là đủ cho

xác nhận những điều sau:

Để máy thông dịch thực hiện được chương trình, cần phải mở và đóng các mạch cổng theo một trình tự nhất định; mô tả về mạch van nào và khi nào mở,

soạn một chương trình (phần sụn) cho máy, hệ thống lệnh trong đó bao gồm lệnh "Mở mạch cổng".

Phép nhân logic được thực hiện bằng cách sử dụng một mạch cổng, đến các đầu vào mà các mã toán hạng được cấp, và kết quả mong muốn được hình thành ở đầu ra (xem đoạn 1.2 và 2.5).

Trong ALU, bộ cộng và mạch nhân logic được kết hợp thành một khối. Trong khối, các bus đầu vào được phân nhánh thành cả hai thiết bị này và đầu ra của các thiết bị được kết nối thông qua các mạch cổng tới các đầu vào của thanh ghi đệm. Do đó, khi toán hạng đến

Các hoạt động của phép tính tổng và phép nhân logic được thực hiện đồng thời, nhưng vì chỉ một trong các mạch cổng đầu ra được mở trên tín hiệu từ B9 (xem Hình 4.5 và 4.7), nên chỉ một trong hai kết quả sẽ được gửi đến thanh ghi đệm. Các mạch cổng đầu ra được điều khiển giống như VI1 và VI2 được điều khiển trong Hình. 4.6: trong trường hợp không có tín hiệu điều khiển trên B9, tổng được ghi vào BR và khi có tín hiệu điều khiển trên B9, kết quả

phép nhân logic.

Thực hiện chuyển đổi theo chu kỳ một chữ số sang phải hoặc trái bằng cách áp dụng một tín hiệu điều khiển duy nhất cho mạch cổng ~ B1 1 hoặc B12, sau đó đến mạch cổng B13 và B22 (xem Hình. 4.4 và 4.5).

Đặt các dấu hiệu chuyển từ bậc cao của bộ cộng, cũng như Giá trị âm hoặc giá trị không của kết quả được thực hiện bằng cách gửi tín hiệu điều khiển đến các van B13, B14 và B15

tương ứng.

Khi một tín hiệu điều khiển được áp dụng cho cổng B14, nội dung của chữ số (dấu) thứ 15 của BR được ghi lại thành một thanh ghi một bit N. Khi cổng B15 được mở, nội dung của một mạch đặc biệt được ghi lại thành một- thanh ghi bit Z, tín hiệu đầu ra của tín hiệu này chỉ bằng một khi tất cả 16 bit trong thanh ghi đệm (từ 0 đến 15) chứa các số không, tức là khi mã số 0 được lưu trong BR.

Nội dung của thanh ghi mang cũng có thể được thay đổi bằng cách sử dụng các tín hiệu điều khiển. Áp dụng một tín hiệu cho B13 cho phép bạn viết lại nội dung của bit bậc cao của BR (chữ số 16), lưu trữ giá trị mang từ bit bậc cao của bộ cộng, thành C. Gói của người quản lý

Một tín hiệu đến B16 đặt lại C (lệnh CLC xóa thanh ghi mang), và một tín hiệu đến B17 khiến một đơn vị được ghi vào nó.

Đăng kí trạng thái. Sự kết hợp giữa cờ một bit và thanh ghi trạng thái này, mục đích đã được thảo luận ở đây và trong Phần 3.3, được thực hiện để chính thức giảm số lượng thanh ghi mà thiết bị điều khiển vi chương trình hoạt động,

cho phép giảm độ dài từ của vi lệnh điều khiển.

Trên bảng điều khiển gỡ lỗi thứ hai của máy tính cơ sở (xem Hình 4.7), thanh ghi trạng thái (PC) được hiển thị (để thuận tiện) như một thanh ghi độc lập. Trên thực tế, các trạng thái của nó chỉ nhân bản trạng thái của các thanh ghi một bit C, Z, N, v.v ... Sự tương ứng giữa các thanh ghi này và các bit PC được đưa ra trong Bảng. 4.2.

Bảng 4.2

Đăng kí trạng thái

Nội dung	Xả
0	Chuyển (C)
1	Zero (Z)
2	Ký tên (N)
3	0 - được sử dụng để tổ chức các bước nhảy không điều kiện trong MPU
bốn	Cho phép ngắt
5	Ngắt
6	Các trạng thái của WU (F)
7	Chuyển đổi trạng thái VẬN HÀNH / DỪNG
	(1 - CÔNG VIỆC)
trăm	Chương trình
9	Nhóm tìm nạp
mười	Tìm nạp địa chỉ
mười một	Chấp hành
12	Đầu ra đầu vào

Các mạch van. Liệt kê dưới đây là các chức năng của một số mạch cổng được sử dụng để truyền thông tin giữa các thanh ghi, giữa các thanh ghi và ALU và để điều khiển các vi mạch trong ALU. Chức năng của các mạch cổng còn lại trong hình. 4.7 sẽ được thảo luận trong đoạn 4.3.

Lưu ý rằng việc trao đổi dữ liệu giữa các thanh ghi có độ dài khác nhau (ví dụ: giữa BR và SC) được thực hiện bằng cách sử dụng trực tiếp

truyền (xem Hình 4.3), và các bit cao hơn của máy thu dài hơn được lấp đầy bởi các số không.

Các tín hiệu đầu vào ALU. Các mạch cổng B1, B2 và B3 dành cho đầu vào bên phải của ALU, và B4, B5 và B6 dành cho bên trái. Tất cả chúng được sử dụng để chuyển nội dung của các thanh ghi tương ứng sang thanh ghi đệm của ALU và xa hơn nữa tới thiết bị điều khiển phần sụn.

TRONG 1. RD trong ALU. Được sử dụng cho số học và logic các hoạt động, cũng như trong quá trình chuyển qua ALU tới RK, SC, RA và MPU.

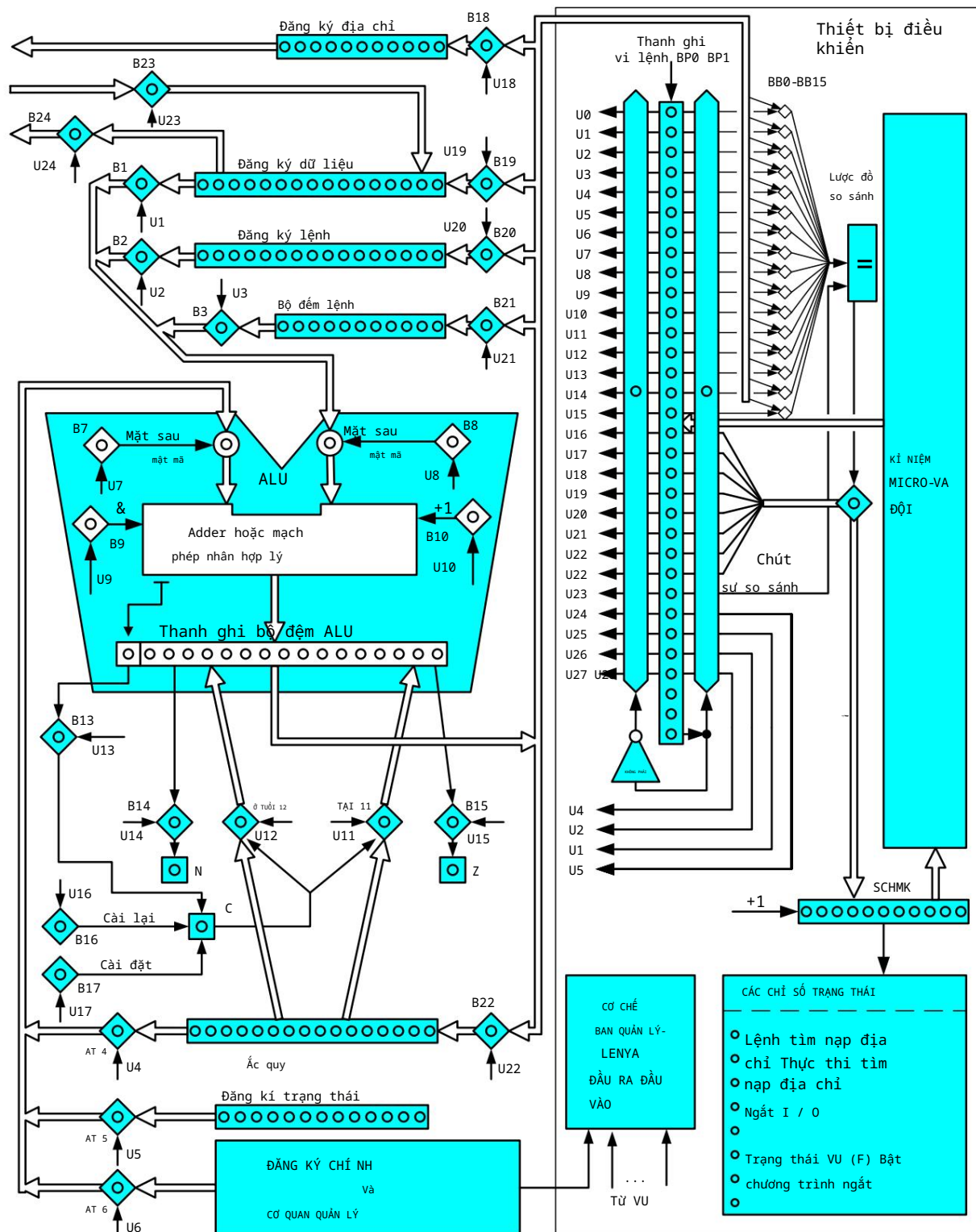
TRONG 2. RK trong ALU. Nó được sử dụng để chuyển nội dung của RK tới MPU.

TAI 3. SC sang ALU. Được sử dụng để tăng hàm lượng SC lên 1 và truyền qua ALU tới RA, RD và MPU.

ĐỢT 4. VÀ trong ALU. Nó được sử dụng cho các phép toán số học và logic, cũng như để chuyển qua ALU tới RD và MPU.

TAI 5. PC sang ALU. Được sử dụng để chuyển nội dung PC sang MPU.

ĐỢT 6. CR trong ALU. Được sử dụng trong khi vận hành điều khiển từ xa Máy tính để chuyển nội dung của thanh ghi khóa (CR) đến SC và RD.



CƠM. 4.7. Bảng điều khiển gỡ lỗi thứ hai của máy tính cơ sở

Các phép toán số học và sự dịch chuyển.

VÀO LÚC 7 GIỜ. Khi mạch cổng này mở, tín hiệu đến đầu vào bên trái bộ cộng đến trong mã ngược lại.

LÚC 8. Khi mạch cổng này mở, tín hiệu đến đầu vào bên phải bộ cộng đến trong mã ngược lại.

LÚC 9 GIỜ. Khi mạch cổng này mở, BR nhận kết quả của phép nhân logic các toán hạng. Với một mạch kín, BR nhận được tổng của các toán hạng.

VÀO LÚC 10 GIỜ. Khi cổng này mở, tổng các toán hạng 1 được thêm vào.

TAI NGÀY 11. Khi cổng này mở, nội dung của A và C được chuyển một cách không đối xứng sang BR: nội dung LSB của A đi vào

Bit thứ 16 của BR, nội dung của C vào bit thứ 15 của BR, nội dung của bit quan trọng nhất của A vào bit thứ 14 của BR, v.v.

Ở TUỔI 12. Khi cổng này mở, nội dung của A và C được chuyển không đối xứng sang BR: nội dung của bit bậc thấp của A chuyển đến bit thứ nhất của BR, nội dung của C chuyển đến bit thứ 0 của BR, nội dung của bit quan trọng nhất của A chuyển đến bit thứ 16 của BR, v.v.

Đặt biển báo.

B13. Khi mạch cổng này mở, một giá trị mang (0 hoặc 1) từ bit quan trọng nhất của bộ cộng, được cố định trong bit quan trọng nhất của BR, đi đến C.

B14. Khi mạch cổng này mở, dấu hiệu của số được lưu trữ trong BR, chuyển đến N.

B15. Khi cổng này mở, Z chuyển sang 1 nếu nội dung BR của tôi là 0 và 0 nếu nội dung BR không phải là 0.

B16. Khi cổng này mở, 0 được ghi vào C.

B17. Khi cổng này mở, số 1 được ghi vào C.

Tín hiệu đầu ra ALU (BR).

B18. BR ở RA. Nó được sử dụng khi chuyển nội dung của RD hoặc SC.

B19. BR trong RD. Nó được sử dụng khi chuyển nội dung của CK, A hoặc CR tới RD.

TRONG 20. BR bằng RK. Nó được sử dụng khi chuyển nội dung của RD sang RC.

TAI NGÀY 21. BR đến SC. Nó được sử dụng khi chuyển nội dung của RD hoặc CR sang SC và để tăng SC lên 1.

B22. BR đến A. Được sử dụng để gửi kết quả của các phép tính số học và chuyển dịch đến A.

Đọc và ghi vào bộ nhớ của máy thông dịch
kì niệm.

B23. Từ bộ nhớ đến RD. Được sử dụng để nạp vào RD từ bộ nhớ có địa chỉ được chỉ định trong RA.

B24. Từ RD đến bộ nhớ. Được sử dụng để ghi nội dung của RD tới từ bộ nhớ có địa chỉ được chỉ định trong RA.

Tổ chức thông tin đầu vào - đầu ra.

B25. Khi mạch cổng này mở, các bộ điều khiển VU từ RD thứ tự cho đầu vào-đầu ra và địa chỉ của VU yêu cầu được gửi.

B26. Khi mạch cổng này mở, tất cả các bộ điều khiển VU nhận được tín hiệu cùng một lúc, đặt lại cờ của chúng.

B27. Khi cổng này mở, bit sẽ bị xóa.
cho phép ngắt trong PC (bit 4).

B28. Khi mạch cổng này mở, cài đặt được thực hiện
ngắt cho phép bit trong PC (bit 4).

Tắt máy tính.

B0. Mạch cổng này dùng để truyền tín hiệu
chấm dứt thực hiện chương trình (lệnh HLT).

Các thành phần bộ xử lý khác (bộ nhớ vi cấu trúc, thanh ghi các lệnh vi mô, v.v.) được thảo luận trong đoạn 4.3.

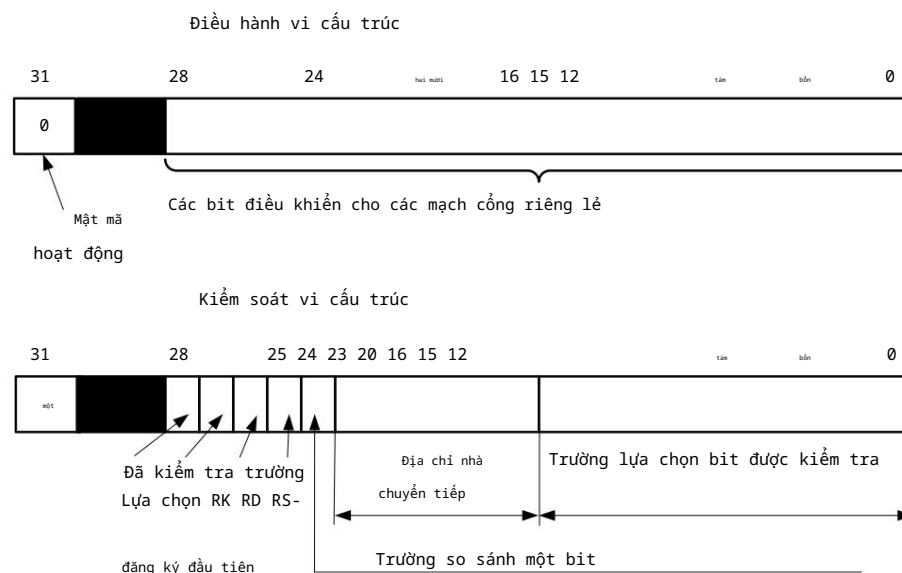
Dễ dàng nhận thấy rằng trong cấu trúc đề xuất, việc truyền dữ liệu giữa các thanh ghi chỉ được thực hiện thông qua thanh ghi đệm ALU. Điều này được thực hiện để giảm số lượng xe buýt và van, tức là, đơn giản hóa và giảm chi phí của bộ xử lý. Tuy nhiên, giải pháp này làm chậm thủ tục trao đổi.

4.3. Điều khiển mạch cổng phần sụn

Các hướng dẫn vi mô. Mức thông dịch (xem Hình 4.7) có một tập hợp các lệnh chỉ bao gồm hai lệnh vi mô: điều hành (OMK) và điều khiển (CMD), các định dạng của chúng được thể hiện trong hình. 4.8. Vì chỉ có hai lệnh nên mã hoạt động một bit là đủ: 0 cho OMK và 1 cho UMK. Mặc dù 30 bit là đủ để chứa thông tin và một mã opcode, các vi lệnh 32 bit (bốn byte) đã được chọn.

Trong một vi lệnh hoạt động, mỗi trong số 29 bit thông tin điều khiển một trong 29 mạch cổng được mô tả trong Phần 4.2. Một có nghĩa là mạch cổng phải được mở, không -

đã đóng cửa. Cấu trúc vi mô này thường được gọi là ngang.



Cơ. 4.8. Hệ thống lệnh của cấp phần sụn được hiển thị trong hình. 4,7
(chỉ gồm hai đội)

Vi lệnh điều khiển được sử dụng để thay đổi

trình tự thực hiện vi lệnh tùy thuộc vào các điều kiện nhất định. Mỗi vi lệnh UMK xác định 1 bit được kiểm tra (bất kỳ trong số 61 bit nằm trong A, RK, RD và PC). Nếu bit được chọn và bit thứ 24 của vi lệnh khớp nhau, trình tự của vi lệnh sẽ được thực thi sẽ thay đổi và địa chỉ của vi lệnh tiếp theo được xác định bởi các bit từ 16 đến 23 của chính CCM. Nếu bit được chọn khác với bit 24, trình tự thực hiện vi lệnh không bị thay đổi.

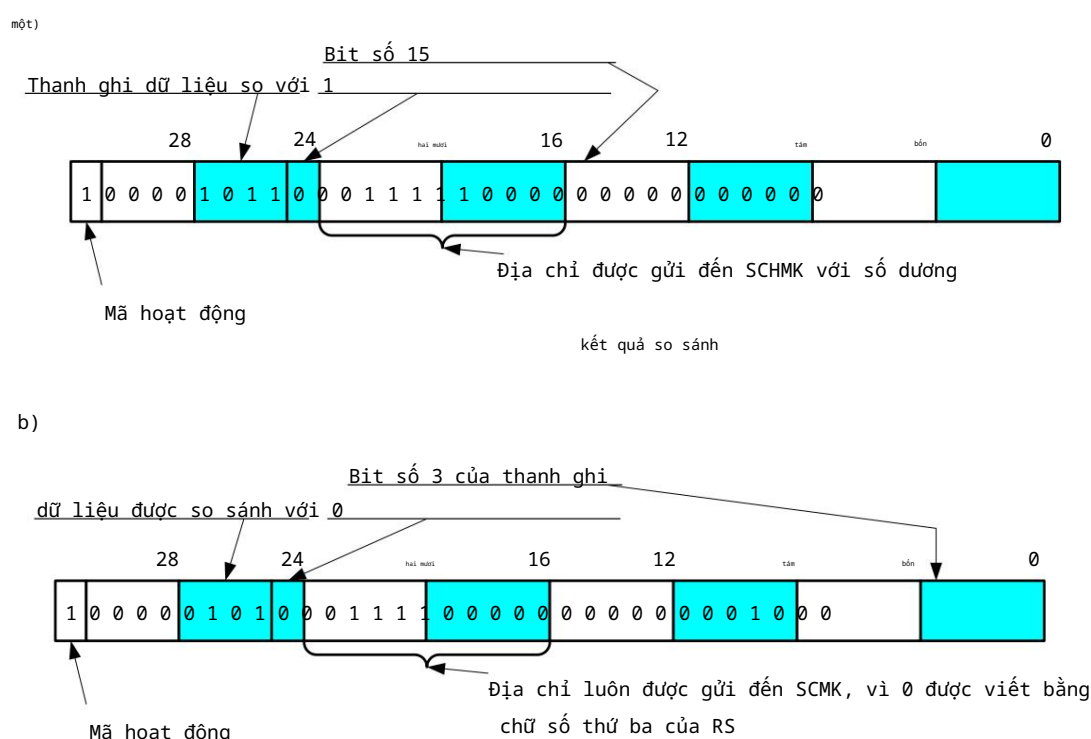
Bit cần kiểm tra được xác định bởi hai trường vi lệnh. Các bit từ 25 đến 28 cho biết thanh ghi nào của máy thông dịch cần được kiểm tra theo những điều sau

quy tắc:

- bit 25 - thanh ghi trạng thái (PC);
- bit 26 - thanh ghi dữ liệu (RD);
- bit 27 - thanh ghi lệnh (RK);
- bit 28 - bộ tích lũy (A).

Các bit vi lệnh từ 0 đến 15 cho biết bit nào của thanh ghi đã chọn sẽ được kiểm tra. Chỉ một bit thuộc nhóm bit 0 đến 15 và một bit của nhóm bit 25 đến 28 phải bằng nhau đơn vị.

Ví dụ, sau khi tăng nội dung của thanh ghi dữ liệu (RD) trong lệnh ISZ (xem Phần 2.4), bạn cần kiểm tra bit đầu của nó (bit số 15). Nếu bit này bằng 1 (nội dung của RD là âm), thì việc thực hiện các vi lệnh của lệnh ISZ đã hoàn thành. Nếu không, cần phải thêm một vào nội dung của bộ đếm chương trình, tức là tổ chức việc bỏ qua lệnh sau ISZ. Việc phân nhánh trong phần sụn của việc thực hiện lệnh ISZ (nhảy đến địa chỉ 8F) được thực hiện bằng cách sử dụng vi lệnh (858F8000) 16. Trên hình. 4.9, và lệnh vi mô này được trình bày dưới dạng nhị phân với dấu hiệu của tất cả các bit được sử dụng.



Cơ. 4.9. Ví dụ về các lệnh vi điều khiển:

a - chuyển tiếp có điều kiện; b - chuyển tiếp vô điều kiện

Để tổ chức một bước nhảy không điều kiện (ví dụ, đến cùng một địa chỉ 8F), bit thứ 3 của thanh ghi trạng thái được sử dụng, chứa hằng số 0. So sánh bit này với hằng số 0 được đặt trong 24-

chữ số thứ của CMC, luôn cho kết quả dương tính và cho phép bạn gửi địa chỉ chuyển tiếp mong muốn đến CMC (Hình 4.9.6).

Thực thi vi chương trình. Vi chương trình được lưu trữ trong bộ nhớ vi lệnh, bao gồm 256 (28) từ 32 bit (xem Hình 4.7). Để kiểm soát việc thực hiện vi chương trình, một bộ đếm vi lệnh (MCC) được sử dụng, tương tự như bộ đếm lệnh của cấp độ máy truyền thống. Để truy xuất vi lệnh từ bộ nhớ

các vi lệnh, để giải mã và thực thi chúng, người ta phải có địa chỉ, dữ liệu và thanh ghi lệnh. Nhưng vì máy thông dịch rất đơn giản, một thanh ghi sẽ thực hiện các chức năng của thanh ghi địa chỉ và bộ đếm vi lệnh, và thanh ghi kia sẽ thực hiện các chức năng của thanh ghi dữ liệu và thanh ghi vi lệnh (RMK).

Hoạt động của bộ xử lý có thể được mô tả như sau. Khi một xung đồng hồ xuất hiện trong RMK, từ được trở đến bởi SMK được truy xuất và tải từ bộ nhớ vi lệnh, và một từ được thêm vào nội dung của bộ đếm này. Nếu CMC được truy xuất từ bộ nhớ, thì bit thứ 31 của RMC chứa 1 (opcode CCD), mở ra mạch cổng VR1 và do đó tạo điều kiện cho CMC thực thi. Nếu OMC được trích xuất, thì bit thứ 31 của RMC bằng không. Tín hiệu này, sử dụng biến tần, KHÔNG mở mạch cổng BP0, và thông qua đó, trạng thái của các bit RMK tương ứng được truyền đến B0-B28. Các bit RMK chứa các bit tạo ra tín hiệu điều khiển mở và các bit chứa số không tạo ra tín hiệu đóng (U0 - U28).

Khi CMD được thực thi, bởi một tín hiệu được tạo ra bởi bất kỳ bit nào của trường lựa chọn thanh ghi kiểm tra (U1, U2, U4 hoặc U5), một trong các mạch cổng B1, B2, B4 hoặc B5 sẽ mở ra và nội dung của thanh ghi tương ứng là được gửi đến các cổng BB0 - BB15 thông qua ALU. Đồng thời, nội dung của trường lựa chọn bit có thể kiểm tra được cung cấp cho các cổng giống nhau từ RMC. Vì chỉ một đơn vị nên được ghi trong trường này, nên chỉ một trong các cổng BB0 - BB15 mở ra, qua đó nội dung của bit được kiểm tra từ thanh ghi đã kiểm tra đi vào mạch so sánh.

Đầu vào thứ hai của mạch so sánh nhận nội dung của trường so sánh một bit (bit thứ 24 của CCM), trong đó, khi mã hóa UMC đã viết ra số 0 hoặc 1. Nếu bit đang được kiểm tra và bit từ trường so sánh giống hệt nhau, thì mạch so sánh tạo ra một tín hiệu duy nhất để mở mạch cổng VA và địa chỉ bước nhảy được gửi đến FMS (bit 16 đến 23). Nếu không, địa chỉ của vi lệnh nằm sau lệnh thực thi được lưu trữ trong CHMK.

Khi địa chỉ của vi lệnh bắt đầu bất kỳ chu kỳ máy nào (xem đoạn 2.5 và 3.3) xuất hiện trong CHMK, thông tin trên chỉ báo trạng thái bộ xử lý sẽ thay đổi. Thông tin này cũng thay đổi trong quá trình chuyển đổi của máy tính từ trạng thái DỪNG sang trạng thái CHƯƠNG TRÌNH hoặc chuyển đổi ngược lại.

4.4. Trình thông dịch máy tính cơ bản

Ngôn ngữ lập trình vi mô. Phần mềm cơ sở có thể được viết dưới dạng một chuỗi các số nhị phân 32-bit, nhưng công việc như vậy rất tẻ nhạt và rất dễ mắc lỗi. Để tạo điều kiện thuận lợi cho việc viết và xác minh chương trình cơ sở, việc thực hiện cá nhân

hướng dẫn máy và vòng lặp sử dụng hệ thập lục phân của chúng

mã hóa. Nhận xét đối với các lệnh vi mô sẽ được ghi lại bằng

Cứu giúp ghi nhớ ngôn ngữ lập trình vi mô, gợi nhớ

đến các ngôn ngữ thuật toán cấp cao (SI, PASCAL

và v.v.). Hãy xem xét các ví dụ về ký hiệu như vậy cho các hoạt động vi mô khác nhau.

Chuyển thông tin từ số đăng ký sang (các) đăng ký

đã nhận xét với một toán tử gán. Ví dụ, việc chuyển nội dung của BR sang RD (mở B19) được viết là

0008 0000; BR => RD.

(Để mã hóa hệ thập lục phân dễ dàng hơn, hãy sử dụng hình.

4.9, hiển thị sự phân chia thành ghi 32 bit thành các tetrads.)

Chuyển nội dung BR sang bộ tích lũy có dấu hiệu cài đặt kết quả (mở B13, B14, B15 và B22) được viết là 0040 E000; BR =>

A, C, N, Z.

Trao đổi thông tin giữa bộ nhớ (OP) và thanh ghi dữ liệu

được thiết lập như sau:

đọc từ bộ nhớ tại địa chỉ được lưu trữ trong RA (mở B23),

0080 0000; OP (RA) => RD;

ghi vào bộ nhớ tại địa chỉ được lưu trữ trong RA (mở B24),

0100 0000; RD => OP (RA).

Các hoạt động được thực hiện trong ALU cũng được chú thích bằng các câu lệnh gán. Trong trường hợp này, mã đảo ngược được viết dưới dạng hàm COM, và dịch chuyển sang phải và trái được viết dưới dạng hàm RAR và RAL, tương ứng. Ví dụ, tính tổng A và RD (mở B1 và B4)

sẽ được viết dưới dạng

0000 0012; A + RD => BR,

phép trừ RD khỏi A (mở B1, B4, B8 và B10)

00000512; A + COM (RD) + 1 => BR,

xoay nội dung của A và C sang phải một bit (mở B11)

0000 0800; RAR (A, C) => BR

và cuối cùng thêm một vào nội dung của SC (mở B3 và B10)

0000 0408; SC + 1 => BR.

Vi lệnh điều khiển được nhân xét bằng cách sử dụng toán tử

bước nhảy có điều kiện

IF BIT (k, reg) \ u003d B THEN nhân (địa chỉ), trong đó k là số bit được kiểm tra (từ 0 đến 15); reg - đăng ký được kiểm tra (A, RK, RD, PC); B - 0 hoặc 1; label (địa chỉ) - nhân và địa chỉ của vi lệnh mà điều khiển được chuyển tới nếu việc kiểm tra cho kết quả dương tính.

Ví dụ: kiểm tra bit 12 của thanh ghi lệnh sẽ được viết như thế này:

89xx 1000; NẾU BIT (12, RK) = 1 THEN E1 (xx),

trong đó "xx" phải được thay thế bằng địa chỉ của lệnh vi mô mà quyền kiểm soát được chuyển đến khi kết quả kiểm tra dương tính; các chú thích của vi cấu trúc này phải được dán nhãn E1.

Kiểm tra bit thứ 15 của bộ tích lũy sẽ được viết là

90xx 8000; NẾU BIT (15, A) = 0 THÌ KHÔNG (xx).

Nhớ lại rằng việc kiểm tra bit thứ 3 của thanh ghi trạng thái có chứa hằng số 0 để xác định xem nó có bằng 0 hay không luôn cho kết quả dương. Điều này tạo ra một sự chuyển giao vô điều kiện

kiểm soát, được viết dưới dạng

82xx 0008; ĐI ĐẾN BR (xx).

Phần mềm cơ sở. Toàn bộ văn bản của trình thông dịch được đưa ra trong Bảng. 4.3. Bảng này có một cột "bổ sung" (VERTICAL), nội dung của cột này được mô tả trong đoạn 4.5.

Các dòng chương trình cơ sở có số 01-0C mô tả các quy trình tìm nạp lệnh của máy thông dịch, gửi đến RC để phân tích và tăng nội dung của SC lên một. Các dòng 07-0B tạo thành một "cây" giải mã kiểm tra các bit 15, 14, 13 và 12 của thanh ghi lệnh để xác định loại lệnh và chuyển quyền điều khiển đến phần thích hợp của phần sụn. Điều này sử dụng thực tế là các mã opcode của lệnh địa chỉ nằm trong phạm vi từ 0000 đến 1100, tức là chúng nhất thiết phải chứa số 0 trong các bit 15, 14 và 13 và các lệnh đầu vào / đầu ra (opcode 1110) khác với không phải địa chỉ lệnh (opcode 1111) 12

phóng điện.

Nếu một lệnh địa chỉ được chọn, thì trong dòng 0C, kiểu địa chỉ được chỉ định trong đó (trực tiếp hoặc gián tiếp) sẽ được chọn. Với địa chỉ trực tiếp, quá trình chuyển đổi sang dòng 1D được thực hiện và với địa chỉ gián tiếp -

chuyển sang vi lệnh của chu trình tìm nạp địa chỉ của toán hạng (dòng 0D - 1C) và trên dòng 1D.

Trong chu kỳ tìm nạp địa chỉ toán hạng, địa chỉ này đầu tiên được tìm nạp vào RD, sau đó kiểm tra xem nó có được lưu trữ trong một trong các ô chỉ mục hay không (các ô có địa chỉ từ 008 đến 00F). Nếu có, thì nội dung của RD được tăng thêm 1, sau đó được chuyển trở lại vị trí chỉ mục, và cuối cùng được khôi phục (1 được trừ vào nội dung của RD). Hơn nữa

bắt đầu thực hiện các lệnh vi mô của chu trình thực thi địa chỉ các lệnh.

Trong chu trình thực thi lệnh địa chỉ, bắt đầu với vi lệnh rẽ nhánh lệnh rẽ nhánh đặt tại địa chỉ 1D, lệnh này được giải mã và thực thi thêm. Sau khi các lệnh nhảy được tách ra, các lệnh số học (SUB, ADD, ADC), ISZ và lệnh AND, v.v. được tách biệt.

Việc thực hiện bất kỳ lệnh địa chỉ nào (tuy nhiên, giống như bất kỳ lệnh nào khác) kết thúc bằng sự chuyển đổi vi lệnh sang chu kỳ ngắt (vi lệnh 8F - 98), trong đó chế độ hoạt động của máy tính được làm rõ và nếu máy đang ở trạng thái "CHƯƠNG TRÌNH", sau đó nó được kiểm tra xem có cần thiết phải ngắt các chương trình đang thực thi theo yêu cầu của thiết bị bên ngoài hay không (xem đoạn 3.5). Nếu không được yêu cầu, thì quá trình chuyển đổi sang phần đầu của vi chương trình được thực hiện (dòng 01), tức là đến việc lựa chọn một lệnh mới của máy thông dịch. Nếu không, một bước nhảy được thực hiện đối với quy trình ngắt.

Các lệnh chưa giải quyết được xử lý theo cùng một cách, các lệnh đầu vào-đầu ra và các lệnh được thiết lập bằng cách nhấn các nút trên bảng điều khiển máy tính.

Bảng 4.3

Trình thông dịch máy tính giáo dục (vi chương trình

Địa chỉ nhà	Nhận xét về hướng dẫn vi mô		Chu	
	Chân trời. Vert.	Hoạt động		
kỳ tìm nạp Lệnh Nhận				
01	0000 0008	0300	sớm	SC ==> BR
02	0004 0000	4001		BR ==> RA
03	0080 0408	0311		OP (RA) ==> RD, SC + 1 ==> BR
04	0020 0000	4004		BR ==> SC
05	0000 0002	0100		RD ==> BR
06	0010 0000	4003		BR ==> RK
Định nghĩa kiểu lệnh				
07	880C8000AF0C			NẾU BIT (15, PK) = 0 THEN ADC (0C)
08	880C4000AE0C			NẾU BIT (14, PK) = 0 THEN ADC (0C)
09	880C2000AD0C			NẾU BIT (13, PK) = 0 THEN ADC (0C)
0A	895E 1000EC5E			NẾU BIT (12, PK) = 1 THEN BAD (5E)
0B	828E 0008 838E			GOTO B / B (8E)
				Xác định loại địa chỉ

Địa chỉ nhà	Horizon		Bình luận	
	Microcommands.	Vert.	Nhân	Hoạt động
0C	881D0800AB1D		ADC Chu	NẾU BIT (11, PK) = 0 THEN ADR (1D)
kỳ tìm nạp địa chỉ toán hạng				
0D	0000 0002 0100 0004			RD ==> BR
0E	0000 4001 0080 0000			BR ==> RA
0F				OP (RA) ==> RD
10	881D0008A31D			NẾU BIT (3, PK) = 0 THEN ADR (1D)
11	891D 0010 E41D			NẾU BIT (4, PK) = 1 THEN ADR (1D)
12	891D 0020 E51D			NẾU BIT (5, PK) = 1 THEN ADR (1D)
13	891D 0040 E61D			NẾU BIT (6, PK) = 1 THEN ADR (1D)
14	891D 0080 E71D			NẾU BIT (7, PK) = 1 THEN ADR (1D)
15	891D 0100 E81D			NẾU BIT (8, PK) = 1 THEN ADR (1D)
16	891D 0200 E91D			NẾU BIT (9, PK) = 1 THEN ADR (1D)
17	891D 0400 EA1D 0000			NẾU BIT (10, PK) = 1 THEN ADR (1D)
18	0402 0110 0008 0000			RD ==> BR
19	0100 0000 0002 0000			BR ==> RD
1A	0140 0008 0000 4002			RD ==> OP (RA)
1B				RD + COM (0) = RD - 1 ==> BR
1C				BR ==> RD
Địa chỉ lệnh thực hiện chu kỳ				
				Giải mã lệnh địa chỉ
1D	892D 8000 EF2D 0000		ADR	NẾU BIT (15, PK) = 1 THEN PKH (2D)
1E	0002 0100 0004 0000			RD ==> BR
1F				BR ==> RA
20	89274000EE27			NẾU BIT (14, PK) = 1 THEN ARF (27)
21	8824 2000AD24			NẾU BIT (13, PK) = 0 THEN A1 (24)
22	8857 1000 AC57 8238			NẾU BIT (12, PK) = 0 THEN JSR (57)
23	0008 8338 0080 0000			GOTO MOV (38)
24			A1	OP (RA) ==> RD
25	8850 1000 AC50 8235			NẾU BIT (12, PK) = 0 THEN ISZ (50)
26	0008 8335 0080 0000			GOTO VÀ (35)
27			ARF	OP (RA) ==> RD
28	882B 2000AD2B			IF BIT (13, PK) = 0 THEN SUM (2B)
29	8843 1000 AC43			NẾU BIT (12, PK) = 0 THEN SUB (43)
2A	82B0 0008 83B0			GOTO P-A (B0)
2B	883C 1000AC3C		TỔNG	NẾU BIT (12, PK) = 0 THÌ THÊM (3C)
2C	823F 0000 833F			GOTO ADC (3F)
2D	88304000AE30		RMP	IF BIT (14, PK) = 0 THEN UPK (30)
2E	8847 1000 AC47			NẾU BIT (12, PK) = 0 THEN BR (47)
2F	82D0 0008 83D0			GOTO P - P (D0)
30	8833 2000 AD33 884C		UPH	NẾU BIT (13, PK) = 0 THEN P1 (33)
31	1000 AC4C			NẾU BIT (12, PK) = 0 THÌ BMI (4C)
32	824E 0008 834E			GOTO BEQ (4E)
33	8846 1000 AC46		P1	NẾU BIT (12, PK) = 0 THEN BCS (46)
34	824A 0008 834A			GOTO BPL (4A)
				Thực thi các lệnh địa chỉ
35	0000 0212 1120		VÀ	A&RD ==> BR
36	0040 C000 4035			BR ==> A, N, Z
37	828F 0008 838F 0000			GOTO PRE (8F)
38	0010 1000 0008 0000		MOV	A ==> BR
39	0100 0000 0002			BR ==> RD
3A				RD ==> OP (RA)
3B	828F 0008 838F 0000			GOTO PRE (8F)
3C	0012 1100		CỘNG	A + RD ==> BR
3D	0040 E000 4075			BR ==> A, C, N, Z
3E	828F 0008 838F			GOTO PRE (8F)
3F	823C 0001 803C 0000		ADC	NẾU BIT (0, PC) = 0 THÌ THÊM (3C)
40	0412 1110			A + RD + 1 ==> BR
41	0040 E000 4075			BR ==> A, C, N, Z

Địa chỉ nhà	Horizon		Bình luận	
	Microcommands.	Vert.	Nhãn mác	Hoạt động
42	828F 0008 838F	0000		GOTO PRE (8F)
43	0512 1190 0040	0000	SUB	A + COM (RD) + 1 = A - RD ==> BR
44	828F 0008 838F	828F		BR ==> A, C, N, Z
45	0001 808F 0000	0002		GOTO PRE (8F)
46	0100 0020 0000	0000	BCS	IF BIT (0, PC) = 0 THEN PRE (8D)
47	0008 838F 838F	0000	BR	RD ==> BR
48	8247 0008 8347	0000		BR ==> SC
49	828F 8247 0008	8347		GOTO PRE (8F)
4A	0128F 0002 8180	0000	BPL	IF BIT (2, PC) = 1 THEN PRE (8F)
4B	0000 8247 0008	0000		GOTO BR (47)
4C	013400 0000 8247	0000	BMI	IF BIT (2, PC) = 0 THEN PRE (8F)
4D	0008 8 0002 8580	0000		GOTO BR (47)
4E	df8f 0000 0408	0310	BEQ	IF BIT (1, PC) = 0 THEN PRE (8F)
4F	0020 0000 4004	0000		GOTO BR (47)
50	838F		ISZ	RD + 1 ==> BR
51				BR ==> RD
52				RD ==> OP (RA)
53				IF BIT (15, RD) = 1 THEN PRE (8F)
54				SC + 1 ==> BR
55				BR ==> SC
56				GOTO PRE (8F)
57			JSR	RD + 1 ==> BR
58				BR ==> RK
59				SC ==> BR
5A				BR ==> RD
5B				RD ==> OP (RA), RK ==> BR
5C				BR ==> SC
5V				GOTO PRE (8F)
Tiếp tục chu kỳ tìm nạp lệnh giải mã và thực hiện các lệnh chưa giải quyết				
5E	8861 0800AB61			NẾU BIT (11, PK) = 0 THEN B0 (61)
5F	886C0400AA6C			NẾU BIT (10, PK) = 0 THEN B1 (6C)
60	82E0 0008 83E0			GOTO R - B (E0)
61	8867 0400AA67		B0	NẾU BIT (10, PK) = 0 THEN B2 (67)
62	8865 0200 A965			NẾU BIT (9, PK) = 0 THEN B3 (65)
63	8882 0100 A882	8285		NẾU BIT (8, PK) = 0 THEN ROL (82)
64	0008 8385			GOTO ROR (85)
65	887B 0100 A87B		B3	NẾU BIT (8, PK) = 0 THEN CMA (7B)
66	827E 0008 837E			GOTO CMC (7E)
67	886A0200A96A		B2	NẾU BIT (9, PK) = 0 THEN B4 (6A)
68	8876 0100 A876	8279		NẾU BIT (8, PK) = 0 THEN CLA (76)
69	0008 8379			GOTO CLC (79)
6A	8888 0100 A888	8287	B 4	NẾU BIT (8, PK) = 0 THEN HLT (88)
6B	0008 8387			GOTO NOP (87)
6C	886F0200A96F		B1	NẾU BIT (9, PK) = 0 THEN B5 (6F)
6D	888A 0100 A88A			NẾU BIT (8, PK) = 0 THEN EI (8A)
6E	828C 0008 838C			GOTO DI (8C)
6F	8873 0100 A873	0000	B5	NẾU BIT (8, PK) = 0 THEN INC (73)
70	0110 1080			A + COM (0) = A - 1 ==> BR
71	0040 E000 4075			BR ==> A, C, N, Z
72	828F 0008 838F	0000		GOTO PRE (8F)
73	0410 1010		INC	A + 1 ==> BR
74	0040 E000 4075			BR ==> A, C, N, Z
75	828F 0008 838F	0000		GOTO PRE (8F)
76	0200 0020		CLA	0 ==> BR
77	0040 C000 4035			BR ==> A, N, Z
78	828F 0008 838F	0001		GOTO PRE (8F)
79	0000 4080		CLC	0 ==> C

Địa chỉ nhà	Horizon		Bình luận	
	Microcommands.	Vert.	Nhãn mác	Hoạt động
7A	828F 0008 838F	0000		GOTO PRE (8F)
7B	0090 1040 0040	0000	CMA	COM (A) ==> BR, đảo ngược A
7C	828F 0008 838F	8280		BR ==> A, N, Z
7D	0001 8080 8279	0008		GOTO PRE (8F)
7E	8379 0002 0000	0000	CMC	NẾU BIT (0, PC) = 0 THEN B6 (80)
7F	0008 838F 0000	0000		GOTO CLC (79)
80	0040 E000 4075	828F	B6	1 ==> C
81	0008 838F 0000	0800		GOTO PRE (8F)
82	0004 0040 E000	0000	ROL	RAL (A) ==> BR dịch sang trái
83	0008 838F 0000	0008		BR ==> A, C, N, Z
84	8308 838F 0000	0008		GOTO PRE (8F)
85	4800 8201 0008	0000	ROR	RAR (A) ==> BR, sang phải
86	0000 4400 8201	0008		BR ==> A, C, N, Z
87	8301 Tiếp tục giải		NOP	GOTO PRE (8F)
88	mã chu kỳ tìm nạp		HLT	Máy dừng
89	và thực hiện cadệnh			GOTO BẮT ĐẦU (01)
8A	I / O		EI	Cho phép ngắt
8B				GOTO BẮT ĐẦU (01)
8C			DI	Tắt ngắt
8D				GOTO BẮT ĐẦU (01)
8E	0200 0000	4100	B / B Thiết lập quan	hệ với trường đại học
Chu kỳ gián đoạn				
8F	8288 0080	8788	TRƯỚC	NẾU BIT (7, PC) = 0 THEN HTL (88)
90	8201 0020	8501		NẾU BIT (5, PC) = 0 THÌ BẮT ĐẦU (01)
91	0000 0200	0020		0 ==> BR
92	0004 0000	4001		BR ==> RA
93	0000 0008	0300		SC ==> BR
94	0008 0000	4002		BR ==> RD
95	0100 0400	0012		RD ==> OP (RA), 1 ==> BR
96	0020 0000	4004		BR ==> SC
97	0800 0000	4400		Tắt ngắt
98	8201 0008	8301		GOTO BẮT ĐẦU (01)
Hoạt động của bảng điều khiển				
				Nhập địa chỉ
99	0000 0040	3000	BA	KR ==> BR
9A	0020 0000	4004		BR ==> SC
9B	828F0008	8388		GOTO TRƯỚC (88)
				Đọc
9C	0000 0008	0300	Thu	SC ==> BR
9D	0004 0000	4001		BR ==> RA
9E	0080 0408	0311		OP (RA) ==> RD, SC + 1 ==> BR
9F	0020 0000	4004		BR ==> SC
A0	828F0008	8388		GOTO TRƯỚC (88)
				ghi âm
A1	0000 0008	0300	ZAP	SC ==> BR
A2	0004 0000	4001		BR ==> RA
A3	0000 0040	3000		KR ==> BR
A4	0008 0000	4002		BR ==> RD
A5	0100 0408	0312		RD ==> OP (RA), SC + 1 ==> BR
A6	0020 0000	4004		BR ==> SC
A7	828F0008	8388		GOTO TRƯỚC (88)
				Bắt đầu
A8	0000 0200 0020		BẮT ĐẦU	0 ==> BR
A9	005C E000 4077	0400		BR ==> A, C, N, Z, RA, RD, RK
AA	0000 4200 0800	0400		Đặt lại cờ RT
AB				Tắt ngắt

Địa chỉ nhà	Horizon		Bình luận	
	Microcommands.	Vert.	Nhãn mác	Hoạt động
AC	828F 0008 838F			GOTO PRE (8F)
...				
B0			R - A	Lệnh số học 7 ###
...				
D0			R - P	Lệnh nhảy D ###
...				
E0			R - B	Lệnh không giải nén FC ##
...				
FF				

Trong tập lệnh của máy tính cơ sở, một số mã quang được dành riêng để bao gồm các lệnh mới. Đây là lệnh số học 7xxx, lệnh nhánh Dxxx và các lệnh không giải mã FC00, FD00, FE00 hoặc FF00. Khi lệnh được giải mã và lệnh 7xxx được chọn, điều khiển được chuyển đến dòng B0. Do đó, phần của vi chương trình mô tả chuỗi các hoạt động vi mô để thực hiện lệnh này phải bắt đầu bằng dòng B0. Ở đây, bạn có thể viết, ví dụ, các phép toán nhân hoặc chia các toán hạng. Phần của chương trình vi mô thực hiện các lệnh không địa chỉ bổ sung bắt đầu từ dòng E0. Các lệnh vi mô đầu tiên của phần này phải giải mã lệnh đã chọn (xác định phần mở rộng opcode: C, D, E hoặc F) và chuyển quyền điều khiển đến các lệnh vi mô tương ứng của đoạn vi chương trình mới. Vì bộ nhớ của các lệnh vi mô có 256 ô, nên chỉ các dòng có số từ AD đến FF mới có thể được sử dụng khi biên dịch phần sụn cho các lệnh mới.

Sự kết luận

Cuối cùng, hãy xem xét cái mà trên thực tế, chúng ta gọi là lập trình vi mô. Ý tưởng chính của cách tiếp cận được nêu ở trên là bắt đầu với một máy có dây cứng rất đơn giản (cấp phần sụn của máy tính cơ sở chỉ có hai lệnh đơn giản) và sử dụng nó để viết trình thông dịch cho một máy phức tạp hơn, được gọi là máy thông dịch. . Đến lượt nó, máy này có thể hoạt động như một thông dịch viên cho một máy ảo (có vẻ) khác, v.v.

4.5. Các tùy chọn khác để xây dựng vi lệnh

Phần 1.5 giải thích tại sao các lệnh ngắn lại tốt hơn

long: các lệnh như vậy tiết kiệm bộ nhớ và tăng số lượng lệnh có thể được tìm nạp từ bộ nhớ trong một

cho tôi một chút. Điều này cũng đúng ở cấp độ vi chương trình. Một lệnh vi mô với một bit trên mỗi mạch cổng (lệnh vi mô ngang) hấp dẫn do tính đơn giản của việc xây dựng và thực hiện (các lệnh vi mô và số không có trong lệnh vi mô đi trực tiếp đến các mạch cổng). Tuy nhiên, trong trường hợp một máy tính có kích thước tương đối lớn, số

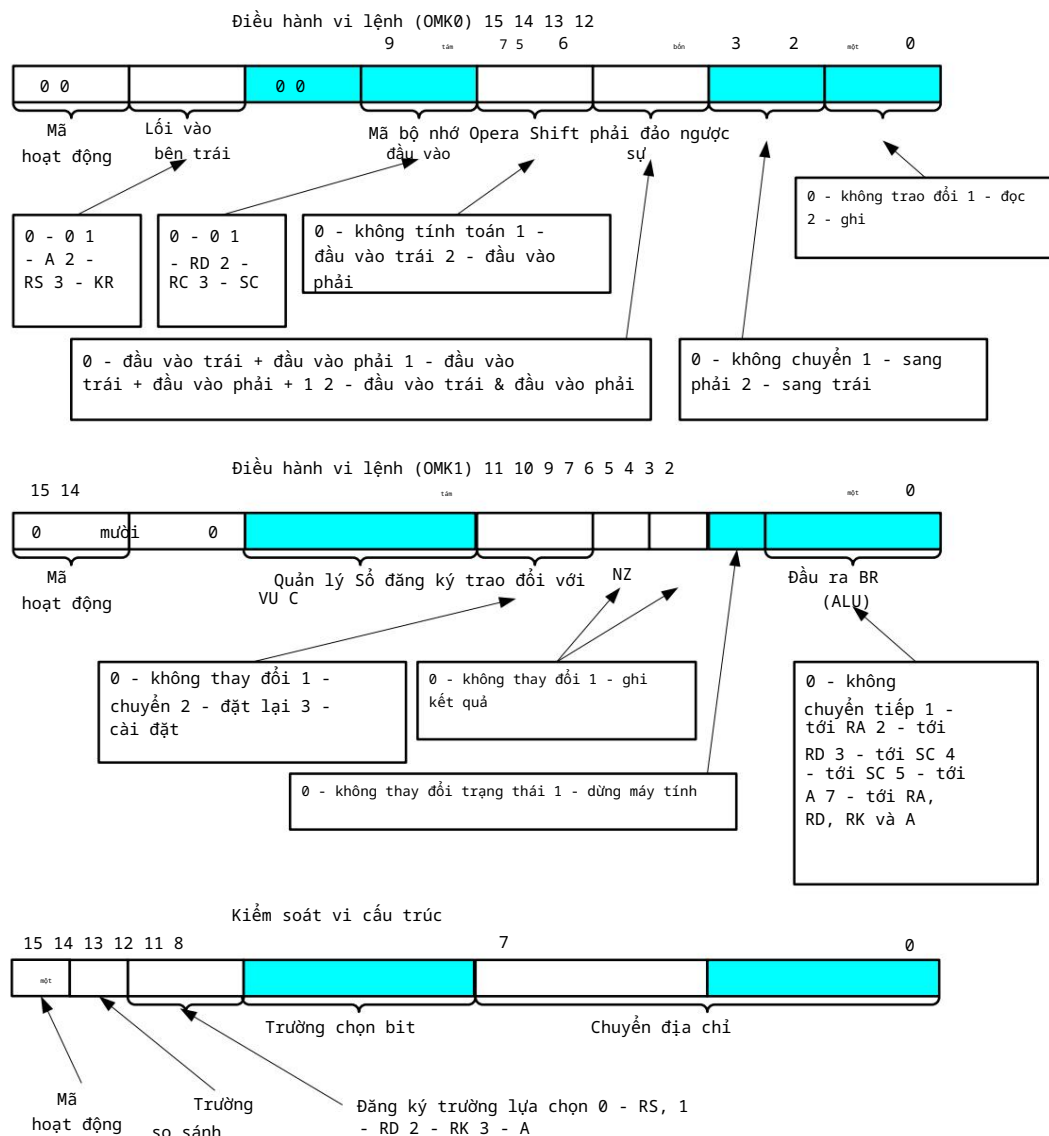
các mạch cổng có thể là vài trăm và vi cấu trúc ngang sẽ dài không thể chấp nhận được.

Các trường được mã hóa. Không phải tất cả các kết hợp có thể có của 32 bit đều có thể được sử dụng trong các lệnh vi mô của máy tính cơ sở được xem xét trước đó (xem Bảng 4.2). Vì vậy, các mạch van 1, 2 và 3; 1 và 19; 23 và 24 và nhiều nhóm khác không thể được mở cùng một lúc (xem hình 4.7). Mặc dù không có hạn chế cơ bản nào đối với các mạch cổng 18, 19, 20, 21 và 22 được mở cùng một lúc, nhưng trên thực tế, hiếm khi cần mở nhiều hơn một trong số chúng tại một thời điểm nhất định.

Không thể chấp nhận nhiều hơn một tín hiệu trên các đầu vào trái và phải của ALU, thiết lập tín hiệu cho các đầu vào ALU và sử dụng kết quả thu được trong cùng một chu kỳ đồng hồ, ghi đồng thời tổng các toán hạng và kết quả của phép nhân logic của chúng trong BR - tất cả điều này cho phép bạn chia vi cấu trúc hoạt động thành hai và giảm tổng số bit vi lệnh được sử dụng.

Trong vi lệnh điều khiển, việc giảm các bit của nó có thể đạt được bằng cách mã hóa các trường lựa chọn của thanh ghi đã kiểm tra và bit được kiểm tra trong thanh ghi này, điều này có thể làm giảm UMC xuống 16 bit, tức là giảm một nửa.

Một trong những biến thể có thể có của vi lệnh 16-bit của máy tính cơ sở được thể hiện trong hình. 4.10.



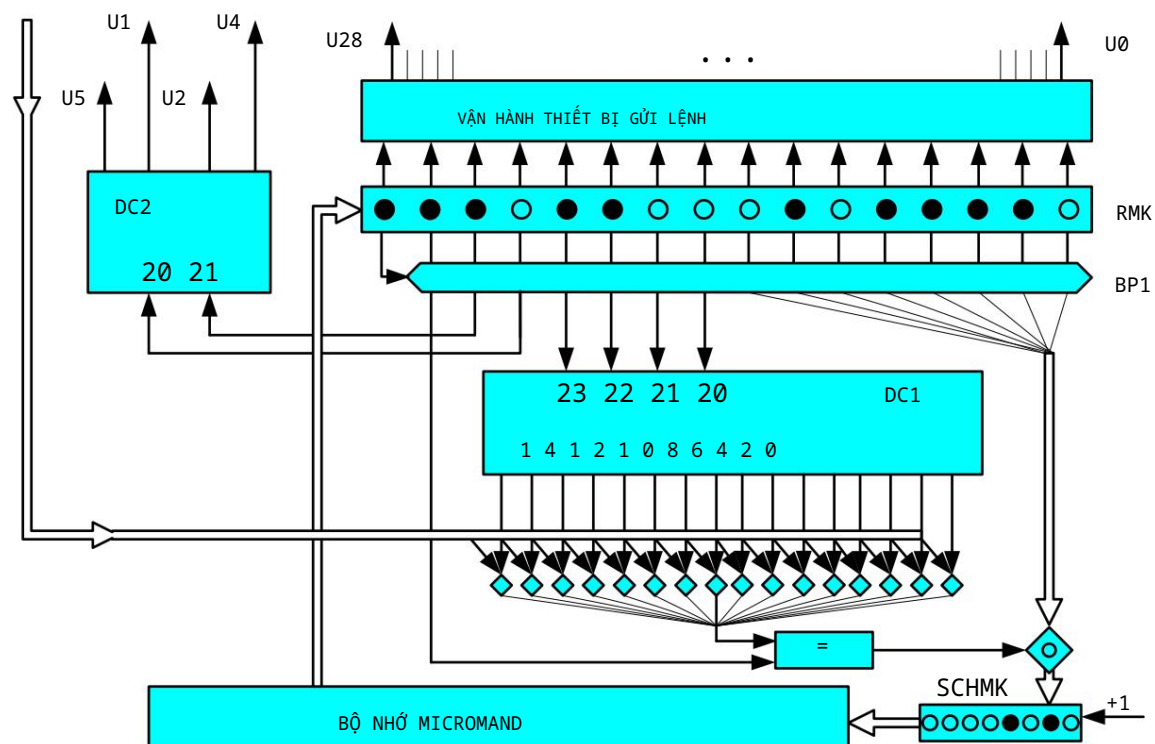
CƠ. 4.10. Các vi lệnh mười sáu-bit của máy tính chính

Ở đây hầu hết mọi thông tin đều được mã hóa. Ví dụ, trường lựa chọn của bit được kiểm tra của UMK mới nhỏ hơn bốn lần so với UMK cũ (xem Hình 4.8). Các lĩnh vực khác cũng bị giảm đi phần nào (lĩnh vực này càng dài thì tiết kiệm từ mã hóa càng lớn). Tuy nhiên, để giải mã các trường nén như vậy, cần phải sử dụng một lược đồ đặc biệt - bộ giải mã.

Bộ giải mã "1 trên $2n$ " là một mạch tổ hợp với n đầu vào và $2n$ đầu ra (Hình 4.11 cho thấy hai bộ giải mã: "1 trên 16", có 4 đầu vào và 24 đầu ra là 24 \ u003d 16 và "1 trên 4", có 2 đầu vào và 22 = 4 đầu ra). Mỗi dòng đầu ra của bộ giải mã tương ứng duy nhất với một trong $2n$ sự kết hợp có thể có của tín hiệu đầu vào.

Trên hình. 4.11 cho thấy một phần của một trong các biến thể của thiết bị điều khiển phần sụn của máy tính cơ sở tại thời điểm này

xử lý vi lệnh điều khiển 16 bit ES5E - IF BIT (12, RK) - 1 THEN BAD (5E).



Cơ. 4.11. Một phần của thiết bị điều khiển phân sụn với 16-bit vi lệnh

Giá trị (10) 2, nằm trong trường lựa chọn của thanh ghi đã kiểm tra (12-13 bit của UMK), tạo ra, đi qua bộ giải mã DC2, một tín hiệu điều khiển duy nhất U2. Tín hiệu này mở ra mạch cổng B2 (xem Hình 4.7), và nội dung của RK được ghi vào thanh ghi đệm (BR). Giá trị (1100) 2, nằm trong trường lựa chọn của bit được kiểm tra (bit thứ 8-11 của UMK), tạo ra một tín hiệu điều khiển duy nhất ở đầu ra thứ 12 của bộ giải mã DC1, được sử dụng để trích xuất và gửi đến so sánh thiết bị là bit thứ 12 của BR, tức là bit thứ 12 của RK. Vì 1 được viết trong trường so sánh một bit, nên với một giá trị duy nhất của bit thứ 12 của RK, địa chỉ của quá trình chuyển đổi 5E (bit 0-7 của UMK) được viết lại trong bộ đếm vi lệnh (SchMK).

Để giải mã các vi lệnh OMK0 và OMK1 (xem Hình 4.10), sẽ cần thêm tám bộ giải mã nữa. Tuy nhiên, chi phí tiết kiệm trong bộ nhớ vi lệnh (bộ nhớ này giảm một nửa do sử dụng vi lệnh 16-bit) có thể sẽ vượt quá chi phí của các bộ giải mã này.

Văn bản của trình thông dịch của máy tính cơ sở, được viết bằng bảng. Các vi lệnh 16-bit, được cung cấp trong 4.3 (cột

THEO CHIỀU DỌC). Tên của biểu đồ là do thực tế là các vi lệnh bao gồm một số lượng lớn các trường được mã hóa thường được gọi là dọc, trái ngược với vi lệnh ngang, trong đó mỗi bit điều khiển một mạch cổng.

Chia nhịp thành các giai đoạn. Dung lượng bộ nhớ vi chương trình cũng có thể được giảm bớt bằng cách sử dụng các vi lệnh ngang. Để làm được điều này, cần phải tăng dung lượng thông tin của chúng: mã hóa một số lượng lớn hơn các hoạt động vi mô trong mỗi lệnh vi mô. Việc thực hiện các vi hoạt động không tương thích về thời gian có thể được đảm bảo bằng cách chia chu kỳ làm việc của máy tính (thời gian thực hiện một lệnh vi mô) thành nhiều chu kỳ con (giai đoạn), trong mỗi chu kỳ đó một hoặc nhiều hành động cơ bản được thực hiện. để thực hiện lệnh vi mô.

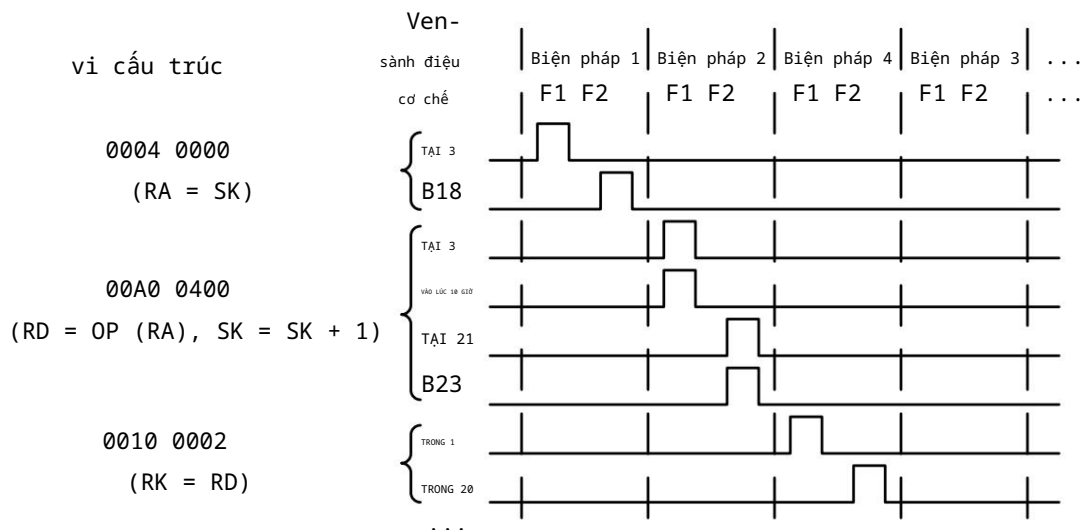
Trên hình. 4.12. ba lệnh vi mô đầu tiên của chu kỳ máy "Lựa chọn lệnh" của máy tính cơ sở và trình tự thực hiện của chúng trong quá trình đồng hồ hai pha (pha F1 và F2) được đưa ra. Đối với các vi hoạt động mới (nhiều dung lượng hơn) này, các hoạt động vi mô tương tự được thực hiện như đối với sáu vi hoạt động đầu tiên trong Table. 4.3.

Nanomemory. Số lượng vi lệnh ngang có nội dung khác nhau thường ít, nhưng một số trong số chúng được sử dụng lặp đi lặp lại (ví dụ, $RA = RK$, $SK = CK + 1$, $SK = RK$ và các lệnh khác của máy tính cơ sở). Tình huống này, cũng như độ dài lớn của các vi lệnh như vậy, đã thúc đẩy các nhà phát triển máy tính nghĩ đến việc chỉ đặt các vi lệnh không lặp lại trong một bộ nhớ bổ sung đặc biệt (nanomemory). Vi chương trình trong trường hợp này sẽ chỉ bao gồm

điều khiển các lệnh vi mô và địa chỉ của các tế bào nano.

Vì số lượng tế bào nanomet nhỏ nên địa chỉ của chúng có độ sâu bit nhỏ, cho phép giảm độ dài của các ô nhớ vi lệnh nơi đặt các địa chỉ như vậy. Trong một số trường hợp, điều này làm giảm đáng kể tổng dung lượng bộ nhớ, mặc dù sự xuất hiện của

loại bộ nhớ mới - nanomemory.



Com. 4.12. Bắt đầu một chuỗi các hoạt động vi mô bằng cách chọn lệnh

Giảm số lượng và dung lượng của các vi lệnh điều khiển. Mỗi vi lệnh điều khiển của máy tính cơ sở chỉ kiểm tra một trong các bit của thanh ghi được chỉ định (xem đoạn 4.3). Do đó, để giải mã một lệnh hoặc xác định một ô chỉ mục, đôi khi cần phải sử dụng chuỗi dài các CCU như vậy (hơn 20 CCU).

Có một số cách để giảm số lượng MCU:

- sửa đổi địa chỉ có trong UMC bằng cách thực hiện một thao tác logic nhất định trên nó và một số trường đã cho đăng ký.

Ví dụ, điều này làm cho nó có thể chỉ sử dụng một CCU để giải mã opcode lệnh thay vì một chuỗi 15-20 CCU;

- kiểm tra sự kết hợp các bit của một thanh ghi đã cho.

Ví dụ, điều này làm cho nó có thể chỉ sử dụng một CCU để xác định một ô chỉ mục thay vì một chuỗi 8 CCU.

Giảm độ sâu bit của CMD chủ yếu đạt được bằng cách thay thế địa chỉ chuyển tiếp bằng một độ lệch ngắn hơn được thêm vào hoặc trừ đi từ địa chỉ CMD hiện tại và tạo ra một số định dạng CMD (CMD chuyên biệt). Điều này giúp bạn có thể lấy các UMK được đặt trong các ô nhớ UMK một byte, điều này cực kỳ hữu ích trong

trường hợp sử dụng nanomemory.

Các lệnh vi mô của các máy tính công nghiệp khác nhau thường đại diện cho sự kết hợp của các cấu trúc ngang và dọc với việc thực hiện các phương pháp giảm thiểu chương trình vi mô nhất định.

kỉ niệm.

5. MÔ PHỎNG CHỨC NĂNG CỦA VI MÔ

5.1. Cách tiếp cận mô hình trong nghiên cứu các thiết bị vi xử lý

Công nghệ

Hiểu biết sâu sắc về nguyên lý cấu tạo và hoạt động của các hệ thống phức tạp (là các thiết bị của công nghệ vi xử lý) chỉ đạt được thông qua công việc thực tế với các hệ thống đó hoặc mô hình của chúng. Ngay cả mô tả chi tiết nhất về công việc với máy tính cơ sở (xem đoạn 3.1) và các minh họa chi tiết về quá trình thực hiện các lệnh (ví dụ, xem Hình 2.2) cũng không thể thay thế công việc thực tế trên máy tính này, được trang bị điều khiển và chỉ dẫn đặc biệt (xem, ví dụ, Hình 3.1 và 4.7).

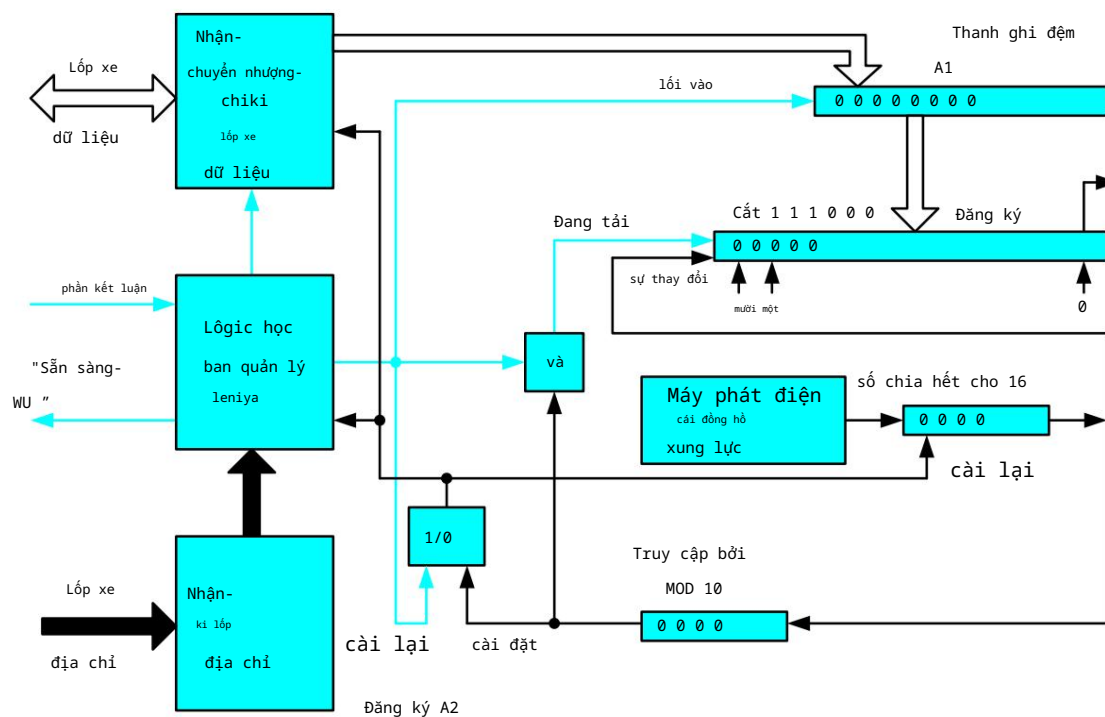
Có vẻ như với mức độ rẻ tương đối và sự phân bố rộng rãi của các thiết bị công nghệ vi xử lý, khá dễ dàng để tạo ra các giá đỡ cho việc nghiên cứu nguyên lý hoạt động của các thiết bị máy tính và điều khiển này. Tuy nhiên, việc thiếu quyền truy cập vào hầu hết các thành phần của thiết bị vi xử lý (thanh ghi, bộ cộng, mạch cổng, v.v.) nằm bên trong một hoặc nhiều LSI dẫn đến nhu cầu sử dụng các mô hình được thiết kế đặc biệt cho các mục đích: vật lý hoặc chức năng.

Mô hình vật lý là một giá đỡ có cấu trúc của thiết bị được nghiên cứu bằng công nghệ vi xử lý, được xây dựng trên các mạch tích hợp có mức độ tích hợp vừa và nhỏ và được trang bị các phương tiện để điều khiển các chế độ hoạt động của thiết bị được nghiên cứu, cũng như chỉ thị trạng thái của bất kỳ thành phần nào của nó. Do sự phát triển nhanh chóng của công nghệ vi xử lý, việc sản xuất hàng loạt các giá đỡ chuyên dụng như vậy là không thực tế. Không có lợi nhuận về mặt kinh tế và quy mô nhỏ sản xuất.

Mô hình chức năng là một máy vi tính, trong đó một chương trình được giới thiệu mô phỏng hoạt động của một thiết bị của công nghệ vi xử lý đang nghiên cứu. Mệnh lệnh được đưa ra từ bảng điều khiển được thiết kế đặc biệt hoặc, ví dụ, từ bàn phím máy tính vi mô tiêu chuẩn. Trạng thái của các thành phần của thiết bị đang nghiên cứu được hiển thị trên các chỉ số của một bảng điều khiển đặc biệt hoặc trên màn hình của máy vi tính. Do đó, với tư cách là một công cụ kỹ thuật để mô hình hóa chức năng của các thiết bị kỹ thuật số, người ta có thể

sử dụng máy tính cá nhân, ví dụ, máy tính gia đình "Electronics BK-0010" [1] - một hộp nhỏ có bàn phím (370x180x70 mm), được kết nối với TV gia đình (đen trắng hoặc màu) và máy ghi cassette gia đình.

Nếu có một chương trình mô phỏng cho bất kỳ thiết bị kỹ thuật số nào, được biên dịch bằng ngôn ngữ của máy tính cá nhân "Điện tử BK 0010" và được ghi trên băng từ cassette, thì sau khi nhập băng như vậy vào máy, cấu trúc của thiết bị đang nghiên cứu sẽ được vẽ trên màn hình TV, ví dụ, cấu trúc của bộ điều khiển đầu ra nối tiếp không đồng bộ (Hình 5.1). Ngoài ra, màn hình sẽ hiển thị (dưới dạng ký hiệu 0 và 1) trạng thái của các thanh ghi và bộ đếm của bộ điều khiển, cũng như các tín hiệu trên các bus dữ liệu và điều khiển.



Cơ. 5.1. Bộ điều khiển đầu ra nối tiếp không đồng bộ

Đồng thời, các phím bàn phím máy tính nhất định sẽ trở thành cơ quan thay đổi thông tin trên bus dữ liệu (ví dụ: phím "0", "1", "7") và dòng "Đầu ra" (ví dụ: "B " Chìa khóa). Bằng cách thiết lập nội dung của byte dữ liệu đầu ra (nội dung của thanh ghi đệm A1) với các phím "0", "7", bạn có thể (bằng cách nhấn nút "B") cung cấp cho bộ điều khiển một tín hiệu để chuyển đổi byte này thành một chuỗi các bit và chuyển chúng (cùng với các bit bắt đầu và dừng) sang VU. Những hành động này có thể dễ dàng theo dõi bằng sự thay đổi (trên màn hình TV)

nội dung của thanh ghi và bộ đếm của bộ điều khiển mô phỏng.

Có thể dễ dàng nhận thấy rằng các mô hình chức năng vô cùng phổ quát: cùng một phương tiện kỹ thuật giúp bạn có thể nghiên cứu quá trình chuyển đổi thông tin trong bất kỳ thiết bị kỹ thuật số nào và với bất kỳ mức độ chi tiết nào. Để nghiên cứu một thiết bị mới, chỉ cần thay đổi chương trình mô hình hóa, chương trình hiển thị hình ảnh cấu tạo của thiết bị mô phỏng trên màn hình, chương trình hiển thị trạng thái của các thành phần của thiết bị này. Hạn chế chính của mô hình chức năng - không thể nghiên cứu hoạt động của mô hình gốc với tín hiệu đầu vào thực, tải và nhiễu - cũng cố hữu trong các mô hình vật lý, vì mô hình sau được xây dựng dựa trên các phần tử khác (so với mô hình gốc), theo một cách khác thiết kế và các thiết bị phụ được kết nối với chúng (trạng thái mạch chỉ báo của các thành phần của bản gốc).

Phù hợp với mục đích của nó, mô hình chức năng phải phản ánh chính xác và đầy đủ trạng thái của chỉ các thành phần được hiển thị của bản gốc và chỉ tại các điểm được hiển thị trong thời gian. Do đó, mô hình hóa chức năng của các thiết bị kỹ thuật số đơn giản nhất có thể được thực hiện bằng cách sử dụng một chương trình, theo tín hiệu của bộ điều khiển (ví dụ: bàn phím của máy tính cá nhân), chọn từ bộ nhớ và "in" trên màn hình TV tương ứng.

trạng thái của các thành phần của bản gốc.

Do đó, khi tạo một mô hình chức năng, trước hết người ta phải quan tâm đến các phương pháp nhập tín hiệu điều khiển và xuất thông tin về các trạng thái của thiết bị mô phỏng, tức là về các yếu tố xác định mức độ cần thiết và mức độ hoàn chỉnh của nghiên cứu. Phần tương tự của phần mềm mô hình

trạng thái hiện tại của nó, sự hiện diện của các tín hiệu bắt đầu

quá trình chuyển đổi sang trạng thái tiếp theo và nội dung mới của các thành phần được chỉ định của bản gốc được xác định, có thể được xây dựng theo bất kỳ cách nào thuận tiện (hoặc có thể truy cập) cho nhà phát triển.

5.2. Tương tác với mô hình chức năng

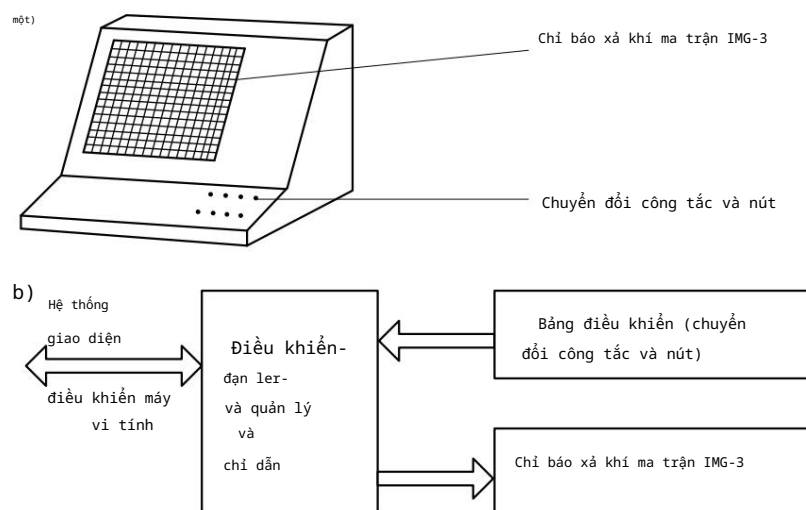
Nếu mô hình chức năng của thiết bị kỹ thuật số được thực hiện bằng máy tính cá nhân, thì việc tương tác với mô hình đó là dễ dàng và rẻ nhất để tổ chức thông qua bàn phím của máy tính này và màn hình của nó. Tương tác với các mô hình được xây dựng trên hệ thống tính toán đa cấp (trong lớp hiển thị) có thể được tổ chức theo cách tương tự. Tuy nhiên, con đường này có một số nhược điểm.

1. Màn hình áp dụng các hạn chế nghiêm trọng về âm lượng thông tin đầu ra. Ví dụ, khi nghiên cứu nguyên lý bất đồng bộ trao đổi thông tin tuần tự trên màn hình phải được hiển thị cấu trúc của không chỉ máy phát (xem Hình 5.1), mà còn cả máy thu thông tin, điều này rất khó thực hiện, vì màn hình có thể hiển thị mỗi dòng chỉ có 24 dòng gồm 64 hoặc 80 ký tự. Ngoài ra, nhiều máy tính đang sử dụng chỉ có thể hiển thị các chữ cái, số và ký hiệu màu đen và trắng, làm giảm đáng kể khả năng hiển thị của các mô hình đã tạo.

2. Sử dụng bàn phím PC thông thường (bàn phím đi kèm với màn hình) khiến việc giao tiếp với kiểu máy trở nên khó khăn, vì bạn phải nhớ chức năng của một phím cụ thể được lập trình cho chức năng gì. Tình hình có phần khả quan hơn trong trường hợp bàn phím có một số phím chức năng được lập trình để thực hiện các thao tác mong muốn.

Do đó, chỉ một bảng điều khiển được chế tạo đặc biệt cho mô hình và chỉ báo các trạng thái của nó sẽ cho phép tổ chức tương tác hiệu quả nhất với mô hình và thu được tất cả thông tin cần thiết về hoạt động của nó.

Trên hình. 5.2, a cho thấy sự xuất hiện và trong hình. 5.2, b - cấu trúc sơ đồ của bảng điều khiển và chỉ dẫn được sử dụng để nghiên cứu hoạt động của máy tính cơ sở. Điều khiển từ xa bao gồm một bộ công tắc bật tắt và nút (xem Hình 3.1), một chỉ báo xả khí ma trận (IMG-3), một bộ điều khiển để kết nối bàn phím và một chỉ báo với một máy tính điều khiển (mô phỏng).



Cơ. 5.2. Bảng điều khiển và chỉ báo của các mô hình máy tính vi mô chức năng:

a - ngoại hình; b - sơ đồ khối

IMG-3 là bảng phóng điện có thiết bị điều khiển có thể đốt cháy bất kỳ phần tử hình ảnh nào trong số 4096 (64X64) (EI) có kích thước 3X3 mm, nằm cách nhau 3 mm. Một mặt nạ có cấu trúc của máy tính cơ sở (xem Hình 3.1) hoặc thiết bị điều khiển vi chương trình của nó (xem Hình 4.6) được chồng lên bảng điều khiển. Trong hình ảnh của các thanh ghi và các thiết bị khác của các mặt nạ này, các lỗ tròn được đục lỗ, nằm phía trên bảng EI. Chương trình mô phỏng làm sáng một số EI với màu xanh lá cây tươi sáng, tương ứng với các trạng thái đơn lẻ của phóng điện nhất định của các thiết bị máy tính cơ sở.

Bàn điều khiển như vậy có thể được sử dụng thành công để nghiên cứu hoạt động của bất kỳ máy vi tính nào. Nó cho phép bạn hiển thị nội dung của các thiết bị riêng lẻ cho các cấu trúc khá phức tạp. Các điều khiển của bảng điều khiển tương tự như các điều khiển của một máy tính mini, điều này tạo ra ảo giác giao tiếp với một máy tính dựa trên phần cứng chứ không phải với mô hình phần mềm của nó. Tuy nhiên, việc sản xuất một bàn điều khiển như vậy và các thiết bị để kết nối với máy tính mô hình hóa là một công việc khá tốn công sức. Do đó, một số kỹ thuật sẽ được xem xét dưới đây cho phép sử dụng VU tiêu chuẩn của máy tính mô hình hóa - bàn phím và màn hình - để điều khiển hoạt động của mô hình và chỉ ra các trạng thái của nó.

Công tắc và nút. Là một phần của bàn phím tiêu chuẩn mô hình hóa máy tính cá nhân không có hai hoặc nhiều công tắc vị trí. Do đó, để mô phỏng các công tắc như vậy của hệ thống kỹ thuật số mô phỏng, cần phải sử dụng các nút (phím không cố định vị trí cơ học) và một phần của màn hình hiển thị của máy tính mô hình.

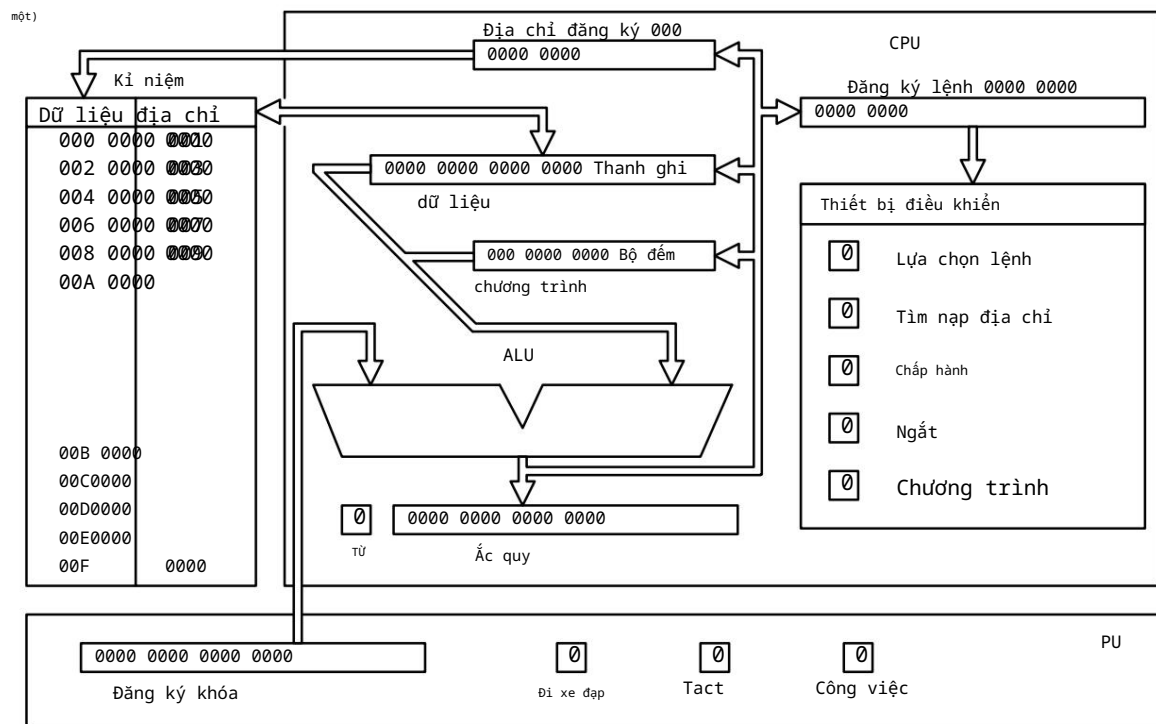
Sơ đồ chuyển mạch được vẽ trên màn hình hoặc các ký hiệu đặc trưng cho trạng thái hiện tại của nó và mỗi lần nhấn phím (nút) tương ứng sẽ bắt đầu thay đổi trạng thái của công tắc và hình ảnh của nó trên màn hình. Thay đổi trạng thái của công tắc mô phỏng được thực hiện theo chương trình: sau khi nhận được tín hiệu từ phím được nhấn, chương trình sẽ phân tích "vị trí" hiện tại của nó và thay đổi nó thành ngược lại (công tắc hai vị trí), hoặc, ví dụ, sang vị trí tiếp theo theo thứ tự (công tắc đa vị trí).

Mô hình đầu tiên của máy tính cơ sở được thực hiện trên máy tính cá nhân của những năm 80 của thế kỷ trước - Iskra 226 (Hình 5.3)

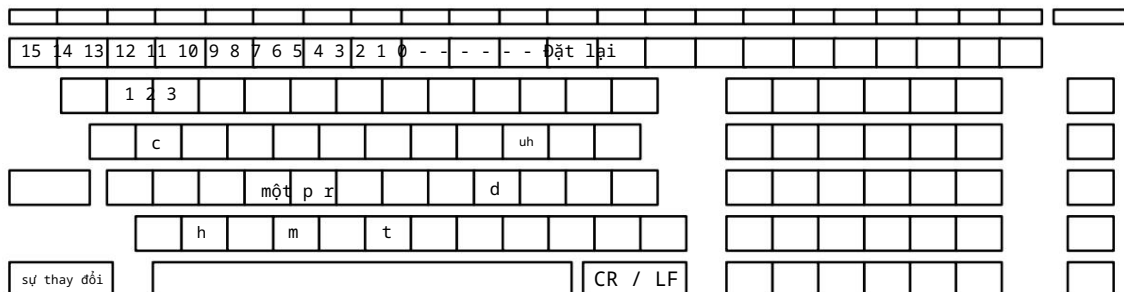


Cm. 5.3. Cá nhân chuyên nghiệp "Iskra 226" [4]

Để đặt địa chỉ của ô nhớ hoặc nhập dữ liệu vào mô hình máy tính cơ sở được xây dựng trên Iskra 226, các phím chức năng của bàn phím của máy tính cá nhân này được sử dụng (Hình 5.4, b).



b)



Cm. 5.4. Các công cụ máy tính Iskra 226 được sử dụng để làm việc với mô hình máy tính cơ bản:

a - màn hình với cấu trúc bộ xử lý; b - bàn phím

Sử dụng từng phím, bạn có thể thay đổi trạng thái của một bit trong thanh ghi phím được hiển thị trên màn hình hiển thị (Hình 5.4, a). Nếu bất kỳ khóa nào ở "vị trí 0" (trong ký hiệu "0" được hiển thị trong chữ số tương ứng của thanh ghi phím), sau đó sau khi nhấn nó một lần, phím sẽ được chuyển sang "vị trí 1" (ký hiệu "0" sẽ được thay thế bằng ký hiệu "1"). Lần nhấn tiếp theo của cùng một phím sẽ di chuyển nó đến "vị trí 0", tức là nó sẽ đảo ngược lại bit tương ứng của thanh ghi bàn phím. Các đoạn của chương trình mô phỏng được sử dụng để thực hiện các hoạt động được mô tả có dạng sau:

```

10 DIM R=1, K=2, K= (16) 1, ...
15 MÔ TẢ TỪ XA VỀ BIẾN TẦN LÀM VIỆC R, CŨNG NHƯ BIẾN SỐ BINARY K VÀ MỘT SẮP XẾP ĐẶC
TRUNG K ( ) ĐƯỢC SỬ DỤNG ĐỂ LƯU TRỮ NỘI DUNG HIỆN TẠI CỦA BỘ ĐĂNG KÝ CHỈ NH ("ĐỊA
CHỈ / DỮ LIỆU")

200 KEYIN R, 240,210: GOTO 300
205 TRẠNG THÁI BÀN PHÍ M KIỂM TRA REM

210 NẾU "R> HEX (0F) THÌ 228: GOSUB '50: GOTO 300
215 REM, THAM KHẢO CHƯƠNG TRÌNH SUB "ĐĂNG KÝ CHỈ NH",
KHI MỘT TRONG 16 CHÌA KHÓA ĐẶC BIỆT CÓ MÃ 00 - FF ĐƯỢC ÉP
.
1000 DEFF N'50
1005 REM SUBPROGRAM "ĐĂNG KÝ CHỈ NH"
1810 I = VAL (R) + 1
1015 PHÁT HIỆN SỐ TỪ KHÓA TỪ XA
1020 NẾU K (I) = "0" THEN 1030: K (I) = "0": GOTO 1040
1030 K (I) = "1"
1835 BỎ QUA PHẦN TỬ 1 CỦA ARRAY K ( )
1040 CHO J = 1 TO 2: R = 0: FOR I = 1 TO 8: NẾU K (I) = "0" THÌ 1050: R = R + 2^(0-1)
1050 TIẾP THEO I: BIN (STR (K, J, 1)) = R: TIẾP THEO J
1055 ĐỊNH NGHĨA NHỎ VỀ GIÁ TRỊ MỚI CỦA BIẾN SỐ CHỈ NH K

1060 IN AT (22,0): CHO J = 1 TO 4: FOR I = 1 TO 4: IN K ((J1) * 4 + 1);
TIẾP THEO I: IN "": TIẾP THEO J
1065 NỘI DUNG ĐĂNG KÝ TỪ KHÓA REM

1070 TRỞ LẠI

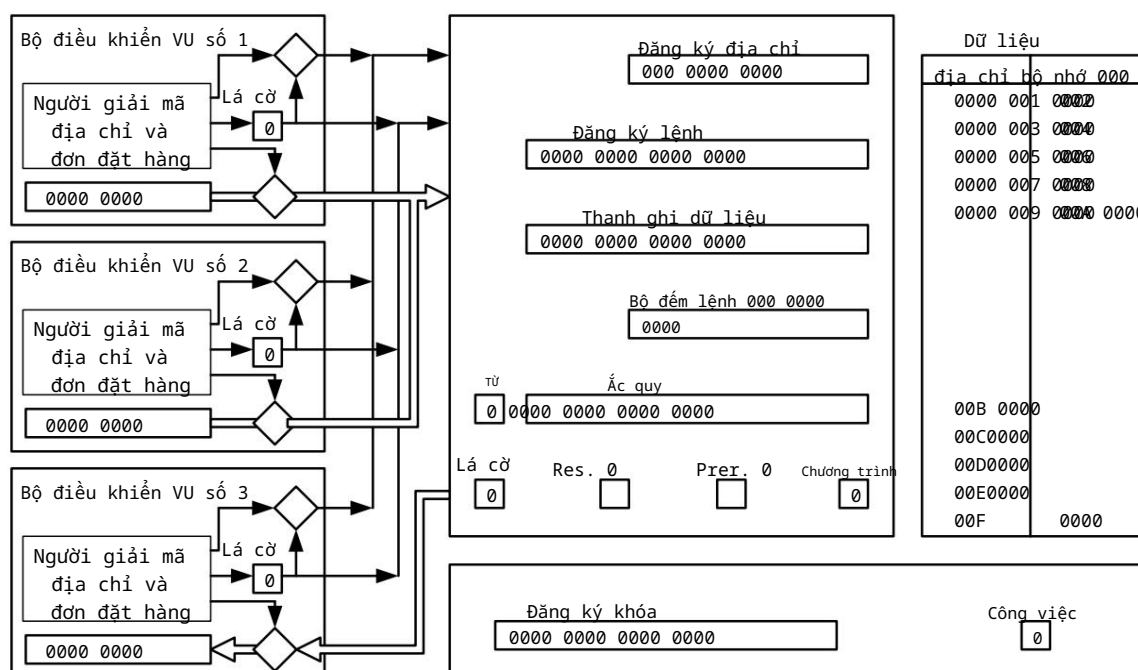
```

Chỉ định. Hiệu quả của việc nghiên cứu hoạt động của một hệ thống điều khiển máy tính kỹ thuật số phần lớn phụ thuộc vào hình thức trình bày thông tin về trạng thái của các thiết bị của nó trong

thời điểm mà nhà nghiên cứu quan tâm. Do đó, việc xây dựng một mô hình nên bắt đầu bằng việc tạo ra một sơ đồ cấu trúc đủ chi tiết của hệ thống đang nghiên cứu và một danh sách các thiết bị của nó và các kết nối giữa chúng, trạng thái sẽ được hiển thị ở mỗi bước được hiển thị của hệ thống. Nếu hình ảnh của tất cả các thiết bị và kết nối này không thể được đặt trên màn hình hiển thị, thì cần phải đơn giản hóa phần nào cấu trúc hiển thị hoặc tiếp tục nghiên cứu theo từng giai đoạn của nó. Loại thứ hai thậm chí còn thích hợp hơn, vì một cấu trúc quá chi tiết rất khó xử lý.

sự nhận thức.

Ví dụ, màn hình của máy tính cá nhân Iskra 226 không cho phép đặt hình ảnh của máy tính cơ sở được hiển thị trong hình. 3.1. Do đó, việc nghiên cứu các phương pháp trao đổi dữ liệu giữa bộ xử lý và các thiết bị bên ngoài của máy tính cơ sở được thực hiện theo cấu trúc như trong Hình. 5.5, khác với cấu trúc trong hình. 5.4, được sử dụng để nghiên cứu hoạt động của bộ xử lý của máy tính cơ sở.

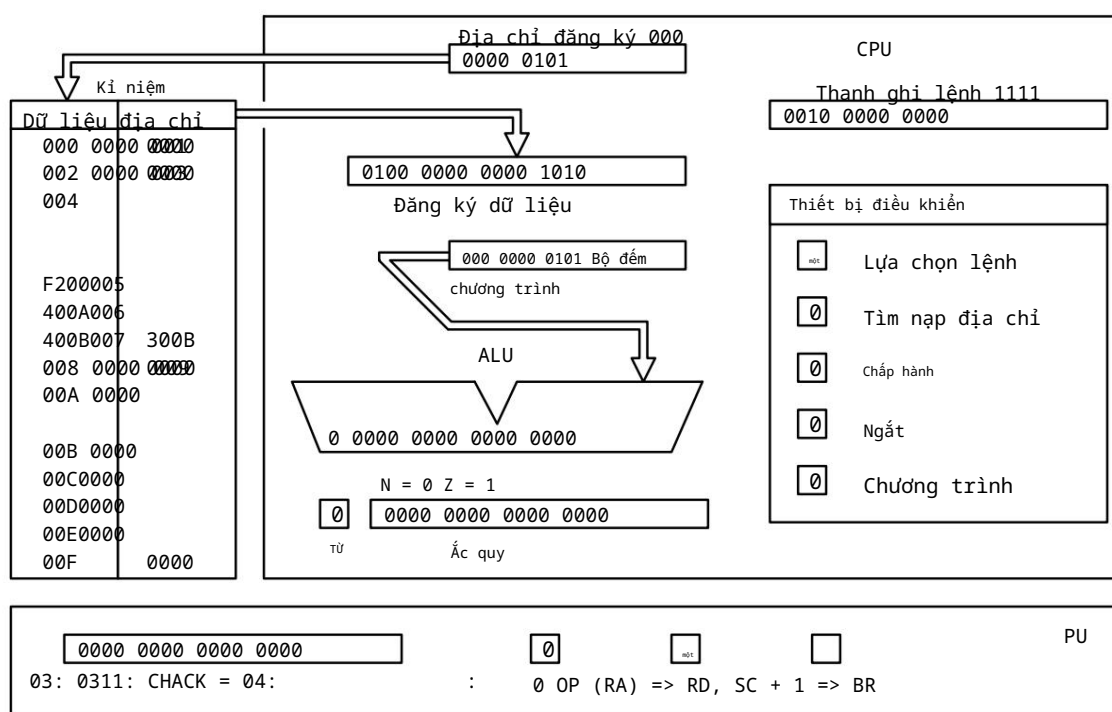


Com. 5.5. Màn hình của máy tính Iskra 226 với cấu trúc được sử dụng trong nghiên cứu trao đổi dữ liệu giữa bộ xử lý và các thiết bị bên ngoài của máy tính cơ sở

Trong trường hợp không có màn hình biểu tượng-đồ họa, cấu trúc của hệ thống kỹ thuật số đang nghiên cứu có thể được mô tả trên một mặt nạ chồng lên màn hình hiển thị biểu tượng. Mặt nạ như vậy tốt nhất nên được vẽ trên phim trong suốt, nhưng cũng có thể trên giấy, trong đó các lỗ sau đó được cắt để chỉ ra nội dung của các thiết bị của hệ thống đang nghiên cứu.

Tốt nhất là hiển thị nội dung này dưới dạng trạng thái của các chữ số riêng lẻ của các thiết bị được hiển thị. Trạng thái này có thể là "" (khoảng để hiển thị, ví dụ, các ký hiệu để giảm độ phức tạp của cấu trúc được hiển thị, cũng có thể xuất ra dưới dạng số thập lục phân (đây là cách nội dung của các ô nhớ của máy tính cơ sở được hiển thị trong Hình 5,4 và 5,5).

Hoạt động theo chu kỳ của hệ thống được minh họa rõ ràng nhất bằng cách chỉ ra các kết nối được thiết lập giữa các thiết bị riêng lẻ của hệ thống ở mỗi trạng thái của nó. Việc nghiên cứu điều khiển vi chương trình của máy tính cơ sở được tổ chức theo cách tương tự (Hình 5.5).



Cơ. 5.5. Màn hình của máy tính Iskra 226 với cấu trúc được sử dụng trong nghiên cứu điều khiển vi chương trình của máy tính cơ sở

5.3. Xây dựng mô hình chức năng

Mục đích của mô hình chức năng (xem Phần 5.1) xác định trước các chế độ hoạt động chính của nó: chế độ từng bước và chế độ thực thi khá chậm các chương trình nhỏ, trong đó có thể theo dõi bản chất của sự thay đổi trong trạng thái của tất cả các thiết bị được hiển thị của hệ thống máy tính mô phỏng. Do đó, tiêu chí "thời gian chạy tối thiểu" không phải là tiêu chí chính trong việc phát triển các chương trình mô hình. Các chương trình như vậy có thể được viết bằng ngôn ngữ cấp cao ngay cả khi chúng được thông dịch trên máy tính mô phỏng trong quá trình hoạt động.

Nên bắt đầu tạo một mô hình chức năng với các chương trình cung cấp sự tương tác với nhà nghiên cứu: các chương trình xử lý tín hiệu điều khiển và nhập dữ liệu từ bàn phím của máy tính mô hình hóa, cũng như các chương trình vẽ cấu trúc đang nghiên cứu và xuất ra các trạng thái của nó. các thiết bị.

Các yêu cầu ít nghiêm ngặt nhất được đặt ra đối với các chương trình vẽ cấu trúc đang được nghiên cứu, vì hoạt động này được thực hiện trước khi bắt đầu nghiên cứu và có thể mất một khoảng thời gian khá dài (hàng chục giây). Theo quy định, khi tạo các chương trình như vậy cho máy tính cá nhân, không có giới hạn nghiêm trọng nào về dung lượng bộ nhớ được sử dụng (không cần lưu nó ở chế độ hoạt động một chương trình của máy tính).

Các chương trình xử lý tín hiệu điều khiển và nhập dữ liệu (lệnh) từ bàn phím của máy tính mô hình hóa phải được thiết kế và được đặt giữa các chương trình khác của tổ hợp mô hình sao cho khoảng thời gian giữa việc nhấn phím bất kỳ và sự thay đổi tương ứng trong mô hình cấu trúc của hệ thống đang nghiên cứu không vượt quá một phần giây. Trong những trường hợp khi mô hình cần một khoảng thời gian dài hơn để xử lý đơn đặt hàng được đưa ra từ bàn phím, nên bằng cách nào đó thông báo cho nhà nghiên cứu rằng đơn đặt hàng đã được nhận và mô hình đã bắt đầu thực hiện. Điều này có thể được thực hiện bằng cách tô sáng stencil.

Ví dụ: trong kiểu máy tính cơ bản, sau khi nhấn các phím mô phỏng hoạt động của các nút "NHẬP ĐỊA CHỈ", "VIẾT", "ĐỌC", "BẮT ĐẦU" và "TIẾP TỤC", các chữ khắc tương ứng được hiển thị trên nền trắng bên cạnh hình ảnh của thanh ghi khóa: "ADDR.", "REC", "READ", "START" và "WAIT". Giấy nền ngừng hiển thị sau khi hoàn thành tất cả các thao tác xử lý đơn đặt hàng và hiển thị trên màn hình

trạng thái mới của mô hình.

Đầu ra của các trạng thái thiết bị của hệ thống đang nghiên cứu có thể là nhận ra:

- khi những trạng thái này thay đổi;
- sau khi hoàn thành tất cả các hành động để thực hiện một bước nhất định (ví dụ: các lệnh hoặc lệnh vi mô);
- ở chế độ hỗn hợp.

Phương pháp đầu tiên chỉ thích hợp cho các mô hình đơn giản, trong đó ở mỗi bước nhất định, một thay đổi được thực hiện

trạng thái của bất kỳ thiết bị nào. Nếu không, thời gian chạy bước (hoặc trình tự các bước) sẽ tăng lên đáng kể, vì thông tin in (ngay cả trên màn hình hiển thị) là thao tác mô hình hóa dài nhất.

Tuy nhiên, không phải lúc nào cũng có lợi khi nhớ thiết bị nào đã thay đổi trạng thái của chúng ở một bước cụ thể để hiển thị nội dung mới của chúng ở cuối bước. Đôi khi, sẽ có ý nghĩa hơn khi hiển thị nội dung của tất cả các thiết bị trong mô hình ở cuối bước, bất kể

liệu họ có thay đổi trạng thái của mình ở bước này hay không.

Do đó, việc lựa chọn một hoặc một phương thức xuất thông tin khác được xác định bởi bản chất của việc xây dựng mô hình. Mô hình máy tính cơ sở sử dụng chế độ hỗn hợp: nội dung của các thanh ghi được chỉ định ô bộ xử lý và bộ nhớ được xuất ra ở cuối một bước nhất định (ví lệnh, chu trình máy hoặc lệnh) và phần còn lại của thiết bị - khi trạng thái của chúng thay đổi.

Nội dung của các chương trình cung cấp việc xử lý thông tin trong một mô hình chức năng không chỉ được xác định bởi độ phức tạp của bản gốc mà còn bởi các thiết bị của bản gốc và với bước lấy mẫu nào sẽ được hiển thị khi mô hình đang chạy. Vì vậy, các chương trình mô hình hóa máy tính vi mô với khả năng nghiên cứu hoạt động của nó trong

chế độ lệnh dễ dàng hơn nhiều so với các chương trình tương tự, trong bổ sung cung cấp cho việc nghiên cứu việc thực hiện các lệnh máy tính vi mô riêng lẻ cho các chu trình và chu trình máy. Điều này là do thực tế là mô hình chức năng phải tái tạo chính xác trạng thái của bản gốc chỉ tại thời điểm hiển thị nội dung của các thiết bị được hiển thị. Do đó, trong trường hợp đầu tiên, không cần thiết phải tạo lại mức phân sụn của bản gốc, đơn giản hóa các chương trình mô phỏng.

5.4. Nghiên cứu được thực hiện trên mô hình chức năng

Như đã đề cập ở trên, các mô hình chức năng không thể được sử dụng để chỉ nghiên cứu những khía cạnh của hoạt động ban đầu có liên quan đến sự biến đổi của các tín hiệu vật lý trong thời gian thực. Nhưng ứng dụng tiện lợi nhất của chúng là nghiên cứu nguyên lý cấu tạo và hoạt động của các thiết bị tính toán.

Do đó, mô hình của máy tính cơ sở được sử dụng để thực hiện các nghiên cứu khá chi tiết về sự tương tác của các thiết bị của nó trong quá trình lựa chọn và thực hiện các lệnh [3, 8]. Đối với điều này, nhỏ

chuỗi mã chứa các lệnh của chương trình tuyến tính (Bảng 5.1) và phân nhánh (Bảng 5.2), cũng như mã dữ liệu. Chuỗi mã được nhập vào bộ nhớ của máy tính cơ sở (xem đoạn 3.1), địa chỉ bắt đầu được nhập vào bộ đếm chương trình (địa chỉ lệnh, + "), máy được đặt ở chế độ được đánh dấu bằng ký hiệu

thực thi từng lệnh của chương trình và thực thi từng bước của nó được thực hiện. Nội dung của tất cả các thanh ghi bộ xử lý sau khi thực hiện mỗi lệnh được nhập vào một bảng tương tự như Table. 3.2, đã phân tích và bình luận.

Bảng 5.1				Bảng 5.2			
Chương trình tuyến tính Các ô				Chương trình phân nhánh			
nhớ Các ô nhớ				tế bào bộ nhớ tế bào bộ nhớ			
Địa chỉ	Nội dung	Địa chỉ	Nội dung	Địa chỉ	Nội dung	Địa chỉ	Nội dung
017	0000	01E	301A	017	54F7	01E	F100
018	0255	01F	F200	018	+ F200	01F	F000
019	7C99	020	4019	019	4022	020	3017
01A	0000	021	101A	01A	4023	021	C01F
01B	+ F200	022	301A	01B	9020	022	FFD1
01C	4024	023	F000	01C	F200	023	002F
01D	6018	024	C3CF	01D	3017	024	7C99

Sau đó, nhà nghiên cứu nên phản ánh: mục đích của chương trình; các hàm (công thức) được thực hiện bởi nó; khu vực nguồn dữ liệu và kết quả; vị trí trong bộ nhớ của chương trình, dữ liệu ban đầu và kết quả; địa chỉ của lệnh đầu tiên và lệnh cuối cùng của chương trình, v.v. Theo cách tương tự (sử dụng chương trình đã hoàn thiện hoặc đã biên dịch), hiệu quả của nhiều cách khác nhau để tổ chức các chương trình tuần hoàn (những công cụ máy tính cơ bản cho phép bạn xây dựng các thuật toán như vậy), hoạt động với các chương trình con (phương thức truyền tham số), v.v ... Nếu sử dụng một mô hình được xây dựng trên máy tính cá nhân Iskra 226, thì cấu trúc được hiển thị trong hình. 5,4, a,

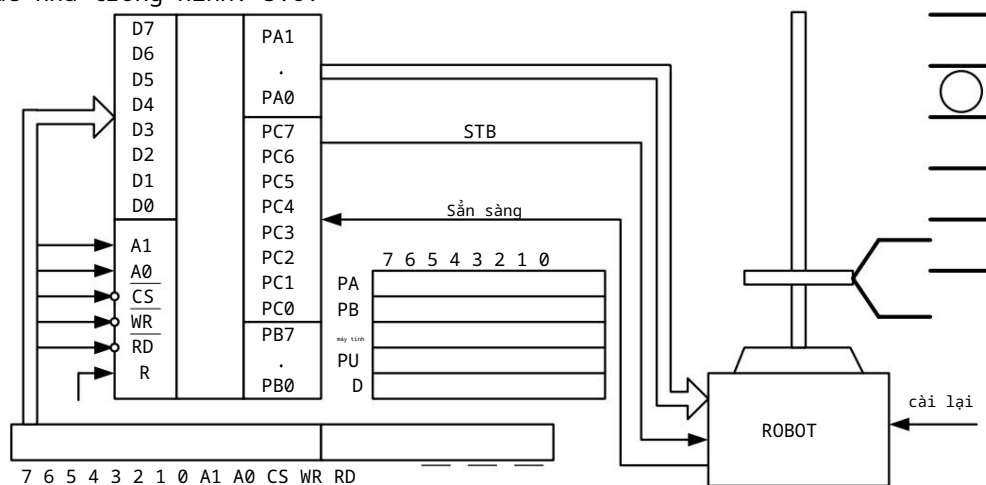
Với sự trợ giúp của cấu trúc được hiển thị trong Hình. 5.4, các thuật toán trao đổi thông tin với thiết bị ngoại vi (trao đổi không đồng bộ và trao đổi khi ngắt chương trình) được nghiên cứu. Nhà nghiên cứu đóng vai trò của các thiết bị đầu vào thông tin, tức là đặt mã của các ký tự được nhập bằng bàn phím và đưa ra tín hiệu sẵn sàng (đặt cờ bộ điều khiển).

Cuối cùng, sử dụng cấu trúc trong Hình. 5.5, hoạt động của máy tính cơ sở được nghiên cứu ở cấp độ vi chương trình. Vì trước tiên bạn nên tự làm quen với việc thực thi theo chu kỳ một số lệnh máy, và sau đó viết, nhập vào bộ nhớ vi lệnh và thực hiện một số lệnh mới (các lệnh có mã quang 7, D, FC, FD, FE và FF). Các lệnh như vậy có thể là các lệnh: nạp bộ tích lũy (opcode 7), đi chặn hoặc lẻ (opcode D). lấy thêm mã nội dung pin (mã hoạt động FC), v.v.

Các mô hình chức năng rất thuận tiện cho việc minh họa quá trình trao đổi dữ liệu tuần tự giữa máy tính và VU. Trong đó có thể dễ dàng chỉ ra những ảnh hưởng của nhiều đối với sự biến dạng của dữ liệu truyền, hoạt động của mạch điều khiển chặn lẻ (lẻ), lỗi do không đồng bộ của máy phát trong quá trình tổ chức trao đổi nối tiếp không đồng bộ, v.v.

P.

Công việc của bộ điều khiển nối tiếp cũng được minh họa rõ ràng. Vì thế. để nghiên cứu hoạt động của giao diện song song của bộ điều khiển "K" (ở đây, bộ điều khiển "K" của những năm 70-80 của thế kỷ trước - KR580VV55) sử dụng một mô hình có cấu trúc như trong hình. 5.6.



Com. 5.6. Màn hình của PC "Iskra 226" với cấu trúc được sử dụng khi nghiên cứu chip giao diện song song "K"

Một mặt, giao diện "K" được liên kết với bộ xử lý, vai trò của bộ xử lý này do nhà nghiên cứu thực hiện và mặt khác, với mô hình rô bốt nhà kho. Thông qua đó, các mệnh lệnh được truyền đi để thực hiện bất kỳ hành động nào của robot (lên, xuống, tiến, lùi, bóp, không rõ). Cấu trúc bên trong của giao diện "K" được biểu diễn như một tập các thanh ghi đầu ra của các cổng I / O, một thanh ghi điều khiển và một thanh ghi dữ liệu đệm.

Người mẫu song song giao diện được lập trình bởi nhà nghiên cứu ở chế độ cần thiết cho việc trao đổi. Sau đó, nhà nghiên cứu ghi vào giao diện song song thứ tự mà robot nhận được và hai từ tạo thành công. Sau khi nhận được nhấp nháy, robot đọc lệnh và thực hiện một trong các hành động trên. Để di chuyển một phần tử ô này sang ô khác, nhà nghiên cứu (mô phỏng hoạt động của bộ xử lý) cần chuyển từ 20 đến 30 8-

từ bit. Các lỗi xảy ra do lệnh không hợp lệ hoặc các nguyên nhân khác được loại bỏ bằng cách truyền tín hiệu "Reset Robot Control".

Việc tạo ra các mô hình thiết bị xử lý riêng lẻ là vô cùng dễ dàng. Chúng thể hiện rất rõ ràng công việc của các ALU, bộ chuyển số, các mạch tổ hợp khác nhau, v.v.

ỨNG DỤNG

phụ lục A

Các bảng chuyển đổi số từ hệ thống số này sang hệ thống số khác

Bảng A.1 So

sánh các mã số thập phân, nhị phân, bát phân và thập lục phân

0	0	0	0		1011	13	B		22	1 0110	26	16
				12	1100	14	C		23	1 0111	27	17
2	1	2	2	13	1101	15	D		24	1 1000	30	18
3	10 11	3	3	14	1110	16	E		25	1 1001	31	19
bốn	100	bốn	bốn	15	1111	17	F		26	1 1010	32	1A
5	101	5	5	16	10000	18	mười		27	1 1011	33	1B
6	110	6	6	17	1 0001	19			28	1 1100	34	1C
7	111		7	18	1 0010	20	12		29	1 1101	35	1D
8	1000	7	8	19	1 0011	21	13		30	1 1110	36	1E
9	1001	10 11	9	20	1 0100	22	14		31	1 1111	37	1F
mười	1010	12	Một	21	1 0101	23	15		32	10.000	40	

Bảng A.2

Giá trị của lũy thừa của một số 2

N	2 ^N		N	16 ^N	32 ^N	64 ^N	128 ^N		N		N	
0			bốn						12	24096	16	2 65
1	1		5				8		13	8192	17	536 131
2	2		6				9		14	16384	18	072 262
3	4 8		7				10 11	2.256.512	15	024 2.048	19	144 524 288

Bảng A.3

Chuyển đổi số thập phân thành số thập lục phân 3

nhân ĐT		2		bốn	5	6	7	tám	9
1		2 14	1E	4	5 32	3C	7 46	8	5A
10	A64	C8	12C	28 190	1F4	258	2BC	50 320	384
102	3E8	7D0	BB8	FA0	1388	1770	1B58	1F40	2328
103	2710	4E20	7530	9C40	C350	EA60	11170	13880	15F90
104	186A0	30D40	493E0	61A80	7A120	927C0	AAE60	C3500	DBBA0
105 106	F4240	1E8480	2DC6C0	3D0900	4C4B40	5B8D80	6ACFC0	7A1200	895440
Ví dụ: 1234 = 1000 + 200 + 30 + 4 = (3E8) 16+ (C8) 16+ (1E) 16+ (4) 16 = (4D2) 16									

PHỤ LỤC B

Mã hóa ký tự

Như đã lưu ý trong Chương 1, máy tính lưu trữ tất cả thông tin dưới dạng byte nhị phân, nghĩa là, các đơn vị 8 bit có thể nhận giá trị từ 0 đến 255. Để lưu trữ thông tin dạng văn bản chứ không phải dạng số trong bộ nhớ máy tính, cần để xác định, byte hoặc các byte nào sẽ mã hóa từng ký tự có thể xuất hiện trong văn bản của chúng ta. Sự tương ứng này giữa các ký tự và các byte mã hóa chúng được gọi là mã hóa ký tự.

(bộ ký tự). Có thể hiểu đơn giản, thứ nhất, mỗi bảng mã được phát triển cho một ngôn ngữ cụ thể của con người (chính xác hơn là cho một hệ thống chữ viết cụ thể), và thứ hai, cho bất kỳ ngôn ngữ nào bạn có thể tạo ra bao nhiêu bảng mã như bạn muốn. Biết được bản chất con người, không khó để đoán rằng họ sẽ nghĩ ra nhiều thứ hơn mức cần thiết. Đương nhiên, điều này đã xảy ra: Thư viện tiên tiến nhất của IBM về các hàm chuyển đổi ICU (International Components for Unicode) cho đến nay hỗ trợ hơn 170 bảng mã khác nhau.

Bảng mã Latinh

Hãy để chúng tôi xem xét chi tiết hơn về các mã hóa của những tập lệnh mà người dùng Nga thường gặp nhất, tức là tiếng Latinh và Cyrillic. Đối với tiếng Latinh ngày nay, hai bảng mã chính được sử dụng: ASCII và EBCDIC. ASCII (American Standard Code for Information Interchange) là một bảng mã bảy bit (mã ký tự 00 - 7F hoặc 0 - 127 thập phân) đã trở thành tiêu chuẩn cho các máy tính vừa và nhỏ. Trong đó, các byte có mã thập lục phân 00 - 1F và 7F được dùng để mã hóa các ký tự điều khiển (không hiển thị), phần còn lại mã hóa các ký tự sau:

20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F
	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F
0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F
‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	7F
p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Bảng mã kirin

Bảng mã của các chữ cái "không phải Latinh" được sắp xếp như sau. Chúng được mã hóa bằng bảng tám bit (1 byte = 1 ký tự), tức là số 00 - FF (0 - 255 thập phân) để nửa dưới của bảng mã (mã 00 - 7F hoặc 0 - 127 thập phân) khớp với ASCII và nửa cao (mã 80 - FF hoặc 128 -

255 thập phân) chứa mã hóa quốc gia, tức là các chữ cái tiếng Nga trong bảng mã tiếng Nga, tiếng Thổ Nhĩ Kỳ trong tiếng Thổ Nhĩ Kỳ, v.v. Tổ chức bảng mã quốc gia này cho phép bạn hiển thị và xử lý chính xác các chữ cái, số và dấu câu Latinh trên bất kỳ máy tính nào, bất kể hệ thống của nó. cài đặt. Đặc biệt, đây chính xác là cách sắp xếp các bảng mã của người Nga, để sau này chúng ta chỉ có thể xem xét nửa cũ của họ.

Lịch sử mã hóa tiếng Nga là một ví dụ về sự nhầm lẫn, hiếm gặp ngay cả đối với thực tế máy tính của chúng ta. Các tổ chức tiêu chuẩn hóa của Liên Xô đã chấp nhận GOST, các nhà sản xuất máy tính (Apple) và hệ điều hành (Microsoft) nhất trí bỏ qua chúng và giới thiệu các bảng mã của riêng họ. Do đó, chúng tôi kế thừa từ bốn GOST khác nhau, hai mã hóa từ Microsoft (cho DOS và Windows) và một mã hóa từ Apple cho Mac (tất nhiên, tất cả đều không tương thích với nhau).

May mắn thay, ngày nay không cần phải mô tả chi tiết tất cả các mã hóa này, vì chỉ có hai trong số chúng tồn tại trên RuNet. Đầu tiên là KOI8-R (KOI là viết tắt của Mã trao đổi và xử lý thông tin, 'P' phân biệt bằng mã tiếng Nga với bảng mã tiếng Ukraina, KOI8-U. Bảng mã này trông giống như sau:

80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
–		Г	г	Л	л	Т	т	Т	т	+	■	■	■	■	■
90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
▤	▥	▦	▧	■	●	√	≈	≤	≥	Ј	Ѓ	◌◌	2	3	÷
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF
=		Г	ё	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF
Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г	Г
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
ю	а	б	ц	д	е	ф	г	х	и	й	к	л	м	н	о
D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF
п	я	р	с	т	у	ж	в	ь	з	ш	э	щ	ч	ф	ъ
E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF
ю	а	б	ц	д	е	ф	г	х	и	й	к	л	м	н	о
F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF
п	я	р	с	т	у	ж	в	ь	з	ш	э	щ	ч	ф	ъ

KOI8-R là tiêu chuẩn thực tế cho tất cả các dịch vụ Internet ngoại trừ WWW. Đặc biệt, tất cả các dịch vụ e-mail và tin tức của Runet đều hoạt động ở chế độ mã hóa này. Đối với Internet, tình hình phức tạp hơn. Thực tế là hơn 90% máy khách trên Web đang chạy Windows với các phiên bản khác nhau. Windows sử dụng bảng mã riêng của các chữ cái tiếng Nga, thường được đặt tên theo số trang mã Windows-1251 hoặc CP1251:

80	81	82	83	84	85	86	87	88	89	9A	9B	9C	9D	9E	9F
Ъ	Г	,	Г	,,	...	†	‡	€	%	Ь	<	Ь	Ќ	Т	У
Ъ	Г	Г	Г	Г	•	-	-		™	Ь	>	Ь	Ќ	Т	У
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF
Ў	Ў	Ў	Ј	Х	Г	!	§	Ё	©	€	«	¬	-	®	Ї
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF
°	±	І	і	Г	μ	¶	•	ё	№	»	Ј	Ѕ	Ѕ	Ѕ	Ѕ
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О	П
D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF
Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю	Я
E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF
а	б	в	г	д	е	ж	з	и	й	а	л	м	н	о	п
F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF
р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я

Vì các trình soạn thảo văn bản và các công cụ phát triển trang HTML trong Windows hoạt động ở chế độ mã hóa này, nên phần lớn các tài liệu Web Runet được lưu trữ trong mã hóa Windows-1251.

KHOA KỸ THUẬT MÁY TÍNH

Về bộ phận

Khoa Khoa học Máy tính tại SPbGU ITMO được thành lập vào năm 1937 và là một trong những các trường khoa học và sư phạm lâu đời nhất và có thẩm quyền nhất ở Nga.

Ban đầu, bộ phận này được gọi là Khoa Toán học và Máy tính và Thiết bị và đã tham gia vào việc phát triển các thiết bị tính toán cơ điện và các thiết bị điều khiển. Bộ phận nhận được tên hiện tại của nó vào năm 1963.

Khoa Kỹ thuật Máy tính là một trong những khoa lớn nhất của trường, sử dụng các chuyên gia có trình độ cao, bao gồm 8 giáo sư và 15 phó giáo sư, giảng dạy cho khoảng 500 sinh viên và 30 nghiên cứu sinh.

Khoa có 4 lớp máy tính kết hợp hơn 70 máy tính vào mạng cục bộ của khoa và cung cấp cho sinh viên quyền truy cập vào tất cả các nguồn thông tin của khoa và truy cập Internet. Ngoài ra, khoa có các phòng thí nghiệm giáo dục và nghiên cứu về công nghệ máy tính, trong đó sinh viên của khoa làm việc.

Chúng tôi dạy gì

Theo truyền thống, tại Khoa Khoa học Máy tính, trọng tâm chính trong việc đào tạo các chuyên gia là đào tạo cơ bản nền tảng trong khuôn khổ các chuyên môn chung và các chuyên ngành đặc biệt bao gồm các phần quan trọng nhất của công nghệ máy tính: mạch chức năng và công nghệ vi xử lý, thuật toán hóa và lập trình, thông tin. hệ thống và cơ sở dữ liệu, công nghệ đa phương tiện, mạng máy tính và các phương tiện viễn thông, bảo vệ thông tin và an ninh thông tin. Đồng thời, khoa cung cấp cho sinh viên năm cuối cơ hội chuyên sâu vào các lĩnh vực chuyên môn hẹp hơn phù hợp với sở thích của họ.

Chuyên môn để lựa chọn

Là một phần của đào tạo kỹ sư và thạc sĩ, Khoa Khoa học Máy tính của ITMO cung cấp cho sinh viên sự lựa chọn trong 3 chuyên ngành.

1. Chuyên ngành hệ thống thông tin và điều khiển nhằm đào tạo các chuyên gia có khả năng thiết kế và phát triển các hệ thống điều khiển thời gian thực dựa trên công nghệ vi xử lý. Đồng thời, sinh viên theo học chuyên ngành này có cơ hội duy nhất để tham gia vào các phát triển cụ thể của thiết bị thực, nghiên cứu

tất cả các giai đoạn thiết kế và sản xuất, cho đến sản phẩm cuối cùng. Với mục đích này, một phòng thí nghiệm đào tạo và sản xuất đặc biệt được trang bị các thiết bị hiện đại nhất đã được tổ chức tại khoa. Cần lưu ý rằng gần đây, do sự trỗi dậy của ngành công nghiệp trong nước, các chuyên gia phát triển và thiết kế hệ thống thông tin và điều khiển ngày càng trở nên có nhu cầu nhiều hơn, không chỉ ở Nga mà còn ở nước ngoài.

2. Khoa Kỹ thuật Máy tính là một trong những khoa đầu tiên bắt đầu đào tạo các chuyên gia trong lĩnh vực thông tin mở và máy tính, ngày nay, sinh viên chuyên về lĩnh vực này có cơ hội học tập và làm chủ một trong những công nghệ tiên tiến nhất - hệ quản trị cơ sở dữ liệu Oracle. Đồng thời, các yêu cầu gia tăng về tài nguyên máy tính, với sự trợ giúp của cơ sở dữ liệu được thực hiện trong môi trường Oracle, được đáp ứng bằng cách tổ chức một lớp máy tính chuyên biệt tại bộ phận, được trang bị máy tính mạnh mẽ của SUN Microsystems, kết nối với mạng cục bộ của bộ phận. Đồng thời, sinh viên chuyên ngành này được đào tạo cơ bản tốt về lĩnh vực hệ thống thông tin, cho phép họ phát triển thành công cơ sở dữ liệu và kiến thức không chỉ trong môi trường Oracle mà còn trên nền tảng của bất kỳ hệ quản trị cơ sở dữ liệu nào khác. hoàn thành các nghiên cứu của họ.

3. Và, tất nhiên, bộ phận không thể tránh khỏi sự tấn công của mạng máy tính và viễn thông trong lĩnh vực công nghệ máy tính. Sự hiện diện của các nhân sự chuyên môn cao trong lĩnh vực này và cơ sở kỹ thuật phù hợp tại bộ phận (hai mạng cục bộ kết hợp khoảng 70 máy tính và cung cấp khả năng làm việc trong các môi trường hoạt động khác nhau -

Solaris, Linux, Windows), cho phép tổ chức đào tạo các chuyên gia trong lĩnh vực này, bao gồm nghiên cứu về bảo mật máy tính, quản trị, tối ưu hóa và thiết kế mạng máy tính.

THƯ MỤC

1. Máy vi tính cá nhân gia dụng "Điện tử BK0010" / S.M. Kosenkov, A.N. Polosin, Z.A. Schepitsky và những người khác // Các công cụ và hệ thống vi xử lý. Năm 1985 # 1. trang 22-25.
2. Giới thiệu về máy vi tính / S.A.Mayorov, V.V.Kirillov, A.A.Pribluda. L.: Kỹ thuật cơ khí. Chi nhánh Leningrad, 1988. 304 trang: bìa.
3. Hướng dẫn làm việc trong phòng thí nghiệm trên máy tính cơ sở. L.: Viện cơ khí chính xác và quang học, 1986, 54 tr.
4. Máy tính cá nhân chuyên nghiệp "Iskra 226" / S.N. Abramovich, V.V. Boyko, B.P. Buturin và những người khác // Các công cụ và hệ thống vi xử lý. 1985 số 2. trang 29-36.
5. Souchek B. Vi xử lý và vi máy tính / Trans. từ tiếng Anh. Ed. A.I. Petrenko. M.: Sov. Radio, 1979, 520 tr.
6. Tanenbaum E. Tổ chức đa cấp của máy tính / TRANS. từ tiếng Anh. Ed. V.K. Pototsky và M.B. Ignatiev. M.: Mir, 1979. 547 tr.
7. Kỷ yếu của Viện Kỹ sư Điện tử và Vô tuyến điện tử (TIIER) / Per. từ tiếng Anh. M: Mir, 1984. T.72. Số 3 (Chuyên đề "Máy tính cá nhân"). 187 tr.
8. Hướng dẫn giáo dục và phương pháp cho công việc phòng thí nghiệm trong các ngành "Tin học" và "Nhập môn chuyên ngành". - St.Petersburg: SPbGU ITMO, 2010. - 120 giấy.

CHỈ SỐ ĐỐI TƯỢNG

bộ tích lũy, 24, 25, 36, 37
đơn vị logic số học (ALU), 10, 21, 29, 34, 36 đơn
vị logic số học (ALU), 38

B

byte,
6 lệnh không giải nén, 27, 33, 43,
44 bit, 6

TẠI

mạch cổng, 9, 102 mã hóa
bát phân và thập lục phân, 41 tìm nạp
địa chỉ, 68 tìm nạp lệnh, 67 thực
thi phần sụn, 108 máy tính, 3

D

nhị phân một, hệ thống
số nhị phân 7, dạng nhị phân 6, 16
lệnh, 41 nhị phân không, 7

Và

ô chỉ mục, 59 trình
thông dịch máy chủ, 110 thông dịch,
92 thực thi lệnh, 30, 34, 39

Đến

phân loại lệnh, 42 mã hoạt
động, 23, 37, 82 lệnh máy
tính, 23 lệnh I / 0, 43, 44,
82 lệnh chương trình con, 63, 71
biên dịch, 92

M

vi cấu trúc
vận hành, điều khiển
106, 107 vi chương
trình, 33, 95, 111 máy tính vi
lập trình, thiết bị điều khiển vi
chương trình 95, mã hóa 36, 72, 102, 103 ghi nhớ
(tượng trưng),

41
máy tính đa cấp, 93

H

nanomemory, 120

O

trao đổi thông tin
không đồng bộ, 81
chương trình gián đoạn, 81 đồng bộ,
81
lệnh unicast
số học, 28 I / 0, 28
chuyển điều khiển, 28
chuyển giao, 28 gỡ lỗi chương
trình, 77

P

bộ nhớ máy tính,
truyền dữ liệu 22, 33 giữa các thanh ghi,
xử lý dữ liệu 96 bit, truyền dữ liệu 60
điều khiển chương trình, bộ xử lý 80, 20, 21, 24

R

thanh ghi
địa chỉ (RA), 30, 37
dữ liệu (RD), 30, 37
lệnh (RK), 29, 37 lệnh
(IC), 30, 36 vi lệnh
(RMC), 109 định nghĩa, 7
chuyển giao, 37 trạng thái
(RS), 29

TỪ

trao đổi đồng bộ, 81
từ, 6 cấu trúc máy tính,
35 bộ đếm lệnh, 30 bộ
đếm lệnh, 36 bộ đếm vi
lệnh, 95

T

xung đồng hồ, 9 bộ
dịch, 3

Tại

thiết bị đầu vào-đầu ra, 20
thiết bị điều khiển, 10, 21

F

mô hình vật lý, 122 dạng
tương tự biểu diễn thông tin, 5 rời
rạc, 5 liên tục, 5 kỹ thuật số, 5
định dạng lệnh và phương pháp định
địa chỉ, 44 mô hình chức năng, 122

W

lốp xe, 9

E

phần tử bộ nhớ, 7

Tôi

ngôn ngữ vi lập trình, 110 vị trí bộ
nhớ, 7

Mục lục GIỚI

THIỆU	một
1. THÔNG TIN CHUNG VỀ ĐẠI DIỆN VÀ XỬ LÝ THÔNG TIN TRONG MÁY TÍNH NH	3
1.1. Hai hình thức trình bày thông tin	3
1.2. Cách trình bày thông tin rời rạc	5
1.3. Hệ thống số được sử dụng trong công nghệ máy tính	16
1.4. Cấu trúc và nguyên lý hoạt động của máy tính	hai mươi
2. MÁY TÍNH CƠ BẢN	35
2.1. Mục đích và cấu trúc của máy tính cơ sở	35
2.2. Hệ thống lệnh và mã hóa chương trình	41
2.3. Các phép tính toán học	46
2.4. Điều khiển quá trình điện toán,	54
2.5. Chương trình con và tham số	63
2.6. Thực hiện các lệnh máy	66
3. TỔ CHỨC CÔNG TÁC ĐẦU VÀO THÔNG TIN	73
BẢN	73
3.1. Bảng điều khiển gỡ lỗi của máy tính cơ sở	73
3.2. Thiết bị I / O của máy tính cơ sở	79
3.3. Truyền thông tin do chương trình điều khiển	80
3.4. Trao đổi không đồng bộ	84
3.5. Trao đổi ngắt chương trình	86
4. MÁY TÍNH CƠ SỞ THIẾT BỊ ĐIỀU KHIỂN PHẦN MỀM	92
4.1. Máy tính đa cấp và mức vi chương trình của chúng	92
4.2. Các thành phần bộ xử lý và các hoạt động cơ bản	96
4.3. Phần mềm điều khiển mạch cổng	106
4.4. Trình thông dịch máy tính cơ sở	110
4.5. Các tùy chọn khác để xây dựng vi cấu trúc	116
5. MÔ PHỎNG CÁC QUÁ TRÌNH CHỨC NĂNG CỦA VI MÔ 122	122
5.1. Phương pháp tiếp cận mô hình trong nghiên cứu các thiết bị công nghệ vi xử lý	122
5.2. Tương tác với mô hình chức năng	124
5.3. Xây dựng mô hình chức năng	130
5.4. Nghiên cứu thực hiện trên mô hình chức năng	132
CÁC ỨNG DỤNG	136
Phụ lục A	136
Các bảng chuyển đổi số từ hệ thống số này sang hệ thống số khác	136
136 Phụ lục B	137
Bảng mã ký tự	137
KHOA KỸ THUẬT MÁY TÍNH NH	139
BIBLIOGRAPHY	141
CHỈ SỐ ĐỐI TƯỢNG	142

Kirillov Vladimir Vasilievich

KIẾN TRÚC MÁY TÍNH CƠ BẢN

Hướng dẫn

Trong ấn bản của tác giả

Thiết kế Kirillov V.V., Pribluda A.A.

Bố cục Kirillov V.V., Pribluda A.A., Lemeshev A.S.

Phòng Biên tập và Xuất bản của Đại học Công nghệ Thông tin, Cơ học và Quang học
St.Petersburg

Cái đầu RIO N.F. Gusarova

Giấy phép ID số 00408 ngày 1.1.199

Đã ký để in Đơn hàng số.

Vòng tuần hoàn

In trên risograph

Phòng Biên tập và Xuất bản của Đại học Công
nghệ Thông tin Bang St.Petersburg,

cơ học và quang học

197101, St.Petersburg, Kronverksky pr., 49

